Autonomous and Mobile Robotics

Comprehensive Lecture Notes

Vittorio Cava

A.Y. 2026/27

Contents

I Lecture 1 **2**

1 Configuration Space and Workspace **2**

1.1 The idea . . . 2
1.2 Robots and workspaces . . . 2
1.3 Configuration and generalized coordinates . . . 2
1.4 Configuration space . . . 3

2 Examples of Configuration Spaces **3**

3 Multi-Body Robots: Manipulators **5**

3.1 Planar manipulator with n_b links . . . 5
3.2 Spatial manipulator with n_b links . . . 6

4 Topology of $\mathcal{C}$ **7**

4.1 The problem with a flat picture of the 2R robot . . . 7
4.2 Wrapping the space: the torus . . . 8
4.3 Manifolds . . . 8

5 Distances in $\mathcal{C}$ **9**

5.1 What a distance must satisfy . . . 9
5.2 Distances on manifolds and geodesics . . . 9
5.3 The displacement metric . . . 10
5.4 The max over infinitely many points . . . 11
5.5 A practical alternative: control points . . . 12

6 Summary **13**

6.1 Configuration spaces seen in this lecture . . . 13
6.2 Distances on $\mathcal{C}$ . . . 13
6.3 Key take-aways . . . 13

II Lecture 2 **15**

III Mechanics of Mobile Robots **15**

1 Outline of this lecture *(slides 1–2)* **15**

2 Ground locomotion *(slide 3)* **15**

3 Balance (not falling) *(slides 4–5)* **16**
3.1 Static balance and the support polygon . . . 16
3.2 Dynamic balance and the Zero Moment Point *(slides 4–5)* . . . 17

4 Wheels: 3 basic types *(slides 6–8)* **18**
4.1 Fixed wheel *(slide 6)* . . . 18
4.2 Orientable (steerable) wheel *(slide 7)* . . . 18
4.3 Caster wheel *(slide 8)* . . . 18

5 WMR kinematic structures *(slides 9–15)* **19**
5.1 Differential-drive mobile robot *(slide 9)* . . . 19
5.2 Synchro-drive mobile robot *(slide 10)* . . . 19
5.3 Tricycle and car-like robots *(slide 11)* . . . 20
5.4 The differential *(slide 12)* . . . 21
5.5 Firetruck *(slide 13)* . . . 21
5.6 Omnidirectional robots with actuated caster wheels *(slide 14)* . . . 22
5.7 Mecanum (Swedish) wheels *(slide 15)* . . . 22

IV Wheels: Constraints and Mobility **23**

6 Wheels (constraints & mobility) *(page 1)* **23**

7 Geometric constraints (GC) *(pages 1–2)* **24**
7.1 Constraints on the configuration *(page 1)* . . . 24
7.2 Mobility limitation and reduced coordinates *(page 2)* . . . 25

8 Kinematic constraints (KC) *(page 3)* **26**

9 Relationships between GC and KC *(pages 4–6)* **26**
9.1 Any GC implies a KC *(pages 4–5)* . . . 26
9.2 The converse: integrable vs. non-integrable *(pages 5–6)* . . . 27

10 Example: rolling wheels *(pages 7–9)* **28**
10.1 The rolling disk and the pure rolling constraint *(page 7)* . . . 28
10.2 Is pure rolling integrable? *(page 8)* . . . 29

10.3 No global mobility limitation *(page 9)* 30

11 Summary **32**

11.1 Part I: kinematic structures of WMRs 32

11.2 Part II: constraints and mobility 33

11.3 Key take-aways 33

V Lecture 3 **34**

1 Kinematic constraints and admissible motions **34**

2 Explicit representation: the kinematic model **35**

2.1 The unconstrained robot 35

3 Controllability of kinematic models **36**

4 The unicycle **37**

4.1 The pure rolling constraint 37

4.2 Interpretation of the inputs 38

5 Controllability of the unicycle **38**

6 Mechanically equivalent vehicles **40**

6.1 Differential drive 40

6.2 Synchro-drive 41

7 Summary **41**

VI Lecture 4 **43**

1 Recap: kinematic models and the unicycle **43**

2 The bicycle model **43**

2.1 Configuration and constraints 43

2.2 Writing the front constraint with rear-wheel coordinates 44

2.3 Rear-wheel drive (RWD) kinematic model 45

3 The front-wheel drive case **45**

4 Controllability **46**

4.1 Is there a universal manoeuvre? 46

4.2 Nonlinear controllability 47

5 The Lie bracket **47**

6 Interpretation of the Lie bracket **48**

7 Distributions, accessibility and Chow's theorem **49**

8 Application to the unicycle **50**

9 Application to the bicycle **51**

10 Summary **53**

10.1 Comparison of the models 53

10.2 Key take-aways 53

VII Lecture 5 **55**

1 Vehicles Kinematically Equivalent to the Bicycle **55**

1.1 Ackermann steering 55

2 Chained Forms **56**

2.1 Controllability 57

2.2 Why chained forms are useful 57

3 Chained Form of the Unicycle **58**

4 Chained Form of the Bicycle **59**

5 Adding Trailers **59**

6 Class Test 2009 – Problem 1: Tricycle with a Trailer **59**

6.1 Generalized coordinates 60

6.2 Pfaffian kinematic constraints 60

6.3 Kinematic model 61

7 Worked Exercise: A Rolling Wheel on the Plane **62**

7.1 Configuration space and kinematic model 62

7.2 Constraints 62

7.3 Controllability 63

7.4 A sequential manoeuvre 63

8 Summary **64**

VIII Lecture 6 **66**

1 Planning admissible motions *(slides 3)* **66**

2 Space-time separation *(slide 4)* **66**

3 Admissible paths and geometric model *(slides 5–8)* **67**
3.1 Geometric path admissibility *(slides 5–6)* . . . 67
3.2 The unicycle example *(slides 7–8)* . . . 69

4 Flat outputs *(slides 9–13)* **70**
4.1 Differential flatness *(slides 9–10)* . . . 70
4.2 Flatness and the chained form *(slides 11–12)* . . . 72
4.3 An algorithm for planning with flat outputs *(slide 13)* . . . 73

5 Unicycle path planning via Cartesian polynomials *(slides 14–15)* **73**

6 Unicycle path planning via the chained form *(slides 16–17)* **74**

7 Path planning via parametrized inputs *(slides 18–21)* **76**
7.1 Chained form with parametrized inputs *(slides 18–20)* . . . 76
7.2 Other parametrizations and trajectory planning *(slide 21)* . . . 77

8 Parking a unicycle: numerical results *(slides 22–27)* **77**
8.1 Planner based on Cartesian polynomials *(slides 22–24)* . . . 77
8.2 Planner based on parametrized inputs *(slides 25–27)* . . . 78

9 Choice of timing law *(slides 28–30)* **79**
9.1 Choosing $s(t)$ *(slide 28)* . . . 79
9.2 Linear timing law for a unicycle with velocity bounds *(slides 29–30)* . . . 80

10 Summary **81**

IX Lecture 7 **83**

1 Toolbox: the Recurring Ideas **83**
1.1 Pfaffian constraints and kinematic models . . . 83
1.2 Controllability by Lie brackets . . . 83
1.3 Differential flatness and reconstruction . . . 84
1.4 Dynamic feedback linearization . . . 84

2 The Single-Legged Hopping Robot **84**
2.1 Setting . . . 85
2.2 (a) Geometry of the configuration space . . . 85
2.3 (b) A kinematic model with physically meaningful inputs . . . 86
2.4 (c) Local and global mobility . . . 86
2.5 (d) Feedback control in flight: $\theta \to 0$, $\ell \to h$ . . . 87
2.6 (e) Differential flatness for $d = 0$ and path planning . . . 88

3 Planning for a $(2,3)$ Chained Form **90**

3.1 Setting . . . 90
3.2 (a) The algorithm . . . 91
3.3 (b) Free parameters and how to choose them . . . 92
3.4 (c) The case $_i = (0,0,0)$, $_f = (1,1,1)$ . . . 92

4 Front-Wheel-Drive Bicycle: Modelling, Flatness and Control 93
4.1 Setting . . . 94
4.2 (a) Kinematic model and controllability . . . 94
4.3 (b) Are x_f, y_f flat outputs? . . . 95
4.4 (c) Tracking controller for (x_f, y_f): dynamic feedback linearization . . . 96

5 Summary 98
5.1 Comparison of the three problems . . . 98
5.2 Key points . . . 98

X Lecture 8 100

1 Motion Control: Setting the Problem *(slides 2–4)* 100
1.1 Why feedback, and why a kinematic model . . . 100
1.2 Two motion control problems . . . 101

2 Trajectory Tracking: State Error Feedback *(slides 5–7)* 101
2.1 Admissible trajectories and flatness . . . 101
2.2 The control scheme . . . 102
2.3 Error coordinates and error dynamics . . . 102

3 Approximate Linearization: Brush-up *(slides 8–9)* 103

4 Tracking via Approximate Linearization *(slides 10–16)* 104
4.1 Input transformation: making the origin an equilibrium . . . 104
4.2 The linear approximation . . . 104
4.3 Gain tuning . . . 105
4.4 Actual inputs, limitations and a global version . . . 106

5 Trajectory Tracking: Output Error Feedback *(slide 17)* 107

6 Exact I/O Linearization: Brush-up *(slides 18–19)* 108

7 Tracking via I/O Linearization: Static Feedback *(slides 20–24)* 109
7.1 The singularity for the output (x, y) . . . 109
7.2 Displacing the output to a point B . . . 109
7.3 What happens to θ: zero dynamics . . . 110
7.4 Properties and block scheme . . . 111

8 Tracking via I/O Linearization: Dynamic Feedback *(slides 25–28)* **111**
8.1 Dynamic extension . . . 111
8.2 Control law . . . 112

9 Simulations *(slides 29–40)* **113**

10 Summary **114**

XI Lecture 9 **115**

XII Motion control: regulation **115**

1 Regulation: the problem and its limits *(regulation slide 2)* **115**

2 Cartesian regulation *(regulation slides 3–6)* **116**
2.1 Problem and geometry *(regulation slide 3)* . . . 116
2.2 Control law *(regulation slide 4)* . . . 117
2.3 Brush-up: Lyapunov stability of nonlinear systems *(background to regulation slide 5)* . . . 117
2.4 Convergence analysis *(regulation slide 5)* . . . 121
2.5 Simulation *(regulation slide 6)* . . . 122

3 Posture regulation *(regulation slides 7–10)* **123**
3.1 Polar coordinates *(regulation slide 7)* . . . 123
3.2 Kinematic model in polar coordinates *(regulation slide 8)* . . . 124
3.3 Control law design via Lyapunov's direct method *(regulation slides 8–9)* . . . 126
3.4 Convergence analysis and Brockett's theorem *(regulation slide 9)* . . . 127
3.5 Simulation *(regulation slide 10)* . . . 128

XIII Mobile manipulators **128**

4 Mobile manipulators *(manipulator slides 1–2)* **128**

5 Kinematic model *(manipulator slide 3)* **129**

6 Path/trajectory planning *(manipulator slide 4)* **132**

7 Differential kinematics *(manipulator slide 5)* **132**

8 Mobile manipulator Jacobian *(manipulator slide 6)* **132**

9 Example: unicycle base with a 2R arm *(manipulator slides 7–8)* **133**

10 Inverse differential kinematics *(manipulator slides 9–10)* **135**

10.1 Redundancy resolution . . . 135
10.2 Why the gradient cannot be used directly . . . 135
10.3 The right choice of u_N . . . 135

11 Kinematic control *(manipulator slide 11)* 137

12 Summary 138

XIV Lecture 10 140

1 Why localization is needed odometry deck, slide 2 140

2 Odometric localization: integrating the kinematic model odometry deck, slides 3–6 140
2.1 Setting: constant inputs over a sampling interval . . . 140
2.2 Three integration schemes . . . 141
2.3 How different are they? Geometry and error order . . . 142

3 Reconstructing the inputs from proprioceptive sensors odometry deck, slide 7 143

4 Dead reckoning and odometric drift odometry deck, slides 8–9 144

5 From dead reckoning to probabilistic localization EKF deck, slide 2 146

6 Basic concepts: mean, covariance, Gaussians EKF deck, slides 3–4 146

7 Kalman filter without noise EKF deck, slides 5–9 147
7.1 Recursive observer: prediction and correction . . . 147
7.2 The correction as a constrained optimization . . . 147
7.3 Why this naive observer is not enough . . . 148

8 Kalman filter with process noise only EKF deck, slides 10–15 149
8.1 Prediction . . . 149
8.2 State correction: Mahalanobis distance and weighted pseudoinverse . . . 149
8.3 Covariance correction and its flaw . . . 150

9 Kalman filter: the full version EKF deck, slides 16–21 151
9.1 State correction with a noisy measurement . . . 151
9.2 Properties of the KF . . . 152

10 The Extended Kalman Filter EKF deck, slides 22–24 153

11 EKF localization of a unicycle with range-and-bearing landmarks book, Sect. 7.7.2 154
11.1 Motion model . . . 154

11.2 Measurement model 154
11.3 The EKF equations for this problem 155

12 Summary **156**
12.1 Comparison of methods 156
12.2 Key take-aways 156

Part I

Lecture 1

1 Configuration Space and Workspace

(notes p. 1–2)

1.1 The idea

The central idea is to *define a suitable space in which the robot is a single point.* Once the robot is a point, two core tasks become much simpler to formulate:

- **planning** (find a path for the point from a start to a goal),
- **control** (make the point follow that path).

(Fun fact from the notes: the C-space concept was born to do *collision avoidance.*)

Intuition

Think of a huge, awkwardly shaped sofa that has to be moved through a door. Reasoning about every corner of the sofa at the same time is painful. If instead we find a space where the whole sofa is *one dot*, and the "forbidden" places for the dot are exactly the sofa positions that hit a wall, then moving the sofa becomes moving a dot around obstacles.

1.2 Robots and workspaces

Definition 1.1 (Robot and workspace)**.** A *robot* is a collection of n_b rigid bodies (we assume rigid robots) moving in a *workspace* $\mathcal{W} = \mathbb{R}^2$ or $\mathcal{W} = \mathbb{R}^3$, the physical space in which the robot moves.

- If $\mathcal{W} = \mathbb{R}^2$ the robot is *planar*; if $\mathcal{W} = \mathbb{R}^3$ it is *spatial.*
- If $n_b = 1$ the robot is *single-body*; if $n_b > 1$ it is *multi-body.*

Example 1.2. A car can be modelled as a single body, whereas a car with a trailer is a multi-body system ($n_b = 2$).

1.3 Configuration and generalized coordinates

Definition 1.3 (Posture and configuration)**.** The *posture* of a robot is the position of all its points in $\mathcal{W}$. A *configuration* is a *minimal* set of parameters that defines the posture of the robot in $\mathcal{W}$. The corresponding vector

$$q = \begin{pmatrix} q_1 \\ \vdots \\ q_n \end{pmatrix}$$

is called a *configuration vector*, and each q_i is a *generalized coordinate.*

The generalized coordinates are of two kinds:

- **Cartesian coordinates**: identify the position of representative points of the robot; they take values in $\mathbb{R}^2$ or $\mathbb{R}^3$.
- **Angular coordinates**: identify the orientation of the bodies of the robot; they take values in SO(2) (planar: one parameter) or SO(3) (spatial: three parameters).

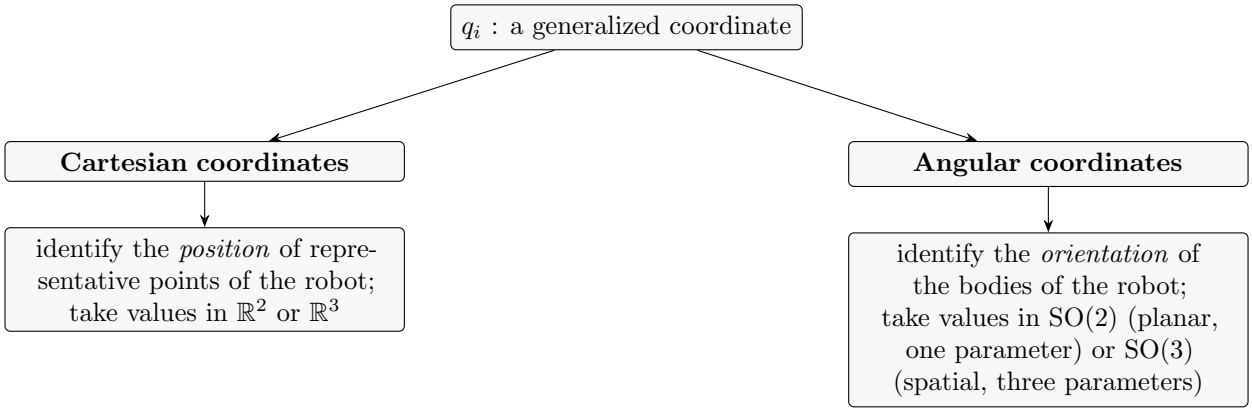


Figure 1: The two kinds of generalized coordinates.

Intuition

"Minimal" is the key word. The posture is the full, redundant description (every point of every body); the configuration is the shortest list of numbers from which the posture can be reconstructed. A rigid rectangle in the plane has infinitely many points, but three numbers (x, y, θ) tell you where all of them are.

1.4 Configuration space

Definition 1.4 (Configuration space)**.** The *configuration space* (C-space) $\mathcal{C}$ of a robot is the set of all possible values of q. Its dimension $n = \dim \mathcal{C}$ is the number of degrees of freedom of the robot.

Intuition

The C-space is the "catalogue of all poses the robot can be in". Each point of $\mathcal{C}$ *is* one pose. A curve in $\mathcal{C}$ is therefore a motion of the robot in the physical space, which is exactly why planning for a robot reduces to planning for a point.

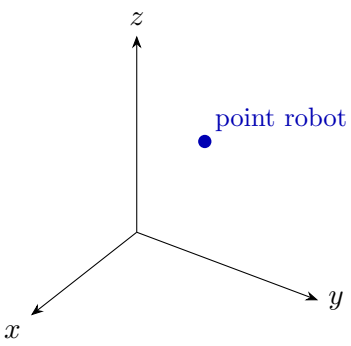
2 Examples of Configuration Spaces

(notes p. 2–3)

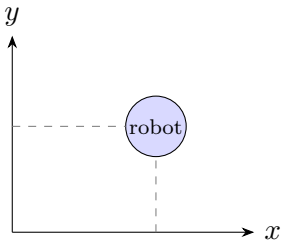
Example 2.1 (Point robot in $\mathcal{W} = \mathbb{R}^3$)**.** $q = (x, y, z)^\top$, so $\mathcal{C} = \mathbb{R}^3 = \mathcal{W}$ and $n = 3$.

Example 2.2 (Disk robot in $\mathcal{W} = \mathbb{R}^2$)**.** Since a disk looks the same under any rotation, its orientation is irrelevant: $q = (x, y)^\top$, $\mathcal{C} = \mathbb{R}^2 = \mathcal{W}$, $n = 2$.

In both examples $\mathcal{C}$ and $\mathcal{W}$ coincide. A case where they differ:



(a) point robot in $\mathcal{W} = \mathbb{R}^3$, $q = (x, y, z)^\top$



(b) disk robot in $\mathcal{W} = \mathbb{R}^2$, $q = (x, y)^\top$

Figure 2: Point robot and disk robot: in both cases $\mathcal{C} = \mathcal{W}$.

Example 2.3 (Disk with a tick)**.** We add a small mark (a "tick") to the disk, so that its orientation now matters. Then

$$q = \begin{pmatrix} x \\ y \\ \varphi \end{pmatrix}, \qquad \mathcal{C} = \mathbb{R}^2 \times \mathrm{SO}(2), \qquad n = 3,$$

where $\times$ is the Cartesian product.

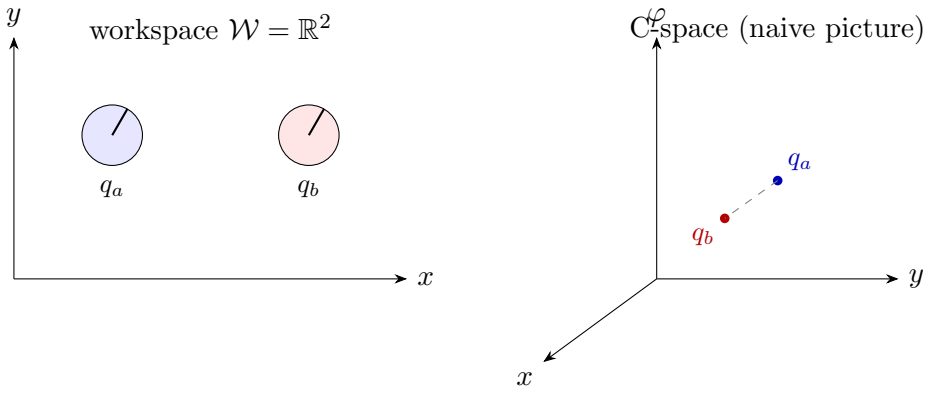


Figure 3: The disk with a tick. Configurations q_a and q_b have the same y and φ but different x. In C-space they differ only along the x-axis. (This picture is *naive*: it is not topologically correct, see Section 4.)

Every change in the configuration space is a path, i.e. a motion, in the physical space.

Example 2.4 (Polygonal robot in $\mathcal{W} = \mathbb{R}^2$)**.** For instance a car moving on the ground. It can rotate about its centre of mass:

$$q = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix}, \quad \mathcal{C} = \mathbb{R}^2 \times \mathrm{SO}(2), \quad n = 3.$$

Example 2.5 (Polyhedral robot in $\mathcal{W} = \mathbb{R}^3$)**.** Treat it like the previous case, but add z and two more orientation angles:

$$q = (x, y, z, \alpha, \beta, \gamma)^\top, \qquad \mathcal{C} = \mathbb{R}^3 \times \mathrm{SO}(3), \qquad n = 6.$$

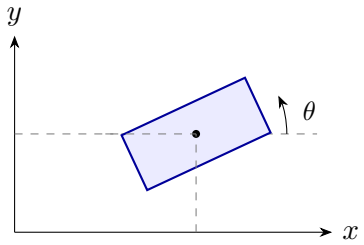


Figure 4: Polygonal robot in the plane: position of the centre of mass (x, y) and orientation θ.

Definition 2.6 (Special Euclidean groups)**.** The spaces $\mathbb{R}^2 \times \mathrm{SO}(2)$ and $\mathbb{R}^3 \times \mathrm{SO}(3)$ are called the *special Euclidean groups*, denoted $\mathrm{SE}(2)$ and $\mathrm{SE}(3)$.

Intuition

$\mathrm{SE}(n)$ is the set of all rigid motions (a rotation followed by a translation) of n-dimensional space. A free rigid body can be moved anywhere and turned any way, so its configuration space is exactly the set of rigid motions: 3 numbers in the plane (2 translations + 1 rotation), 6 in space (3 translations + 3 rotations).

3 Multi-Body Robots: Manipulators

(notes p. 4–5)

3.1 Planar manipulator with n_b links

Consider a planar manipulator with n_b links and n_b (e.g. revolute) joints moving in $\mathcal{W} = \mathbb{R}^2$. Each link is a rigid body with 3 degrees of freedom in the plane, so one might expect $3n_b$ generalized coordinates. This is wrong: the links are *connected*, and every joint introduces constraints.

Proposition 3.1 (Degrees of freedom of a planar serial arm)**.** *A planar arm of n_b links connected to the ground by n_b revolute joints has*

$$3n_b - 2n_b = n_b$$

degrees of freedom. Each revolute joint imposes 2 constraints (the two coordinates of the joint point must coincide on the two bodies it connects).

Hence, to describe the configuration we do not need to remember all the constraints: one angle per joint is enough,

$$q = \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_{n_b} \end{pmatrix}, \quad n = n_b, \qquad \mathcal{C} = \underbrace{\mathrm{SO}(2) \times \cdots \times \mathrm{SO}(2)}_{n_b \text{ times}} = \mathrm{SO}(2)^{n_b}.$$

Intuition

Imagine gluing the links together at the joints. Each glued joint removes two "free sliding" directions. Three links loose in the plane would have 9 degrees of freedom; pin them into a chain anchored at the base and only 3 remain: the three joint angles. The number of angles

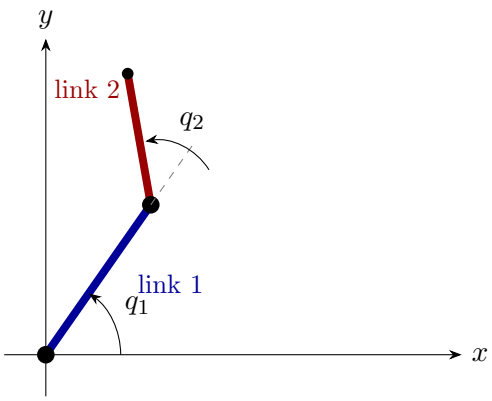


Figure 5: A 2R planar arm: one angle per joint is enough to describe the configuration.

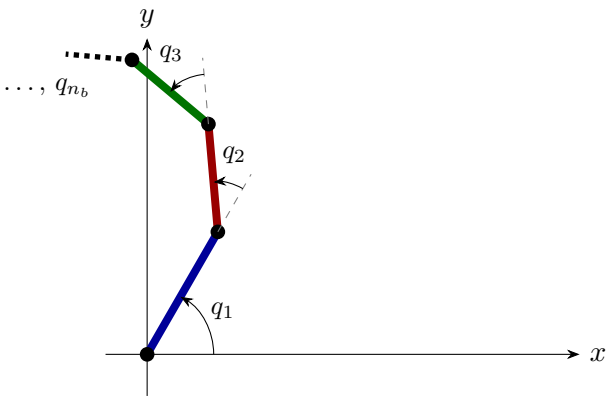


Figure 6: Planar manipulator with n_b links: one joint angle q_i per joint.

= the number of joints.

Proposition 3.2. *For three revolute joints, $\mathcal{C} = \mathrm{SO}(2) \times \mathrm{SO}(2) \times \mathrm{SO}(2)$, which is very different from $\mathrm{SO}(3)$: the two spaces have the same dimension 3 but different topology.*

Intuition

SO(3) describes all orientations of *one* rigid body (roll, pitch, yaw). Three planar joints are three *independent* angles each living on its own circle; they cannot all mix into one 3D rotation. Same number of parameters, different shape: it is the shape (topology), not just the count, that matters later.

3.2 Spatial manipulator with n_b links

Proposition 3.3 (Degrees of freedom of a spatial serial arm)**.** *A spatial arm with n_b links and n_b revolute joints in $\mathcal{W} = \mathbb{R}^3$ has*

$$6n_b - 5n_b = n_b$$

degrees of freedom: each link has 6 degrees of freedom, but a revolute joint imposes 5 constraints (3 positions and 2 orientations are fixed; only rotation about the joint axis remains).

Even though the manipulator is spatial, each joint has a *planar* orientation (a single angle), so

again

$$\mathcal{C} = \mathrm{SO}(2) \times \cdots \times \mathrm{SO}(2) = \mathrm{SO}(2)^{n_b} \qquad (n_b \text{ times}).$$

Intuition

A hinge (revolute joint) always allows exactly one motion: turning about its axis. Whether the world is 2D or 3D, that one motion is a single angle. The ambient dimension changes the *constraint count* (2 vs. 5) but not the end result: one angle per joint.

4 Topology of $\mathcal{C}$

(notes p. 5–6)

4.1 The problem with a flat picture of the 2R robot

Take the 2R planar robot, $\mathcal{C} = \mathrm{SO}(2) \times \mathrm{SO}(2)$, $q = (q_1, q_2)^\top$. Since angles are periodic, q_1 and $q_1 + 2\pi$ give the *same posture*. So if we naively draw $\mathcal{C}$ as the plane $\mathbb{R}^2$ of angle pairs, then two different points give the same posture: the map from this plane to postures is *not injective*, which is bad for us.

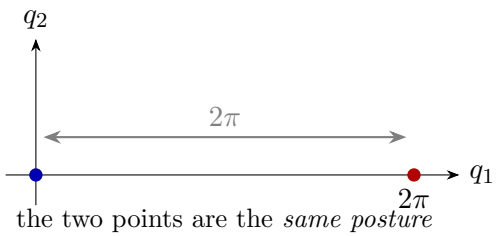


Figure 7: On the plane of angle pairs, $q_1 = 0$ and $q_1 = 2\pi$ (same q_2) correspond to the same posture: the map to postures is many-to-one.

A new attempt: restrict each angle to $[0, 2\pi)$, i.e. draw $\mathcal{C}$ as the square $[0, 2\pi) \times [0, 2\pi)$. Now the correspondence with postures is one-to-one (considering only the two "bad sides" of the square, $q = 0$ and $q = 2\pi$, as the same). But this is still not good enough: two configurations q_a and q_b that are almost the same posture (one just below 2π, the other just above 0) look *far apart* in the square.

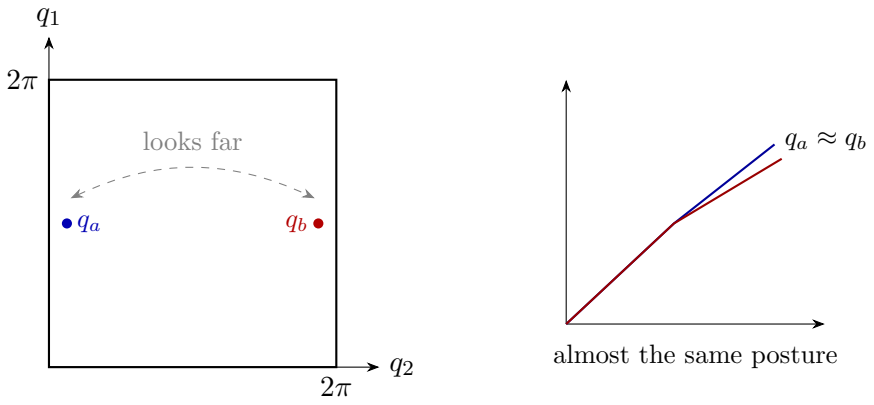


Figure 8: The square picture of $\mathcal{C}$: q_a and q_b are close as postures but far in the flat picture.

So this topology does not describe exactly what is happening.

4.2 Wrapping the space: the torus

The fix is to *wrap* the space: identify the opposite sides of the square (left with right, bottom with top) and embed the resulting 2D object in 3D. We obtain a *torus* (from Latin, "pillow").

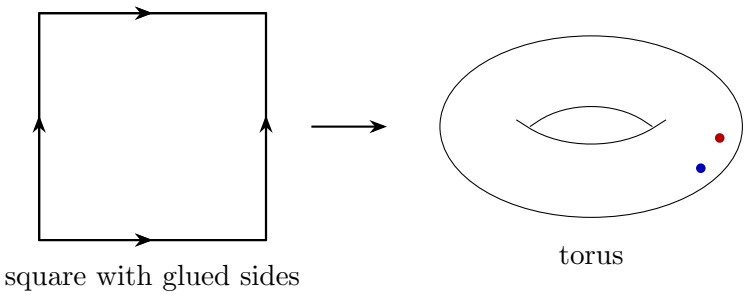


Figure 9: Gluing opposite sides of the square gives a torus. Points that were far apart in the square (across the seam) become neighbours on the torus.

Proposition 4.1. $\mathrm{SO}(2) \cong \mathbb{S}^1$ *(the circle), hence the configuration space of the 2R planar robot is* $\mathbb{S}^1 \times \mathbb{S}^1 = \mathbb{T}^2$*, the torus, and in general* $\mathrm{SO}(2)^{n_b} \cong \mathbb{T}^{n_b}$ *is an* n_b*-dimensional torus.*

Intuition

A joint angle is like a clock hand: after 2π you are back where you started, so one angle lives on a *circle*, not on a line. Two joints give two circles, and a product of two circles is the surface of a doughnut. Walking off the right side of the square and reappearing on the left is just going around the doughnut the short way.

4.3 Manifolds

Definition 4.2 (Manifold)**.** A *manifold* of dimension n is a space in which the neighbourhood of each point "looks like" a neighbourhood of $\mathbb{R}^n$. (For the torus, $\mathbb{R}^2$.)

Proposition 4.3. *Configuration spaces are manifolds (in general* not *Euclidean anymore):* $\mathbb{R}^m$*,* $\mathrm{SO}(2)$*,* $\mathrm{SO}(3)$*,* $\mathrm{SE}(2)$*,* $\mathrm{SE}(3)$ *and their products are all manifolds.*

Intuition

The Earth is the classical example: it is a sphere, yet a small enough patch looks flat, so a city map works. Locally we can use ordinary coordinates and calculus; globally the space can wrap around, and that is what a flat picture cannot show.

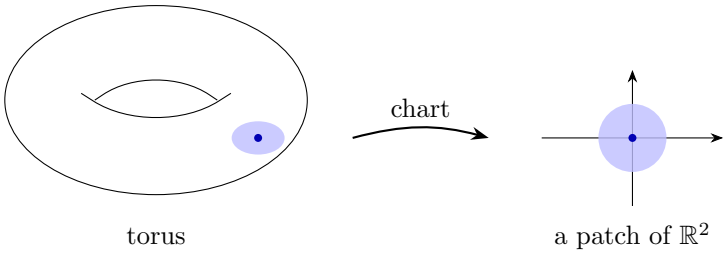


Figure 10: A small neighbourhood of any point of the torus can be flattened (charted) onto a neighbourhood in $\mathbb{R}^2$: this is what "looks like $\mathbb{R}^n$" means.

5 Distances in $\mathcal{C}$

(notes p. 6–8)

5.1 What a distance must satisfy

Definition 5.1 (Distance (metric))**.** A function $d : \mathcal{C} \times \mathcal{C} \to \mathbb{R}$ is a *distance* if for all $q_A, q_B, q_C \in \mathcal{C}$:

(i) $d(q_A, q_B) \geq 0$;
(ii) $d(q_A, q_B) = 0$ if and only if $q_A = q_B$;
(iii) $d(q_A, q_B) = d(q_B, q_A)$ (symmetry);
(iv) $d(q_A, q_B) \leq d(q_A, q_C) + d(q_C, q_B)$ (triangle inequality).

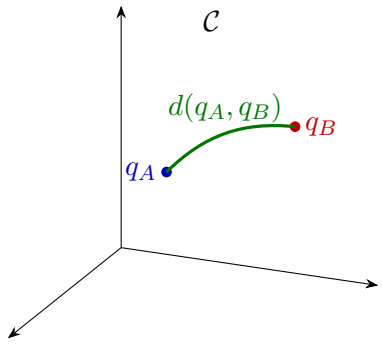
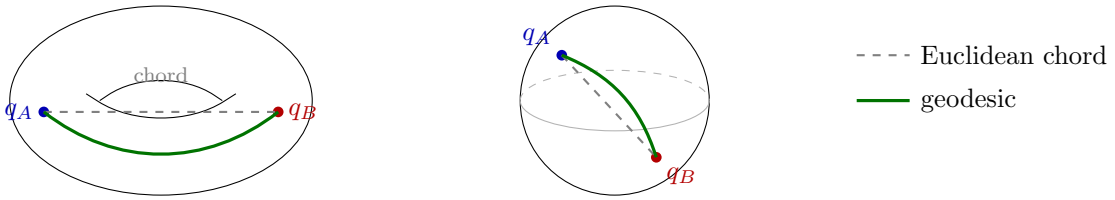


Figure 11: The distance $d(q_A, q_B)$ between two points of the configuration space.

5.2 Distances on manifolds and geodesics

The classic (Euclidean) distance completely ignores the curvature of the space: for two points on a torus, the straight segment between them passes *through the hole*, outside the manifold. So we cannot use the Euclidean distance.



torus: tricky in general

sphere: great circles

Figure 12: The chord between two points leaves the manifold (it crosses the hole of the torus, or the inside of the sphere); the geodesic stays on the surface. On a sphere the geodesic is an arc of a great circle; on a general manifold it is hard to compute.

Definition 5.2 (Geodesic)**.** A *geodesic* is a minimum-length path between two points *on the manifold*. The geodesic distance is the length of that path.

Intuition

For two cities on Earth, the geodesic is the great-circle route an aircraft follows, not the tunnel through the Earth. Geodesics are the "honest" distances on a curved space.

5.3 The displacement metric

Geodesics are easy on a sphere but can be tricky in general (and on a torus). Because of this difficulty, roboticists do not use them. They use instead a *displacement metric*.

Let p denote a point of the robot (e.g. the centre of mass, or the end point of a link), and $p(q)$ its position in $\mathcal{W}$ when the configuration is q.

Definition 5.3 (Displacement metric)**.**

$$d(q_A, q_B) = \max_{p \in \text{robot}} \|p(q_A) - p(q_B)\|.$$

It is the *maximum displacement* of the points of the robot when moving from q_A to q_B.

Proposition 5.4. *The displacement metric satisfies the four axioms of a distance (provided distinct configurations give distinct postures, which is the case for a minimal parametrization).*

Proof. (i),(iii) hold because a norm is non-negative and symmetric, and the maximum of such quantities is too. (ii) If $q_A = q_B$ every point has zero displacement, so $d = 0$; conversely $d = 0$ means every point is at the same place in the two configurations, i.e. the postures coincide, hence $q_A = q_B$. (iv) For every point p,

$$\|p(q_A) - p(q_B)\| \le \|p(q_A) - p(q_C)\| + \|p(q_C) - p(q_B)\| \le d(q_A, q_C) + d(q_C, q_B);$$

taking the maximum over p on the left gives the claim. □

Intuition

Instead of measuring a length *inside* the curved configuration space, we go back to the physical world and ask: "how far does the worst-behaved point of the robot have to travel?" We use ordinary Euclidean distances there, where they are perfectly fine, and the curvature of $\mathcal{C}$ takes care of itself. Nice side effect: this measure is automatically periodic, since q_1 and $q_1 + 2\pi$ give the same positions.

Example 5.5 (1R robot)**.** A single link of length L rotating about the origin; q_A, q_B are two angles differing by $\Delta \in [0, \pi]$. A point at distance $r \le L$ from the joint moves by the chord $2r \sin(\Delta/2)$, largest at the tip, so

$$d(q_A, q_B) = 2L \sin\left(\tfrac{\Delta}{2}\right).$$

We are effectively using the *Euclidean* distance between the two tip positions to measure distance on the manifold.

Example 5.6 (Numbers for the 1R robot, $L = 1$)**.** For $\Delta = \pi/2$: $d = 2\sin(\pi/4) = \sqrt{2} \approx 1.41$. For two angles that differ by $2\pi - 0.1$ (almost a full turn): the wrapped difference is 0.1, so $d = 2\sin(0.05) \approx 0.10$ – correctly tiny, whereas the flat difference of angles (6.18) would call them far apart.

Example 5.7 (2R robot)**.** With two links we take the maximum over the points of both links. The displacement of a point on a link is an affine function of its position along the link, so its norm is convex and the maximum is attained at a link endpoint: the base (which never moves), the elbow joint, or the tip. In Figure 15 (link lengths 1.6 and 1) the tip moves by about 0.64 but the elbow moves by about 1.60, so $d(q_A, q_B) \approx 1.60$: the maximum is *not*

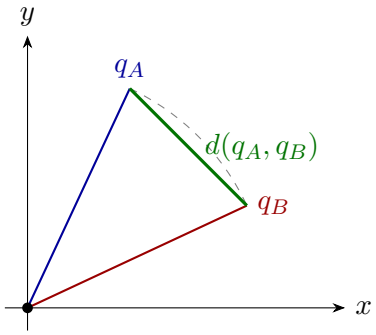


Figure 13: 1R robot: the displacement metric is the Euclidean distance between the tip in the two configurations.

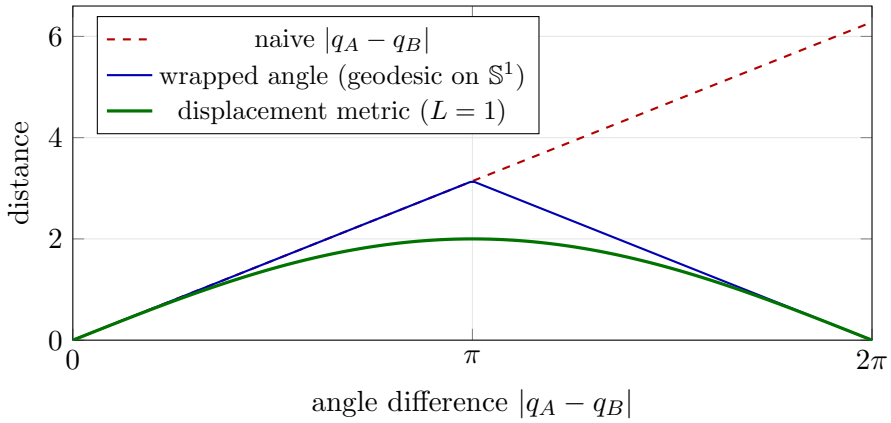


Figure 14: Three ways to measure the distance between two configurations of a 1R robot. The naive difference grows up to 2π; the other two vanish again at 2π because the configuration wraps around.

necessarily at the tip.

5.4 The max over infinitely many points

There is a catch: the definition asks for a maximum over an *infinite* set of points. Intuitively, for polygonal robots we should only look at the *vertices*, but it is not always like this.

Proposition 5.8. *If the robot is a convex polygon (or a union of segments, as a chain of links), the maximum in the displacement metric is attained at a vertex (link endpoint).*

Proof. Under a rigid motion, the position of a body point r is affine in r, so $r \mapsto p(q_A) - p(q_B)$ is affine on the body. The norm of an affine function is convex, and a convex function on a polytope attains its maximum at a vertex. □

Intuition

The point that moves most is the one sticking out the farthest – a corner, or the tip of a link. For a curved body (e.g. a disk with attached parts) there is no finite list of corners to check, so we need an approximation.

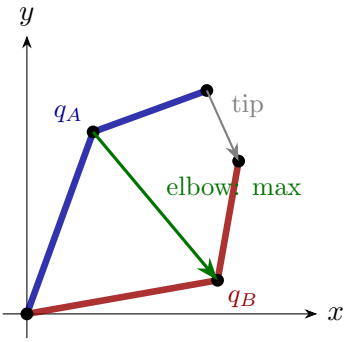


Figure 15: 2R robot in two configurations. The displacement of each characteristic point is drawn; the displacement metric takes the largest one (here the elbow).

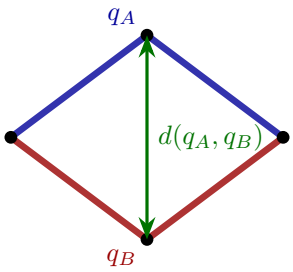


Figure 16: 2R robot with equal links, "elbow up" vs "elbow down", same tip: the tip does not move, and the displacement metric equals the distance between the two elbows.

5.5 A practical alternative: control points

We can use a predetermined number of points, called *control points*, which, if chosen correctly, give a good approximation.

Definition 5.9 (Control-point distance)**.** Fix control points $\{p_1, p_2, \ldots, p_c\}$ on the robot. Define

$$d(q_A, q_B) = \sum_{i=1}^{c} \|p_i(q_A) - p_i(q_B)\|.$$

Algorithm 5.10 (Computing the control-point distance)**.**
1. Choose c representative points $p_1, \ldots, p_c$ on the robot (e.g. centre of mass, link endpoints).
2. For each i, compute the forward kinematics $p_i(q_A)$ and $p_i(q_B)$.
3. Return $\sum_{i=1}^{c} \|p_i(q_A) - p_i(q_B)\|$.

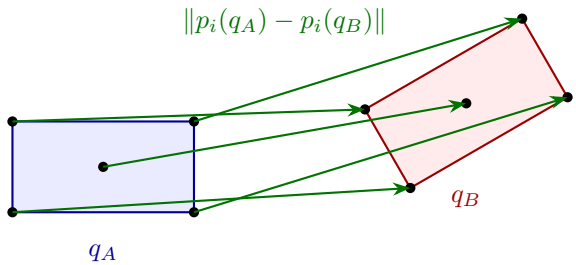


Figure 17: Control points $p_1, \ldots, p_5$ (four corners and the centre) of a rectangular robot. The distance is the sum of the lengths of the displacement arrows.

Proposition 5.11. *The control-point distance is symmetric and satisfies the triangle inequality. It is a genuine distance if the control points are chosen so that $p_i(q_A) = p_i(q_B)$ for all i implies $q_A = q_B$; otherwise it is only a pseudo-distance.*

Proof. Each summand is a norm of a difference, so symmetry holds, and the triangle inequality holds termwise and is preserved by the sum. The condition on control points gives axiom (ii). □

Intuition

Rather than tracking every point, you pin a few "sensors" on the robot (say, the tip and the middle) and add up how far they each move. A moving robot cannot fool three well-placed sensors: if they all end up in the same places, the robot is in the same pose. The cost: an approximation of the true worst-case displacement, but cheap and easy to differentiate.

6 Summary

6.1 Configuration spaces seen in this lecture

Robot	$\mathcal{C}$	$n = \dim \mathcal{C}$	Euclidean?
Point in $\mathbb{R}^3$	$\mathbb{R}^3$	3	yes
Disk in $\mathbb{R}^2$	$\mathbb{R}^2$	2	yes
Disk with tick / polygon in $\mathbb{R}^2$	$\mathrm{SE}(2) = \mathbb{R}^2 \times \mathrm{SO}(2)$	3	no
Polyhedron in $\mathbb{R}^3$	$\mathrm{SE}(3) = \mathbb{R}^3 \times \mathrm{SO}(3)$	6	no
Planar n_b-R arm	$\mathrm{SO}(2)^{n_b} = \mathbb{T}^{n_b}$	n_b	no
Spatial n_b-R arm	$\mathrm{SO}(2)^{n_b} = \mathbb{T}^{n_b}$	n_b	no

6.2 Distances on $\mathcal{C}$

Distance	Definition	Pros / cons
Euclidean in parameters	$\|q_A - q_B\|$	simple, but ignores wrapping (bad on angles)
Geodesic	shortest path on the manifold	exact, but hard to compute in general
Displacement metric	$\max_p \|p(q_A) - p(q_B)\|$	respects topology, physically meaningful; max over infinitely many points
Control points	$\sum_i \|p_i(q_A) - p_i(q_B)\|$	cheap approximation; needs well-chosen points

6.3 Key take-aways

- The configuration space $\mathcal{C}$ is the space of all values of a *minimal* set of parameters describing the robot; in it the robot is a single point, and planning/control become problems about that point.

- Joints reduce degrees of freedom: a planar arm has $3n_b - 2n_b = n_b$ and a spatial arm $6n_b - 5n_b = n_b$ degrees of freedom – one angle per joint.
- Because angles are periodic, $\mathcal{C}$ is generally a *manifold* (e.g. a torus for a 2R arm), not $\mathbb{R}^n$: same dimension does not mean same shape ($\mathrm{SO}(2)^3 \neq \mathrm{SO}(3)$).
- Euclidean distance in parameter space is misleading on a manifold; geodesics are correct but hard to compute.
- Roboticists use the *displacement metric* (max point displacement) or its practical version with a few *control points*.

Part II

Lecture 2

About these notes

These notes have two parts. **Part I** is based on the lecture slides (`WheeledMobileRobots1_Slides.pdf`, slide ranges in parentheses). **Part II** is based on the second half of the lecture, provided as handwritten notes (`AMR02.pdf`, page ranges in parentheses). No companion book chapter was provided for either part, so the formal statements (Definitions, Propositions, Lemmas, Theorems, proofs) formalise the content of the slides/notes with standard results. Whatever is *not* in the source is explicitly tagged *(beyond the slides)* or *(beyond the notes)*. In Part II I corrected a typo in the notes ($\partial h/\partial k$ read as $\partial h/\partial q$) and added the solution of the example left open on page 6. Red dashed or framed boxes marked "Blank" are spots for photographs/videos on the slides that were deliberately not redrawn; all other figures are original TikZ drawings.

Part III

Mechanics of Mobile Robots

1 Outline of this lecture *(slides 1–2)*

The lecture opens the part of the course on *wheeled mobile robots* (WMRs). It covers four topics:

- **ground locomotion**: how a robot moves by contact with the ground;
- **balance**: what it takes for the robot not to fall;
- **wheels**: the three basic wheel types;
- **kinematic structures**: the classical ways of combining wheels on a chassis.

Intuition

Think of this lecture as the *hardware vocabulary* of the course. Later lectures write equations of motion (kinematics, control) for each structure; to do so one first needs to know which wheel is where, which wheels are motorised, and which can steer. Everything here is about that bookkeeping.

2 Ground locomotion *(slide 3)*

Moving on the ground requires *contact* between the robot and the terrain. The slides distinguish two ways of establishing it.

Definition 2.1 (Wheeled mobile robot)**.** A *wheeled mobile robot* (WMR) typically consists of a single rigid body (the *base* or *chassis*) plus a set of wheels that connect it to the ground.

Definition 2.2 (Legged robot)**.** A *legged robot* typically consists of *several* rigid bodies articulated through joints; the contact with the ground is made through the feet.

Some robots achieve ground locomotion with neither wheels nor feet, for instance *snake robots*.

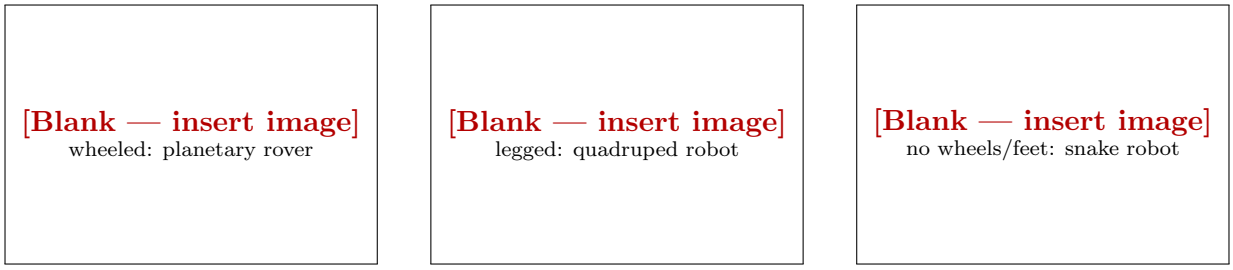


Figure 18: Photographs on slide 3 (not redrawn).

Intuition

The difference is in the *mechanical complexity*: a WMR is one rigid body plus wheels, so its configuration is just the position and orientation of the chassis (plus steering angles); a legged robot is an articulated system, and every joint adds a degree of freedom to control. This is why the course starts with WMRs.

3 Balance (not falling) *(slides 4–5)*

3.1 Static balance and the support polygon

Definition 3.1 (Support polygon)**.** The *support polygon* is the convex hull of the points where the robot touches the ground: the wheel contact points for a WMR, the footprints for a legged robot.

Definition 3.2 (Static balance)**.** A robot is *statically balanced* when the vertical projection of its Center of Mass (CoM) on the ground falls inside the support polygon.

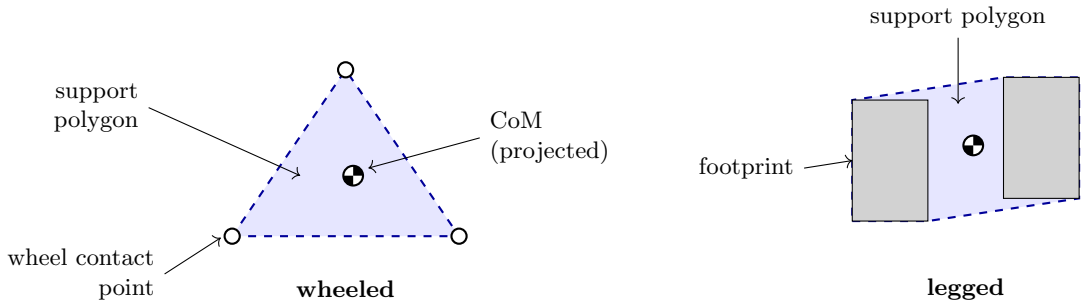


Figure 19: Support polygons (slide 4). Wheeled robot: triangle through three wheel contact points. Legged robot: convex hull of the two footprints. The projected CoM (⊗-like symbol) must lie inside.

For WMRs the slides state a consequence: *one needs 3 wheels*. Here is the formal reason.

Proposition 3.3 (Three wheels are necessary)**.** *A robot whose support polygon has non-empty interior needs at least three ground contact points that are not collinear. Consequently, a WMR that must be statically balanced with a robust margin needs at least 3 wheels.*

Proof. The convex hull of one point is a point; of two points, a segment; of three or more collinear points, still a segment. All of these have empty interior in the plane, so the CoM projection can

lie in them only by an unstable coincidence. Three non-collinear points span a triangle, which has non-empty interior. □

Intuition

A stool with one or two legs falls; a three-legged stool stands. With two wheels the robot can rotate about the line through the two contact points (like a seesaw), unless extra dynamic control is used. This is why the differential-drive robot of slide 9 adds a third, passive wheel (a caster) "for balance".

The support polygon also tells us how the weight is shared among the wheels.

Example 3.4. A differential-drive robot has driving wheels at $p_1 = (0, 0.3)$, $p_2 = (0, -0.3)$ and a caster at $p_3 = (0.4, 0)$ (metres), so $A(p_1, p_2, p_3) = 0.12\,\mathrm{m}^2$. With $c = (0.1, 0.05)$ the algorithm gives $\lambda = (0.458,\ 0.292,\ 0.25)$: all positive, so the robot is balanced and the caster carries 25% of the weight. After loading a payload that moves the CoM to $c = (0.3, 0.1)$, we get $\lambda = (7/24,\ -1/24,\ 3/4)$. Since $\lambda_2 < 0$, wheel 2 lifts and the robot tips over the edge p_1p_3.

Intuition

The weight splits among the wheels like a seesaw: the closer the CoM is to a wheel, the more load that wheel carries. In the example the caster carries a quarter of the weight, so 75% of the weight presses on the driving wheels, which is what gives them traction. If the load moves far from the centre, one wheel is relieved of all weight and the robot tips. With four or more wheels the loads are no longer unique (like a four-legged table that wobbles on uneven ground), which is why real robots with more than three wheels need suspensions.

3.2 Dynamic balance and the Zero Moment Point *(slides 4–5)*

Definition 3.5 (Dynamic balance)**.** In *dynamic balance*, the CoM in the static criterion is replaced by the *Zero Moment Point* (ZMP), the point of the ground about which the horizontal components of the moment of gravity and inertial forces vanish. The robot is dynamically balanced when the ZMP lies inside the support polygon.

Example 3.6. With $z_c = 0.8\,\mathrm{m}$, $\ddot{z}_c = 0$ and $\ddot{x}_c = 1\,\mathrm{m/s}^2$: $x_{\mathrm{zmp}} = x_c - 0.8/9.81 \approx x_c - 8.2\,\mathrm{cm}$. The ZMP lies 8.2 cm behind the CoM projection.

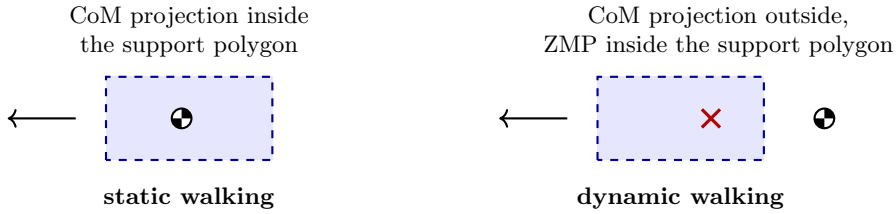


Figure 20: Static vs. dynamic walking (slide 5), shown for a single stance foot, top view. Arrows: walking direction. Red cross: ZMP.

Intuition

Carrying a tray while walking fast: you do not keep the tray's weight right above your feet, you lean and let motion help. Static walking is the cautious version, the CoM never leaves the foot area, so the robot could stop at any instant without falling; it is slow. In dynamic

walking the CoM may be momentarily outside the foot, and the robot stays up because the accelerations move the ZMP back inside. The slide contrasts two humanoids walking one way or the other.

4 Wheels: 3 basic types *(slides 6–8)*

Wheels are classified by how their orientation relates to the chassis and by whether they are motorised (*active*) or free (*passive*).

4.1 Fixed wheel *(slide 6)*

Definition 4.1 (Fixed wheel)**.** A wheel with *fixed orientation* with respect to the chassis. It may be *active* (used for driving) or *passive* (used for balance).

4.2 Orientable (steerable) wheel *(slide 7)*

Definition 4.2 (Orientable wheel)**.** A wheel whose orientation with respect to the chassis is *variable*. It is typically *active* (used for steering).

4.3 Caster wheel *(slide 8)*

Definition 4.3 (Caster wheel)**.** A wheel with variable orientation with respect to the chassis, typically *passive* (used for balance) but possibly active, that *automatically aligns with the direction of motion*.

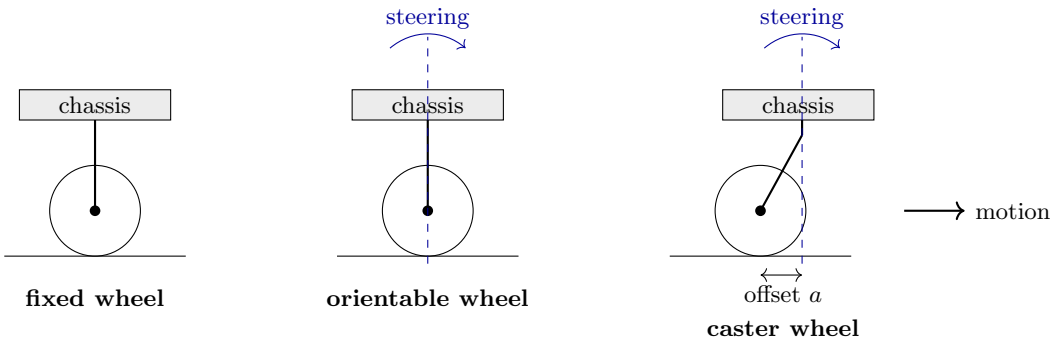


Figure 21: Side views of the three wheel types. The steering axis of the orientable wheel passes through the wheel centre; that of the caster is offset by a distance a, with the wheel trailing behind it.

Wheel	Orientation w.r.t. chassis	Typical role
Fixed	constant	active (driving) or passive (balance)
Orientable	variable	active (steering)
Caster	variable, self-aligning	passive (balance), possibly active

Table 1: The three basic wheel types (slides 6–8).

Intuition

It is the same reason a shopping-cart wheel swings round when you start pushing, or why a trailer follows the car: the wheel is *pulled* by a pivot ahead of it, and a pulled object trails behind. With $v = 0.5\,\mathrm{m/s}$ and $a = 5\,\mathrm{cm}$ the time constant is $a/v = 0.1\,\mathrm{s}$: the wheel aligns in about a tenth of a second. The *offset* is essential: with $a = 0$ the wheel would be an ordinary orientable wheel, which stays wherever it is steered. Fixed wheels give grip and balance, orientable wheels give steering, casters give free support.

5 WMR kinematic structures *(slides 9–15)*

The wheel types are combined on a chassis in a few classical ways. The slides present seven structures.

5.1 Differential-drive mobile robot *(slide 9)*

Definition 5.1 (Differential-drive WMR)**.** A WMR with **2 fixed wheels** on a common axle, **independently actuated**, plus **1 caster** added for balance.

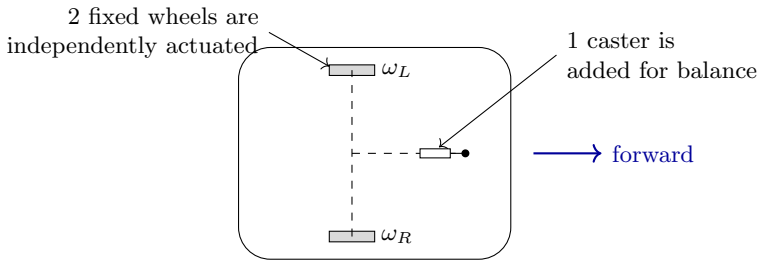


Figure 22: Differential-drive robot (slide 9), top view.

Intuition

It is a tank (or a wheelchair): equal speeds drive straight, different speeds curve towards the slower wheel, opposite speeds spin on the spot. With $r = 0.1\,\mathrm{m}$, $d = 0.4\,\mathrm{m}$: $\omega_R = \omega_L = 5\,\mathrm{rad/s}$ gives $v = 0.5\,\mathrm{m/s}$, $\omega = 0$; $\omega_R = -\omega_L = 5\,\mathrm{rad/s}$ gives $v = 0$, $\omega = 2.5\,\mathrm{rad/s}$. The slides show an indoor mobile base and a robot vacuum cleaner as examples. The caster only keeps the robot from tipping, as the balance section requires.

5.2 Synchro-drive mobile robot *(slide 10)*

Definition 5.2 (Synchro-drive WMR)**.** A WMR with **3 orientable wheels** that are **simultaneously actuated**. The orientation of the chassis remains constant.

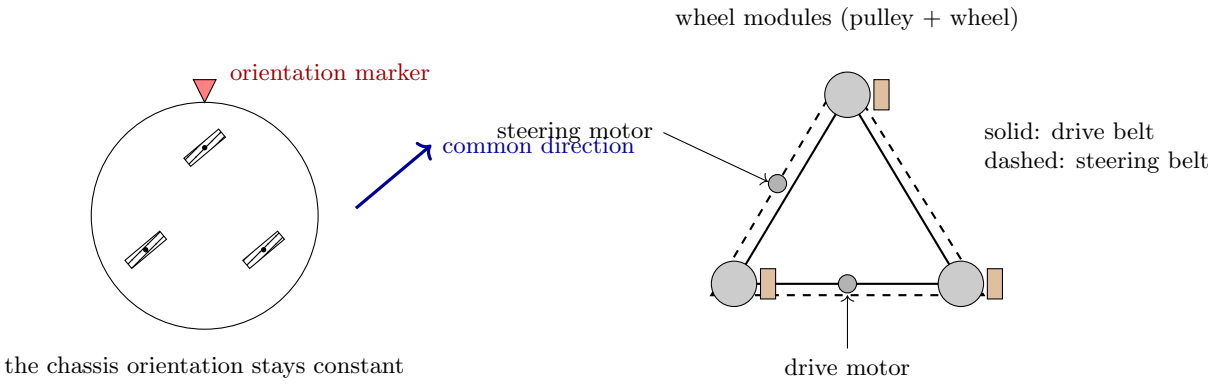


Figure 23: Synchro-drive (slide 10). Left: three orientable wheels, always parallel. Right: simplified drive train redrawn from the slide: one drive motor and one steering motor act on all the wheels through belts.

Intuition

The three wheels are like three people pushing a table in exactly the same direction at the same speed: the table translates and never turns. The robot changes its direction of travel by turning all wheels together, but its body keeps pointing the same way; that is why the sensors or the "head" of a synchro-drive robot would need their own rotation mechanism (the slide shows a laptop riding on top). The price of this simplicity is a drive train with belts, but only two motors for three wheels.

5.3 Tricycle and car-like robots *(slide 11)*

Definition 5.3 (Tricycle)**. 2 fixed wheels** on a common axle and **1 orientable wheel**.

Definition 5.4 (Car-like)**. 2 fixed wheels** on a common axle and **2 orientable wheels** on a common axle.

Both may be front-wheel drive or rear-wheel drive.

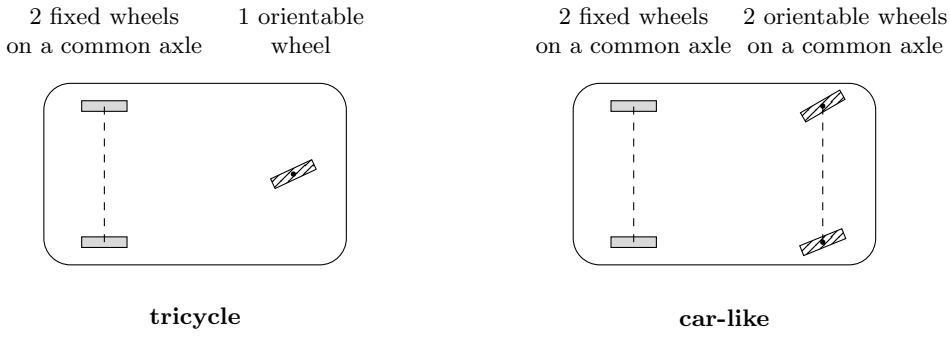


Figure 24: Tricycle (left) and car-like robot (right), top views (slide 11). The tilted wheels are steered.

Intuition

When a car turns, each wheel must roll along a circle with the same centre, and the inner circle is smaller; the inner wheel therefore has to turn more than the outer one. Numbers:

$\ell = 2.5\,\text{m}$, $w = 1.5\,\text{m}$, $\delta_i = 30°$: $\cot\delta_o = 1.732 + 0.6 = 2.332$, so $\delta_o \approx 23.2°$. A tricycle, with only one steered wheel, does not have this issue. Front-wheel drive means the steered wheel(s) are also driven; rear-wheel drive means the fixed axle is driven, which requires the device of the next subsection when the two rear wheels share a motor.

5.4 The differential *(slide 12)*

Definition 5.5 (Differential)**.** A mechanical device needed whenever two *driving* wheels are mounted on a *common axle* (driven by the same motor), which allows the two wheels to move at *different speeds*.

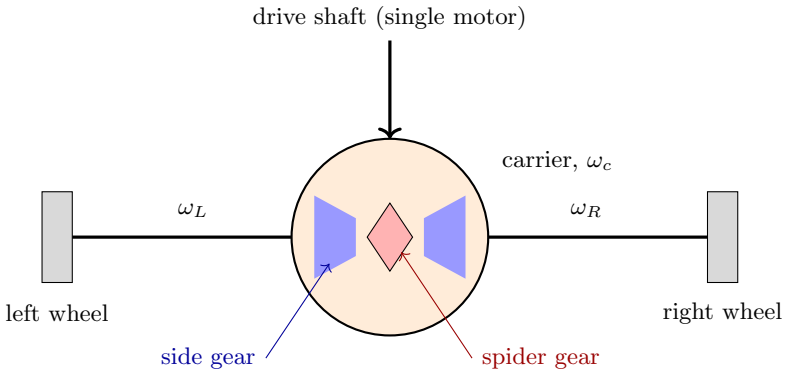


Figure 25: Schematic of a differential (idealised). The slide shows an animated 3D model.

Intuition

Why is it needed? When a car turns, the outer wheel travels along a longer arc than the inner one, so the two must rotate at different speeds; if rigidly tied to one shaft they would drag and slip. The differential lets the wheels share the motor while spinning differently. Numbers: wheels at 4 and 6 rad/s $\Rightarrow$ the carrier turns at 5 rad/s. A known side effect of equal torque split: if one wheel is on ice, it can only transmit a tiny torque, and so can the other.

5.5 Firetruck *(slide 13)*

Definition 5.6 (Firetruck)**.** A **tractor** (car-like) coupled to a **trailer** having **2 orientable wheels on a common axle**.

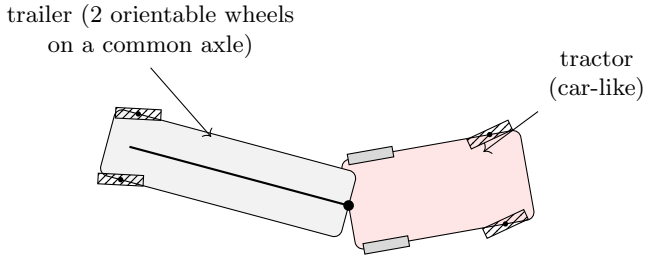


Figure 26: Firetruck (slide 13), top view; the dot is the hitch.

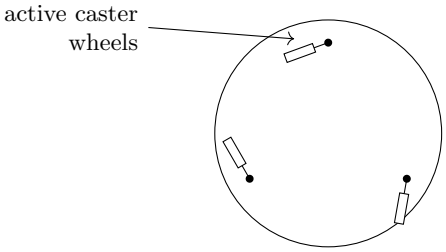
Intuition

A ladder firetruck is very long. A second person (the "tiller operator", as the slide's overlay says) steers the *rear* axle so that the trailer does not cut the corner and can pass around cars and curbs. In kinematic terms, it is a car-like robot pulling a trailer with its own steering.

5.6 Omnidirectional robots with actuated caster wheels *(slide 14)*

Definition 5.7 (Omnidirectional WMR)**.** A WMR that can instantaneously move in any direction of the plane, *without first re-orienting the chassis*, and rotate independently of translation.)

The structure on the slide uses **active caster wheels**, i.e. caster wheels that are both steered and driven.



[**Blank — insert image**]
actuated caster unit
(steering motor + driving motor)
and demo video on slide 14

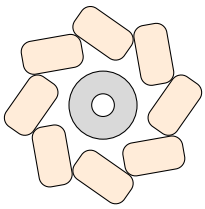
Figure 27: Omnidirectional robot with three active casters (slide 14, left drawing) and a spot for the slide's photo/video.

Intuition

Each wheel can point wherever it likes and be driven at the right speed, so it can follow the velocity that its own mount point must have. If all wheels point the same way, the robot translates in that direction; if their directions differ, the robot rotates. The effect is like a shopping cart that can instantly go sideways.So something that a car can't do, however the cost is mechanical: two motors per wheel. So more expensive, more complex and also more difficult to control than a differential-drive robot, but it can move in any direction without turning first.

5.7 Mecanum (Swedish) wheels *(slide 15)*

Definition 5.8 (Mecanum wheel)**.** A wheel with a ring of free-spinning rollers mounted around its rim, with the roller axes oblique to the wheel axle. Omnidirectional mobile robots can be built using Mecanum (Swedish) wheels.



[Blank — insert image]
Mecanum robot (slide 15, top photo)

[Blank — insert image]
Mecanum wheelchair (slide 15, bottom photo)

Figure 28: A Mecanum wheel seen along its axle (slide 15, left drawing) and spots for the two photos.

Intuition

The rollers turn the "wheel" into one that can roll forward *and* slide sideways freely: the ground force of each wheel acts along the axis of its roller instead of along the wheel plane. Choosing the speeds of several such wheels, one can obtain any combination of forward, sideways and rotational motion of the chassis, with no steering mechanism at all, a purely fixed-axle structure that is nonetheless omnidirectional.The problem is that the rollers have to be really close to the ground to transmit force, so the robot has to be very low to the ground. The slide shows a small robot and a wheelchair as examples.

Part IV
Wheels: Constraints and Mobility

6 Wheels (constraints & mobility) *(page 1)*

Wheels do not merely support the robot: they *introduce constraints* on its motion. We can study them in four steps:

- **geometric constraints** (GC), which limit the configurations q the robot can take;
- **kinematic constraints** (KC), which limit the velocities $\dot{q}$ it can have;
- the **relationships** between GC and KC;
- the resulting **mobility** of the robot,

and then apply everything to the example of **rolling wheels**.

Intuition

Compare a bead on a wire with an ice skater. The bead can never leave the wire: a restriction on *where* it can be. The skater's blade cannot slide sideways at any instant: a restriction on *how* she can move, yet she can still reach every point of the rink. Both are "constraints" but of two different natures, and this part of the lecture is about telling them apart. Wheels turn out to be of the skater type.

7 Geometric constraints (GC) *(pages 1–2)*

7.1 Constraints on the configuration *(page 1)*

Definition 7.1 (Configuration space)**.** The configuration of the robot is a vector $q \in \mathcal{C}$ of generalized coordinates, where the configuration space $\mathcal{C}$ is (locally) $\simeq \mathbb{R}^n$. The number n is the number of coordinates.

Constraints are classified along two axes: *equality* (bilateral) vs. *inequality* (unilateral), and *time-variant* vs. *time-invariant*.

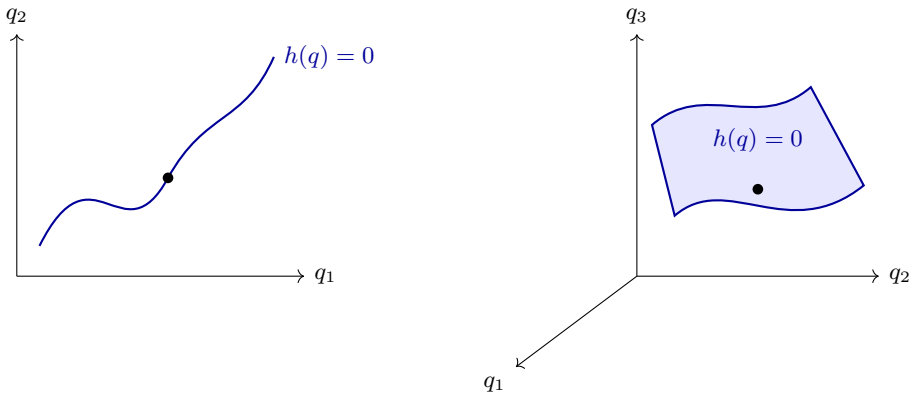
	time-invariant	time-variant
equality (bilateral)	$h(q) = 0$	$h(q,t) = 0$
inequality (unilateral)	$h(q) \geq 0$	$h(q,t) \geq 0$

Table 2: Classification of constraints on q (page 1). Only the first column of the first row is studied here.

Definition 7.2 (Geometric constraint)**.** Among the equality, time-invariant constraints, a *geometric constraint* (GC) has the form

$$h_i(q) = 0, \qquad i = 1, \ldots, k,$$

and *depends only on* q.



$\mathcal{C} \simeq \mathbb{R}^2$, $k = 1$
the robot can only stay on the curve

$\mathcal{C} \simeq \mathbb{R}^3$, $k = 1$
the robot can only stay on the surface

Figure 29: Geometric constraints (page 1): the dot is the robot, which must remain on the curve (left) or surface (right) $h(q) = 0$.

Intuition

A bead threaded on a wire has $n = 2$ coordinates in the plane but one constraint ($k = 1$): it lives on a curve. A point constrained to a surface in space ($n = 3$, $k = 1$) lives on a surface. In general, k constraints leave an $(n - k)$-dimensional world for the robot to live in.

7.2 Mobility limitation and reduced coordinates *(page 2)*

Proposition 7.3 (Global mobility limitation)**.** *If the constraints $h_1(q) = \cdots = h_k(q) = 0$ are independent, i.e. $\partial h/\partial q$ has rank k, the set $\{q : h(q) = 0\}$ is (locally) an $(n-k)$-dimensional subset of $\mathcal{C}$, and the robot configuration must remain in it for all times.*

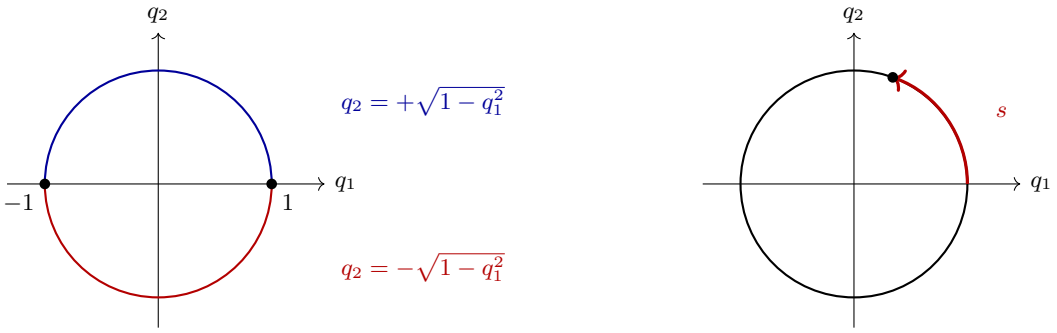
Proof. This is the implicit function theorem (regular value theorem): around any solution, k coordinates can be expressed as smooth functions of the remaining $n - k$. □

We call this a *global* mobility limitation: wherever the robot is, it is confined to a lower-dimensional subset.

Intuition

Are we then "wasting" the k remaining coordinates? Yes: they are determined by the other $n - k$. The number $n - k$ is the *number of degrees of freedom* (DOF), and one may try to describe the robot with a "reduced" configuration vector of $n - k$ coordinates: solve the GC for k coordinates as functions of the independent ones and keep only the latter.

Example 7.4 (The circle)**.** Take $q = (q_1, q_2) \in \mathbb{R}^2$ with $q_1^2 + q_2^2 = 1$. Solving for q_2 gives $q_2 = \pm\sqrt{1 - q_1^2}$. This is only a *local* resolution: it needs a choice of branch, and at $q_1 = \pm 1$ the derivative $\partial h/\partial q_2 = 2q_2$ vanishes, so the square root breaks down (a *singularity*). So *we don't do that.* We use different coordinates instead, here the arc length $s \in [0, 2\pi)$, with $q = (\cos s, \sin s)$.



local resolution: two branches, singular at $q_1 = \pm 1$ — arc-length coordinate $s \in [0, 2\pi)$

Figure 30: The circle $q_1^2 + q_2^2 = 1$ (page 2): solving the GC for one coordinate versus using a single new coordinate s.

Intuition

It is the difference between describing a point on the Earth's equator by "the latitude as a square root of something" and simply by its *longitude*: the longitude (our arc length s) is one honest coordinate that wraps around, with no branches and no singular points. In general one picks new coordinates that automatically satisfy the GC, instead of solving the constraint equations.

8 Kinematic constraints (KC) *(page 3)*

Definition 8.1 (Kinematic constraint)**.** A *kinematic constraint* (KC) has the form

$$a_i(q, \dot{q}) = 0, \qquad i = 1, \dots, k,$$

where $\dot{q}$ are the *generalized velocities* (time derivatives). It depends on q *and* $\dot{q}$; at any q it is a constraint on $\dot{q}$.

Definition 8.2 (Pfaffian form)**.** A KC is in *Pfaffian form* when $\dot{q}$ appears *linearly*:

$$a_i^T(q)\,\dot{q} = 0, \quad i = 1, \dots, k \qquad \Longleftrightarrow \qquad A(q)\,\dot{q} = 0, \quad A(q) = \begin{bmatrix} a_1^T(q) \\ \vdots \\ a_k^T(q) \end{bmatrix}.$$

KC are typically found in this form.

Proposition 8.3 (Admissible velocities)**.** *At each configuration q the admissible generalized velocities are exactly those in the null space $\mathcal{N}(A(q))$. If $A(q)$ has rank k, this is a linear subspace of dimension $n - k$.*

Proof. $A(q)\dot{q} = 0$ is the definition of the null space, whose dimension is $n - \operatorname{rank} A(q) = n - k$ by rank–nullity. □

Example 8.4. For $n = 3$ and the single constraint $a^T = (2, -1, 0)$, i.e. $2\dot{q}_1 - \dot{q}_2 = 0$, the admissible velocities are $\dot{q} = \alpha(1, 2, 0)^T + \beta(0, 0, 1)^T$: a plane ($n - k = 2$ dimensions) in velocity space.

Intuition

The KC does not forbid any *place*; it forbids some *ways of moving* at each place: at every q the allowed velocities form a plane (or subspace), and the plane can change with q. This is a **local** mobility limitation. The number of instantaneous degrees of freedom is still $n - k$, but *we cannot "reduce" the configuration vector*: all n coordinates are still needed (whether this is really so is the subject of the next section).

9 Relationships between GC and KC *(pages 4–6)*

9.1 Any GC implies a KC *(pages 4–5)*

Proposition 9.1 (GC ⇒ KC)**.** *Every GC $h(q) = 0$ implies the Pfaffian KC*

$$\dot{h}(q) = \frac{\partial h}{\partial q}\,\dot{q} = \left(\nabla_q h\right)^T \dot{q} = 0,$$

i.e. $a^T(q) = \partial h / \partial q$. Hence admissible velocities belong to the tangent plane of the surface $h(q) = 0$ at every q.

Proof. For the constraint to hold as time goes by, it must hold continuously: $h(q(t)) \equiv 0$, so

$\dot{h} = 0$. Since q depends on time, the chain rule gives $\frac{\partial h}{\partial q}\dot{q} = 0$, where $\left(\frac{\partial h}{\partial q_1}, \dots, \frac{\partial h}{\partial q_n}\right) = (\nabla_q h)^T$ is the gradient. This is a KC in Pfaffian form. □

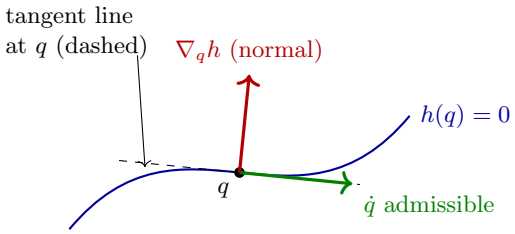


Figure 31: The gradient $\nabla_q h$ is normal to the surface $h(q) = 0$; admissible velocities are tangent to it, so $(\nabla_q h)^T \dot{q} = 0$ (page 4).

Intuition

Imagine hiking on a hillside: your velocity must be tangent to the slope, never pointing into the ground or up into the air. The gradient is the arrow sticking out of the surface (the normal), and "$a^T \dot{q} = 0$" simply says that the velocity is *orthogonal to the normal*: a dot product between orthogonal vectors is zero. So every geometric constraint secretly hides a velocity constraint.

9.2 The converse: integrable vs. non-integrable *(pages 5–6)*

Can we say the contrary, i.e. that a KC $a^T(q)\dot{q} = 0$ implies a GC $h(q) = 0$? In general no, sometimes yes: it depends on whether the KC can be *integrated*.

Definition 9.2 (Holonomic / nonholonomic)**.** A Pfaffian KC $a^T(q)\dot{q} = 0$ is *integrable* (*holonomic*) if there exist a function $h(q)$ and a function $\gamma(q) \neq 0$ such that

$$\frac{\partial h}{\partial q} = \gamma(q)\, a^T(q),$$

so that the KC is equivalent to $\dot{h} = 0$, i.e. to the GC $h(q) = \text{const}$. Otherwise it is *non-integrable* (*nonholonomic*), which is the general case.

Example 9.3 (Integrable: circles)**.** In $\mathbb{R}^2$, $q = (q_1, q_2)^T$, take $q_1\dot{q}_1 + q_2\dot{q}_2 = 0$, i.e. $a^T(q) = (q_1,\ q_2)$. Consider $q_1^2 + q_2^2 = C$ and differentiate: $2q_1\dot{q}_1 + 2q_2\dot{q}_2 = 0$; dividing by 2 gives the KC. So it is *integrable*, with $h = q_1^2 + q_2^2$ ($\gamma = 2$) and C an integration constant.

Proposition 9.4 (Local limitation ⇒ global limitation)**.** *If a KC is integrable, every admissible trajectory stays on a level set of h: $h(q(t)) = h(q(0))$ for all t.*

Proof. $\frac{d}{dt}h(q(t)) = \frac{\partial h}{\partial q}\dot{q} = \gamma(q)\, a^T(q)\dot{q} = 0$. □

Example 9.5 (Left open by the professor: $q_2\dot{q}_1 + q_1\dot{q}_2 = 0$ *(solution by me, not in the notes)*)**.** Here $a^T = (q_2,\ q_1)$ and $h(q) = q_1 q_2$ satisfies $\partial h/\partial q = (q_2, q_1) = a^T$ (with $\gamma = 1$): the KC is integrable and the robot is confined to a hyperbola $q_1 q_2 = c$.

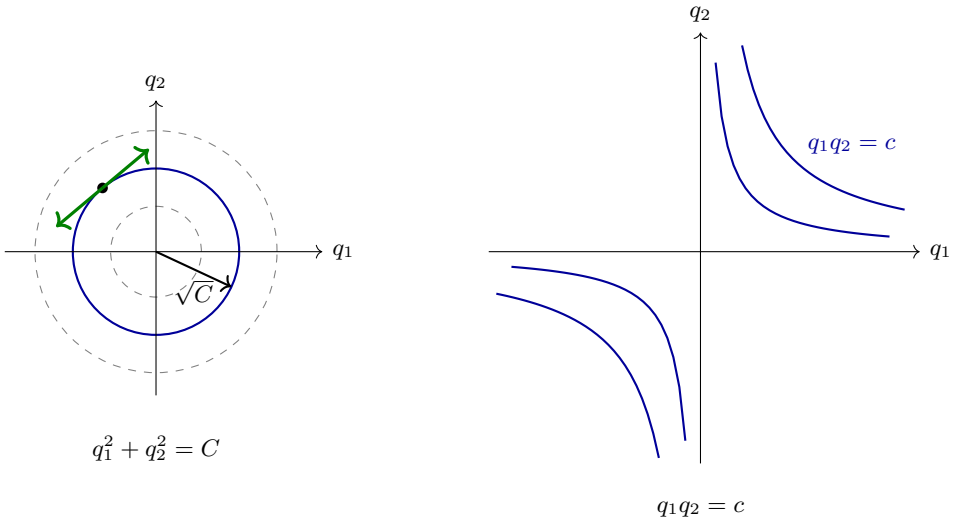


Figure 32: Left: the KC $q_1\dot{q}_1 + q_2\dot{q}_2 = 0$ is tangent to circles: starting at a point we cannot leave the circle (page 6). Right: the KC $q_2\dot{q}_1 + q_1\dot{q}_2 = 0$ integrates to hyperbolas.

Intuition

At each point the admissible velocity is tangent to a curve; chaining these tiny tangent steps, you simply trace the curve out: you can never hop onto another circle. This is why we can say: *the local limitation (of the KC) implies a global limitation (of the GC)*. The function γ is just a rescaling: multiplying the constraint by a non-zero factor does not change which velocities are admissible. How does one know if a constraint is integrable without guessing h? There are mathematical tools, such as **Frobenius' theorem**.

Theorem 9.6 (Frobenius, single constraint in $\mathbb{R}^3$ *(stated without proof)*)**.** *Let $a(q) \neq 0$, $q \in \mathbb{R}^3$. The constraint $a^T(q)\dot{q} = 0$ is integrable if and only if $a \cdot (\nabla \times a) = 0$.*

10 Example: rolling wheels *(pages 7–9)*

10.1 The rolling disk and the pure rolling constraint *(page 7)*

If the KC is **not integrable** (nonholonomic), the mobility limitation is *purely local*. The key example is the rolling disk.

Definition 10.1 (Rolling disk)**.** A disk (wheel) that stays upright while rolling on the plane. Its configuration is $q = (x, y, \theta)^T$, where (x, y) is the contact point and θ the orientation of the wheel plane (heading).

Lemma 10.2 (Pure rolling constraint)**.** *Pure rolling without lateral slip means that the instantaneous velocity of the contact point has no component along the unit normal $n = (\sin\theta, -\cos\theta)^T$ to the wheel plane:*

$$\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0, \quad \textit{i.e.} \quad \underbrace{(\sin\theta,\ -\cos\theta)}_{n^T} \begin{pmatrix}\dot{x}\\ \dot{y}\end{pmatrix} = 0.$$

In the full configuration space this is the Pfaffian KC

$$a^T(q)\,\dot{q} = (\sin\theta,\ -\cos\theta,\ 0)\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = 0.$$

Proof. A velocity $(\dot{x},\dot{y})$ orthogonal to n is parallel to the heading $(\cos\theta,\sin\theta)$; the formula is the orthogonality condition $n^T(\dot{x},\dot{y})^T = 0$, with a zero entry for $\dot{\theta}$ since the constraint does not involve it. □

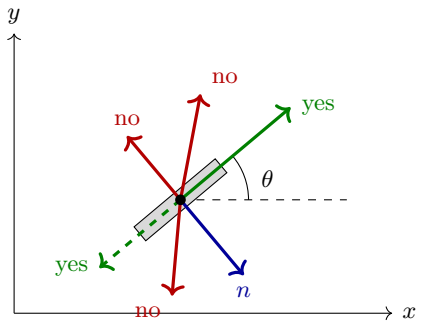


Figure 33: Top view of the rolling disk (page 7); the dot is the contact point (x,y). Velocities of the contact point with a component along the normal n are forbidden ("no"); only motion along the wheel plane is allowed ("yes").

Intuition

It is the ice-skater's blade: the contact point can glide forwards or backwards along the blade, but can never slide sideways, so the velocity is always lined up with θ. Since the condition concerns *instantaneous* velocity, it is a *local* mobility constraint. In fact the null space of $a^T(q)$ is spanned by $g_1 = (\cos\theta,\sin\theta,0)^T$ (roll) and $g_2 = (0,0,1)^T$ (spin): $\dot{q} = g_1 v + g_2\omega$, with $n-k = 3-1 = 2$ admissible directions at each configuration.

10.2 Is pure rolling integrable? *(page 8)*

Proposition 10.3 (Pure rolling is non-integrable)**.** *The pure rolling constraint is non-integrable.*

We decide to avoid the use of Frobenius' theorem and prove it *constructively*, by proposing a maneuver that moves the disk from anywhere to everywhere: the *universal maneuver*.

Algorithm 10.4 (Universal maneuver)**.** Input: start $q_s = (x_s,y_s,\theta_s)$ and goal $q_g = (x_g,y_g,\theta_g)$, with $(x_g,y_g) \neq (x_s,y_s)$. Let $\varphi = \operatorname{atan2}(y_g - y_s,\ x_g - x_s)$.

1. **Pure rotation** about the vertical axis, $\dot{x} = \dot{y} = 0$, from θ_s to φ (the KC is satisfied).
2. **Pure rolling**, $\dot{q} = v(\cos\varphi,\sin\varphi,0)^T$, until (x_g,y_g) is reached (the KC is satisfied because the contact point velocity is always aligned with $\theta = \varphi$).
3. **Pure rotation** from φ to θ_g (the KC is satisfied).

(If $(x_g,y_g) = (x_s,y_s)$, only a rotation is needed.)

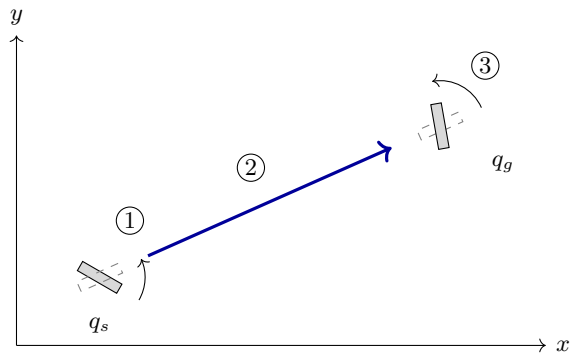


Figure 34: The universal maneuver (page 8): ① rotate in place towards the goal, ② roll straight to (x_g, y_g), ③ rotate to the final orientation. Dashed: intermediate configurations.

Proposition 10.5 (The maneuver is universal)**.** *For any $q_s, q_g \in \mathbb{R}^2 \times S^1$, Algorithm 10.5 produces a path that satisfies the pure rolling constraint at every instant and connects q_s to q_g.*

Proof. During phases 1 and 3, $\dot{x} = \dot{y} = 0$ so $a^T \dot{q} = 0 \cdot \sin\theta - 0 \cdot \cos\theta + 0 \cdot \dot{\theta} = 0$. During phase 2, $\theta = \varphi$ is constant and $\dot{q} = v(\cos\varphi, \sin\varphi, 0)$, so $a^T \dot{q} = v(\sin\varphi\cos\varphi - \cos\varphi\sin\varphi) = 0$. The three phases end respectively at heading φ, at position (x_g, y_g) and at heading θ_g. □

Corollary 10.6 (Non-integrability)**.** *The pure rolling constraint is non-holonomic.*

Proof. If it were integrable, by Proposition 9.4 every admissible trajectory would stay on a level set of a function h with $\partial h/\partial q = \gamma a^T \neq 0$, so h would not be constant; but by the universal maneuver 10.5 admissible paths connect any two configurations, so h would be constant on $\mathbb{R}^2 \times S^1$. Contradiction. □

Example 10.7. Let $q_s = (0, 0, 0)$ and $q_g = (1, 1, \pi/2)$ (metres, radians). Then $\varphi = \pi/4$: rotate by $\pi/4$ in place, roll $\sqrt{2} \approx 1.41\,\text{m}$ along the diagonal, then rotate by another $\pi/4$ to reach $\theta_g = \pi/2$.

Intuition

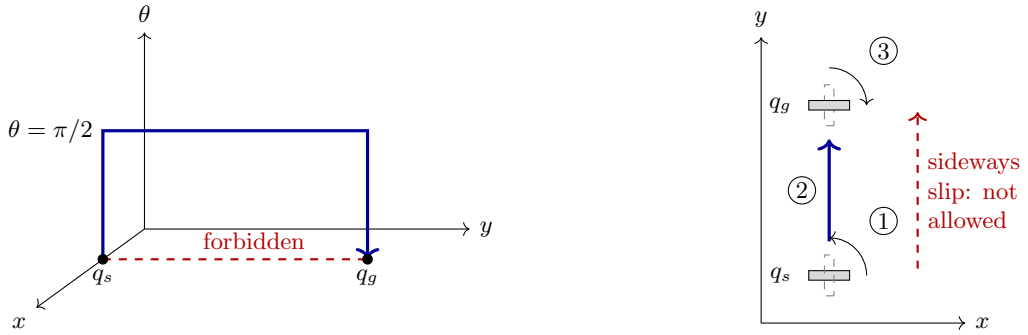
It is the recipe of "turn towards the goal, walk straight, turn to the final direction". Each of the three motions is allowed by the KC, and together they reach *any* configuration, even though at every instant only 2 of the 3 directions of q-space are admissible. A constraint that could be integrated would trap the robot on a curve or surface, whereas here nothing traps it.

10.3 No global mobility limitation *(page 9)*

Hence there is **no global mobility limitation**: the KC *must be non-holonomic*. Here we illustrate this with a "naive representation" of $\mathcal{C}$ and with parallel parking.

Example 10.8 (Parallel parking of a rolling disk)**.** Let $q_s = (0, 0, 0)$ and $q_g = (0, 2, 0)$: the goal is 2 m to the side, with the same orientation. The direct motion $\dot{y} \neq 0$ with $\theta = 0$ is not admissible, since $a^T \dot{q} = -\dot{y} \neq 0$: parallel parking *cannot be done instantaneously*. It can be

done with three admissible moves: (1) rotate to $\theta = \pi/2$; (2) roll 2 m along y (at $\theta = \pi/2$ the constraint reads $\dot{x} = 0$, so y is free); (3) rotate back to $\theta = 0$.



(a) path in $\mathcal{C}$

(b) top view

Figure 35: Parallel parking (page 9). (a) In $\mathcal{C} = \mathbb{R}^2 \times S^1$ the direct path at $\theta = 0$ is forbidden; the admissible path climbs along the θ axis, moves along y, and comes back down. (b) The same maneuver seen from above.

Intuition

Think of the three coordinates (x, y, θ) as a 3D world in which the disk may move only along two of the three directions at each point; yet by combining the motions, like a sailor tacking against the wind, every point becomes reachable. So the kinematic constraint *reduces how you can move* (local), but *not where you can get to* (global). Compare the two kinds of constraint: an integrable KC is a GC in disguise (global limitation, the dimension of the reachable set drops to $n - k$); a non-integrable KC (rolling) keeps all n coordinates alive and limits only the instantaneous velocities.

11 Summary

11.1 Part I: kinematic structures of WMRs

Structure	Wheels	Key feature	Slide examples
Differential-drive	2 fixed, independently actuated, + 1 caster (balance)	Steering through the speed difference of the two wheels	indoor base, robot vacuum cleaner
Synchro-drive	3 orientable, simultaneously actuated	Wheels always parallel; chassis orientation constant	cylindrical robot with laptop
Tricycle	2 fixed on a common axle + 1 orientable	Front- or rear-wheel drive	—
Car-like	2 fixed + 2 orientable, each pair on a common axle	Front- or rear-wheel drive; differential for a driven common axle	—
Firetruck	car-like tractor + trailer with 2 orientable wheels on a common axle	Trailer steered separately by the tiller operator	ladder truck
Omni, active casters	active caster wheels (steered and driven)	Any direction without re-orienting; 2 motors per wheel	actuated caster unit
Omni, Mecanum	wheels with oblique free rollers	Omnidirectional with fixed wheel axles	small robot, wheelchair

Table 3: Part I: comparison of the WMR kinematic structures (slides 9–15).

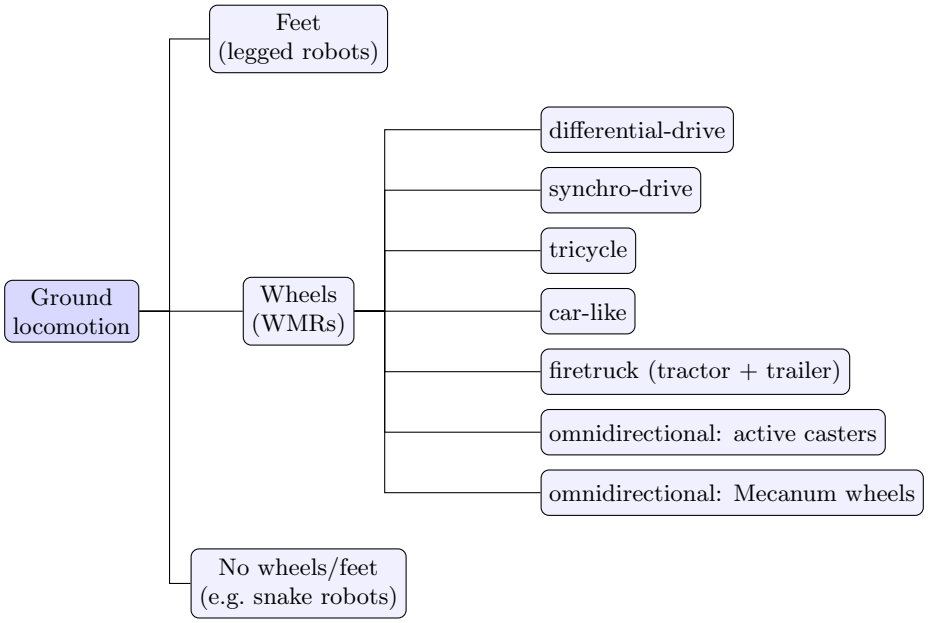


Figure 36: Part I map: from ground locomotion to the WMR kinematic structures.

11.2 Part II: constraints and mobility

Constraint	Form	Mobility limitation	Example
Geometric (GC)	$h(q) = 0$, depends only on q	*Global*: q confined to an $(n-k)$-dimensional set; reduced coordinates possible	circle $q_1^2 + q_2^2 = 1$, arc length s
Kinematic, integrable (holonomic)	$a^T(q)\dot{q} = 0$ with $\partial h/\partial q = \gamma\, a^T$	Equivalent to a GC: local limitation implies a global one	$q_1\dot{q}_1 + q_2\dot{q}_2 = 0$: circles
Kinematic, non-integrable (nonholonomic)	$a^T(q)\dot{q} = 0$, no such h	*Purely local*: $\dot{q} \in \mathcal{N}(A(q))$; every configuration is reachable; q cannot be reduced	rolling disk $\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0$

Table 4: Part II: types of constraints and their effect on mobility (pages 1–9).

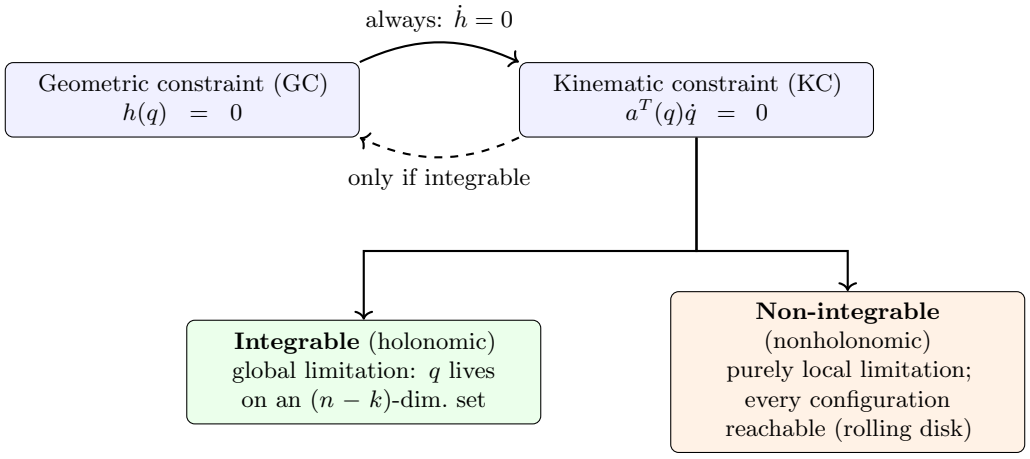


Figure 37: Part II map: relationships between geometric and kinematic constraints.

11.3 Key take-aways

- Ground locomotion needs contact (wheels or feet). Static balance requires the CoM projection inside the support polygon, hence at least 3 wheels; dynamic balance replaces the CoM with the ZMP.
- Wheels come in three types (fixed, orientable, caster) which combine into the classical structures: differential-drive, synchro-drive, tricycle, car-like, tractor–trailer, and omnidirectional robots (active casters or Mecanum wheels); a differential is needed when two driven wheels share an axle.
- Wheels introduce constraints on q. A geometric constraint $h(q) = 0$ confines the robot globally to an $(n-k)$-dimensional set; a kinematic constraint $a^T(q)\dot{q} = 0$ restricts the velocities locally to the null space of $A(q)$.
- Every GC implies a KC (the velocity must be tangent to the surface, $\dot{h} = 0$); the converse holds only for integrable (holonomic) KC, for which the local limitation implies a global one.
- Pure rolling, $\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0$, is non-integrable (Frobenius, or the universal "turn, roll, turn" maneuver): there is no global mobility limitation, all of (x, y, θ) is reachable, but not instantaneously, as in parallel parking.

Part V

Lecture 3

1 Kinematic constraints and admissible motions

(page 1)

Let the robot have a configuration vector $q \in \mathcal{C} \simeq \mathbb{R}^n$, where n is the number of generalized coordinates. The wheels impose *kinematic constraints* (KCs) that are linear in the generalized velocities.

Definition 1.1 (Kinematic constraints)**.** A set of k kinematic constraints is written in Pfaffian form

$$A(q)\ = 0, \qquad A(q) \in \mathbb{R}^{k\times n},$$

where k is the number of constraints and n the number of generalized coordinates.

Definition 1.2 (Admissible motions)**.** A generalized velocity is *admissible* at q if $\in (A(q))$, the null space of $A(q)$. If the constraints are independent, $\dim(A(q)) = n - k =: m$.

Intuition

The constraints do not forbid places, they forbid *instantaneous directions of motion.* At each configuration q the robot may move only along the directions in the null space. Think of a skate on ice: from any position it may glide along its blade, but not sideways. The set of allowed directions changes with the configuration, which is why A depends on q.

Pick a basis $\{g_1(q), \dots, g_m(q)\}$ of $(A(q))$. Each $g_i : \mathcal{C} \to \mathbb{R}^n$ is a *vector field*, and every admissible velocity is a combination

$$= \sum_{i=1}^{m} g_i(q)\, u_i = \begin{pmatrix} g_1(q) & \cdots & g_m(q) \end{pmatrix} \begin{pmatrix} u_1 \\ \vdots \\ u_m \end{pmatrix},$$

where the u_i are scalar coefficients, i.e. the *admissible velocities* (also called pseudo-velocities).

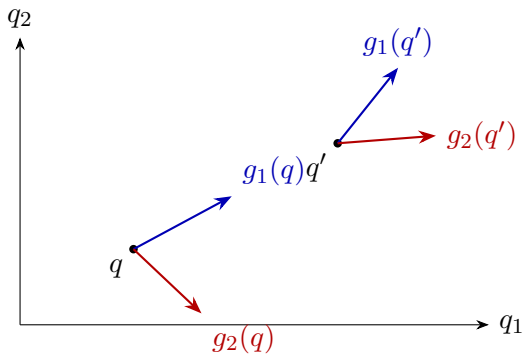


Figure 38: The basis vectors $g_i(q)$ are vector fields: at each configuration they give (generally different) admissible directions.

2 Explicit representation: the kinematic model

(pages 2–3)

Definition 2.1 (Kinematic model)**.** The *kinematic model* (KM) of a robot with m independent admissible directions is

$$= \sum_{i=1}^{m} g_i(q)\, u_i = G(q)\, u, \qquad G(q) \in \mathbb{R}^{n\times m} \text{ (a tall matrix)},\ u \in \mathbb{R}^m.$$

For a chosen input u, at each q it returns the instantaneous motion the robot executes.

The KM is a *dynamical system* with

- state $q \in \mathcal{C} \simeq \mathbb{R}^n$ and input $u \in \mathbb{R}^m$;
- *nonlinear* dependence on the state q (linear in the input u);
- *driftless*: $u = 0 \Rightarrow = 0$, i.e. with no input the robot stays where it is.

Intuition

"Driftless" means there is no momentum in a kinematic model: stop commanding and the robot stops. (Momentum and forces appear only in the *dynamic* model, which is a different level of description.) "Tall matrix" simply says that we have more coordinates (n) than inputs (m): fewer commands than degrees of freedom of the configuration, which is exactly what the constraints cause.

Proposition 2.2 (The KM is not unique)**.** *Let $= G(q)u$ be a kinematic model and $T(q) \in \mathbb{R}^{m\times m}$ any smooth invertible matrix. Then $= G(q)T(q)\,u'$ is another kinematic model of the same robot, and the two describe the same set of admissible velocities.*

Proof. The columns of $G' = GT$ are linear combinations of the columns of G, hence lie in $(A(q))$; they are independent because T is invertible, so they are m independent vectors in an m-dimensional space, i.e. a basis. Conversely any admissible $= Gu = GT(T^{-1}u)$ is generated by $u' = T^{-1}u$. □

Intuition

The basis $\{g_1, \dots, g_m\}$ can be chosen in infinitely many ways (rotate, rescale, mix the vector fields), so *all these models are equivalent*. Some choices are nevertheless better because the inputs u_i get a physical *interpretation* (e.g. "forward speed" and "turning speed"). We will see this with the unicycle.

2.1 The unconstrained robot

(page 3)

Example 2.3 (KM of an unconstrained robot)**.** If there are no kinematic constraints ($k = 0$) and no geometric constraints, then $m = n$, A is absent and a natural basis of $\mathbb{R}^n$ is the canonical one:

$$= \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} u_1 + \cdots + \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix} u_n = u.$$

This is the case of, e.g., a free-flying robot. Every generalized velocity is directly commanded: the robot is *fully actuated*.

Block schemes. The unconstrained robot is a bank of n simple integrators; a constrained robot places the matrix $G(q)$ between the pseudo-velocities u and the integrators, with q fed back to G.

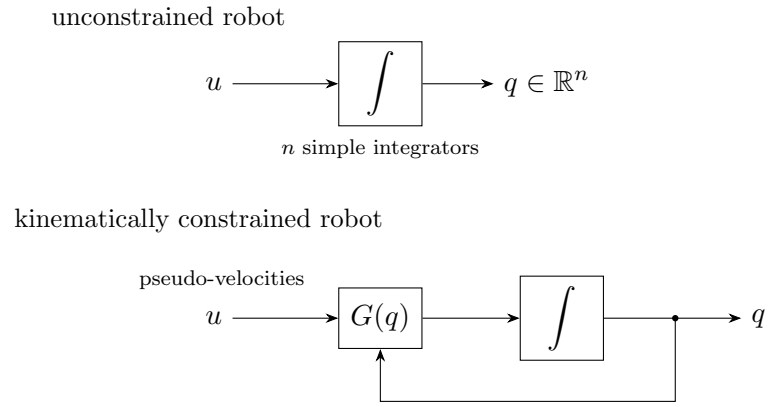


Figure 39: Block schemes of the kinematic model.

3 Controllability of kinematic models

(page 4)

Definition 3.1 (Controllable KM)**.** A kinematic model is *controllable* if for any initial configuration q_i and any final configuration q_f there exist a time T and an input $u : [0, T] \to \mathbb{R}^m$ such that the solution of $= G(q)u$ starting at q_i reaches q_f at time T.

Controllability can be established in two ways: *constructively*, by looking for a *universal maneuver* that connects any pair of configurations, or by applying a *test for controllability* (a rank condition, see Remark 5.3).

Link with the constraints. The notes relate controllability to the integrability of the kinematic constraints:

- the KM is controllable $\iff$ the KCs are *nonholonomic* (not integrable);
- the KM is not controllable $\iff$ the KCs are *holonomic*, i.e. integrable (partially or completely).

Intuition

If a constraint is integrable, it can be rewritten as a restriction on positions, $h(q) = \text{const}$ (for example $y = 0$): the robot is trapped on a lower-dimensional surface and can never reach the other configurations, so the KM cannot be controllable. If the constraints are not integrable they leave the robot able to reach every configuration by sequencing admissible motions, as a car does when parallel parking: it cannot move sideways instantaneously, but it can reach any pose.

4 The unicycle

(pages 4–6)

Definition 4.1 (Unicycle)**.** The unicycle is a vehicle with a single *orientable* wheel. Its configuration is $q = (x, y, \theta)^\top$, where (x, y) is the contact point and θ the orientation of the wheel's *sagittal axis* with respect to the x-axis. The configuration space is $\mathcal{C} = \mathbb{R}^2 \times \mathrm{SO}(2)$, locally $\simeq \mathbb{R}^3$ (hence $n = 3$).

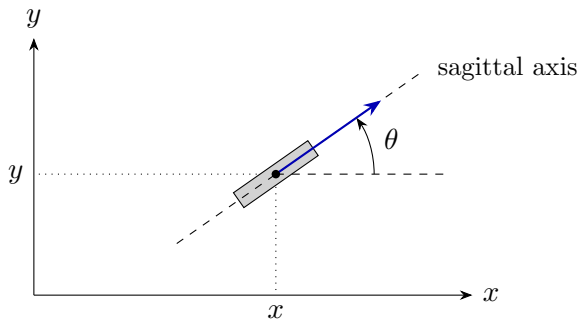


Figure 40: The unicycle seen from above: contact point (x, y), sagittal axis at angle θ.

4.1 The pure rolling constraint

(page 5)

The wheel rolls without slipping, so the contact point cannot move along the wheel axis (orthogonal to the sagittal axis):

$$\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0.$$

Equivalently, $(\dot{x}, \dot{y})$ must be *aligned with the sagittal axis*, i.e. it has no component along the direction orthogonal to it. In Pfaffian form

$$\underbrace{\begin{pmatrix}\sin\theta & -\cos\theta & 0\end{pmatrix}}_{A(q)} \begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = 0, \qquad k = 1,\ n = 3,\ m = n - k = 2.$$

Intuition

Worked numbers. At $\theta = 0$: $A = (0, -1, 0)$, so the constraint reads $\dot{y} = 0$: the wheel may translate along x and spin in place, never sideways. At $\theta = \pi/2$: $A = (1, 0, 0)$, so $\dot{x} = 0$: now it may move along y only. The forbidden direction rotates together with the wheel.

Proposition 4.2 (Basis of the admissible velocities)**.** *A basis of* $\begin{pmatrix}\sin\theta & -\cos\theta & 0\end{pmatrix}$*, a 2-dimensional linear space, is*

$$g_1(q) = \begin{pmatrix}\cos\theta\\ \sin\theta\\ 0\end{pmatrix}, \qquad g_2(q) = \begin{pmatrix}0\\ 0\\ 1\end{pmatrix}.$$

Proof. $Ag_1 = \sin\theta\cos\theta - \cos\theta\sin\theta = 0$ and $Ag_2 = 0$. The two vectors are linearly independent for every θ, and $\dim(A) = 3 - 1 = 2$. □

Kinematic model of the unicycle.

$$= \begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix} u_1 + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} u_2 \quad \Longleftrightarrow \quad \begin{cases} \dot{x} = \cos\theta\, u_1 \\ \dot{y} = \sin\theta\, u_1 \\ \dot{\theta} = u_2. \end{cases}$$

4.2 Interpretation of the inputs

(pages 5–6)

From the model, $\dot{x}^2 + \dot{y}^2 = \cos^2\theta\, u_1^2 + \sin^2\theta\, u_1^2 = u_1^2$, hence

$$u_1 = \pm\sqrt{\dot{x}^2 + \dot{y}^2} \quad \text{(equivalently } u_1 = \cos\theta\,\dot{x} + \sin\theta\,\dot{y}\text{)},$$

the magnitude of the velocity of the contact point, with sign + for forward and − for backward motion. We therefore rename

$$u_1 = v \text{ (driving velocity)}, \qquad u_2 = \dot{\theta} = \omega \text{ (steering velocity)}.$$

The driving velocity is related to the angular velocity of the wheel about its own axis by $v = r\,\omega_{\text{roll}}$ (r = wheel radius, side view in Fig. 41); ω is the angular velocity of the wheel about the vertical axis.

Definition 4.3 (Unicycle kinematic model)**.**

$$= \begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix} v + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \omega, \qquad \text{i.e.} \quad \begin{cases} \dot{x} = v\cos\theta \\ \dot{y} = v\sin\theta \\ \dot{\theta} = \omega. \end{cases}$$

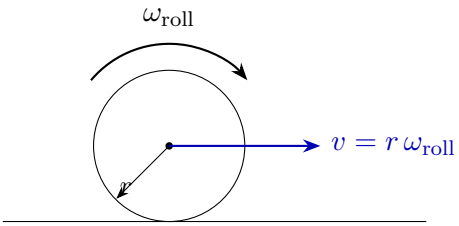


Figure 41: Side view of the wheel: the driving velocity is $v = r\,\omega_{\text{roll}}$.

Intuition

The basis $\{g_1, g_2\}$ was chosen so that the inputs are exactly what a driver feels: v is "how fast I roll along where I am pointing" and ω is "how fast I turn". Any other basis (Proposition 2.2) would give an equivalent but less readable model. Constant inputs give circles: with $v = 1$ m/s and $\omega = 0.5$ rad/s the contact point moves on a circle of radius $v/\omega = 2$ m (see the figure in the Summary).

5 Controllability of the unicycle

(page 7)

Proposition 5.1. *The unicycle KM is controllable: it can be steered from any $q_i = (x_i, y_i, \theta_i)$ to any $q_f = (x_f, y_f, \theta_f)$.*

The notes answer "yes" by applying a universal maneuver, the same idea used earlier for the disk, in three steps.

Algorithm 5.2 (Universal maneuver for the unicycle)**.** Let $\varphi = \operatorname{atan2}(y_f - y_i,\ x_f - x_i)$ and $d = \sqrt{(x_f - x_i)^2 + (y_f - y_i)^2}$.

1. $v = 0$, $\omega =$ constant, until $\theta = \varphi$ (rotate in place to point at the goal).
2. $v =$ constant, $\omega = 0$, until $(x, y) = (x_f, y_f)$ (drive straight, distance d).
3. $v = 0$, $\omega =$ constant, until $\theta = \theta_f$ (rotate in place to the final orientation).

Proof of Proposition 5.1. During phases 1 and 3, $v = 0$ gives $\dot{x} = \dot{y} = 0$ and $\dot{\theta} = \omega$, so the position is frozen and θ changes at will; phase 1 takes θ from θ_i to φ in time $T_1 = (\varphi - \theta_i)/\omega$. During phase 2, $\omega = 0$ keeps $\theta = \varphi$ constant, so $(\dot{x}, \dot{y}) = v(\cos\varphi, \sin\varphi)$ is a straight line, reaching (x_f, y_f) after $T_2 = d/v$. Phase 3 takes θ from φ to θ_f. (If $d = 0$ phase 2 is skipped.) □

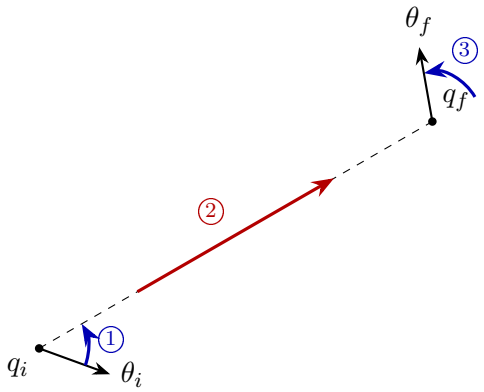


Figure 42: Universal maneuver: rotate in place, drive straight, rotate in place.

Intuition

It is a "point, drive, align" strategy. Worked numbers: from $q_i = (0, 0, 0)$ to $q_f = (3, 4, \pi/2)$ we have $\varphi = \arctan(4/3) \approx 0.927$ rad (53.1°) and $d = 5$. With $|\omega| = 1$ rad/s and $v = 1$ m/s: rotate for 0.927 s, drive for 5 s, rotate for $\pi/2 - 0.927 \approx 0.644$ s (36.9°). The two rotations "cost" nothing in position, which is possible only because the model lets us set $v = 0$ and turn freely.

Intuition

[Consistency with Section 3] Despite the constraint $\dot{y} = 0$ at $\theta = 0$ (no sideways motion), the unicycle can reach any pose, because the pure rolling constraint is *not integrable*: this is exactly the controllable/nonholonomic case.

Lemma 5.3 (Rank test, supplementary)**.** *Let* $[g_1, g_2] = \frac{\partial g_2}{\partial q} g_1 - \frac{\partial g_1}{\partial q} g_2$ *be the Lie bracket. For the unicycle* $[g_1, g_2] = (\sin\theta, -\cos\theta, 0)^\top$*, and* $\{g_1, g_2, [g_1, g_2]\} = 3 = n$ *for every* q*, so the controllability rank condition (Chow) holds.*

Proof. $\partial g_2/\partial q = 0$ and $\partial g_1/\partial q\, g_2 = (-\sin\theta, \cos\theta, 0)^\top$, hence $[g_1, g_2] = (\sin\theta, -\cos\theta, 0)^\top$, which is orthogonal to g_1 and independent of g_2. □

(This is the "test for controllability" mentioned on page 4; the explicit computation is not in the handwritten notes.)

Intuition

$[g_1, g_2]$ points *sideways*, exactly the forbidden direction: the wiggle "drive, turn, back up, turn back" produces a net sideways displacement, the same effect as parallel parking.

6 Mechanically equivalent vehicles

(pages 7–8)

Several real vehicles have the unicycle KM as soon as the right inputs are chosen.

6.1 Differential drive

(page 7)

Two parallel wheels (left and right) with a common axis; the reference point is the *midpoint between the two contact points*. Its KM is the unicycle one,

$$\begin{cases} \dot{x} = \cos\theta\, v \\ \dot{y} = \sin\theta\, v \\ \dot{\theta} = \omega \end{cases} \qquad v = r\,\frac{\omega_R + \omega_L}{2}, \qquad \omega = r\,\frac{\omega_R - \omega_L}{d},$$

where ω_R, ω_L are the roll velocities of the right and left wheel, r the wheel radius and d the distance between the wheels (the denominator is cut off in the handwritten page).

Proposition 6.1. *For a differential drive, the physical inputs (ω_R, ω_L) are related to the unicycle inputs (v, ω) by an invertible linear map.*

Proof. The contact points move with speeds $v_R = r\omega_R$ and $v_L = r\omega_L$. The midpoint has velocity $v = (v_R + v_L)/2$ and the body rotates with $\omega = (v_R - v_L)/d$. The matrix $\frac{r}{2}\begin{pmatrix} 1 & 1 \\ 2/d & -2/d \end{pmatrix}$ has determinant $-2r^2/d \neq 0$. □

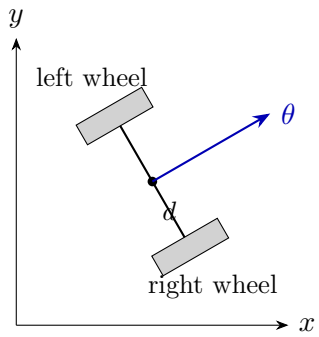


Figure 43: Differential drive seen from above; the reference point is the midpoint of the axle.

Intuition

Worked numbers: $r = 0.1$ m, $d = 0.4$ m, $\omega_R = 10$ rad/s, $\omega_L = 6$ rad/s. Then $v = 0.1 \cdot 16/2 = 0.8$ m/s and $\omega = 0.1 \cdot 4/0.4 = 1$ rad/s, a turn of radius $v/\omega = 0.8$ m. Equal wheel speeds give $\omega = 0$ (straight line); opposite speeds give $v = 0$ (rotation in place, the first and third phase of the universal maneuver).

6.2 Synchro-drive

(page 8)

A synchro-drive has three (or more) wheels that are steered and driven in a synchronized way, so that all wheels always point in the same direction θ. The notes state that the KM *is* the unicycle one and that *no input transformation is needed*:

$$\begin{cases} \dot{x} = \cos\theta\, v \\ \dot{y} = \sin\theta\, v \\ \dot{\theta} = \omega, \end{cases}$$

with v the common driving velocity and ω the common steering velocity.

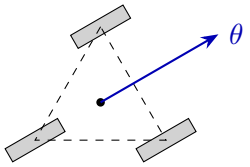


Figure 44: Synchro-drive: three wheels steered together, all with orientation θ.

Intuition

Because every wheel is always aligned with the others, the vehicle behaves as one big unicycle: the *body orientation* that the model calls θ is the common steering angle. Unlike the differential drive, the physical inputs (driving and steering velocity of the synchronized wheels) already coincide with (v, ω), so the model works as is.

7 Summary

System	n	k	m	Inputs u	Model
Unconstrained robot	n	0	n	$u \in \mathbb{R}^n$ (fully actuated)	$= u$ (canonical basis)
Unicycle	3	1	2	v (driving), ω (steering)	$= g_1 v + g_2 \omega$
Differential drive	3	1	2	$\omega_R, \omega_L \Rightarrow v = r\frac{\omega_R+\omega_L}{2}$, $\omega = r\frac{\omega_R-\omega_L}{d}$	unicycle KM
Synchro-drive	3	1	2	v, ω directly (no transformation)	unicycle KM

Table 5: Kinematic models covered. All are driftless and nonlinear in q; the three wheeled vehicles are controllable.

Key take-aways

- Wheels impose Pfaffian constraints $A(q) = 0$; admissible velocities form $(A(q))$, of dimension $m = n - k$.
- The kinematic model $= G(q)u$ is a nonlinear, driftless system; G is not unique, and bases are chosen for interpretability.

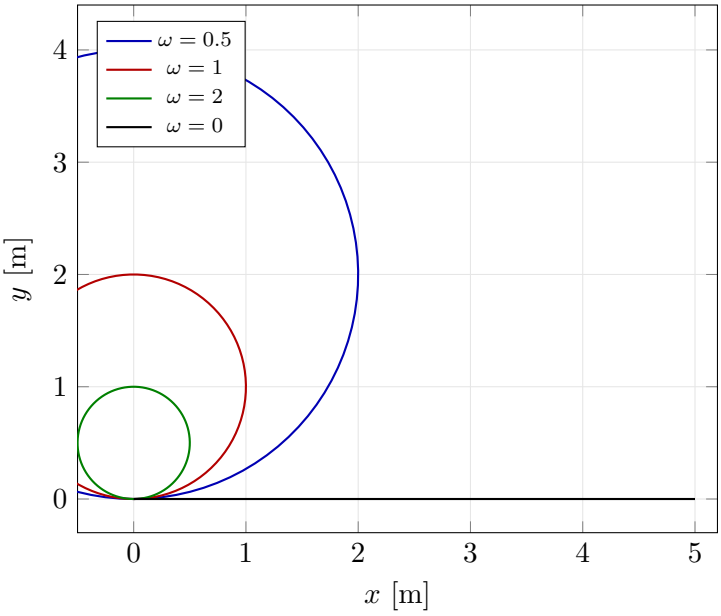


Figure 45: Unicycle with $v = 1$ m/s, $q_i = (0, 0, 0)$ and constant ω: circles of radius v/ω (straight line for $\omega = 0$).

- The unicycle $(n = 3, k = 1, m = 2)$ has $\dot{x} = v\cos\theta,\ \dot{y} = v\sin\theta,\ \dot{\theta} = \omega$; $v = r\omega_{\text{roll}}$ is the driving velocity and ω the steering velocity.
- Controllability of a KM is tied to nonholonomy of the constraints; the unicycle is controllable via the *rotate–drive–rotate* maneuver.
- Differential drive and synchro-drive share the unicycle KM (the former after the linear input change $v = r\frac{\omega_R+\omega_L}{2}$, $\omega = r\frac{\omega_R-\omega_L}{d}$).

Part VI

Lecture 4

Conventions. The driving and steering inputs are written u_1, u_2 (for the unicycle also v, ω). The Lie bracket is defined as $g_1 g_2 = \frac{\partial g_2}{\partial q} g_1 - \frac{\partial g_1}{\partial q} g_2$. $g_1 g_2 = \frac{\partial g_2}{\partial q} g_1 - \frac{\partial g_1}{\partial q} g_2$.

1 Recap: kinematic models and the unicycle

Definition 1.1 (Kinematic model)**.** Let a wheeled robot with configuration $q \in \mathbb{R}^n$ be subject to k independent Pfaffian (kinematic) constraints

$$A(q)\,\dot{q} = 0, \qquad A(q) \in \mathbb{R}^{k \times n}.$$

Admissible velocities are exactly $\dot{q} \in (A(q))$, a subspace of dimension $m = n - k$. Choosing a basis $g_1(q), \ldots, g_m(q)$ of the null space gives the *kinematic model*

$$\dot{q} = \sum_{i=1}^{m} g_i(q)\, u_i.$$

Example 1.2 (Unicycle)**.** With $q = (x, y, \theta)$ the single pure-rolling constraint is $\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0$, and

$$\dot{q} = \underbrace{\begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix}}_{\text{drive VF } g_1} v + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}}_{\text{steer VF } g_2} \omega.$$

Intuition

The two vector fields are the two "elementary motions" of the robot: g_1 rolls it along its heading without changing the heading, g_2 spins it in place. Every admissible velocity is a mix of the two. Example: at $\theta = \pi/4$ with $v = 1$, $\omega = 0$ the robot moves with $\dot{q} = (0.71, 0.71, 0)^T$, a straight diagonal line. We have $n = 3$ coordinates but only $m = 2$ inputs: the missing direction (sideways) is exactly what is forbidden by the constraint.

2 The bicycle model

2.1 Configuration and constraints

The bicycle has a fixed rear wheel and a steerable front wheel. Let (x, y) be the rear-wheel contact point, θ the orientation of the body, ϕ the steering angle, ℓ the wheelbase, and (x_f, y_f) the front-wheel contact point:

$$q = \begin{pmatrix} x \\ y \\ \theta \\ \phi \end{pmatrix}, \qquad \mathcal{C} = \mathbb{R}^2 \times SO(2) \times SO(2), \qquad n = 4.$$

Both wheels roll without lateral slipping:

$$\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0 \qquad \text{(pure rolling, rear wheel)} \qquad \text{(R)}$$
$$\sin(\theta+\phi)\,\dot{x}_f - \cos(\theta+\phi)\,\dot{y}_f = 0 \qquad \text{(pure rolling, front wheel)} \qquad \text{(F)}$$

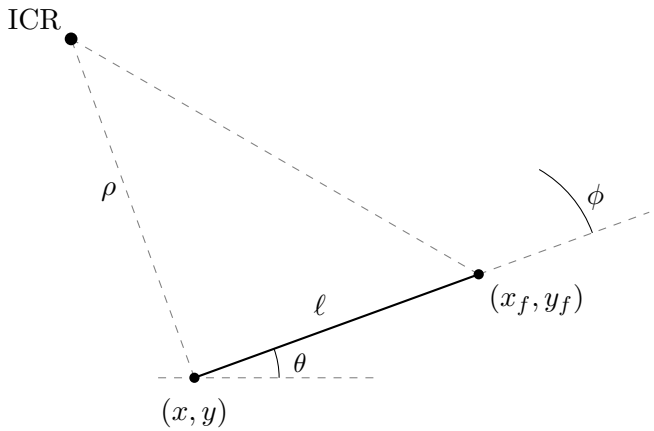


Figure 46: Bicycle geometry. The axes of the two wheels (perpendicular to the rolling directions) meet at the instantaneous centre of rotation (ICR); the whole robot rotates instantaneously about it, with turning radius $\rho = \ell/\tan\phi$.

Intuition

At every instant a rigid body that moves in the plane is rotating about a single point, the ICR. A rolling wheel can only move perpendicular to its axle, so the ICR must lie on the line through the axle. With two wheels, the ICR is the intersection of the two axle lines (Fig. 46) and *all* points of the robot contribute to this one rotation. Numbers: for $\ell = 1$ and $\phi = 30^\circ$ the radius is $\rho = 1/\tan 30^\circ \approx 1.73$; as $\phi \to 0$ the axle lines become parallel and the ICR escapes to infinity (straight motion).

2.2 Writing the front constraint with rear-wheel coordinates

Lemma 2.1 (Front-wheel constraint)**.** *Constraint (F) is equivalent to*

$$\sin(\theta+\phi)\,\dot{x} - \cos(\theta+\phi)\,\dot{y} - \ell\cos\phi\,\dot{\theta} = 0.$$

Proof. The front contact point is $x_f = x + \ell\cos\theta$, $y_f = y + \ell\sin\theta$, hence

$$\dot{x}_f = \dot{x} - \ell\sin\theta\,\dot{\theta}, \qquad \dot{y}_f = \dot{y} + \ell\cos\theta\,\dot{\theta}.$$

Substituting in (F):

$$\sin(\theta+\phi)\dot{x} - \cos(\theta+\phi)\dot{y} - \ell\dot{\theta}\big[\sin(\theta+\phi)\sin\theta + \cos(\theta+\phi)\cos\theta\big] = 0,$$

and the bracket equals $\cos\big((\theta+\phi)-\theta\big) = \cos\phi$. □

Proposition 2.2 (Constraint matrix)**.** *With $k = 2$ constraints, $A(q)\dot{q} = 0$ where*

$$A(q) = \begin{pmatrix} \sin\theta & -\cos\theta & 0 & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & -\ell\cos\phi & 0 \end{pmatrix}.$$

Hence $\dim(A(q)) = n - k = 2$ *and two linearly independent vector fields are needed.*

2.3 Rear-wheel drive (RWD) kinematic model

Proposition 2.3 (RWD bicycle)**.** *A basis of* $(A(q))$ *is*

$$g_1(q) = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \frac{1}{\ell}\tan\phi \\ 0 \end{pmatrix}, \qquad g_2(q) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix},$$

so that

$$\dot{q} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{pmatrix} = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \frac{1}{\ell}\tan\phi \\ 0 \end{pmatrix} u_1 + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u_2, \qquad u_1 = \pm\sqrt{\dot{x}^2 + \dot{y}^2}, \quad u_2 = \dot{\phi}.$$

Proof. $g_2 \in (A)$ trivially, because the last column of A is zero. For g_1 take $(\cos\theta, \sin\theta, \alpha, 0)^T$; the first row gives $\sin\theta\cos\theta - \cos\theta\sin\theta = 0$ for any α, and the second gives

$$\sin(\theta+\phi)\cos\theta - \cos(\theta+\phi)\sin\theta - \ell\cos\phi\,\alpha = \sin\phi - \ell\cos\phi\,\alpha = 0 \implies \alpha = \tfrac{1}{\ell}\tan\phi.$$

Finally $\dot{x}^2 + \dot{y}^2 = (\cos^2\theta + \sin^2\theta)u_1^2 = u_1^2$, so u_1 is the driving speed of the rear wheel, and $u_2 = \dot{\phi}$ is the steering velocity. □

Intuition

Think of the rear wheel as the "engine" and the front wheel as just a rudder. The rear point moves along the heading with speed u_1 (first two components of g_1), while the heading changes at the rate $\dot{\theta} = \frac{u_1}{\ell}\tan\phi = u_1/\rho$: exactly the angular rate of a point travelling at speed u_1 on a circle of radius $\rho = \ell/\tan\phi$. The steering vector field g_2 only turns the handlebar. Numbers: $\ell = 1$, $\phi = 30°$, $u_1 = 1$ gives $\dot{\theta} \approx 0.58$ rad/s.

Validity and singularity. The model is good *if the bicycle is rear-wheel drive.* At $\phi = \pi/2$ the vector field g_1 "blows up" ($\tan\phi \to \infty$): a singularity. Physically, with the front wheel perpendicular to the body, a nonzero rear speed $v \neq 0$ would require the front wheel to *slip* sideways, which the pure-rolling hypothesis excludes.

3 The front-wheel drive case

For a front-wheel-drive (FWD) vehicle the natural control is the speed of the front wheel, $u_1 = \pm\sqrt{\dot{x}_f^2 + \dot{y}_f^2}$.

Proposition 3.1 (FWD bicycle). *Rescaling $g_1 \mapsto g_1 \cos\phi$ and keeping g_2,*

$$\dot{q} = \begin{pmatrix} \cos\theta\cos\phi \\ \sin\theta\cos\phi \\ \frac{1}{\ell}\sin\phi \\ 0 \end{pmatrix} u_1 + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u_2, \qquad \dot{x}_f^2 + \dot{y}_f^2 = u_1^2, \quad u_2 = \dot{\phi}.$$

Proof. From $\dot{x}_f = \dot{x} - \ell\sin\theta\,\dot{\theta}$ and $\dot{y}_f = \dot{y} + \ell\cos\theta\,\dot{\theta}$ we get

$$\dot{x}_f = u_1(\cos\theta\cos\phi - \sin\theta\sin\phi) = u_1\cos(\theta+\phi), \qquad \dot{y}_f = u_1\sin(\theta+\phi),$$

so $\dot{x}_f^2 + \dot{y}_f^2 = u_1^2$. Since $\cos\phi\, g_1 \in (A)$ and $g_2 \in (A)$, this is still a basis of the null space wherever $\cos\phi \neq 0$, and the rescaled field is also valid (smooth, nonzero) at $\phi = \pi/2$. □

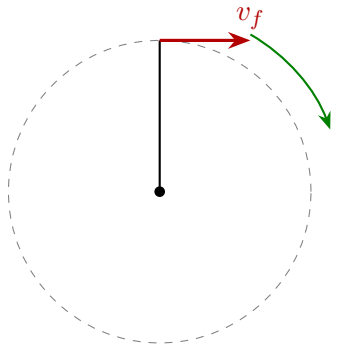


Figure 47: FWD bicycle at $\phi = \pi/2$: the front wheel pushes the vehicle sideways and it pivots about the rear-wheel contact point.

Intuition

If you *drive* the front wheel you can steer it all the way to $\phi = \pi/2$ and still move: the front wheel rolls in its own direction, perpendicular to the body, and the body simply rotates about the rear contact point (Fig. 47) with $\dot{\theta} = u_1/\ell$. So you can move in circle around the rear point *without hitting any singularity*. In the RWD model the same configuration would require infinite $\dot{\theta}$ per unit of rear speed.

4 Controllability

4.1 Is there a universal manoeuvre?

Algorithm 4.1 (Universal manoeuvre for the bicycle). To go from q_i to any q_f:

(1) use the ICR: with the wheel steered, drive along a circle until the heading points towards the goal;

(2) steer the wheel back to straight;

(3) go straight along a line, then finish with another circular arc to match the final heading and steering.

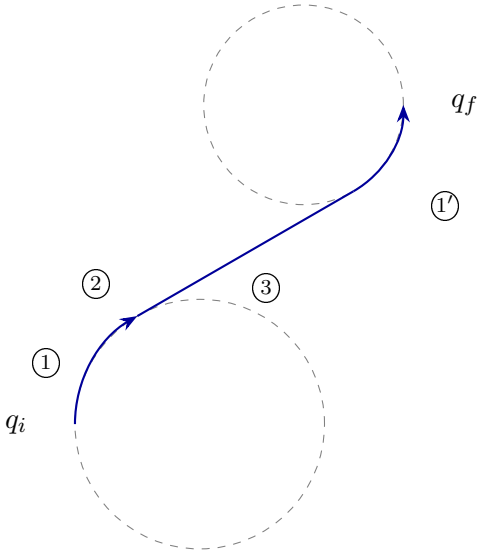


Figure 48: Schematic universal manoeuvre (arc around an ICR, straight line, arc). The numbers refer to the steps of the algorithm; $1'$ is the final arc that matches the target heading.

Intuition

It is the same idea as parking: any pose can be reached by combining only two primitives, rotating about an ICR and driving straight. Hence, although the bicycle has only two inputs for four configuration variables, *it can reach every configuration.* This is possible precisely because the velocity constraints are *non-integrable* (nonholonomic): if they were integrable, the robot would be confined to a lower-dimensional surface of $\mathcal{C}$ and could not reach everything.

4.2 Nonlinear controllability

We study driftless nonlinear systems

$$\dot{x} = \sum_{i=1}^{m} g_i(x) u_i, \qquad x \in \mathbb{R}^n,\ u \in \mathbb{R}^m,$$

("driftless": no term independent of u; for us x is the configuration q).

Definition 4.2 (Controllability)**.** The system is *controllable* if, for any pair (x_i, x_f), there exist $T > 0$ and an input $u(t)$, $t \in [0, T]$, such that the trajectory starting at $x(0) = x_i$ satisfies $x(T) = x_f$.

5 The Lie bracket

Definition 5.1 (Lie bracket)**.** Let g_1, g_2 be two vector fields on $\mathbb{R}^n$. The vector field

$$g_1 g_2(x) = \frac{\partial g_2}{\partial x}\, g_1(x) - \frac{\partial g_1}{\partial x}\, g_2(x)$$

(where $\partial g_i / \partial x$ is the Jacobian) is the *Lie bracket* (LB) of g_1, g_2.

Immediate properties: $g_1 g_2 = -g_2 g_1$ (hence $gg = 0$), and $g_1 g_2 = 0$ for constant fields.

Example 5.2 (A bracket in $\mathbb{R}^3$)**.** Let $g_1 = (\cos x_3, \sin x_3, 0)^T$ and $g_2 = (0, 0, x_1)^T$. Then

$$g_1 g_2(x) = \underbrace{\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}}_{\partial g_2/\partial x} \begin{pmatrix} \cos x_3 \\ \sin x_3 \\ 0 \end{pmatrix} - \underbrace{\begin{pmatrix} 0 & 0 & -\sin x_3 \\ 0 & 0 & \cos x_3 \\ 0 & 0 & 0 \end{pmatrix}}_{\partial g_1/\partial x} \begin{pmatrix} 0 \\ 0 \\ x_1 \end{pmatrix}$$
$$= \begin{pmatrix} 0 \\ 0 \\ \cos x_3 \end{pmatrix} - \begin{pmatrix} -x_1 \sin x_3 \\ x_1 \cos x_3 \\ 0 \end{pmatrix} = \begin{pmatrix} x_1 \sin x_3 \\ -x_1 \cos x_3 \\ \cos x_3 \end{pmatrix}.$$

This is a *new* vector field which, in this case, is linearly independent of g_1, g_2 (the determinant of $[g_1\ g_2\ g_1 g_2]$ is x_1^2, nonzero for $x_1 \neq 0$).

Intuition

The bracket measures *how much the flows of two vector fields fail to commute*: moving first along g_1 then g_2 ends in a different place than g_2 then g_1. The difference of the two endpoints is (to second order) the bracket. If the bracket points in a direction that g_1, g_2 cannot generate directly, then "wiggling" between the two fields gives access to new directions of motion, which is the key to controllability.

6 Interpretation of the Lie bracket

Consider the 2-input system $\dot{x} = g_1(x)u_1 + g_2(x)u_2$ and the piecewise-constant input of duration 4ε.

Algorithm 6.1 (Lie-bracket manoeuvre)**.**

$$(u_1, u_2) = \begin{cases} (1, 0) & t \in [0, \varepsilon) \\ (0, 1) & t \in [\varepsilon, 2\varepsilon) \\ (-1, 0) & t \in [2\varepsilon, 3\varepsilon) \\ (0, -1) & t \in [3\varepsilon, 4\varepsilon). \end{cases}$$

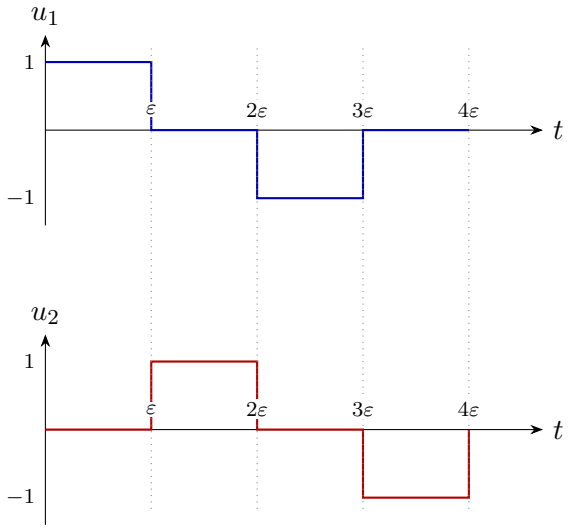


Figure 49: Control sequence of Algorithm 6.1: forward g_1, forward g_2, backward g_1, backward g_2.

Proposition 6.2 (Displacement of the LB manoeuvre)**.** *Starting from $x(0) = x_0$, the state after the sequence of Algorithm 6.1 is*

$$x(4\varepsilon) = x(0) + \varepsilon^2\, g_1 g_2(x_0) + O(\varepsilon^3),$$

where the remainder goes to zero as ε^3 or faster when $\varepsilon \to 0$.

Proof. Write $a = g_1(x_0)$, $b = g_2(x_0)$, $A = \partial g_1/\partial x(x_0)$, $B = \partial g_2/\partial x(x_0)$. The flow of a field g for time t is $x + t\,g(x) + \frac{t^2}{2}\frac{\partial g}{\partial x}g(x) + O(t^3)$. Applying the four phases in turn, up to terms $O(\varepsilon^2)$:

$$\begin{aligned} x_1 &= x_0 + \varepsilon a + \tfrac{\varepsilon^2}{2} Aa, \\ x_2 &= x_1 + \varepsilon g_2(x_1) + \tfrac{\varepsilon^2}{2} Bb = x_0 + \varepsilon(a+b) + \varepsilon^2\big(\tfrac{1}{2}Aa + Ba + \tfrac{1}{2}Bb\big), \\ x_3 &= x_2 - \varepsilon g_1(x_2) + \tfrac{\varepsilon^2}{2} Aa = x_0 + \varepsilon b + \varepsilon^2\big(Ba + \tfrac{1}{2}Bb - Ab\big), \end{aligned}$$

using $g_1(x_2) = a + \varepsilon A(a+b) + O(\varepsilon^2)$. Finally $g_2(x_3) = b + \varepsilon Bb + O(\varepsilon^2)$ and

$$x_4 = x_3 - \varepsilon g_2(x_3) + \tfrac{\varepsilon^2}{2} Bb = x_0 + \varepsilon^2(Ba - Ab) + O(\varepsilon^3) = x_0 + \varepsilon^2 g_1 g_2(x_0) + O(\varepsilon^3). \qquad \square$$

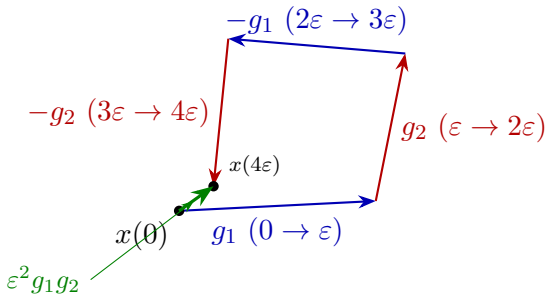


Figure 50: The four legs of the manoeuvre almost close a loop; the gap is the displacement $\varepsilon^2 g_1 g_2(x_0)$ (schematic, exaggerated).

Intuition

Going forward-sideways-backward-sideways "almost" returns to the start, like the parking manoeuvre. The residual gap is second order in the time, ε^2, and is the Lie bracket. Unicycle check ($\theta = 0$): go forward ε, rotate by $+\varepsilon$, go backward ε along the rotated heading, rotate back. The net motion in y is $-\varepsilon \sin\varepsilon \approx -\varepsilon^2$, i.e. a sideways shift, exactly $\varepsilon^2 g_1 g_2 = \varepsilon^2(\sin\theta, -\cos\theta, 0) = \varepsilon^2(0, -1, 0)$, even though the unicycle cannot move sideways directly. Numbers: with $\varepsilon = 0.1$ the path costs $4\varepsilon = 0.4$ time units but moves only $\varepsilon^2 = 0.01$ sideways: bracket motions are cheap in inputs but slow.

7 Distributions, accessibility and Chow's theorem

Definition 7.1 (Input distribution)**.** For $\dot{x} = \sum_{i=1}^{m} g_i(x)u_i$ the *input distribution* $= \{g_1(x), \ldots, g_m(x)\}$ associates to each x the subspace $\{g_1(x), \ldots, g_m(x)\}$ (all directions in which the inputs can push directly).

Definition 7.2 (Accessibility distribution)**.** The *accessibility distribution* is the distribution

generated by the input fields and all their iterated brackets:

$$= \Big\{ g_1, \dots, g_m,\ \dots, g_i g_j,\ \dots,\ g_i g_j g_k,\ \dots \Big\},$$

with all 1st-order brackets, all 2nd-order brackets, and so on.

Theorem 7.3 (Chow). *Consider the driftless system $\dot{x} = \sum_i g_i(x) u_i$ on $\mathbb{R}^n$ and assume $\dim(x)$ is constant for all x. Then the system is controllable if and only if*

$$\dim = n.$$

This result is known as *Chow's theorem*, the *Lie algebra rank condition* or the *accessibility rank condition*. In practice, for low-order brackets one computes

$$\begin{pmatrix} g_1 & g_m & \cdots & g_i g_j & \cdots & g_i g_j g_k & \cdots \end{pmatrix} = n,$$

adding brackets of increasing order until the rank reaches n (or no new independent direction appears).

Intuition

The inputs give you m directions for free. Each bracket manoeuvre buys you one more direction (Prop. 6.2), brackets of brackets buy yet more. If after collecting all of them the directions span the whole state space, there is no direction in which you are stuck, so you can steer anywhere. Analogy: you can only drive a car forward/backward and turn, yet by combining these ($\to$ parallel parking) you effectively obtain a "slide sideways" control.

8 Application to the unicycle

Example 8.1 (Controllability of the unicycle). With $q = (x, y, \theta)^T$, $g_1 = (\cos\theta, \sin\theta, 0)^T$ (drive) and $g_2 = (0, 0, 1)^T$ (steer):

$$g_1 g_2(q) = \underbrace{0}_{\partial g_2/\partial q = 0} \cdot g_1 - \begin{pmatrix} 0 & 0 & -\sin\theta \\ 0 & 0 & \cos\theta \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \sin\theta \\ -\cos\theta \\ 0 \end{pmatrix} =: g_3(q),$$

and

$$\begin{pmatrix} \cos\theta & 0 & \sin\theta \\ \sin\theta & 0 & -\cos\theta \\ 0 & 1 & 0 \end{pmatrix} = 3 = n.$$

(The determinant is 1 for all θ.)

Corollary 8.2. *The unicycle is controllable by Theorem 7.3.*

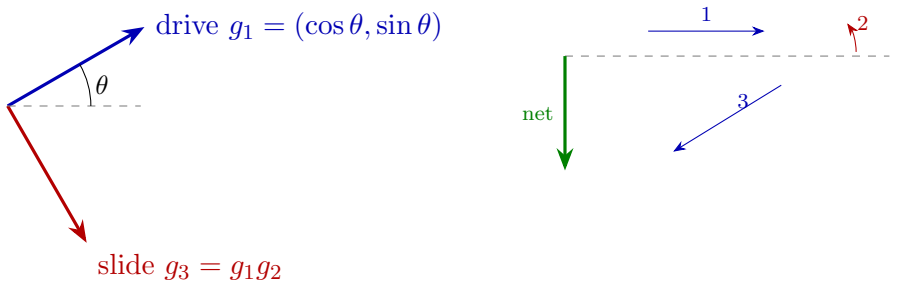


Figure 51: Left: drive and slide vectors of the unicycle. Right: LB manoeuvre (1 forward, 2 rotate, 3 backward, then rotate back): the net displacement is sideways (angles exaggerated).

Intuition

The new vector field $g_3 = g_1 g_2$ points perpendicular to the heading: it is the *slide* that the wheel forbids. Together g_1 (drive), g_2 (steer) and g_3 (slide) span all of $\mathbb{R}^3$ at every configuration, so the unicycle can reach every (x, y, θ).

9 Application to the bicycle

Take the RWD bicycle ($n = 4$)

$$\dot{q} = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \tan\phi/\ell \\ 0 \end{pmatrix} v + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \omega.$$

The same computation applies to the FWD model. Since $n = 4$ and only two inputs are available, we need the brackets up to the second order.

Proposition 9.1 (Brackets of the bicycle)**.** *With g_1, g_2 as above,*

$$g_1 g_2 = -\frac{\partial g_1}{\partial \phi} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ -\dfrac{1}{\ell \cos^2 \phi} \\ 0 \end{pmatrix},$$

$$g_1 g_1 g_2 = \begin{pmatrix} -\dfrac{\sin\theta}{\ell \cos^2 \phi} \\ \dfrac{\cos\theta}{\ell \cos^2 \phi} \\ 0 \\ 0 \end{pmatrix}.$$

Proof. Only g_1 depends on ϕ, and $g_2 = \partial/\partial\phi$ is constant, so $g_1 g_2 = -\frac{\partial g_1}{\partial q} g_2 = -\frac{\partial g_1}{\partial \phi} = -(0, 0, \frac{1}{\ell \cos^2 \phi}, 0)^T$. Let $w = g_1 g_2 = (0, 0, -\sigma, 0)^T$ with $\sigma = 1/(\ell \cos^2 \phi)$. Then $\frac{\partial w}{\partial q} g_1 = 0$ (since w depends only on ϕ and g_1 has no ϕ-component) and $\frac{\partial g_1}{\partial q} w = -\sigma \frac{\partial g_1}{\partial \theta} = -\sigma(-\sin\theta, \cos\theta, 0, 0)^T$. Hence $g_1 w = -\frac{\partial g_1}{\partial q} w = \sigma(-\sin\theta, \cos\theta, 0, 0)^T$. □

With the bracket convention above, the first-order bracket has a negative sign; the second-order bracket is then exactly as stated. The sign does not affect any rank computation.

Theorem 9.2. *The bicycle is controllable: for $\phi \neq \pm\pi/2$,*

$$\big(g_1,\ g_2,\ g_1 g_2,\ g_1 g_1 g_2\big) = 4 = n.$$

Proof. Order the coordinates (x, y, θ, ϕ). The matrix has g_2 as the only vector with a ϕ-component, $w = g_1 g_2$ with a θ-component $-\sigma \neq 0$ that also appears in g_1, and the last vector spans the (x, y)-plane orthogonally to the heading. Direct expansion (along the ϕ-row, then the column of w) gives $\det = \sigma^2 = \frac{1}{\ell^2 \cos^4 \phi} \neq 0$. □

The determinant of the matrix with the columns in the order above is σ^2, nonzero wherever the model is defined, which is all that matters. For the FWD model, one checks that the same rank-4 test holds, also at $\phi = \pi/2$, where the second-order bracket is $\frac{1}{\ell}(-\sin\theta, \cos\theta, 0, 0)^T$.

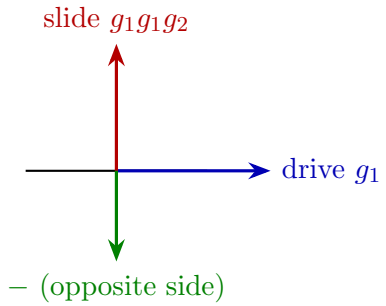


Figure 52: Bicycle at $\theta = 0$: the second-order bracket $g_1 g_1 g_2 \propto (-\sin\theta, \cos\theta, 0, 0)$ is the *sliding* direction, perpendicular to the heading.

Intuition

The picture is the same as for the unicycle but one level deeper: *steering* the wheel changes the *turning rate* (first bracket, $\partial_\phi \dot{\theta}$ direction: a pure rotation of the body), and driving while rotating then produces a net *sideways slide* (second-order bracket, with magnitude growing as $1/\cos^2\phi$). This is the car's parallel-parking motion. Four independent directions (drive, steer, rotate, slide) cover the four-dimensional state space.

	Unicycle	**Bicycle RWD**	**Bicycle FWD**
State q (n)	(x, y, θ) (3)	(x, y, θ, ϕ) (4)	(x, y, θ, ϕ) (4)
Constraints k / inputs m	1 / 2	2 / 2	2 / 2
Drive field g_1	$(\cos\theta, \sin\theta, 0)$	$(\cos\theta, \sin\theta, \frac{\tan\phi}{\ell}, 0)$	$(\cos\theta\cos\phi, \sin\theta\cos\phi, \frac{\sin\phi}{\ell}, 0)$
Steer field g_2	$(0, 0, 1)$	$(0, 0, 0, 1)$	$(0, 0, 0, 1)$
Drive input u_1	body speed v	rear-wheel speed	front-wheel speed
Singularity	none	$\phi = \pm\pi/2$	none
Brackets needed	$g_1 g_2$	up to $g_1 g_1 g_2$	same
Rank / controllable	$3 = n$, yes	$4 = n$, yes	$4 = n$, yes

Table 6: Kinematic models and their controllability.

10 Summary

10.1 Comparison of the models

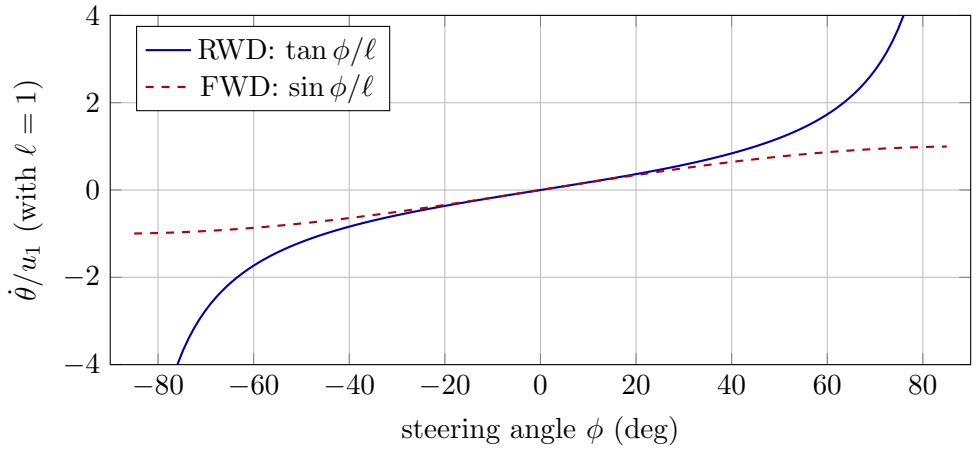


Figure 53: Rotation rate per unit of drive speed: it blows up at $\phi = \pm\pi/2$ for the rear-wheel-drive model but stays bounded for the front-wheel-drive one.

10.2 Key take-aways

- The bicycle has $n = 4$ coordinates and 2 independent pure-rolling constraints: the null space has dimension 2, giving the drive and steer vector fields. RWD uses $g_1 = (\cos\theta, \sin\theta, \tan\phi/\ell, 0)$ and is singular at $\phi = \pi/2$; FWD uses $g_1 \cos\phi$ and has no singularity.
- Controllability of driftless nonlinear systems is about *directions*: the Lie bracket $g_1 g_2 = \frac{\partial g_2}{\partial x} g_1 - \frac{\partial g_1}{\partial x} g_2$ gives new directions, and the four-leg manoeuvre displaces the state by $\varepsilon^2 g_1 g_2 + O(\varepsilon^3)$.
- Chow's theorem (Lie algebra / accessibility rank condition): the system is controllable iff the accessibility distribution generated by g_i and all iterated brackets has dimension n.
- Unicycle: $g_1 g_2 = (\sin\theta, -\cos\theta, 0)$ is the forbidden sliding direction, so rank 3 and it is controllable. Bicycle: $g_1 g_2$ and $g_1 g_1 g_2$ supply the missing rotation and slide, rank 4, hence controllable.

- Controllability with fewer inputs than states is equivalent to the kinematic constraints being non-integrable, i.e. nonholonomic.

Part VII

Lecture 5

1 Vehicles Kinematically Equivalent to the Bicycle

Several vehicles that look different share the same kinematic model as the bicycle: the *tricycle* (front-wheel or rear-wheel drive) and the *car-like* robot. In all of them the pair of front (steering) wheels can be replaced by a single *virtual* wheel placed at the midpoint between the two wheels, and the configuration is

$$= \begin{pmatrix} x \\ y \\ \theta \\ \phi \end{pmatrix},$$

where (x, y) is the position of the midpoint of the rear axle, θ the orientation of the body and ϕ the steering angle. A *differential-drive* robot has instead the same model as the unicycle, with the reference point centred on the axis of the driving wheels.

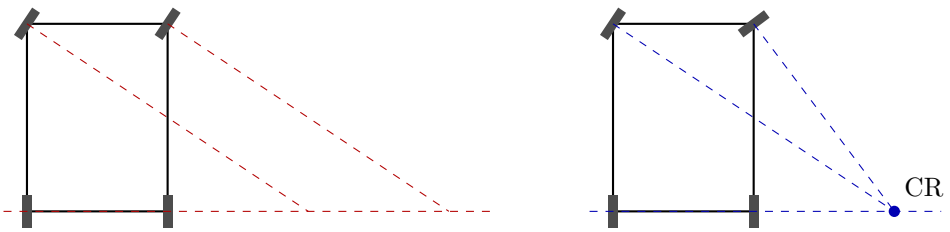
Intuition

Replacing two wheels by one is legitimate because of the *instantaneous centre of rotation*: at any instant a rigid body moves as if rotating about a single point (the CR), and pure rolling forces the motion of every wheel to be perpendicular to the line joining the wheel to that point. Whatever happens on the left and the right is therefore described by one "average" wheel on the vehicle's axis of symmetry.

1.1 Ackermann steering

Consider a car-like robot whose two front wheels are steered. Each wheel, rolling without slipping, cannot move along its axle, so its *zero-motion line* is the line containing the axle. For the vehicle to move, all zero-motion lines must meet at one common point, the CR.

If the two front wheels were steered by the *same* angle, they would be parallel: the two zero-motion lines would also be parallel, no common point could exist together with the rear-axle line, and the car would necessarily *slip*. The cure is the **Ackermann steering geometry**: the external front wheel is steered slightly *less* than the internal one, so that the three axle lines meet in a single CR lying on the rear axle line. In practice this is obtained mechanically with a four-bar linkage (the steering trapezoid).



(a) parallel wheels: no common point (b) Ackermann: single CR

Figure 54: Zero-motion lines of the wheels (dashed) of a car turning right. With parallel front wheels (a) they never meet in one point; with Ackermann steering (b) the outer (left) wheel is turned less than the inner one and everything converges in one CR.

Intuition

Think of the car going around a bend: the inner front wheel travels on a smaller circle than the outer one, so it must be turned *more*. If both were turned equally, one of them would be fighting the other, dragging sideways on the road, and that is exactly the slipping that the kinematic model forbids.

2 Chained Forms

The kinematic models of many wheeled robots can be brought into a (rather) *canonical* form, the chained form.

Definition 2.1 ($(2,n)$ chained form)**.** A driftless system with two inputs and n state variables is in $(2,n)$ *chained form* if (the first number is the number of inputs, the second the number of coordinates)

$$\begin{aligned}\dot{z}_1 &= w_1,\\ \dot{z}_2 &= w_2,\\ \dot{z}_3 &= z_2\, w_1,\\ &\vdots\\ \dot{z}_n &= z_{n-1}\, w_1,\end{aligned}$$

or, in vector form,

$$\dot{} = \begin{pmatrix}1\\0\\z_2\\\vdots\\z_{n-1}\end{pmatrix} w_1 + \begin{pmatrix}0\\1\\0\\\vdots\\0\end{pmatrix} w_2 = \gamma_1() \, w_1 + \gamma_2 \, w_2.$$

The structure is a *cascade of integrators*: w_1 is integrated to give z_1; w_2 is integrated to give z_2; and every following variable is obtained by multiplying the previous one by w_1 and integrating.

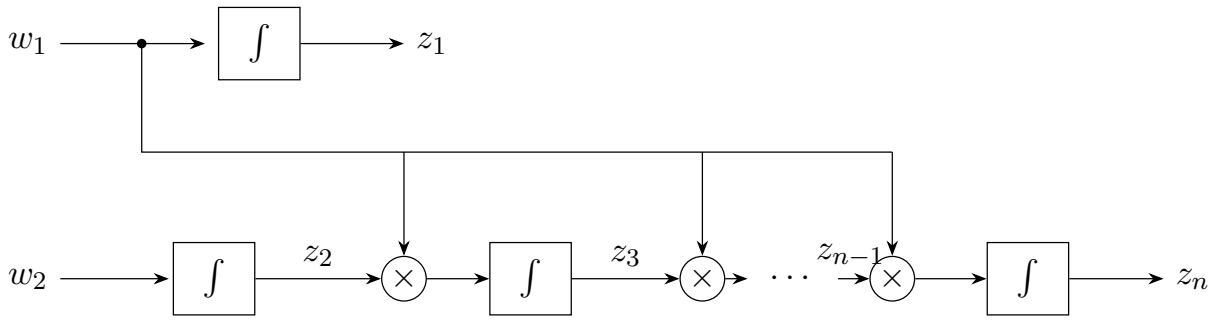


Figure 55: Block diagram of a $(2,n)$ chained form: the signal w_1 is distributed to all the multipliers.

2.1 Controllability

Proposition 2.2. *Every $(2, n)$ chained form is controllable: the accessibility distribution has dimension $\dim \Delta_A = n$ at every .*

Proof. With $\gamma_2 = e_2$ constant, the Lie bracket is $[\gamma_1, \gamma_2] = \frac{\partial \gamma_2}{\partial} \gamma_1 - \frac{\partial \gamma_1}{\partial} \gamma_2 = -\frac{\partial \gamma_1}{\partial} e_2 = -e_3$, because the only non-zero derivative of γ_1 with respect to z_2 is that of its third entry. Repeating the same argument,

$$\overset{k}{\gamma_1} \gamma_2 = (-1)^k e_{k+2}, \qquad k = 0, \ldots, n-2.$$

Hence $\gamma_1, \gamma_2, \overset{}{\gamma_1} \gamma_2, \ldots, \overset{n-2}{\gamma_1} \gamma_2$ are n vectors that span $\mathbb{R}^n$ (from $e_2, \ldots, e_n$ one gets e_1 out of γ_1), i.e. $\dim \Delta_A = n$. By Chow's theorem the system is controllable. □

Intuition

The two inputs directly move only z_1 and z_2. But w_1 "feeds" z_3 through z_2, z_3 feeds z_4, and so on: the effect of w_2 propagates *down the chain* through the interplay with w_1. Each extra Lie bracket (an infinitesimal "wiggle": move along γ_1, then γ_2, then back) opens up exactly one new direction, and there are just enough of them to reach all n coordinates.

2.2 Why chained forms are useful

Proposition 2.3 (Linearity under constant input)**.** *If $w_1(t) \equiv c$ is constant, the chained form becomes a* linear *time-invariant system in $(z_2, \ldots, z_n)$ with input w_2 (and $z_1 = ct + z_1(0)$). If w_1 is piecewise constant, the system is piecewise linear.*

Proof. For $w_1 = c$ the equations read $\dot{z}_1 = c$, $\dot{z}_2 = w_2$, $\dot{z}_i = c\, z_{i-1}$, $i = 3, \ldots, n$: all right-hand sides are linear in $(z_2, \ldots, z_n, w_2)$. □

Intuition

The nonlinearity of the chained form is entirely due to the *product* of a state with the input w_1. Freeze w_1 and the product becomes a plain gain. This is very handy in digital control, where inputs are held constant between sampling instants: within each sampling interval the system is linear, and one can use all of linear-systems theory.

Proposition 2.4 (Transformability, fact)**.** *All controllable driftless systems with $n \leq 4$ and 2 inputs can be put in chained form (under standard regularity assumptions). This means there exist*

- *a* coordinate transformation $= \alpha()$ *from the original coordinates to the chained-form coordinates, and*
- *an* input transformation $= \beta()$ *, with $\beta()$ invertible (so that are the original, linearly independent, inputs).*

Intuition

The chained form is a change of "language": the robot is the same, but written in coordinates and with inputs that make its structure transparent. The transformations are built so that the system after the change is exactly a cascade of integrators.

3 Chained Form of the Unicycle

The unicycle kinematics

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega$$

is a $(2,3)$ system: it should be mapped to $\dot{z}_1 = w_1$, $\dot{z}_2 = w_2$, $\dot{z}_3 = z_2 w_1$.

Proposition 3.1. *The coordinate and input transformations*

$$\begin{aligned} z_1 &= \theta, & w_1 &= \omega, \\ z_2 &= x\cos\theta + y\sin\theta, & w_2 &= v - z_3\,\omega, \\ z_3 &= x\sin\theta - y\cos\theta, \end{aligned}$$

bring the unicycle into $(2,3)$ *chained form.*

Proof. Direct check. First $\dot{z}_1 = \dot{\theta} = \omega = w_1$. Then

$$\begin{aligned} \dot{z}_2 &= \dot{x}\cos\theta - x\sin\theta\,\dot{\theta} + \dot{y}\sin\theta + y\cos\theta\,\dot{\theta} \\ &= v(\cos^2\theta + \sin^2\theta) - (x\sin\theta - y\cos\theta)\dot{\theta} = v - z_3\,\omega = w_2, \\ \dot{z}_3 &= \dot{x}\sin\theta + x\cos\theta\,\dot{\theta} - \dot{y}\cos\theta + y\sin\theta\,\dot{\theta} \\ &= v(\cos\theta\sin\theta - \sin\theta\cos\theta) + (x\cos\theta + y\sin\theta)\dot{\theta} = z_2\,\omega = z_2\,w_1. \end{aligned}$$

The map is invertible: $x = z_2\cos z_1 + z_3\sin z_1$, $y = z_2\sin z_1 - z_3\cos z_1$. □

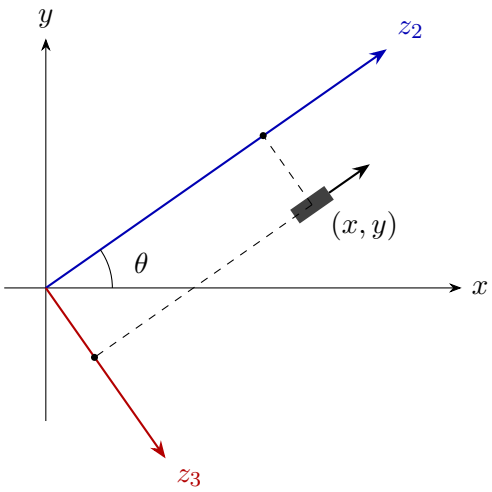


Figure 56: Meaning of the chained coordinates of the unicycle: z_2 and z_3 are the components of the position (x,y) along the axis z_2 (the direction of heading θ) and along the perpendicular axis z_3 obtained by rotating it by $-90°$; $z_1 = \theta$.

Intuition

The new coordinates are simply the robot's position *expressed in a frame rotated by the heading angle*. Rotating the robot ($w_1 = \omega$) changes the frame, and thereby changes z_2 and z_3: the position in the rotating frame changes even if the robot does not translate, which is the origin of the product term $z_2\,w_1$. The input $w_2 = v - z_3\omega$ is the forward speed corrected by this frame-rotation effect.

4 Chained Form of the Bicycle

The bicycle (equivalently the car-like robot) has $n = 4$ coordinates and is therefore a $(2, 4)$ system:

$$\dot{z}_1 = w_1, \quad \dot{z}_2 = w_2, \quad \dot{z}_3 = z_2 w_1, \quad \dot{z}_4 = z_3 w_1.$$

Proposition 4.1. *For the rear-wheel drive car with model $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \frac{v}{\ell}\tan\phi$, $\dot{\phi} = \omega$, in the region $|\theta| < \pi/2$ the transformation*

$$z_1 = x, \quad z_2 = \frac{\tan\phi}{\ell\cos^3\theta}, \quad z_3 = \tan\theta, \quad z_4 = y, \qquad w_1 = v\cos\theta, \quad w_2 = \dot{z}_2$$

gives the $(2, 4)$ chained form.

Proof. $\dot{z}_1 = v\cos\theta = w_1$ and $\dot{z}_2 = w_2$ by definition (explicitly, $w_2 = \dfrac{\omega}{\ell\cos^3\theta\cos^2\phi} + \dfrac{3\sin\theta\tan^2\phi}{\ell^2\cos^4\theta}\, v$, invertible with respect to ω). Next

$$\dot{z}_3 = \frac{\dot{\theta}}{\cos^2\theta} = \frac{v\tan\phi}{\ell\cos^2\theta} = v\cos\theta\cdot\frac{\tan\phi}{\ell\cos^3\theta} = z_2 w_1, \qquad \dot{z}_4 = v\sin\theta = v\cos\theta\tan\theta = z_3 w_1. \quad \square$$

Intuition

Here z_1 and z_4 are the Cartesian coordinates, z_3 is (the tangent of) the heading, and z_2 is (a scaled version of) the curvature of the path. The chain reads: speed along x drives the slope of the path; the slope drives y; and the steering rate changes the curvature. It is the familiar picture of "curvature → heading → position" written as a cascade.

5 Adding Trailers

Consider adding one (or more) **trailers** to the unicycle or to the bicycle. The new vehicle is *still transformable to chained form*, as long as each trailer is *hinged at the midpoint between the two wheels* of the preceding body (the axle midpoint of the car or of the previous trailer).

Intuition

Hitching at the axle midpoint keeps the geometry "rolling-friendly": each trailer wheel is again the point of a body that can only move along its own heading direction, exactly like the rear wheel of the car. The new body therefore adds one new coordinate (its angle θ_t) and one new link in the chain, rather than breaking the structure. A fully worked example (a tricycle with a trailer, with its constraints and kinematic model) is given in Section 6.

6 Class Test 2009 – Problem 1: Tricycle with a Trailer

Example 6.1 (Problem 1)**.** Consider the mobile robot obtained by attaching a trailer to a rear-wheel drive tricycle. The trailer is a rigid body with an axle carrying two fixed wheels, and is connected to the midpoint of the tricycle rear axle through a revolute joint. Both two-wheel axles can be assimilated to a single wheel located at the midpoint of the axle.

1. Find a set of generalized coordinates for the robot, and clearly indicate them on the drawing.

2. Write the Pfaffian kinematic constraints to which the robot is subject.
3. Derive a kinematic model of the system.

6.1 Generalized coordinates

A convenient choice is

$$= (x \;\; y \;\; \theta \;\; \phi \;\; \theta_t)^T,$$

i.e. the generalized coordinates of the tricycle plus one additional coordinate for the trailer. Here (x, y) is the midpoint P of the rear axle, θ the heading of the tricycle, ϕ the steering angle of the front wheel, and θ_t the orientation of the trailer; P_f is the centre of the front wheel (at distance ℓ from P) and P_t the midpoint of the trailer axle (at distance ℓ_t from P). The robot has three wheels: the tricycle front wheel, the tricycle rear wheel and the trailer wheel.

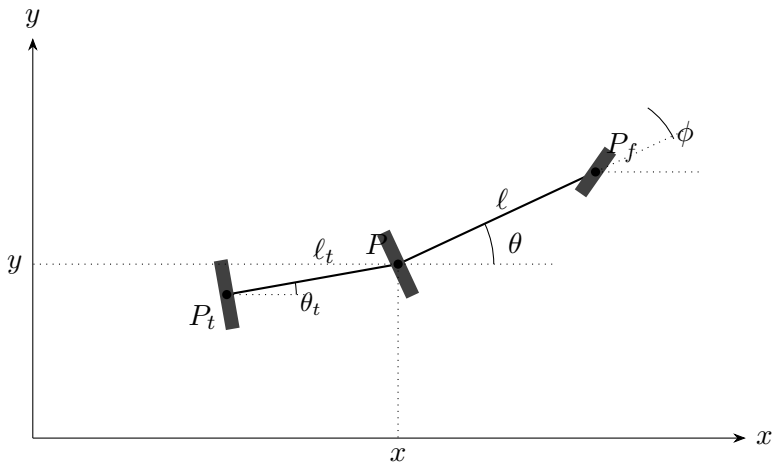


Figure 57: Tricycle with trailer. Generalized coordinates $= (x, y, \theta, \phi, \theta_t)$.

6.2 Pfaffian kinematic constraints

There are three kinematic constraints (one pure-rolling condition per wheel):

$$\begin{aligned} \dot{x}_f \sin(\theta + \phi) - \dot{y}_f \cos(\theta + \phi) &= 0, \\ \dot{x} \sin\theta - \dot{y} \cos\theta &= 0, \\ \dot{x}_t \sin\theta_t - \dot{y}_t \cos\theta_t &= 0, \end{aligned}$$

where (x_f, y_f) and (x_t, y_t) are the Cartesian coordinates of P_f and P_t:

$$x_f = x + \ell\cos\theta, \quad y_f = y + \ell\sin\theta, \qquad x_t = x - \ell_t\cos\theta_t, \quad y_t = y - \ell_t\sin\theta_t.$$

Differentiating and substituting (for instance $\dot{x}_f = \dot{x} - \ell\dot{\theta}\sin\theta$, $\dot{y}_f = \dot{y} + \ell\dot{\theta}\cos\theta$, and the identities $\sin\theta\sin(\theta+\phi) + \cos\theta\cos(\theta+\phi) = \cos\phi$ and $\sin^2\theta_t + \cos^2\theta_t = 1$) one obtains

$$\begin{aligned} \dot{x}\sin(\theta + \phi) - \dot{y}\cos(\theta + \phi) - \dot{\theta}\,\ell\cos\phi &= 0, \\ \dot{x}\sin\theta - \dot{y}\cos\theta &= 0, \\ \dot{x}\sin\theta_t - \dot{y}\cos\theta_t + \ell_t\dot{\theta}_t &= 0, \end{aligned}$$

or, in Pfaffian form $A^T(\,)\dot{} = 0$,

$$\begin{pmatrix} \sin\theta & -\cos\theta & 0 & 0 & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & -\ell\cos\phi & 0 & 0 \\ \sin\theta_t & -\cos\theta_t & 0 & 0 & \ell_t \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \\ \dot{\theta}_t \end{pmatrix} = 0.$$

Intuition

Each wheel contributes one equation of the form "velocity of the wheel centre $\perp$ axle direction" (no lateral slipping). The first two rows (rear and front wheel of the tricycle) are the constraints of the bicycle; the third row is new and says that the trailer wheel is, again, forced to move along the direction of the trailer.

6.3 Kinematic model

Note that the submatrix made of the first two rows and the first four columns of A^T coincides with the constraint matrix of the bicycle. A basis of the null space of A^T can then be written by "completing" the basis found for the bicycle (in particular the rear-wheel drive version, since the tricycle has rear-wheel drive) with a suitable fifth entry:

$$G(\,) = \begin{pmatrix} g_1 & g_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ \tan\phi/\ell & 0 \\ 0 & 1 \\ -\sin(\theta_t-\theta)/\ell_t & 0 \end{pmatrix}.$$

The kinematic control system is

$$\dot{} = g_1(\,)\,v + g_2(\,)\,\omega,$$

where v and ω are respectively the driving velocity and the steering velocity of the tricycle.

Check. The third constraint applied to g_1 gives $\cos\theta\sin\theta_t - \sin\theta\cos\theta_t + \ell_t \cdot \frac{-\sin(\theta_t-\theta)}{\ell_t} = \sin(\theta_t-\theta) - \sin(\theta_t-\theta) = 0$, and the other two rows vanish exactly as in the bicycle.

Intuition

The driving input v moves the tricycle exactly as in the bicycle and, in addition, *pulls* the trailer: $\dot{\theta}_t$ is proportional to $\sin(\theta-\theta_t)$, so the trailer tends to align behind the tricycle when it moves forward (the further it is from alignment, the faster it rotates, and it stops rotating when aligned). The steering input ω only changes ϕ and leaves the others untouched.

A different choice of coordinates. The model depends on the chosen ; a different choice leads to an equivalent model via a change of coordinates. For example with $' = (x\ y\ \theta\ \phi\ \delta)^T$, $\delta = \theta - \theta_t$, one has $\dot{'} = g_1'(')v + g_2'(')\omega$ with

$$G'(') = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ \tan\phi/\ell & 0 \\ 0 & 1 \\ \tan\phi/\ell - \sin\delta/\ell_t & 0 \end{pmatrix},$$

since $\dot{\delta} = \dot{\theta} - \dot{\theta}_t = v\big(\tan\phi/\ell - \sin\delta/\ell_t\big)$.

7 Worked Exercise: A Rolling Wheel on the Plane

Setting. A unicycle (a disk of radius R) moves on a plane. Extend the configuration vector by including the *wheel (rolling) angle* ϕ. We want to: (1) define the configuration space; (2) write the constraints; (3) show controllability; (4) describe a sequential manoeuvre.

7.1 Configuration space and kinematic model

The configuration is

$$= \begin{pmatrix} x \\ y \\ \theta \\ \phi \end{pmatrix} \in \mathbb{R}^2 \times SO(2) \times SO(2), \qquad \dim = 4,$$

(θ is the heading, ϕ the angle of rotation of the wheel about its horizontal axis). The unicycle equations $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$ are completed by $\dot{\phi} = \omega_\phi$, where ω_ϕ is the angular velocity of the wheel about its horizontal axis. Since the wheel rolls, $v = R\,\omega_\phi$, so

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{pmatrix} = \begin{pmatrix} R\cos\theta \\ R\sin\theta \\ 0 \\ 1 \end{pmatrix} \omega_\phi + \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \omega.$$

7.2 Constraints

Two pure-rolling conditions act on the wheel:

$$\begin{aligned} \dot{x}\sin\theta - \dot{y}\cos\theta = 0 \qquad & \text{(rolling laterally: no side slip, along the axle)}, \\ v = R\,\omega_\phi \qquad & \text{(rolling along the sagittal axis)}. \end{aligned}$$

Kinematically, the second one reads $\pm\sqrt{\dot{x}^2 + \dot{y}^2} - R\dot{\phi} = 0$, which is *not* Pfaffian (it is not linear in $\dot{}$).

How to solve this. Use $v = R\dot{\phi}$ and $v(\cos^2\theta + \sin^2\theta) = \dot{x}\cos\theta + \dot{y}\sin\theta$ to obtain

$$\dot{x}\cos\theta + \dot{y}\sin\theta - R\dot{\phi} = 0,$$

which *is* Pfaffian. The constraint matrix is

$$A^T() = \begin{pmatrix} \sin\theta & -\cos\theta & 0 & 0 \\ \cos\theta & \sin\theta & 0 & -R \end{pmatrix},$$

the first row being the lateral rolling condition and the second the rolling condition along the sagittal axis. The null space of $A^T()$ gives the same kinematic model as before, $\dot{} = g_1\omega_\phi + g_2\omega$ with

$$g_1 = \begin{pmatrix} R\cos\theta \\ R\sin\theta \\ 0 \\ 1 \end{pmatrix}, \qquad g_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}.$$

Intuition

The "physical" rolling condition is a statement about *speeds*: the distance covered on the ground equals the radius times the angle turned. A speed contains a square root of the velocity components, which is not linear. The trick is to project the velocity on the heading

direction, $\dot{x}\cos\theta + \dot{y}\sin\theta$, which is the signed forward speed, and which *is* linear in $\dot{}$. This keeps the sign (forward/backward) and brings the problem back to the Pfaffian world.

7.3 Controllability

Proposition 7.1. *The rolling wheel on the plane is controllable (the accessibility distribution has dimension 4 everywhere).*

Proof. Compute the Lie brackets with $[g_i, g_j] = \frac{\partial g_j}{\partial} g_i - \frac{\partial g_i}{\partial} g_j$. Since g_2 is constant,

$$g_3 := [g_1, g_2] = -\frac{\partial g_1}{\partial} g_2 = -\begin{pmatrix} 0 & 0 & -R\sin\theta & 0 \\ 0 & 0 & R\cos\theta & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} R\sin\theta \\ -R\cos\theta \\ 0 \\ 0 \end{pmatrix}.$$

This is linearly independent of g_1, g_2, so already 3 directions are available. Further brackets:

$$[g_1, g_3] = 0, \qquad [g_2, g_3] = \frac{\partial g_3}{\partial \theta} = \begin{pmatrix} R\cos\theta \\ R\sin\theta \\ 0 \\ 0 \end{pmatrix} =: g_4.$$

Finally, the matrix $\begin{pmatrix} g_1 & g_2 & g_3 & g_4 \end{pmatrix}$ has the structure (reordering rows and columns, a block triangular matrix) in which only the 2×2 blocks on the diagonal matter. Its determinant is

$$\det \begin{pmatrix} R\cos\theta & 0 & R\sin\theta & R\cos\theta \\ R\sin\theta & 0 & -R\cos\theta & R\sin\theta \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} = -R^2 \neq 0.$$

So $\{g_1, g_2, g_3, g_4\} = 4$ and by Chow's theorem the system is controllable. □

Intuition

Four coordinates, two inputs. Rolling forward and turning generate the sideways motion g_3 (the parking manoeuvre: roll, turn, roll back, turn back); and turning the wheel's heading while a sideways displacement is present generates a further direction g_4. Crucially, ϕ can only change when the wheel *moves* (rolling), which is why all four directions need the brackets and not only the two inputs.

7.4 A sequential manoeuvre

Algorithm 7.2 (Reaching a desired $(x_g, y_g, \theta_g, \phi_g)$)**.** Let the initial configuration be $(x_s, y_s, \theta_s, \phi_s)$.

1. Steps 1–3 are as for the unicycle: drive the position and heading to the goal values, $x \to x_g$, $y \to y_g$, $\theta \to \theta_g$. At the end of this phase the wheel angle ϕ has some value which, in general, differs from ϕ_g.
2. Perform a *circular motion* that brings ϕ to ϕ_g and leaves the robot in the same position and orientation. The wheel rolls along a circle of radius r once around, so the rolled distance $2\pi r$ equals the arc length corresponding to the required change of the wheel

angle:

$$2\pi r = R\,(\phi_g - \phi_s),$$

where r is the radius of the circle and R is the wheel radius (ϕ_s denotes the wheel angle at the start of this last phase).

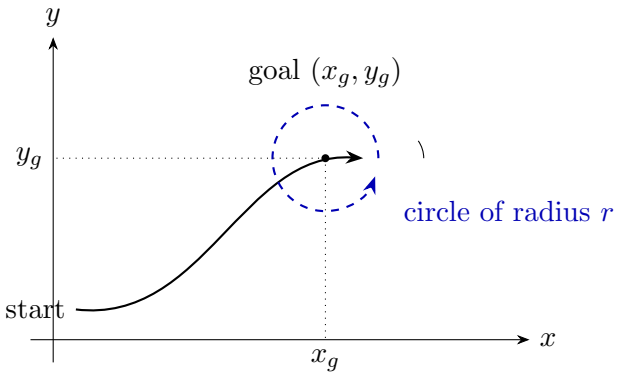


Figure 58: Sequential manoeuvre: first the unicycle-like motion to the goal position and heading (solid), then a circular motion at the goal (dashed) that only changes ϕ.

Intuition

The wheel angle ϕ is only changed by rolling, but rolling also moves the robot. The trick is to roll *in a closed loop*: after one full circle the robot is back at the same position and (modulo 2π) the same heading, while the wheel has turned by an amount proportional to the length of the circle. Choosing r gives exactly the missing wheel rotation. Systematic ways to do this will be seen in the next lecture, when we discuss planning.

8 Summary

System	n	Constraints	Controllable	Chained form
Unicycle / differential drive	3	1	yes	$(2,3)$ via $z_1 = \theta$, $z_2 = x\cos\theta + y\sin\theta$, $z_3 = x\sin\theta - y\cos\theta$
Bicycle, tricycle, car-like (rear-wheel drive)	4	2	yes	$(2,4)$ via $z_1 = x$, $z_2 = \frac{\tan\phi}{\ell\cos^3\theta}$, $z_3 = \tan\theta$, $z_4 = y$ $(\lvert\theta\rvert < \pi/2)$
Tricycle with trailer (hinged at the axle midpoint)	5	3	yes	still transformable to chained form
Rolling wheel on the plane (with ϕ)	4	2 (after linearization of rolling)	yes (brackets g_3, g_4)	reached with a sequential manoeuvre

Key take-aways.

- Bicycle, tricycle and car-like robots share the same kinematic model; an Ackermann steering geometry (outer front wheel turned less than the inner one) guarantees a single CR and therefore pure rolling.
- The $(2,n)$ chained form $\dot z_1 = w_1$, $\dot z_2 = w_2$, $\dot z_i = z_{i-1}w_1$ is a cascade of integrators that is always controllable (via the Lie brackets $\overset{k}{\gamma_1}\gamma_2 \propto e_{k+2}$).

- With w_1 constant (or piecewise constant, as with digital control) the chained form is linear (piecewise linear), which makes control design easy.
- Unicycle and bicycle are put in chained form through a coordinate transformation = $\alpha()$ and an input transformation = $\beta()$; adding trailers hinged at the axle midpoint preserves transformability.
- Constraints that are not Pfaffian at first sight (rolling of the wheel with ϕ) can be made Pfaffian by projecting the velocity on the heading direction; controllability then follows from the Lie algebra rank condition and non-trivial configurations are reached by sequential manoeuvres.

Part VIII

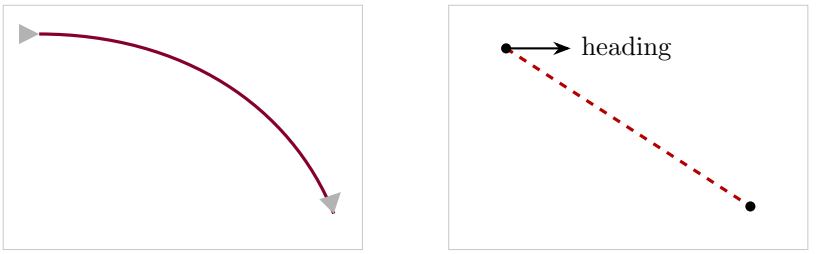
Lecture 6

Roadmap. The lecture is about computing motions for a wheeled mobile robot (WMR) that *respect its nonholonomic constraints*, in free space (no obstacles). The plan is: separate space from time; characterize which *paths* are admissible (geometric model); exploit *differential flatness* to generate admissible paths by pure interpolation; apply this to the unicycle via Cartesian polynomials, via the chained form, and via parametrized inputs; look at numerical parking maneuvers; and finally see how a timing law turns a path into a trajectory.

1 Planning admissible motions *(slides 3)*

Definition 1.1 (Motion planning without obstacles)**.** Given an initial configuration and a final configuration , find a motion (t) that transfers the WMR from to and is *admissible*, i.e., satisfies the nonholonomic constraints $A()\dot{} = 0$ at every time instant.

Meeting the boundary conditions is not enough: a curve that joins to but demands, say, a sideways slip of the wheels is useless. Admissibility is what makes this problem different from plain curve interpolation.



admissible path tangent ≠ heading: not admissible

Figure 59: A unicycle-like robot starting along the horizontal direction can follow a smooth curve that bends toward (left), but not the straight segment (right), whose tangent disagrees with the heading at .

Intuition

Think of parking a car: you cannot simply draw the straight line from where you are to the target spot, because the car cannot move sideways. The planner must hand the robot a curve it is *capable* of tracking. Obstacles are deliberately left out here; the whole difficulty already lives in the nonholonomic constraints.

2 Space-time separation *(slide 4)*

Definition 2.1 (Path, timing law, trajectory)**.** A configuration-space *trajectory* (t), $t \in [t_i, t_f]$, with $(t_i) =$ and $(t_f) =$, can be split into:

- a geometric *path* (s), where s is a path parameter running from $s_i = s(t_i)$ to $s_f = s(t_f)$;
- a *timing law* $s = s(t)$, strictly monotonic (increasing).

One possible choice of s is the arc length.

- **Path planning:** plan an admissible path from to .
- **Trajectory planning:** plan an admissible trajectory from to . It can be *direct*, or *indirect*: first plan a path, then add a timing law.

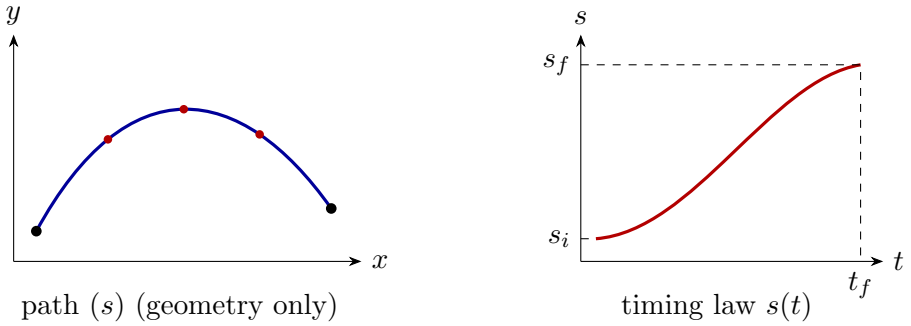


Figure 60: Space-time separation: the same path (left) can be run with many timing laws (right). The dots on the path are equally spaced in s, hence equally spaced in time only when $s(t)$ is linear.

Intuition

A path is the *railway track*, the timing law is the *timetable*. The same track can be traveled quickly or slowly, but the track itself decides whether the train can follow it at all. This is why a robot that cannot track a given curve cannot be rescued by changing speed.

In symbols. A trajectory (t), $t \in [t_i, t_f]$, with $(t_i) =$ and $(t_f) =$, is rewritten as the composition $(t) \to (s(t))$. The first factor, (s) with $s \in [s_i, s_f]$, is the *path*: it carries only the *geometry* of the motion. The second factor, $s(t)$ with $t \in [t_i, t_f]$, is the *timing law*: it says *when* each point of the path is reached.

3 Admissible paths and geometric model *(slides 5–8)*

3.1 Geometric path admissibility *(slides 5–6)*

Intuitively, the local mobility limitation caused by the nonholonomic constraints also shows up on *paths*: if a motion is not admissible, speeding up or slowing down will not make it admissible. To see this, start from the space-time separation:

$$\dot{} = \frac{}{t} = \frac{}{s}\dot{s} =' \dot{s}, \tag{1}$$

where the prime denotes differentiation with respect to s. The vector $'$ is directed as the *tangent* to the path in configuration space, and $\dot{s}$ only changes its modulus.

Example 3.1 (Tangent vector and arc length)**.** Let $\mathcal{C} = \mathbb{R}^2$, with coordinates $(q_1, q_2) = (x, y)$, and consider the path

$$x(s) = \sqrt{1 - s^2}, \qquad y(s) = s, \qquad s \in [0, 1],$$

a quarter of the unit circle going from $= (1, 0)$ to $= (0, 1)$. The tangent vector is

$$'(s) = \begin{pmatrix} x'(s) \\ y'(s) \end{pmatrix} = \begin{pmatrix} -s/\sqrt{1 - s^2} \\ 1 \end{pmatrix}, \qquad \|'(s)\| = \frac{1}{\sqrt{1 - s^2}} \neq 1,$$

so this parameter s is *not* the arc length (for which one would have $\|\mathcal{C}'\| = 1$, e.g., $x = \cos\sigma$, $y = \sin\sigma$).

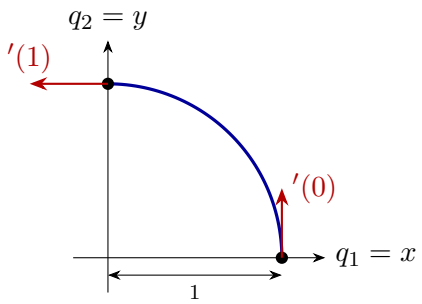


Figure 61: The path of Example 3.1: the tangent direction is the same for any parametrization, only the modulus of $\mathcal{C}'$ depends on s.

Intuition

Reparametrizing a path (e.g., from s to arc length σ) leaves the curve unchanged; it only rescales the tangent vector $\mathcal{C}'$. This is why, in the geometric model, only the *direction* of $\mathcal{C}'$ matters for admissibility, and why in all the planners below we are free to use a convenient s rather than the arc length.

Proposition 3.2 (Geometric path admissibility)**.** *A path $\mathcal{C}(s)$ is the support of an admissible trajectory (for any timing law with $\dot{s} > 0$) if and only if*

$$A(\mathcal{C}(s))\,\mathcal{C}'(s) = 0 \qquad \text{at all points of the path.} \tag{2}$$

Proof. Using (1), the constraints become $A(\mathcal{C}(s))\,\dot{\mathcal{C}}(s) = A(\mathcal{C}(s))\,\mathcal{C}'(s)\,\dot{s} = 0$. Since $\dot{s}(t) > 0$ for all t we may divide by $\dot{s}$, which gives $A(\mathcal{C}(s))\,\mathcal{C}'(s) = 0$; conversely, if $A(\mathcal{C}(s))\,\mathcal{C}'(s) = 0$ holds along the path, then $A(\mathcal{C}(s))\,\dot{\mathcal{C}}(s) = 0$ for any $\dot{s}$. □

Definition 3.3 (Geometric model and geometric inputs)**.** Let the columns of $G(\mathcal{C}(s))$ span $\ker(A(\mathcal{C}(s)))$. All geometrically admissible paths are solutions of the *geometric model*

$$\mathcal{C}' = G(\mathcal{C})\,\mathcal{U}, \tag{3}$$

where $\mathcal{U}$ are the *geometric inputs*. They are related to the velocity inputs of the kinematic model by

$$\boldsymbol{u}(t) = \mathcal{U}(s)\,\dot{s}(t). \tag{4}$$

Once the geometric inputs $\mathcal{U}(s)$ are chosen, the path is determined; selecting a timing law $s = s(t)$ then identifies a particular trajectory along that path.

Kinematic versus geometric model. The geometric model is built exactly in parallel to the kinematic model (KM). Letting $\boldsymbol{g}_1, \ldots, \boldsymbol{g}_m$ be the columns of $G(\mathcal{C})$:

$$\text{(KM)} \quad A(\mathcal{C})\,\dot{\mathcal{C}} = 0 \;\Rightarrow\; \dot{\mathcal{C}} \in \ker(A(\mathcal{C})) \;\Rightarrow\; \dot{\mathcal{C}} = \sum_{i=1}^{m} \boldsymbol{g}_i(\mathcal{C})\,u_i, \tag{5}$$

$$\text{(GM)} \quad A(\mathcal{C})\,\mathcal{C}' = 0 \;\Rightarrow\; \mathcal{C}' \in \ker(A(\mathcal{C})) \;\Rightarrow\; \mathcal{C}' = \sum_{i=1}^{m} \boldsymbol{g}_i(\mathcal{C})\,\tilde{u}_i. \tag{6}$$

Multiplying the geometric model by $\dot{s}$ and using $\dot{} = {}' \dot{s}$,

$$\dot{} = {}' \ \dot{s} = \sum_{i=1}^{m} \boldsymbol{g}_i() \, \tilde{u}_i \, \dot{s},$$

and comparing with the kinematic model (the $\boldsymbol{g}_i$ are linearly independent) gives $u_i = \tilde{u}_i \, \dot{s}$. The u_i are the *pseudovelocities* (or *velocity inputs*); the $\tilde{u}_i$ are the *geometric inputs*. In short: the kinematic model is a *trajectory generator*, the geometric model is a *path generator*.

Intuition

The condition $A()' = 0$ is the kinematic model with time "factored out". The nonholonomic constraints say which *directions* of motion are allowed; speed has no say in direction. So the geometric model is just the kinematic model run with a different clock: s instead of t, and instead of $\boldsymbol{u}$.

3.2 The unicycle example *(slides 7–8)*

For the unicycle the pure-rolling constraint is $\dot{x} \sin\theta - \dot{y} \cos\theta = 0$, i.e., $(\sin\theta \quad -\cos\theta \quad 0)\dot{} = 0$. Replacing $\dot{}$ by $'\dot{s}$ and dividing by $\dot{s} > 0$ gives $(\sin\theta \quad -\cos\theta \quad 0)' = 0$, the *path admissibility* condition, which reads

$$x' \sin\theta - y' \cos\theta = 0, \tag{7}$$

i.e., the Cartesian path tangent must align with the sagittal axis of the robot. The robot may travel along the tangent in either sense: forward motion if the heading points along the tangent, backward motion if it points opposite.

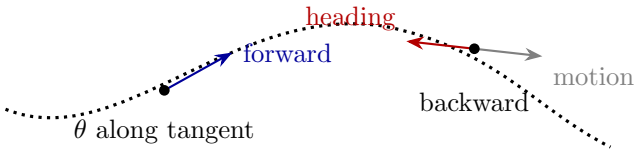


Figure 62: Along a Cartesian path the unicycle heading is tangent to the path; it points the same way as the motion (forward) or the opposite way (backward).

A Cartesian path with a *discontinuous tangent* is not admissible, unless the unicycle is allowed to stop at the discontinuity points to realign with the new tangent.

The geometric model of the unicycle is

$$\begin{aligned} x' &= \cos\theta, \\ y' &= \sin\theta, \\ \theta' &=, \end{aligned} \tag{8}$$

with geometric inputs related to the velocity inputs by $v(t) = (s) \, \dot{s}(t)$ and $\omega(t) = (s) \, \dot{s}(t)$.

Example 3.4 (A Cartesian path with a corner)**.** Consider a Cartesian path made of a horizontal segment followed by a vertical one. Is it admissible for a unicycle? Two paths in the configuration space (x, y, θ) have this same *footprint* in the (x, y) plane:

(1) θ stays at 0 (or jumps at the corner). On the horizontal segment $y' = 0$ and $x' \sin 0 - y' \cos 0 = 0$ holds, but on the vertical segment $x' = 0$, $y' \neq 0$ and the condition fails; a jump of θ would make the path discontinuous in $\mathcal{C}$. **Not admissible.**

(2) $\theta = 0$ on the horizontal segment; at the corner θ goes from 0 to $\pi/2$ while $x' = y' = 0$ (rotation on the spot); then $\theta = \pi/2$ on the vertical segment. The condition holds at

every point, so the path **is admissible**, but only if the unicycle *stops* at the corner.

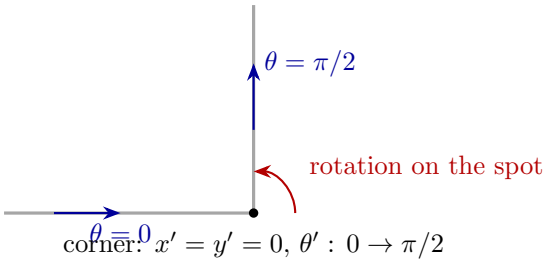


Figure 63: Path (2) of Example 3.4: the footprint has a corner, but the robot realigns with the new tangent by rotating at the corner.

Intuition

A corner in the Cartesian path is a place where the robot would have to change heading instantly. A unicycle can do that only by stopping and rotating on the spot, i.e., by "cheating" the pure path picture.

Planning formulation. Path planning for WMRs is generally formulated and solved with the *geometric* model. Trajectory planning can be solved *directly* with the kinematic model, or *indirectly* by first planning a geometric path and then choosing a timing law.

4 Flat outputs *(slides 9–13)*

4.1 Differential flatness *(slides 9–10)*

Definition 4.1 (Differential flatness)**.** The system $\dot{\boldsymbol{x}} = f(\boldsymbol{x}) + G(\boldsymbol{x})\boldsymbol{u}$ is *differentially flat* if there exists a set of outputs $\boldsymbol{y}$, called *flat outputs*, such that the state and the control inputs can be expressed *algebraically* as functions of $\boldsymbol{y}$ and its time derivatives:

$$\boldsymbol{x} = \boldsymbol{x}\big(\boldsymbol{y}, \dot{\boldsymbol{y}}, \ddot{\boldsymbol{y}}, \ldots, \boldsymbol{y}^{(r)}\big), \tag{9}$$

$$\boldsymbol{u} = \boldsymbol{u}\big(\boldsymbol{y}, \dot{\boldsymbol{y}}, \ddot{\boldsymbol{y}}, \ldots, \boldsymbol{y}^{(r)}\big). \tag{10}$$

The definition applies equally to *geometric* models if s replaces t and derivatives with respect to t are replaced by derivatives with respect to s. If the path (trajectory) of $\boldsymbol{y}$ is known, the path (trajectory) of $\boldsymbol{x}$ and the history of $\boldsymbol{u}$ in s (or t) can be *reconstructed*.

Intuition

Flat outputs are "free handles": pick any sufficiently smooth curve for $\boldsymbol{y}$, and everything else follows by differentiation, with no differential equation to integrate. For a unicycle, picking a smooth planar curve $(x(s), y(s))$ is enough: the heading is the direction of the curve, and the inputs are its speed and curvature.

Proposition 4.2 (Flatness of the unicycle and the bicycle)**.** *The unicycle and the bicycle are differentially flat with x and y as flat outputs. For the geometric model of the unicycle, a Cartesian path $(x(s), y(s))$ determines the path of the whole state $(s) = (x(s), y(s), \theta(s))$ and*

the geometric inputs:

$$\theta(s) = \arctan\frac{y'(s)}{x'(s)} = \big(y'(s), x'(s)\big) + k\pi, \tag{11}$$

$$(s) = \pm\sqrt{(x'(s))^2 + (y'(s))^2}, \tag{12}$$

$$(s) = \frac{y''(s)x'(s) - x''(s)y'(s)}{(x'(s))^2 + (y'(s))^2}. \tag{13}$$

For forward (backward) motion take $k = 1$ ($k = 0$) and the + (−) sign. For the bicycle, let L be the wheelbase and delta the front-wheel steering angle; its kinematic model is

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \frac{v}{L}\tan\delta.$$

On intervals where the vehicle does not stop, the same Cartesian outputs reconstruct its state and inputs as

$$\begin{aligned}
\theta &= \operatorname{atan2}(\dot{y}, \dot{x}) \quad \textit{(modulo } \pi\textit{)},\\
v &= \pm\sqrt{\dot{x}^2 + \dot{y}^2},\\
\delta &= \arctan\left(L\,\frac{\operatorname{sgn}(v)(\dot{x}\ddot{y} - \dot{y}\ddot{x})}{(\dot{x}^2 + \dot{y}^2)^{3/2}}\right).
\end{aligned}$$

The sign of v selects forward or backward motion, with the corresponding heading branch.

Derivation for the unicycle. From $x' = \cos\theta$ and $y' = \sin\theta$ we get $(x')^2 + (y')^2 = ^2$, hence $= \pm\sqrt{(x')^2 + (y')^2}$, and $y'/x' = \tan\theta$, hence $\theta = \arctan(y'/x')$ up to a multiple of π. Differentiating $\tan\theta = y'/x'$:

$$\frac{\theta'}{\cos^2\theta} = \frac{y''x' - x''y'}{(x')^2} \implies = \theta' = \frac{y''x' - x''y'}{(x')^2}\cos^2\theta = \frac{y''x' - x''y'}{(x')^2 + (y')^2},$$

since $\cos^2\theta = (x')^2/\big((x')^2 + (y')^2\big)$. □

Derivation for the bicycle. The first two bicycle equations have the same form as the unicycle equations, so they give v and θ directly from (x, y) and their first derivatives, as above. Differentiating the heading yields

$$\dot{\theta} = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}.$$

Using $\dot{\theta} = (v/L)\tan\delta$ and $|v| = \sqrt{\dot{x}^2 + \dot{y}^2}$ (with the heading branch chosen consistently with the sign of v), we obtain

$$\tan\delta = L\frac{\operatorname{sgn}(v)(\dot{x}\ddot{y} - \dot{y}\ddot{x})}{(\dot{x}^2 + \dot{y}^2)^{3/2}},$$

which gives the steering reconstruction above. Thus state and inputs are algebraic functions of the flat outputs and their derivatives, proving flatness wherever $v \neq 0$. □

Intuition

is the "speed along the curve" and is times the *curvature* of the Cartesian path: the formula for is the familiar planar curvature formula $\kappa = (y''x' - x''y')/\big((x')^2 + (y')^2\big)^{3/2}$ multiplied by . The sign choice only decides whether the robot runs the curve nose-first or tail-first.

4.2 Flatness and the chained form *(slides 11–12)*

Theorem 4.3 (Flatness, chained form, linearization)**.** *For driftless dynamic systems, differential flatness is a necessary and sufficient condition for transformability into chained form; hence all WMRs with dimension up to 4 are flat. For general dynamic systems, differential flatness is a necessary and sufficient condition for linearizability via dynamic feedback.*

Singular points of the reconstruction.

- If $x'(s) = y'(s) = 0$ at an isolated point (as at cusps or motion inversions), then $(s) = 0$ there; at these points $\theta(s)$ and (s) can be computed by continuity arguments.
- The reconstruction of θ and $$ *fails* if the Cartesian path degenerates to a point ($x'(s) = y'(s) = 0$ identically). In fact, the unicycle may then have any orientation at the contact point, and may even be rotating on the spot.

Geometrically: at an isolated cusp the Cartesian tangent vanishes only at the tip, so $= 0$ there, the robot stops, inverts its motion and leaves along the other branch; if instead $x' \equiv y' \equiv 0$ over an interval, the contact point does not move and θ, cannot be recovered from (x, y) (they are free in the geometric model $\theta' =$).

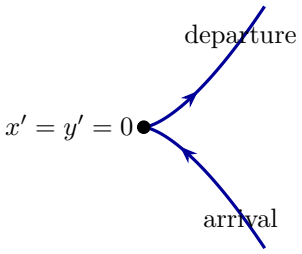


Figure 64: A cusp of the Cartesian path: the tangent vanishes only at the tip, where the unicycle inverts its motion.

Proposition 4.4 (Flatness of the $(2, n)$ chained form)**.** *The $(2, n)$ chained form is differentially flat, with flat outputs z_1 and z_n; all the other state variables and the associated control inputs can be computed from them.*

Example 4.5 (The $(2, 3)$ chained form)**.** Consider

$$\begin{aligned} z_1' &= {}_1, \\ z_2' &= {}_2, \\ z_3' &= z_2\, {}_1. \end{aligned} \tag{14}$$

The reconstruction formulas are

$$z_2 = \frac{z_3'}{z_1'} \quad \text{(state)}, \qquad {}_1 = z_1', \qquad {}_2 = \frac{z_1' z_3'' - z_1'' z_3'}{(z_1')^2} \quad \text{(input)}. \tag{15}$$

Derivation. The first equation gives ${}_1 = z_1'$. Substituting in the third, $z_3' = z_2 z_1'$, so $z_2 = z_3'/z_1'$ (when $z_1' \neq 0$). The second equation then gives ${}_2 = z_2' = \big(z_3'' z_1' - z_3' z_1''\big)/(z_1')^2$ by the quotient rule. □

Intuition

In chained form the first variable z_1 acts as "clock": it advances at the rate $_1$. Dividing the third equation by the first, $z_3/z_1 = z_2$: the slope of z_3 against z_1 *is* z_2. Choose the curve $z_3(z_1)$ and z_2 is forced; its derivative then gives the remaining input. This is exactly the same "choose the curve, read off the rest" mechanism as for (x, y) in the unicycle.

4.3 An algorithm for planning with flat outputs *(slide 13)*

Reconstruction formulas associate admissible paths (trajectories) to the evolution of the flat outputs, which can be chosen *arbitrarily*. This leads to the following algorithm.

Algorithm 4.6 (Path planning via flat outputs)**.**
1. Compute $\boldsymbol{y}_i$ and $\boldsymbol{y}_f$ corresponding to and .
2. Find $\boldsymbol{y}(s)$ by interpolating $\boldsymbol{y}_i$ and $\boldsymbol{y}_f$ with the appropriate boundary conditions for .
3. Use the reconstruction formulas to find (s) and (s).

The resulting path is *automatically* admissible, i.e., it satisfies the nonholonomic constraints at all points. The same algorithm can be used for *trajectory planning* by interpolating over t rather than over s.

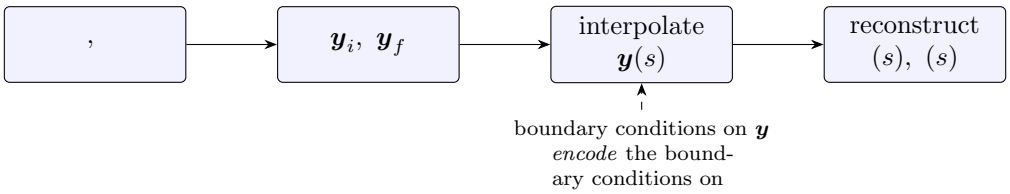


Figure 65: Planning with flat outputs: admissibility is built into the reconstruction step.

Intuition

Normally planning for a nonholonomic robot means solving a constrained problem. Flatness turns it into an *unconstrained* one: the constraints are already "absorbed" by the reconstruction formulas. The only subtlety is that orientation (and higher-order variables) are *not* flat outputs, so their boundary values must be translated into *derivative* conditions on $\boldsymbol{y}$; this is the role of the "appropriate boundary conditions" in step 2.

5 Unicycle path planning via Cartesian polynomials *(slides 14–15)*

We apply Algorithm 4.6 to the unicycle, whose flat outputs are x and y. Given $= (x_i, y_i, \theta_i)$ and $= (x_f, y_f, \theta_f)$, find an admissible path (s), $s \in [s_i, s_f]$, with $(s_i) =$ and $(s_f) =$.

Let $s_i = 0$, $s_f = 1$ and use *pre-fitted cubic polynomials*

$$x(s) = s^3 x_f - (s-1)^3 x_i + \alpha_x s^2(s-1) + \beta_x s(s-1)^2, \tag{16}$$

$$y(s) = s^3 y_f - (s-1)^3 y_i + \alpha_y s^2(s-1) + \beta_y s(s-1)^2, \tag{17}$$

which already satisfy the boundary conditions on x, y (indeed $x(0) = x_i$, $x(1) = x_f$). Since θ_i and θ_f are given, we must also impose

$$\arctan \frac{y'(0)}{x'(0)} = \theta_i, \qquad \arctan \frac{y'(1)}{x'(1)} = \theta_f. \tag{18}$$

Since the interpolation is done independently in x and y, these are separated as

$$\begin{aligned} x'(0) &= k_i \cos\theta_i, \quad x'(1) = k_f \cos\theta_f, \\ y'(0) &= k_i \sin\theta_i, \quad y'(1) = k_f \sin\theta_f, \end{aligned} \tag{19}$$

where $k_i \neq 0$, $k_f \neq 0$ are free parameters. In other words, the flat outputs of the boundary configurations are (x_i, y_i) and (x_f, y_f), while the orientations θ_i, θ_f translate into conditions on the first derivatives. Each of $x(s)$ and $y(s)$ thus carries four boundary conditions (value and derivative at both ends), which matches the four coefficients of a cubic.

Proposition 5.1 (Cartesian cubic planner)**.** *Under the simplifying assumption $k_i = k_f = k$, the additional boundary conditions give*

$$\begin{aligned} \alpha_x &= k\cos\theta_f - 3x_f, \quad \beta_x = k\cos\theta_i + 3x_i, \\ \alpha_y &= k\sin\theta_f - 3y_f, \quad \beta_y = k\sin\theta_i + 3y_i. \end{aligned} \tag{20}$$

Moreover $(0) = (1) = k$; $k > 0$ gives forward motion, $k < 0$ backward motion.

Proof. Differentiating, $x'(s) = 3s^2 x_f - 3(s-1)^2 x_i + \alpha_x(3s^2 - 2s) + \beta_x\big((s-1)^2 + 2s(s-1)\big)$. Then $x'(0) = -3x_i + \beta_x = k\cos\theta_i$ and $x'(1) = 3x_f + \alpha_x = k\cos\theta_f$, which give β_x, α_x. The computation for y is identical. At the extremes $= \pm\sqrt{(x')^2 + (y')^2} = \pm|k|$, with the sign of k for the chosen direction of motion. □

From $x(s)$ and $y(s)$ one obtains $\theta(s)$, (s) and (s) via the reconstruction formulas of Proposition 4.2.

Intuition

This is cubic *Hermite interpolation*: (x_i, y_i) and (x_f, y_f) are the endpoints, and the vectors $k(\cos\theta_i, \sin\theta_i)$ and $k(\cos\theta_f, \sin\theta_f)$ are the end tangents. The heading is forced to be tangent to the path (so the path is admissible), and k is a "tangent length": a large k makes the robot leave and arrive at *straight and for a long stretch* before bending in the middle, a small k gives a more direct, tighter curve.

Example 5.2 (Worked numbers: forward parking)**.** Take $= (0, 0, 0)$ and $= (4, -3, -\pi/2)$. Then $\alpha_x = -12$, $\beta_x = k$, $\alpha_y = 9 - k$, $\beta_y = 0$, so

$$x(s) = (k-8)s^3 + (12-2k)s^2 + ks, \qquad y(s) = (6-k)s^3 - (9-k)s^2.$$

One checks $x'(0) = k$, $y'(0) = 0$ (heading 0, speed k) and $x'(1) = 0$, $y'(1) = -k$ (heading $-\pi/2$, speed k). At the start, $(0) = y''(0)/k = 2(k-9)/k$: for $k = 5$ this is -1.6 (the robot immediately steers right), whereas for $k = 20$ it is $+1.1$ (the robot first swings slightly *left*, up, and then comes down).

Intuition

The sign change of (0) at $k = 9$ is the numerical face of "long tangents overshoot": with a very large k the cubic is stretched along the initial heading, so it first bulges forward and upward before turning down to the target.

6 Unicycle path planning via the chained form *(slides 16–17)*

As an alternative one may transform the unicycle into the $(2, 3)$ chained form, whose flat outputs are z_1 and z_3. (The standard unicycle transformation is $z_1 = \theta$, $z_2 = x\cos\theta + y\sin\theta$, $z_3 = x\sin\theta - y\cos\theta$, with inputs $w_1 = \omega$, $w_2 = v - z_3\omega$.)

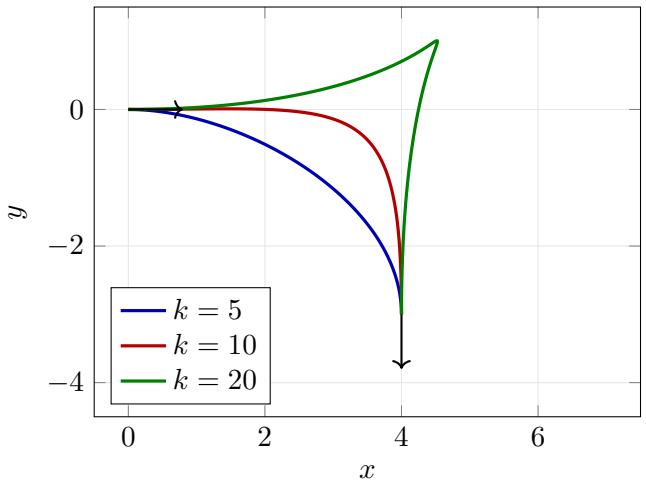


Figure 66: Cartesian cubic planner for the forward-parking problem of Example 5.2, for increasing k (own computation from the formulas above). Larger k produces longer take-off and landing phases.

First compute the initial and final values of z_1, z_2, z_3 corresponding to and . Then, assuming $z_{1,i} \neq z_{1,f}$, define z_1 and z_3 over $[0, 1]$ as

$$z_1(s) = z_{1,f}\, s - (s-1)\, z_{1,i}, \tag{21}$$

$$z_3(s) = s^3 z_{3,f} - (s-1)^3 z_{3,i} + \alpha_3 s^2(s-1) + \beta_3 s(s-1)^2, \tag{22}$$

which already satisfy the boundary conditions on z_1, z_3. Since $z_{2,i}$ and $z_{2,f}$ are also given, we must impose

$$\frac{z_3'(0)}{z_1'(0)} = z_{2,i}, \qquad \frac{z_3'(1)}{z_1'(1)} = z_{2,f}. \tag{23}$$

Proposition 6.1 (Chained-form cubic planner)**.** *Imposing these conditions gives*

$$\alpha_3 = z_{2,f}(z_{1,f} - z_{1,i}) - 3z_{3,f}, \qquad \beta_3 = z_{2,i}(z_{1,f} - z_{1,i}) + 3z_{3,i}. \tag{24}$$

Proof. $z_1'(s) = z_{1,f} - z_{1,i} =: \Delta$ is constant. As in Proposition 2.11, $z_3'(0) = -3z_{3,i} + \beta_3$ and $z_3'(1) = 3z_{3,f} + \alpha_3$; setting them equal to $z_{2,i}\Delta$ and $z_{2,f}\Delta$ respectively gives the result. □

From $z_1(s)$ and $z_3(s)$ one obtains $z_2(s)$ and $_1(s)$, $_2(s)$ via the reconstruction formulas of Example 4.5. Finally (s), (s) and (s) are computed by inverting the chained form coordinate and input transformations.

The singular case. If $z_{1,i} = z_{1,f}$ (i.e., $\theta_i = \theta_f$) the construction breaks down (z_1 would be constant). One can introduce a *via point* $_v = (x_v, y_v, \theta_v)$ with $\theta_v \neq \theta_i$ and split the original problem in two; alternatively, set $z_{1,f} = z_{1,i} + 2\pi$ (a full turn).

Direct trajectory planning. Both previous schemes (Cartesian polynomials and chained form) can be used for direct trajectory planning: just let $s = t$ and use the kinematic model.

Intuition

Here $z_1 = \theta$ is used as the interpolation "clock" and it moves *linearly* between θ_i and θ_f: the heading changes monotonically. That is why $\theta_i = \theta_f$ is a problem: the clock does not move

at all and the curve $z_3(z_1)$ has nothing to be a function of. Adding a via point or a full 2π turn restores a clock that ticks.

Remark on the numerical results. For the unicycle chained form, the position enters z_2, z_3 through the current (x, y). Placing the origin of z_2, z_3 at (so that $z_{2,i} = z_{3,i} = 0$) changes the plan: with the global origin a reorientation generates a Cartesian displacement, whereas with the origin at pure rotations on the spot are representable (in the numerical check for $= (1, 1, 0) \to = (1, 1, \pi/2)$ with origin at the integration gives $c_0 = c_1 = 0$, i.e. $x \equiv y \equiv$ const).

7 Path planning via parametrized inputs *(slides 18–21)*

7.1 Chained form with parametrized inputs *(slides 18–20)*

A different approach, not directly based on flatness, consists in writing the inputs in *parametrized form* and then imposing that the robot goes from to . Assume the robot can be put in chained form,

$$\begin{aligned} z_1' &= {}_1, \\ z_2' &= {}_2, \\ z_3' &= z_{21}, \\ &\vdots \\ z_n' &= z_{n-11}, \end{aligned} \tag{25}$$

which becomes *linear* (hence easily integrable) if ${}_1(s)$ is constant.

Let $\Delta = z_{1,f} - z_{1,i}$ and choose the geometric inputs for $s \in [s_i, s_f] = [0, |\Delta|]$ as

$${}_1 = (\Delta), \tag{26}$$

$${}_2 = c_0 + c_1 s + \cdots + c_{n-2} s^{n-2}. \tag{27}$$

Integrating the chained-form equations from $\boldsymbol{z}(0) = \boldsymbol{z}_i$ and imposing $\boldsymbol{z}(|\Delta|) = \boldsymbol{z}_f$ leads to a *linear system*

$$D(\Delta) \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-2} \end{pmatrix} = \boldsymbol{d}(\boldsymbol{z}_i, \boldsymbol{z}_f, \Delta), \tag{28}$$

with $D(\Delta)$ invertible if $\Delta \neq 0$.

Proposition 7.1 (The $(2, 3)$ case)**.** *For the* $(2, 3)$ *chained form,*

$$D = \begin{pmatrix} |\Delta| & \dfrac{\Delta^2}{2} \\ (\Delta)\dfrac{\Delta^2}{2} & \dfrac{\Delta^3}{6} \end{pmatrix}, \qquad \boldsymbol{d} = \begin{pmatrix} z_{2,f} - z_{2,i} \\ z_{3,f} - z_{3,i} - z_{2,i}\Delta \end{pmatrix}, \tag{29}$$

and $\det D = -(\Delta)\,\Delta^4/12 \neq 0$ *whenever* $\Delta \neq 0$.

Proof. With ${}_1 = (\Delta)$, ${}_2 = c_0 + c_1 s$: $z_1(s) = z_{1,i} + (\Delta)s$, which reaches $z_{1,f}$ at $s = |\Delta|$; $z_2(s) = z_{2,i} + c_0 s + c_1 s^2/2$; and $z_3' = (\Delta) z_2$ gives $z_3(s) = z_{3,i} + (\Delta)\big(z_{2,i} s + c_0 s^2/2 + c_1 s^3/6\big)$. Imposing $z_2(|\Delta|) = z_{2,f}$ gives $c_0|\Delta| + c_1\Delta^2/2 = z_{2,f} - z_{2,i}$. Imposing $z_3(|\Delta|) = z_{3,f}$ and using $(\Delta)|\Delta| = \Delta$,

$(\Delta)|\Delta|^3 = \Delta^3$ gives $(\Delta)\frac{\Delta^2}{2}c_0 + \frac{\Delta^3}{6}c_1 = z_{3,f} - z_{3,i} - z_{2,i}\Delta$. Finally $\det D = |\Delta|\Delta^3/6 - (\Delta)\Delta^4/4 = -(\Delta)\Delta^4/12$. □

If $\Delta = 0$ ($z_{1,i} = z_{1,f}$) a singular case is met, which can be handled as before (e.g., with a via point). Once planning on the chained form is complete, (s) and (s) for the original model are computed via the inverse coordinate and input transformations. Although this method does not make explicit use of flat outputs, it still relies on transformability in chained form, which is strongly related to flatness (Theorem 4.3).

Intuition

With $_1$ constant, s simply counts how far the "clock" variable z_1 has advanced, and the remaining equations are a *chain of integrators* driven by the polynomial $_2$. The $n-1$ unknown coefficients are exactly as many as the $n-1$ boundary conditions on $z_2, \ldots, z_n$ (those on z_1 are met by the choice of the interval $[0, |\Delta|]$), so a square linear system results. For $n = 3$ there are two coefficients (c_0, c_1) for two conditions.

7.2 Other parametrizations and trajectory planning *(slide 21)*

Other classes of parametrized inputs can be used, such as sinusoidal and piecewise-constant functions. Note that in *all* the presented path planning methods the path parameter s *does not represent the arc length.* This scheme too can be used for direct trajectory planning by replacing s with t and using the kinematic model in place of the geometric model.

8 Parking a unicycle: numerical results *(slides 22–27)*

8.1 Planner based on Cartesian polynomials *(slides 22–24)*

- **Forward parking** ($k = 5 > 0$): the motion is forward; a negative k would produce a backward motion. There are no motion inversions (a general result for this planner).
- **Effect of k:** increasing k (e.g., $k = 10$ and $k = 20$) leads to elongated take-off and landing phases; still no motion inversions.
- **Parallel parking** and **reorientation** (again $k = 10$ and $k = 20$): since the planner is based on flat outputs, a Cartesian path of *finite length* is needed for *any* kind of reconfiguration.

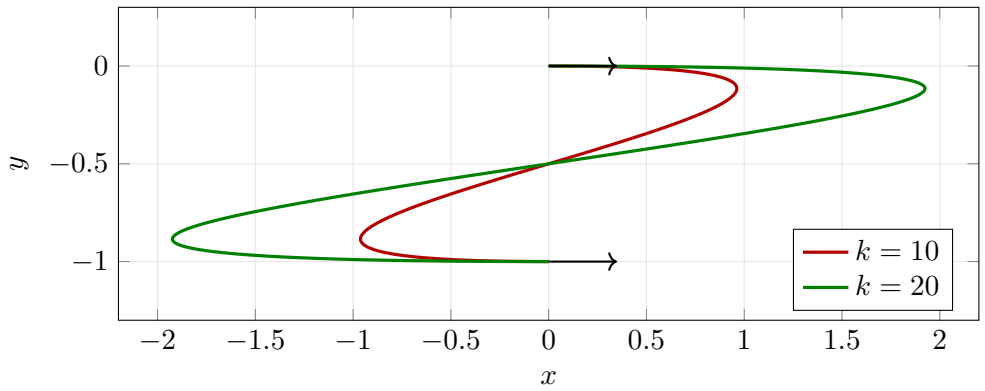


Figure 67: Parallel parking with the Cartesian planner, $= (0, 0, 0) \rightarrow = (0, -1, 0)$ (own computation). A larger k widens the forward and backward excursions needed to shift sideways.

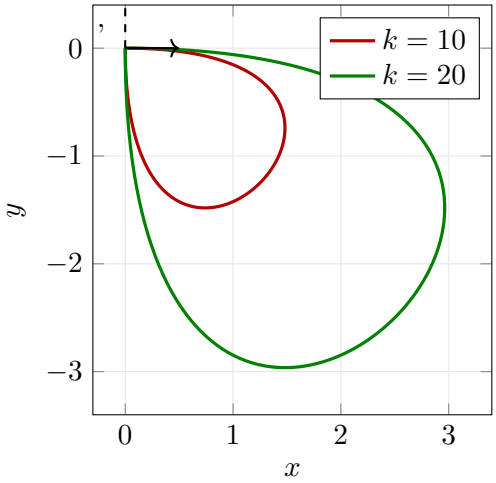


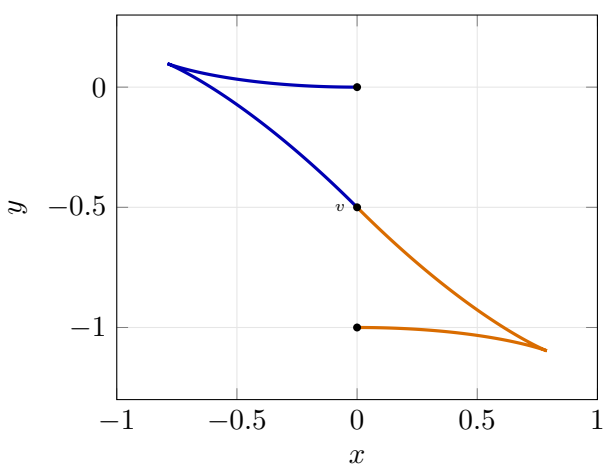
Figure 68: Reorientation $= (0, 0, 0) \rightarrow = (0, 0, \pi/2)$ with the Cartesian planner (own computation): the robot leaves heading east (solid arrow), draws a finite loop and returns heading north (dashed arrow).

Intuition

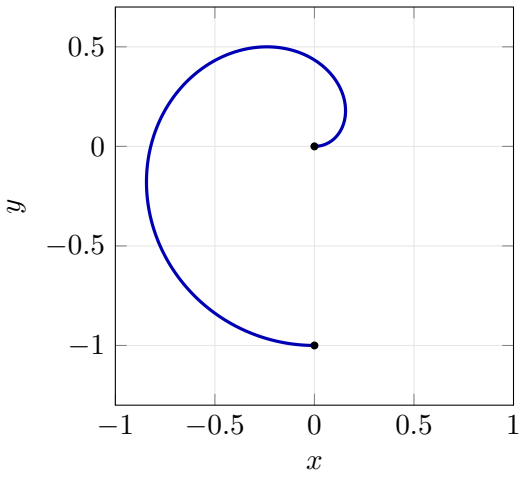
Reorienting a unicycle in place is easy for the physical robot (rotate on the spot), but the Cartesian planner has no way to say "stay at a point". The reconstruction formulas fail when the Cartesian path degenerates to a point (Section 4), so the planner must draw a small loop in the plane to change orientation.

8.2 Planner based on parametrized inputs *(slides 25–27)*

- **Forward parking:** the first maneuver is similar to that of the other planner. In the second maneuver a *cusp* appears: this planner can only generate paths along which the orientation varies monotonically.
- **Parallel parking:** the problem is solved with a via point, or by imposing a full rotation.
- **Reorientation:** using the classical chained form transformation, a Cartesian displacement is generated; placing the origin of z_2, z_3 at avoids it.

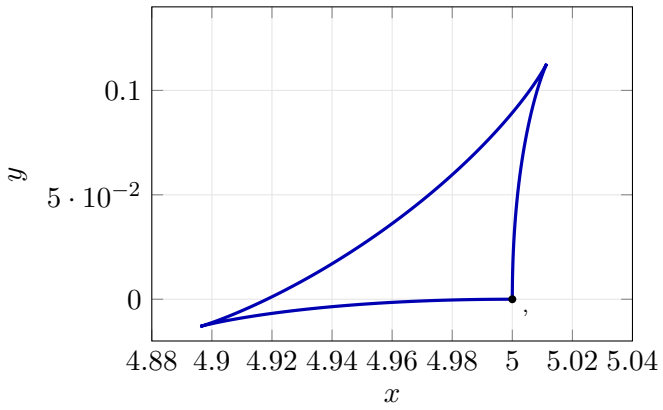


with a via point

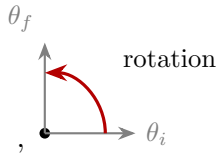


imposing a full rotation ($z_{1,f} = z_{1,i} + 2\pi$)

Figure 69: Parallel parking $= (0, 0, 0) \rightarrow = (0, -1, 0)$ with the parametrized-input planner. Since $\theta_i = \theta_f$ ($\Delta = 0$), the problem is either split at a via point (here $_v = (0, -0.5, -\pi/4)$) or solved with a full rotation of the orientation (own computation).



classical transformation (origin of z_2, z_3 elsewhere)



origin of z_2, z_3 placed at

Figure 70: Reorientation $= (x, y, 0) \rightarrow = (x, y, \pi/2)$ with the parametrized-input planner (own computation). With the classical transformation a small Cartesian displacement is generated (note the scale of the axes); placing the origin at gives $c_0 = c_1 = 0$, i.e., a pure rotation on the spot.

9 Choice of timing law *(slides 28–30)*

9.1 Choosing $s(t)$ *(slide 28)*

Once a configuration-space path (s), $s \in [s_i, s_f]$, has been generated, obtaining a trajectory requires choosing a *timing law*

$$s = s(t), \qquad t \in [t_i, t_f], \qquad s(t_i) = s_i, \quad s(t_f) = s_f. \tag{30}$$

Typically t_f is *not* assigned a priori: it results from the timing law itself. Two simple choices are

- a *constant* $\dot{s}$ over $[t_i, t_f]$, i.e., a *linear* timing law;
- a *trapezoidal* $\dot{s}$, i.e., a *quadratic–linear–quadratic* timing law.

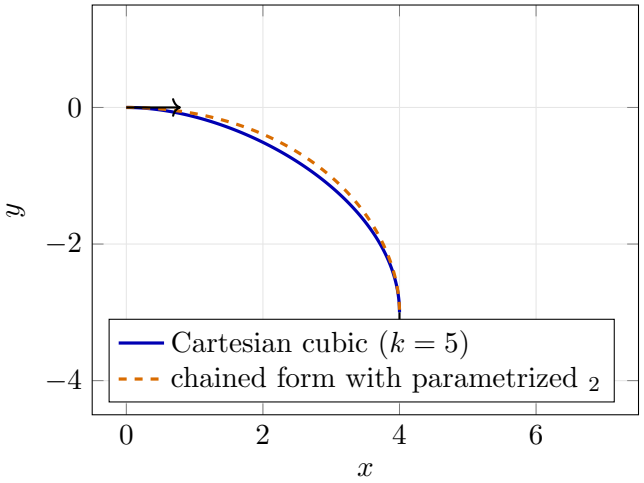


Figure 71: The two planners on the same forward-parking problem $= (0, 0, 0) \rightarrow = (4, -3, -\pi/2)$ (own computation). The parametrized-input planner turns monotonically from 0 to $-\pi/2$ with $= \pm 1$ per unit of s, so s is the angle swept, not the arc length.

Whatever the choice, it must be *feasible*: the resulting velocities must comply with the robot velocity bounds.

Given a geometric path (s) with inputs (s), any $s = s(t)$ with $\dot{s} > 0$ yields an admissible trajectory with inputs $v(t) = (s)\dot{s}$, $\omega(t) = (s)\dot{s}$. For a unicycle with bounds $|v| \le v_{\max}$, $|\omega| \le \omega_{\max}$, feasibility means

$$\dot{s}(t) \le \min\left\{ \frac{v_{\max}}{|(s)|}, \frac{\omega_{\max}}{|(s)|} \right\} \qquad \text{for all } t.$$

Intuition

The trapezoidal $\dot{s}$ is the familiar "accelerate, cruise, decelerate" profile: $s(t)$ is a parabola, then a straight line, then a parabola, so the robot starts and stops without jumps in $\dot{s}$. The linear law is the cruise phase alone. Slowing down scales *both* inputs by the same factor, so the shape of the path never changes; it is the sharp parts of the path (large $||$) that force the robot to slow down.

9.2 Linear timing law for a unicycle with velocity bounds *(slides 29–30)*

Consider a unicycle subject to bounds on the velocity inputs, and assume that a path (s) has been generated, with geometric inputs (s) and (s). Choose the linear timing law

$$s = \frac{t}{k}, \qquad t \in [0, t_f], \qquad t_f = k\, s_f, \tag{31}$$

(taking $s_i = 0$). Since $\dot{s} = 1/k$,

$$v(t) = \frac{(s(t))}{k}, \qquad \omega(t) = \frac{(s(t))}{k}. \tag{32}$$

Proposition 9.1 (Feasible linear timing law)**.** *Let*

$$_{\text{peak}} = \max_{s \in [s_i, s_f]} |(s)|, \qquad _{\text{peak}} = \max_{s \in [s_i, s_f]} |(s)|.$$

The linear timing law is feasible if and only if $k \ge \max\{_{\text{peak}}/v_{\max},\ _{\text{peak}}/\omega_{\max}\}$*, and the feasible*

choice with the shortest duration $t_f = ks_f$ is

$$k = \max\left\{\frac{\text{peak}}{v_{\max}},\ \frac{\text{peak}}{\omega_{\max}}\right\}. \tag{33}$$

With this k, either v or ω saturates at (at least) one instant of time.

Proof. $|v(t)| \le v_{\max}$ for all t is equivalent to $\max_s |(s)|/k \le v_{\max}$, i.e., $k \ge_{\text{peak}} /v_{\max}$; likewise $|\omega(t)| \le \omega_{\max}$ for all t is equivalent to $k \ge_{\text{peak}} /\omega_{\max}$. Both hold iff k is at least the larger of the two ratios, and since $t_f = ks_f$ the smallest such k gives the shortest duration. For that k one of the two inequalities is an equality, so the corresponding input reaches its bound at the instant where || or || attains its peak. □

Example 9.2 (Worked numbers)**.** Let $s_f = 1$, $_{\text{peak}} = 2$, $_{\text{peak}} = 1.5$, $v_{\max} = 0.5$, $\omega_{\max} = 0.6$ (consistent units). Then $k = \max\{2/0.5,\ 1.5/0.6\} = \max\{4,\ 2.5\} = 4$, so $t_f = 4$; the peak linear velocity is $2/4 = 0.5 = v_{\max}$ (saturated), while the peak angular velocity is $1.5/4 = 0.375 < \omega_{\max}$.

Intuition

Dividing both geometric inputs by the same k is like playing the same film in slow motion: the path is unchanged and every velocity shrinks by the factor $1/k$. We slow down just enough so that the most demanding point of the path, in terms of v or of ω, exactly touches its bound. That is why one input saturates: slowing down any further would be feasible but wasteful.

More generally, any choice of a constant $k \neq 1$ based on feasibility or optimality arguments can be seen as a *uniform time scaling* with respect to the straightforward choice $s = t$ (which corresponds to $k = 1$).

10 Summary

	Cartesian polynomials	**Chained form (cubic z_3)**	**Parametrized inputs**
Flat outputs used	$x,\ y$	$z_1,\ z_3$	not directly (needs chained form)
What is chosen	cubic $x(s), y(s)$; k free	z_1 linear, z_3 cubic	$_1 = \Delta$, $_2$ polynomial
Unknowns	$\alpha_x, \beta_x, \alpha_y, \beta_y$ (closed form), k	α_3, β_3 (closed form)	$c_0, \ldots, c_{n-2}$ (linear system $D\boldsymbol{c} = \boldsymbol{d}$)
Singular case	—	$\theta_i = \theta_f$	$\Delta = 0$
Singular case remedy	—	via point or $+2\pi$	via point or $+2\pi$
Orientation	follows the Cartesian curve	monotonic	monotonic
Cusps	none	may appear	may appear
s is arc length?	no	no	no
Direct trajectory planning	$s = t$	$s = t$	$s = t$

Table 7: Comparison of the three planners for the unicycle (all assume obstacle-free motion).

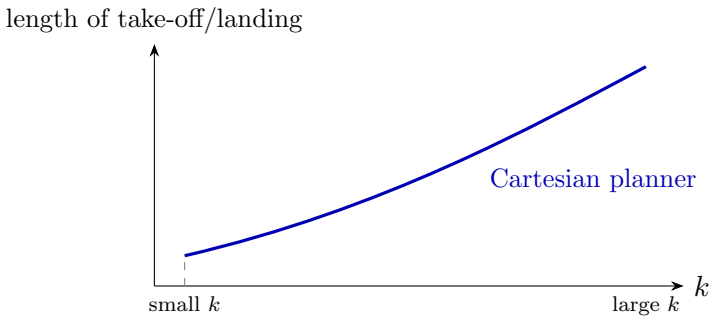


Figure 72: Qualitative trade-off in the Cartesian planner: larger k gives longer straight take-off and landing phases and a possible initial overshoot (qualitative sketch, not measured data).

Key take-aways

- A path is admissible for a nonholonomic WMR iff $A()' = 0$; this is independent of the timing law, which only converts a path into a trajectory ($\boldsymbol{u} = \dot{s}$).
- Differentially flat systems (unicycle, bicycle, all WMRs up to dimension 4) can be planned by interpolating the flat outputs and reconstructing the rest: admissibility comes for free.
- For the unicycle, a cubic Cartesian Hermite curve with tangent length k gives a closed-form planner; the sign of k selects forward or backward motion and a large k elongates take-off and landing.
- In chained form, either interpolate z_1, z_3 or parametrize $_2$ and solve a linear system; both fail when $\theta_i = \theta_f$ and need a via point or a 2π turn, and both produce monotonically varying orientation.
- In none of the methods does s represent the arc length, and every scheme becomes a direct trajectory planner by setting $s = t$.

Part IX

Lecture 7

These notes work out in full three exercises on the kinematic modelling, controllability, flatness, planning and control of nonholonomic robots:

- the single-legged **hopping robot** in flight phase (Midterm 2022/23, Problem 1);
- **planning for a** $(2,3)$ **chained form** with a two-phase algorithm (Midterm 2021/22, Problem 2);
- the **front-wheel-drive bicycle**: kinematic model, controllability, (non-)flatness of the front wheel position and dynamic feedback linearization (Midterm 2024/25, Problem 1).

Each problem is solved part by part. Every non-obvious step is followed by a blue *Intuition* box.

1 Toolbox: the Recurring Ideas

All three exercises are driftless nonholonomic systems, so the same few ideas keep returning. We collect them here.

1.1 Pfaffian constraints and kinematic models

A *kinematic constraint* is linear in the generalized velocities,

$$A^T(q)\,\dot{q} = 0, \qquad A(q) \in \mathbb{R}^{n\times k}, \tag{34}$$

where $n = \dim q$ and k is the number of (independent) constraints. All admissible velocities lie in the $(n-k)$-dimensional null space of $A^T(q)$. Choosing a basis $g_1(q), \dots, g_{n-k}(q)$ of this null space gives the *kinematic model*

$$\dot{q} = \sum_{j=1}^{n-k} g_j(q)\, u_j = G(q)\, u. \tag{35}$$

Intuition

The constraint says "you may not move in these k directions" (a wheel cannot skid sideways; angular momentum cannot be created from nothing). What is left is a space of allowed velocities, and the kinematic model simply parametrizes it with $n-k$ free numbers u_j. The basis is *not unique*: any invertible u-reparametrization gives an equivalent model. The good choice is the one in which the u_j are quantities you can actually command, for instance joint velocities.

1.2 Controllability by Lie brackets

For driftless systems, controllability is decided by the *accessibility rank condition*: if the involutive closure of $\operatorname{span}\{g_1, \dots, g_{n-k}\}$ (obtained by adding Lie brackets $[g_i, g_j] = \frac{\partial g_j}{\partial q} g_i - \frac{\partial g_i}{\partial q} g_j$, then brackets of brackets, ...) has dimension n at every q, the system is controllable (Chow's theorem). If a constraint is *nonholonomic* it is not integrable, so no loss of reachable configurations occurs: local velocity restrictions do not restrict global mobility.

Intuition

Think of parallel parking. You cannot slide the car sideways (a velocity restriction), yet you can reach any position by wiggling forward and backward while steering. The Lie bracket

$[_i,_j]$ is the direction obtained by the sequence "move along $_i$, then $_j$, then back along $_i$, then back along $_j$"; to first order the net displacement is along the bracket. Brackets are therefore the "wiggle directions" that are not available directly.

1.3 Differential flatness and reconstruction

A system $\dot{} = G()u$ is *differentially flat* with *flat outputs* y if y has as many components as there are inputs, and and u can be reconstructed *algebraically* (i.e. without integrating any differential equation) from y and a finite number of its derivatives. For driftless systems, the *geometric model* is obtained by replacing $\dot{(\cdot)}$ with $(\cdot)' = \mathrm{d}(\cdot)/\mathrm{d}s$ with respect to a path parameter s, and inputs u by geometric inputs $\tilde{u}$ (so that $u = \tilde{u}\,\dot{s}$). If y is flat, a feasible path is planned by interpolating $y(s)$ with sufficiently smooth functions (e.g. polynomials) that match the boundary conditions, then reconstructing (s) and $\tilde{u}(s)$.

Intuition

"Flat" means the whole robot behaviour is encoded in a few free functions. Once you pick the curves $y(s)$ freely, nothing else is left to choose: the remaining states and inputs follow by differentiating and solving algebraic equations. This turns a planning problem with a non-holonomic constraint into a simple interpolation problem. The condition to check is whether the *hidden* state variables can be recovered from y without knowing an initial condition. If recovering a variable requires solving an ODE, it carries an arbitrary initial condition and y is *not* flat.

1.4 Dynamic feedback linearization

When the outputs to be controlled are such that the input does not appear in their first derivatives (or a static decoupling matrix is singular), one may *extend the dynamics* by integrating some inputs. If after the extension the *decoupling matrix* in

$$y^{(r)} = (,\xi) + (,\xi)\,\tilde{u} \tag{36}$$

is invertible, the choice $\tilde{u} = ^{-1}\,(w-)$ turns the system into $y^{(r)} = w$, i.e. decoupled chains of integrators, for which linear PD+feedforward tracking is available.

2 The Single-Legged Hopping Robot

(Midterm Class Test 2022/23, Problem 1)

Problem statement A simplified model of a single-legged hopping robot consists of a body with inertia moment J and a leg of mass m. The leg is connected to the body through a revolute joint and a prismatic joint, which are separated by a distance d. Both joints are actuated; in particular, the extension of the length can be changed via the prismatic joint actuator.

When the robot is in flight phase, angular momentum is conserved. This leads to the following constraint

$$J\dot{\theta} + m(d+\ell)^2\dot{\phi} = 0,$$

where we have assumed that the leg mass is concentrated at its extremity (foot) and the initial momentum is zero. Let the configuration of the robot be defined as $= (\phi, \ell, \theta)$ (the translational motion is not of interest).

(a) Discuss the geometry of the robot configuration space.

(b) Derive a kinematic model of the robot for which there is a clear physical interpretation of the associated inputs.
(c) Discuss local and global mobility of the robot in its configuration space.
(d) Using the kinematic model, find a feedback control law for the flight phase that will drive the body orientation θ to zero and the leg length to a constant value h. Provide a block scheme.
(e) Consider for simplicity the case $d = 0$. Show that the system is differentially flat and derive the associated reconstruction formulas. Based on this, devise an algorithm for planning a feasible path between two arbitrary configurations $_s$ and $_g$.

2.1 Setting

A simplified single-legged hopping robot consists of a body with inertia moment J and a leg of mass m concentrated at its extremity (the foot). The leg is attached to the body through a revolute joint and a prismatic joint, separated by a distance d; both are actuated, so the leg length ℓ can be changed. Let ϕ be the leg angle (relative to the horizontal), ℓ the leg extension and θ the body orientation. Only the orientation-related motion is of interest, so

$$= (\phi, \ell, \theta)^T.$$

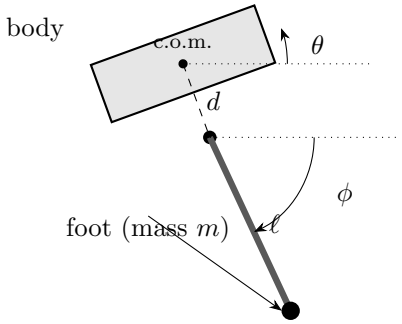


Figure 73: Schematic of the hopping robot: body with inertia J and orientation θ, revolute joint (angle ϕ) and prismatic joint (extension ℓ) separated by d. Original redrawing.

During the flight phase the total angular momentum is conserved. With zero initial momentum and the leg mass concentrated at the foot,

$$J\dot{\theta} + m(d+\ell)^2\dot{\phi} = 0. \tag{37}$$

Intuition

This is the same principle that lets a falling cat turn in mid-air, or an astronaut reorient without touching anything: with no external torque, total angular momentum stays zero, so rotating the leg one way forces the body to rotate the other way. How *much* the body turns depends on the lever arm $(d+\ell)$ of the leg: since $\dot{\theta} = -\frac{m(d+\ell)^2}{J}\dot{\phi}$, a *longer* leg produces *more* body rotation for the same swing. This lever-arm effect is what makes the robot controllable in part (c).

2.2 (a) Geometry of the configuration space

Both ϕ and θ are angles, hence elements of SO(2) (circles), whereas the extension ℓ is a real number (physically $\ell \geq 0$). Therefore

$$\mathcal{C} = \mathrm{SO}(2) \times \mathbb{R} \times \mathrm{SO}(2), \qquad n = \dim \mathcal{C} = 3.$$

There is a single constraint, (37), so $k = 1$.

Intuition

The configuration space is not $\mathbb{R}^3$ but a "cylinder times a circle": angles wrap around (ϕ and $\phi + 2\pi$ are the same configuration). This matters later, when planning: reaching ϕ_g may be done by choosing any representative $\phi_g + 2\pi k$, which changes the path.

2.3 (b) A kinematic model with physically meaningful inputs

Writing (37) in Pfaffian form,

$$\underbrace{\begin{pmatrix} m(d+\ell)^2 & 0 & J \end{pmatrix}}_{A^T()} \begin{pmatrix} \dot{\phi} \\ \dot{\ell} \\ \dot{\theta} \end{pmatrix} = 0. \tag{38}$$

The null space of $A^T()$ has dimension $n - k = 2$. Two vector fields spanning it are

$$_1() = \begin{pmatrix} 1 \\ 0 \\ -\frac{m(d+\ell)^2}{J} \end{pmatrix}, \qquad _2() = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \tag{39}$$

which gives the kinematic model

$$\boxed{\begin{aligned} \dot{\phi} &= u_1 \\ \dot{\ell} &= u_2 \\ \dot{\theta} &= -\frac{m(d+\ell)^2}{J}\, u_1. \end{aligned}} \tag{40}$$

How the basis is found. We want $\dot{\phi}$ and $\dot{\ell}$ to be the inputs, since they are the velocities of the two actuated joints. Hence the first two components of the basis vectors should be $(1, 0)$ and $(0, 1)$. The third component α of $_1 = (1, 0, \alpha)^T$ is fixed by requiring $A_1^T = 0$:

$$m(d+\ell)^2 + J\alpha = 0 \implies \alpha = -\frac{m(d+\ell)^2}{J},$$

while $_2 = (0, 1, 0)^T$ is automatically in the null space since the constraint does not involve $\dot{\ell}$.

Intuition

The model says: command the two joints freely (u_1 swings the leg, u_2 extends it), and the body orientation is *slaved* to the leg swing through the conservation of angular momentum. $\dot{\ell} = u_2$ does not affect θ at all, as extending the leg exerts no torque about the center of mass in this model, but *when* you swing depends on ℓ. Any other basis is equally valid mathematically (e.g. taking $\dot{\theta}$ as an input), but then the inputs would no longer be joint velocities.

2.4 (c) Local and global mobility

Local mobility. At each , the velocity $\dot{}$ must belong to $\mathcal{N}(A^T())$, a 2-dimensional subspace of the 3-dimensional tangent space. So *locally* the robot cannot move in an arbitrary direction: the body cannot rotate independently of the leg.

Global mobility. We check the accessibility rank condition. The only non-trivial bracket is

$$_3() = [_1,_2]() = \frac{\partial_2}{\partial\ _1} - \frac{\partial_1}{\partial\ _2} = -\frac{\partial_1}{\partial\ell} = \begin{pmatrix} 0 \\ 0 \\ \dfrac{2m(d+\ell)}{J} \end{pmatrix}. \tag{41}$$

Its third component never vanishes provided $d + \ell > 0$ (this holds for $\ell \geq 0$ as soon as $d > 0$; if $d = 0$ the point $\ell = 0$ is singular, where the leg has no lever arm). Hence

$$\begin{pmatrix} _1 & _2 & _3 \end{pmatrix} = 3,$$

and the kinematic model is *controllable*. In other words, the angular-momentum conservation constraint is *nonholonomic*: global mobility is not restricted, and every configuration of $\mathcal{C}$ can be reached.

Intuition

Here is the concrete "wiggle" behind the bracket. Take $m/J = 1$, $d = 0$. Extend the leg to $\ell_2 = 2$, swing it by $\Delta\phi = 0.5\,\text{rad}$, retract to $\ell_1 = 1$, and swing back by $-0.5\,\text{rad}$. The body rotation produced while swinging out is $-\ell_2^2\Delta\phi = -2\,\text{rad}$; the one while swinging back is $+\ell_1^2\Delta\phi = +0.5\,\text{rad}$. The leg is back at the same (ϕ, ℓ), but the body has rotated by

$$\Delta\theta = -\frac{m}{J}\Delta\phi\left[(d+\ell_2)^2 - (d+\ell_1)^2\right] = -0.5\,(4-1) = -1.5\ \text{rad}.$$

So a closed cycle in the leg variables (ϕ, ℓ) produces a net rotation of the body: this is why one can reorient arbitrarily in flight (exactly like the cat), and it is the geometric meaning of $_3$ pointing along θ.

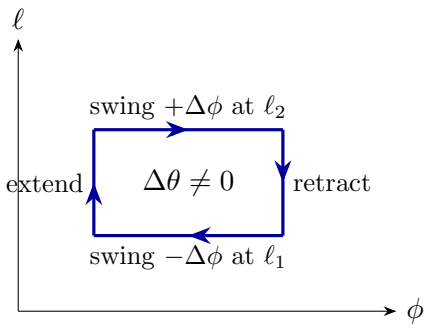


Figure 74: A rectangular loop in the leg variables (ϕ, ℓ) leaves the leg where it started but produces a net body rotation $\Delta\theta$ (nonholonomy).

2.5 (d) Feedback control in flight: $\theta \to 0$, $\ell \to h$

In the model (40) the leg length is a pure integrator, $\dot{\ell} = u_2$. Choosing

$$u_2 = k_\ell\,(h - d - \ell), \qquad k_\ell > 0, \tag{42}$$

gives $\dot{\ell} = -k_\ell(\ell - (h-d))$, i.e. exponential convergence of ℓ to $h - d$, so that the total leg length $d + \ell$ converges to the desired h.

For θ, introduce the input transformation (always invertible, since $d + \ell > 0$)

$$u_1 = -\frac{J}{m(d+\ell)^2}\,v_1,$$

under which $\dot\theta = v_1$. Taking $v_1 = -k_\theta\theta$, $k_\theta > 0$, yields exponential convergence of θ to zero. Going back to the original input:

$$u_1 = k_\theta \frac{J}{m(d+\ell)^2}\,\theta. \tag{43}$$

The control law (42)–(43) only needs measurements of ℓ and θ (not ϕ).

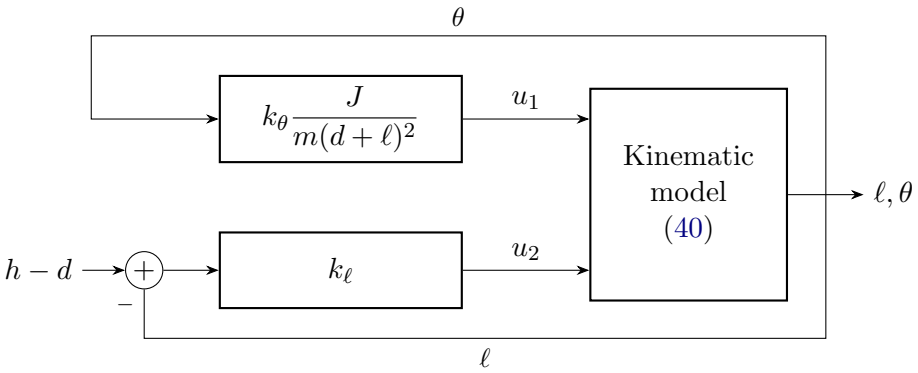


Figure 75: Block scheme of the flight-phase controller: two decoupled loops. The θ-loop applies the state-dependent gain $k_\theta J/(m(d+\ell)^2)$ to θ (regulation to zero); the ℓ-loop is a proportional regulator on $h - d - \ell$.

Intuition

The trick is to make each variable look like a trivial first-order system $\dot x = -kx$. For ℓ this is immediate. For θ, the factor $\frac{m(d+\ell)^2}{J}$ is a (known, positive) "gear ratio" between the swing velocity u_1 and the body rotation rate, so we simply divide by it. The price is that ϕ is left uncontrolled: it drifts. How far? If ℓ were constant, momentum conservation gives $J\Delta\theta = -m(d+\ell)^2\Delta\phi$. Straightening the body from $\theta_0 = 0.3\,\text{rad}$ with $d + \ell = h = 0.5\,\text{m}$, $m/J = 2\,\text{m}^{-2}$ requires the leg to rotate by

$$\Delta\phi = \frac{J}{mh^2}\theta_0 = \frac{0.3}{2 \cdot 0.25} = 0.6 \text{ rad}.$$

2.6 (e) Differential flatness for $d = 0$ and path planning

For $d = 0$ the model (40) becomes

$$\begin{aligned} \dot\phi &= u_1 \\ \dot\ell &= u_2 \\ \dot\theta &= -\frac{m\ell^2}{J}u_1. \end{aligned} \tag{44}$$

Proposition 2.1. *System (44) is differentially flat with flat outputs (ϕ, θ).*

Proof. Using $u_1 = \dot\phi$ in the last equation gives $\ell^2 = -\frac{J}{m}\frac{\dot\theta}{\dot\phi}$. Since $\ell \geq 0$,

$$\ell = \sqrt{-\frac{J}{m}\frac{\dot\theta}{\dot\phi}}. \tag{45}$$

The inputs follow immediately:

$$u_1 = \dot\phi, \qquad u_2 = \dot\ell = \frac{1}{2\ell}\left(-\frac{J}{m}\right)\frac{\ddot\theta\dot\phi - \dot\theta\ddot\phi}{\dot\phi^2} = \frac{J}{2m\ell}\,\frac{\dot\theta\ddot\phi - \ddot\theta\dot\phi}{\dot\phi^2}, \tag{46}$$

with ℓ given by (45). All reconstruction formulas are algebraic in (ϕ, θ) and their derivatives (up to second order), so (ϕ, θ) is a flat output. □

Constraint (37) implies that $\dot{\theta}$ and $\dot{\phi}$ always have *opposite* signs, so the radicand in (45) is nonnegative: the square root is well-posed, and $\ell > 0$ is consistent with the constraint. The formulas require $\dot{\phi} \neq 0$.

Intuition

Why is ℓ *not* a flat output while θ is? Because ℓ can be read off from the *ratio* of how fast the body turns to how fast the leg swings: a long leg turns the body a lot per unit swing. So watching $\phi(t)$ and $\theta(t)$ is enough to infer the leg length at every instant, and then its rate, and the input.

Planning. For path planning we use the geometric version of (44), where $(\cdot)' = \mathrm{d}/\mathrm{d}s$:

$$\phi' = \tilde{u}_1, \qquad \ell' = \tilde{u}_2, \qquad \theta' = -\frac{m\ell^2}{J}\tilde{u}_1. \tag{47}$$

Time has disappeared: only the *shape* of the path matters. The reconstruction formulas hold with s-derivatives:

$$\ell = \sqrt{-\tfrac{J}{m}\tfrac{\theta'}{\phi'}}, \qquad \tilde{u}_1 = \phi', \qquad \tilde{u}_2 = \ell' = \frac{J}{2m\ell}\,\frac{\theta'\phi'' - \theta''\phi'}{\phi'^2}.$$

Algorithm: path planning between $_s = (\phi_s, \ell_s, \theta_s)$ and $_g = (\phi_g, \ell_g, \theta_g)$ $(d = 0)$ [1] Choose a **linear** polynomial $\phi(s) = as + b$, $s \in [0, 1]$, with $\phi(0) = \phi_s$, $\phi(1) = \phi_g$. Hence $b = \phi_s$, $a = \phi_g - \phi_s$, and $\tilde{u}_1(s) = \phi'(s) = a$ for all s. Choose a **cubic** polynomial $\theta(s) = cs^3 + ds^2 + es + f$ with $\theta(0) = \theta_s$, $\theta(1) = \theta_g$, $\theta'(0) = -\dfrac{m\,\ell_s^2}{J}\,a$, $\theta'(1) = -\dfrac{m\,\ell_g^2}{J}\,a$. Reconstruct $\ell(s) = \sqrt{-\tfrac{J}{m}\tfrac{\theta'(s)}{a}}$ and $\tilde{u}_2(s) = \ell'(s)$. (Optional) Pick a time law $s(t)$ to obtain velocities $u = \tilde{u}\,\dot{s}$.

The boundary conditions on θ' come from the last equation of (47) with $\tilde{u}_1 = a$: they *encode the initial and final leg lengths*. Solving the four linear conditions on the cubic, with $\Delta\theta = \theta_g - \theta_s$ and θ'_s, θ'_g the two prescribed slopes,

$$f = \theta_s, \quad e = \theta'_s, \quad d = 3\Delta\theta - 2\theta'_s - \theta'_g, \quad c = \theta'_s + \theta'_g - 2\Delta\theta.$$

Intuition

Why a line for ϕ and a cubic for θ? ϕ only has two boundary conditions (start and end value), so a straight line suffices, and conveniently gives a *constant* geometric input a. θ has four conditions (two values and two slopes), so we need a cubic. The two slope conditions are what carry the information about ℓ_s and ℓ_g: they say "the body must be turning, at the start, at the rate corresponding to leg length ℓ_s, and at the end at the rate corresponding to ℓ_g". Once $\theta(s)$ is fixed, the leg length in between is whatever is needed to make the constraint hold.

[Feasibility check] Since $\ell^2 = -\frac{J}{m}\frac{\theta'}{a}$ must be positive along the whole path, the cubic must satisfy $a\,\theta'(s) < 0$ for all $s \in [0, 1]$: its derivative must not change sign. The boundary slopes already have the right sign; if the interior of θ' crosses zero, the plan is not feasible and one must modify it (e.g. choose $\phi_g + 2\pi k$, i.e. a different number of leg turns, or use a higher-degree polynomial). If $\phi_s = \phi_g$ then $a = 0$ and the formulas degenerate (a nonconstant $\phi(s)$ must be chosen).

Example 2.2 (Numerical example)**.** Take $m/J = 1$, $\phi : 0 \to 1$ (so $a = 1$), $\ell_s = 1$, $\ell_g = 1.5$, and $\theta_s = 0$, $\theta_g = -2$. Then $\theta'_s = -1$, $\theta'_g = -2.25$, $\Delta\theta = -2$, so

$$c = -1 - 2.25 + 4 = 0.75, \quad d = -6 + 2 + 2.25 = -1.75, \quad e = -1, \quad f = 0,$$

$$\theta(s) = 0.75s^3 - 1.75s^2 - s, \qquad \ell(s) = \sqrt{-\theta'(s)} = \sqrt{1 + 3.5s - 2.25s^2}.$$

Here $\theta'(s) = 2.25s^2 - 3.5s - 1$ has its roots at $s \approx -0.25$ and $s \approx 1.8$, outside $[0, 1]$, so $\theta' < 0$ throughout and the path is feasible; indeed $\ell(0) = 1$ and $\ell(1) = 1.5$.

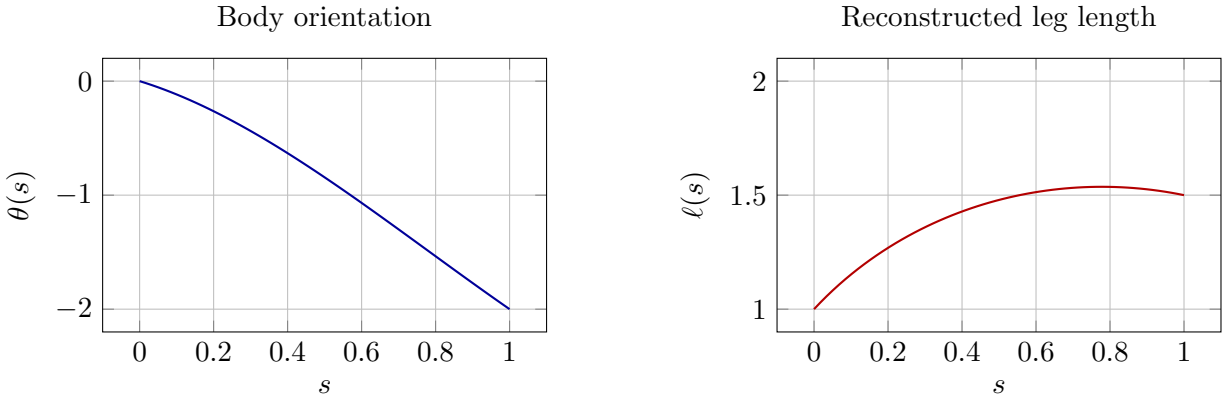


Figure 76: Planned path for the numerical example: the cubic $\theta(s)$ and the leg length $\ell(s)$ reconstructed from it. Note how $\ell(s)$ is never freely chosen: it is imposed by θ.

3 Planning for a $(2, 3)$ Chained Form

(Midterm Class Test 2021/22, Problem 2)

Problem statement Consider a $(2, 3)$ chained form. It is easy to verify that the application of the control inputs

$$v_1 = a \sin \omega t, \qquad v_2 = b \cos \omega t$$

over the time interval $[0, 2\pi/\omega]$ brings z_1, z_2 back to their initial values and causes a displacement of z_3 equal to $ab\pi/\omega^2$. Exploiting this fact (no need to prove it), design a 2-phase algorithm for planning a trajectory that moves the chained form system from any initial configuration $_i$ to any final configuration $_f$. Moreover:

(a) provide a pseudocode description of the algorithm;
(b) discuss which are its free parameters and how to choose their values;
(c) assuming $_i = (0, 0, 0)$ and $_f = (1, 1, 1)$, compute the total time needed for reaching $_f$ and draw the trajectory of the system on the z_1, z_2 plane.

3.1 Setting

The $(2, 3)$ chained form has two inputs and three states:

$$\dot{z}_1 = v_1, \qquad \dot{z}_2 = v_2, \qquad \dot{z}_3 = z_2 \, v_1. \tag{48}$$

We are given the fact that the sinusoidal inputs

$$v_1 = a \sin \omega t, \qquad v_2 = b \cos \omega t$$

applied over $[0, 2\pi/\omega]$ bring z_1, z_2 back to their initial values and produce a displacement $\Delta z_3 = ab\pi/\omega^2$. We exploit it to design a two-phase algorithm moving $_i = (z_{1,i}, z_{2,i}, z_{3,i})$ to any $_f = (z_{1,f}, z_{2,f}, z_{3,f})$.

Controllability check. With $_1 = (1, 0, z_2)^T$ and $_2 = (0, 1, 0)^T$ we get $[_{1,2}] = -\partial_1/\partial z_2 = (0, 0, -1)^T$, so $(_{1,2}, [_{1,2}]) = 3$ everywhere: the system is controllable, so the planning problem is always solvable.

Intuition

The first two equations are *pure integrators*: steering (z_1, z_2) anywhere is trivial. The only difficulty is z_3, which is not directly driven: it accumulates $z_2\,\dot{z}_1$. By Green's theorem, over a closed loop in the (z_1, z_2) plane

$$\Delta z_3 = \oint z_2\,\mathrm{d}z_1 = \pm \text{ (area enclosed by the loop)},$$

with + for a clockwise loop. The sinusoidal inputs trace an ellipse with semi-axes a/ω and b/ω, whose area is $\pi\,\frac{a}{\omega}\,\frac{b}{\omega} = \frac{ab\pi}{\omega^2}$, exactly the given displacement. So z_3 is "the area you have swept": *to change z_3, go around a loop in the z_1–z_2 plane.* This is the same idea as the $\Delta\theta$ produced by the leg cycle of the hopping robot in Section 2, and of parallel parking.

3.2 (a) The algorithm

Phase 1 (move z_1, z_2). Drive z_1, z_2 to $z_{1,f}, z_{2,f}$ in a finite time T_1, ignoring z_3. At the end, z_3 has some value $z_3(T_1)$.

Phase 2 (fix z_3). Apply the sinusoidal inputs, shifted in time, with parameters chosen so that the displacement of z_3 equals $z_{3,f} - z_3(T_1)$. Since z_1, z_2 return to $z_{1,f}, z_{2,f}$, the final configuration is $_f$.

Phase 1 with sign inputs. Set $v_1 = (z_{1,f} - z_{1,i})$, $v_2 = (z_{2,f} - z_{2,i})$ for $t \in [0, T_1]$, with $(0) = 0$ and $T_1 = \max\{|z_{1,f} - z_{1,i}|, |z_{2,f} - z_{2,i}|\}$.

[Caution on the open-loop sign inputs] With a *constant* unit-magnitude input over a common time T_1, the component with the smaller displacement would overshoot its target: this version is exact only if $|\Delta z_1| = |\Delta z_2|$ (as in part (c)). Two ways to fix it:

- **feedback sign controller**: $v_1 = (z_{1,f} - z_1)$, $v_2 = (z_{2,f} - z_2)$, so that each input switches off *automatically* when $(0) = 0$ is reached (each component stops at its target, in time $|\Delta z_i|$);
- **time scaling**: $v_i = \Delta z_i/T_1$ over $[0, T_1]$ with any chosen $T_1 > 0$, so that both components arrive simultaneously. This also yields the closed form

$$z_3(T_1) = z_{3,i} + z_{2,i}\Delta z_1 + \tfrac{1}{2}\Delta z_1 \Delta z_2, \tag{49}$$

  obtained by integrating $\dot{z}_3 = z_2 v_1$.

A third alternative is a **proportional feedback** $v_1 = k_1(z_{1,f} - z_1)$, $v_2 = k_2(z_{2,f} - z_2)$, which is smoother but converges only asymptotically; one then switches to Phase 2 once the error is below a threshold, and uses the *measured* value of z_3 to compute the Phase 2 parameters, which makes the scheme robust to the imperfect Phase 1.

Algorithm: two-phase planner for the $(2, 3)$ chained form [1] $_i = (z_{1,i}, z_{2,i}, z_{3,i})$, $_f = (z_{1,f}, z_{2,f}, z_{3,f})$ **Phase 1** $\Delta_1 \leftarrow z_{1,f} - z_{1,i}$, $\Delta_2 \leftarrow z_{2,f} - z_{2,i}$, $T_1 \leftarrow \max(|\Delta_1|, |\Delta_2|)$ any $T_1 > 0$ if time-scaled apply $v_1 = (\Delta_1)$, $v_2 = (\Delta_2)$ on $[0, T_1]$ (or the feedback/scaled variant) $z_3(T_1) \leftarrow$ integrate $\dot{z}_3 = z_2 v_1$ (closed form (49) for the scaled variant) **Phase 2** $\Delta_3 \leftarrow z_{3,f} - z_3(T_1)$ choose $\omega > 0$

and $a \neq 0$; set $b \leftarrow \dfrac{\omega^2 \Delta_3}{\pi a}$ so that $ab\pi/\omega^2 = \Delta_3$ apply $v_1 = a \sin \omega(t - T_1)$, $v_2 = b \cos \omega(t - T_1)$ for $t \in [T_1,\, T_1 + 2\pi/\omega]$ total time $T = T_1 + 2\pi/\omega$

3.3 (b) Free parameters and how to choose them

Phase 1 contains no parameters (apart from the optional T_1 in the scaled variant). Phase 2 has three parameters (a, b, ω) subject to *one* constraint $ab\pi/\omega^2 = z_{3,f} - z_3(T_1)$, so two remain free.

- a, b are the **input amplitudes**: they should be chosen compatibly with the bounds on the available inputs. They also shape the trajectory: (z_1, z_2) traces an ellipse with semi-axes a/ω (along z_1) and b/ω (along z_2), so $|a| > |b|$ gives an ellipse elongated along z_1 and $|a| < |b|$ along z_2.
- ω sets the **duration** $2\pi/\omega$ of Phase 2, hence the total time.
- The sign of ab is fixed by the sign of Δ_3: $\Delta_3 > 0$ requires $ab > 0$ (clockwise loop), $\Delta_3 < 0$ requires $ab < 0$ (counter-clockwise).

Intuition

There is a trade-off between speed and effort. For the product ab fixed by the area to be swept, the peak input $\max(|a|, |b|)$ is smallest when $|a| = |b|$ (a circle): any elongation raises one amplitude. For $|a| = |b|$ the area constraint gives $a^2 = \Delta_3\, \omega^2/\pi$, i.e. $a \propto \omega$: going faster (larger ω) needs proportionally larger inputs. With a bound $v_{\max}$ on the inputs, the fastest feasible loop has $|a| = v_{\max}$.

3.4 (c) The case $_i = (0, 0, 0)$, $_f = (1, 1, 1)$

Phase 1: $\Delta_1 = \Delta_2 = 1$, so $T_1 = \max(1, 1) = 1$. The inputs $v_1 = v_2 = 1$ drive (z_1, z_2) along the straight line from $(0, 0)$ to $(1, 1)$ (here the open-loop sign version is exact). At the end,

$$z_3(T_1) = \int_0^1 z_2 v_1 \, \mathrm{d}t = \int_0^1 t \, \mathrm{d}t = \tfrac{1}{2},$$

in agreement with (49). Phase 2 must therefore produce $\Delta_3 = 1 - \frac{1}{2} = \frac{1}{2}$, i.e. $ab\pi/\omega^2 = \frac{1}{2}$.
With ω chosen arbitrarily, the total time is

$$T = 1 + \frac{2\pi}{\omega}.$$

In the (z_1, z_2) plane the trajectory is the segment from $(0, 0)$ to $(1, 1)$, followed by the ellipse of Phase 2 closing at $(1, 1)$. For instance, with $a = b$ one has $a = \omega/\sqrt{2\pi}$ and the loop is a circle of radius $a/\omega = 1/\sqrt{2\pi} \approx 0.40$ centered at $(1 + 0.40,\, 1)$; with $\omega = 2\pi$ the amplitudes are $a = b = \sqrt{2\pi} \approx 2.51$ and the total time is $T = 2$.

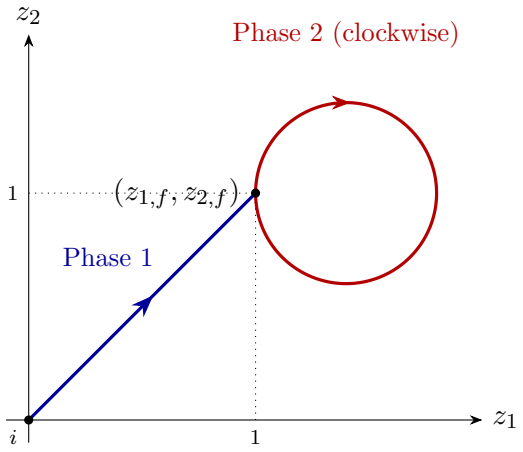


Figure 77: Trajectory in the (z_1, z_2) plane for $_i = (0,0,0)$, $_f = (1,1,1)$ with $a = b$: a line (Phase 1) followed by a circle (Phase 2). The area enclosed by the circle, $\pi/(2\pi) = 1/2$, is exactly the missing Δz_3.

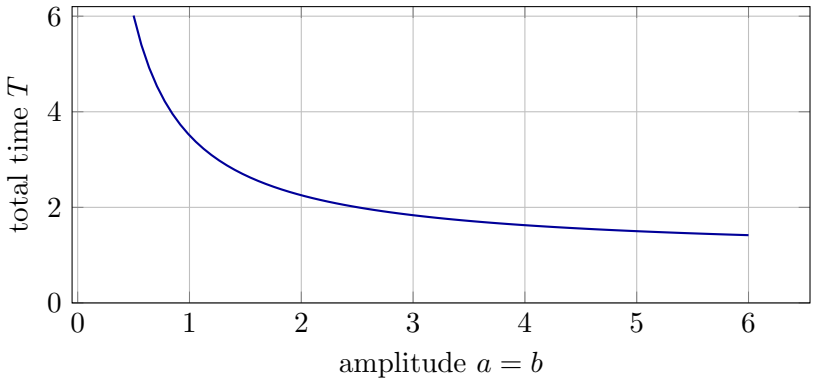


Figure 78: Trade-off for $_i = (0,0,0) \to_f = (1,1,1)$ with $a = b$: $T = 1 + \sqrt{2\pi}/a$. Larger input amplitude shortens Phase 2, which can never be shorter than zero, so $T \to T_1 = 1$.

4 Front-Wheel-Drive Bicycle: Modelling, Flatness and Control

(Midterm Class Test 2024/25, Problem 1)

Problem statement Consider a bicycle robot of length ℓ with front-wheel drive. Define the configuration vector as $= (x_f, y_f, \theta, \phi)$, where (x_f, y_f) are the Cartesian coordinates of the (contact point of the) front wheel, θ is the vehicle orientation, and ϕ is the steering angle of the front wheel.

(a) Starting from the kinematic constraints acting on the system, derive a kinematic model of the vehicle with the front wheel driving and steering velocities as control inputs, and prove its controllability.

(b) Are x_f, y_f flat outputs for this model? Motivate your answer and give an interpretation in terms of state/input reconstructability.

(c) Design a feedback controller for driving x_f, y_f along a desired trajectory $x_f^*(t), y_f^*(t)$ and provide the corresponding block scheme. *Hint: find an invertible mapping between the time derivative (of a suitable order) of the outputs and the dynamically extended control inputs.*

4.1 Setting

Consider a bicycle robot of length ℓ with front-wheel drive and configuration vector $= (x_f, y_f, \theta, \phi)^T$, where (x_f, y_f) are the Cartesian coordinates of the contact point of the front wheel, θ is the vehicle orientation and ϕ is the steering angle of the front wheel.

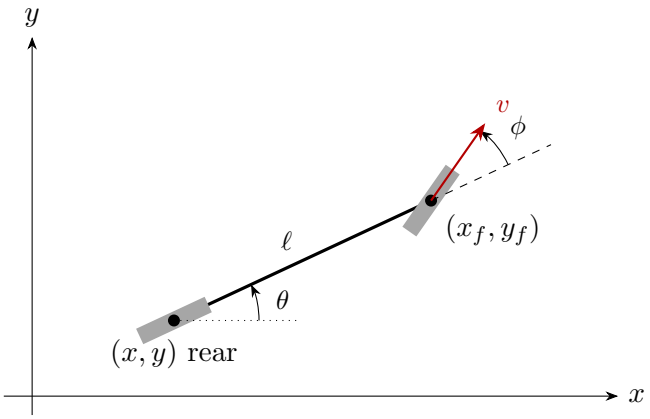


Figure 79: Front-wheel-drive bicycle. The front wheel has orientation $\theta + \phi$ and carries both drive (v) and steering ($\omega = \dot{\phi}$) inputs.

4.2 (a) Kinematic model and controllability

Constraints. There is one pure-rolling condition per wheel (no lateral slip):

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0 \qquad \text{(rear wheel)} \tag{50}$$

$$\dot{x}_f\sin(\theta+\phi) - \dot{y}_f\cos(\theta+\phi) = 0 \qquad \text{(front wheel)}, \tag{51}$$

where (x, y) are the coordinates of the rear wheel contact point, related to the front one by

$$x = x_f - \ell\cos\theta, \qquad y = y_f - \ell\sin\theta.$$

Differentiating, $\dot{x} = \dot{x}_f + \ell\dot{\theta}\sin\theta$, $\dot{y} = \dot{y}_f - \ell\dot{\theta}\cos\theta$, and substituting into the rear constraint ($\sin^2\theta + \cos^2\theta = 1$):

$$\dot{x}_f\sin\theta - \dot{y}_f\cos\theta + \ell\dot{\theta} = 0.$$

In Pfaffian form, with $= (x_f, y_f, \theta, \phi)$,

$$A^T(\dot{)} = \begin{pmatrix} \sin\theta & -\cos\theta & \ell & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & 0 & 0 \end{pmatrix} \begin{pmatrix} \dot{x}_f \\ \dot{y}_f \\ \dot{\theta} \\ \dot{\phi} \end{pmatrix} = 0. \tag{52}$$

Here $n = 4$ and $k = 2$, so the null space is 2-dimensional.

Null-space basis and model. A basis is

$$_1() = \begin{pmatrix} \cos(\theta+\phi) \\ \sin(\theta+\phi) \\ \frac{\sin\phi}{\ell} \\ 0 \end{pmatrix}, \qquad _2() = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

(Check: the second row of A^T annihilates $_1$ because $\sin(\theta+\phi)\cos(\theta+\phi)-\cos(\theta+\phi)\sin(\theta+\phi)=0$; the first row gives $\sin\theta\cos(\theta+\phi)-\cos\theta\sin(\theta+\phi)+\sin\phi=-\sin\phi+\sin\phi=0$.) The kinematic model is

$$\boxed{\begin{aligned}\dot{x}_f &= \cos(\theta+\phi)\,v\\ \dot{y}_f &= \sin(\theta+\phi)\,v\\ \dot{\theta} &= \frac{\sin\phi}{\ell}\,v\\ \dot{\phi} &= \omega\end{aligned}} \tag{53}$$

where v and ω are the driving and steering velocities, both generated at the front wheel.

Intuition

The front wheel can only move along its own heading $\theta+\phi$ (this is v). The rear wheel must follow, it cannot slip sideways, so the body turns at a rate $\frac{v\sin\phi}{\ell}$: it is the component of v perpendicular to the body, divided by the lever arm ℓ. If $\phi=0$ the vehicle goes straight; the larger the steering angle, the tighter the turn; at $\phi=\pm\pi/2$ the front wheel moves perpendicular to the body and the body pivots about the rear wheel. Steering is a separate, directly actuated velocity $\omega=\dot{\phi}$.

Controllability. We compute the Lie brackets:

$$_3 = [_1,_2] = \begin{pmatrix}\sin(\theta+\phi)\\ -\cos(\theta+\phi)\\ -\frac{\cos\phi}{\ell}\\ 0\end{pmatrix}, \qquad _4 = [_1,_3] = \begin{pmatrix}-\frac{\sin\theta}{\ell}\\ \frac{\cos\theta}{\ell}\\ 0\\ 0\end{pmatrix}.$$

Then, reordering the columns,

$$\begin{pmatrix}_1 & _3 & _4 & _2\end{pmatrix} = \begin{pmatrix}\cos(\theta+\phi) & \sin(\theta+\phi) & -\frac{\sin\theta}{\ell} & 0\\ \sin(\theta+\phi) & -\cos(\theta+\phi) & \frac{\cos\theta}{\ell} & 0\\ \frac{\sin\phi}{\ell} & -\frac{\cos\phi}{\ell} & 0 & 0\\ 0 & 0 & 0 & 1\end{pmatrix} = 4,$$

because the determinant of the upper-left 3×3 block is $1/\ell^2\neq0$ (expand along the third column and use $\sin(\theta+\phi)\cos\theta-\cos(\theta+\phi)\sin\theta=\sin\phi$ and $\cos(\theta+\phi)\cos\theta+\sin(\theta+\phi)\sin\theta=\cos\phi$). The rank is 4 for every , so the kinematic model is *controllable*.

Intuition

The bracket $_3$ is the "wiggle the steering" direction (drive a bit, steer, drive back, steer back), and $_4$ is the "lateral" direction perpendicular to the rear wheel. This is precisely the sideways motion that the constraint forbids directly but parallel parking obtains: the determinant $1/\ell^2$ never vanishes, so there is no configuration (not even $\phi=\pm\pi/2$) where this fails.

4.3 (b) Are x_f, y_f flat outputs?

What can be reconstructed. From the first two equations of (53),

$$\theta+\phi=\arctan\frac{\dot{y}_f}{\dot{x}_f}, \qquad v=\pm\sqrt{\dot{x}_f^2+\dot{y}_f^2}$$

(the sign of v distinguishes forward from backward motion), and differentiating the first,

$$\dot{\theta}+\dot{\phi}=\frac{\ddot{y}_f\dot{x}_f-\dot{y}_f\ddot{x}_f}{\dot{x}_f^2+\dot{y}_f^2}=:\kappa, \qquad \text{so} \quad \dot{\theta}=\kappa-\dot{\phi}. \tag{54}$$

The obstruction. From the third equation of (53), $\sin\phi = \ell\dot{\theta}/v$, hence

$$\phi = \arcsin\left[\frac{\ell}{v}\left(\frac{\ddot{y}_f\dot{x}_f - \dot{y}_f\ddot{x}_f}{\dot{x}_f^2 + \dot{y}_f^2} - \dot{\phi}\right)\right], \qquad \text{equivalently} \qquad \dot{\phi} = \kappa - \frac{v}{\ell}\sin\phi. \tag{55}$$

The right-hand side contains $\dot{\phi}$ itself: ϕ is the solution of a *differential equation* driven by the outputs, not an algebraic function of them. Therefore ϕ (and consequently $\theta = \arctan(\dot{y}_f/\dot{x}_f) - \phi$) *cannot be reconstructed algebraically* from x_f, y_f and their derivatives:

$$\boxed{x_f, y_f \text{ are } \textit{not} \text{ flat outputs.}}$$

Interpretation (state/input reconstructability). Along a given front-wheel trajectory, v and the heading $\theta + \phi$ of the front wheel are known, but the *split* of this heading between body orientation θ and steering ϕ depends on the initial condition of (55). For the same front-wheel path there are infinitely many possible rear-wheel paths, one per initial condition.

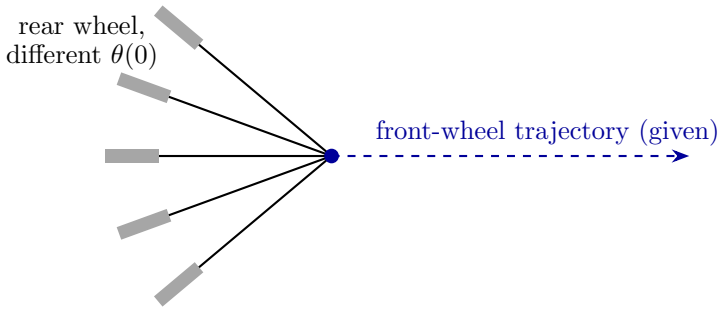


Figure 80: The same front-wheel path is compatible with infinitely many rear-wheel configurations, depending on the initial body orientation: this is why (x_f, y_f) do not determine the state.

Intuition

It is a matter of what the front point alone can tell you. Suppose you only watch the front wheel moving along a straight line: is the car perfectly aligned behind it, or is the body at an angle, with the steering compensating? Both are consistent with the same front-wheel path. Which one is true is "memory" stored in ϕ, and memory means integration, not algebra. Equation (55) adds a further insight: for forward motion $v > 0$ and $|\phi| < \pi/2$, the linearization $-\frac{v}{\ell}\cos\phi$ is negative, so the ϕ-dynamics are *stable* (the body aligns behind the front wheel, like a shopping cart caster); for backward motion the same ODE is *unstable* (jackknifing).

A different choice of output *does* work: for the bicycle with front-wheel drive, the Cartesian position of the *rear* wheel is a flat output, because θ is recovered from the direction of its velocity (the rear wheel moves along the body axis), and ϕ, v then follow algebraically. The non-flatness above is specific to the front-wheel point.

4.4 (c) Tracking controller for (x_f, y_f): dynamic feedback linearization

Differentiate the outputs. The first derivatives are $\dot{x}_f = \cos(\theta+\phi)v$, $\dot{y}_f = \sin(\theta+\phi)v$: only v appears, so no decoupling is possible at this order (one input, two outputs). Differentiating again,

$$\begin{aligned}\ddot{x}_f &= -\sin(\theta+\phi)(\dot{\theta}+\dot{\phi})\,v + \cos(\theta+\phi)\,\dot{v},\\ \ddot{y}_f &= \;\;\cos(\theta+\phi)(\dot{\theta}+\dot{\phi})\,v + \sin(\theta+\phi)\,\dot{v}.\end{aligned}$$

Dynamic extension. Set $\dot{v} = a$ (the acceleration becomes the input, and v becomes a state), and replace $\dot{\theta} = \frac{\sin\phi}{\ell} v$, $\dot{\phi} = \omega$:

$$\begin{pmatrix} \ddot{x}_f \\ \ddot{y}_f \end{pmatrix} = \underbrace{\begin{pmatrix} -\sin(\theta+\phi)\frac{\sin\phi}{\ell} v^2 \\ \cos(\theta+\phi)\frac{\sin\phi}{\ell} v^2 \end{pmatrix}}_{(\,,v)} + \underbrace{\begin{pmatrix} \cos(\theta+\phi) & -\sin(\theta+\phi)\, v \\ \sin(\theta+\phi) & \cos(\theta+\phi)\, v \end{pmatrix}}_{(\,,v)} \begin{pmatrix} a \\ \omega \end{pmatrix}. \tag{56}$$

The decoupling matrix has $\det = v \neq 0$: it is invertible whenever $v \neq 0$. In this case

$$^{-1}(\,, v) = \frac{1}{v} \begin{pmatrix} v\cos(\theta+\phi) & v\sin(\theta+\phi) \\ -\sin(\theta+\phi) & \cos(\theta+\phi) \end{pmatrix}.$$

Linearizing law and tracking. Let

$$\begin{pmatrix} a \\ \omega \end{pmatrix} = ^{-1}(\,, v) \left(\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} - (\,, v) \right). \tag{57}$$

The system from (u_1, u_2) to the outputs is then linear and decoupled,

$$\ddot{x}_f = u_1, \qquad \ddot{y}_f = u_2.$$

For a desired smooth trajectory $x_f^*(t), y_f^*(t)$ use the PD+feedforward law

$$u_1 = \ddot{x}_f^* + k_{p1}(x_f^* - x_f) + k_{d1}(\dot{x}_f^* - \dot{x}_f), \tag{58}$$
$$u_2 = \ddot{y}_f^* + k_{p2}(y_f^* - y_f) + k_{d2}(\dot{y}_f^* - \dot{y}_f). \tag{59}$$

With $e_x = x_f^* - x_f$ the error obeys $\ddot{e}_x + k_{d1}\dot{e}_x + k_{p1}e_x = 0$, whose characteristic polynomial $s^2 + k_{d1}s + k_{p1}$ is Hurwitz iff $k_{p1}, k_{d1} > 0$; the same holds for y. Hence *global exponential tracking* is obtained for any positive gains, as long as $v \neq 0$.

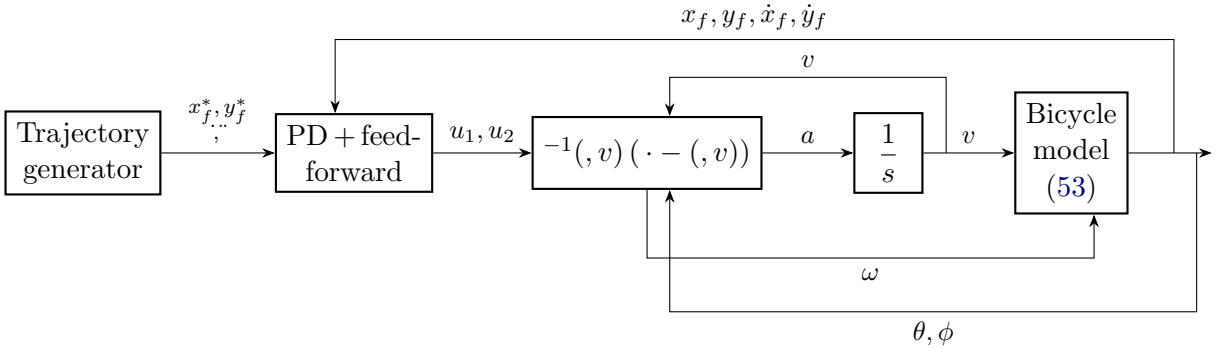


Figure 81: Block scheme of the dynamic feedback linearizing controller for the front wheel. The integrator turns the acceleration input a into the driving velocity v, so the controller is intrinsically dynamic (its state is v).

Practical remarks.

- To compute u_1, u_2 one needs x_f, y_f and their first derivatives. These can be obtained from the original measured states (x, y, θ or x_f, y_f, θ, ϕ), and in fact *without numerical differentiation*, since $\dot{x}_f = \cos(\theta+\phi)v$ and $\dot{y}_f = \sin(\theta+\phi)v$ are known as soon as θ, ϕ, v are.
- To compute (a, ω) through (57), the angles θ, ϕ and the extended state v are needed.
- The driving velocity v is obtained *by integrating* a: the controller has an internal state, and must be initialized with $v(0) \neq 0$.

Intuition

Why do we need to add an integrator? At the level of $\ddot{x}_f, \ddot{y}_f$ we have two outputs but, as long as v is regarded as an input, a single actuator in the first derivatives. Delaying the appearance of v by one step (treating v as a state and a as the input) gives two inputs (a, ω) that both show up in the second derivatives, and the matrix that multiplies them is invertible provided the vehicle is moving. The condition $v \neq 0$ has a clear physical meaning: when the front wheel is at rest it cannot instantaneously accelerate *sideways*: the steering input ω affects the front-wheel acceleration only through v ($\omega\, v$ terms). Finally, the linearized dynamics have order $2 + 2 = 4$ whereas the extended system has order 5: the remaining state is the unobserved internal dynamics which is, again, the steering equation (55), stable for forward motion. This is consistent with part (b).

5 Summary

5.1 Comparison of the three problems

	Hopping robot	$(2,3)$ **chained form**	**Front-wheel bicycle**
State / inputs	$= (\phi, \ell, \theta)$; $u_1 = \dot{\phi}$, $u_2 = \dot{\ell}$	$= (z_1, z_2, z_3)$; v_1, v_2	$= (x_f, y_f, \theta, \phi)$; v, ω
n, k	3, 1 (angular momentum)	3, 0 (already a model)	4, 2 (rolling)
Controllable?	Yes: $[_{1,2}] = (0, 0, \frac{2m(d+\ell)}{J})^T$	Yes: $[_{1,2}] = (0, 0, -1)^T$	Yes: $_{1,2}, [_{1,2}], [_1, [_{1,2}]]$ span, $\det = 1/\ell^2$
Flatness	Flat for $d = 0$; outputs (ϕ, θ)	(Chain form is flat: z_1, z_3)	(x_f, y_f) *not* flat (ϕ via ODE)
Main tool	geometric model, polynomial interpolation of flat outputs	Two phases: integrate z_1, z_2; loop to fix z_3	Dynamic extension $\dot{v} = a$, $^{-1}$
Result	Path planning; regulator $\theta \to 0$, $\ell \to h$	$T = T_1 + 2\pi/\omega$; $ab\pi/\omega^2 = \Delta z_3$	Exponential tracking of x_f^*, y_f^* if $v \neq 0$
Where it fails	$d+\ell = 0$; $\dot{\phi} = 0$; θ' must not change sign	$\omega \to 0$ (slow) or large a, b	$v = 0$

Table 8: Side-by-side view of the three exercises.

5.2 Key points

1. **A velocity constraint is not a loss of reachability.** In all three systems the constraint is nonholonomic; brackets of the input vector fields (a leg cycle, a steering wiggle, an area swept in the (z_1, z_2) plane) supply the "missing" directions, and the rank condition certifies it.

2. **Choose the basis for interpretability.** The null space of the constraint admits many bases; take the one whose inputs are directly actuated ($\dot{\phi}, \dot{\ell}$ for the hopper; v, ω for the bicycle).

3. **Flatness is a statement about algebra versus integration.** If a hidden variable is obtained by an algebraic formula (as ℓ from θ'/ϕ'), the output is flat and planning reduces

to polynomial interpolation. If it requires solving an ODE (as ϕ from the front-wheel path), it is not flat: the same output curve hides infinitely many states.

4. **Closed loops in the base variables generate net motion in the fiber.** The $\Delta\theta$ of the leg cycle and Δz_3 equal to the enclosed area are the same phenomenon, and give simple two-phase planners.

5. **When the outputs are not flat, dynamic extension can still give exact tracking**, at the price of a singularity ($v = 0$) and of leaving an internal dynamics (the steering angle) that must be stable.

Part X

Lecture 8

Conventions. Throughout, the WMR is a *unicycle* with configuration $q = (x, y, \theta)^T$, Cartesian position $p = (x, y)^T$ and kinematic model

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega, \tag{60}$$

where v is the driving velocity and ω the steering velocity. Subscript d denotes desired (reference) quantities.

1 Motion Control: Setting the Problem *(slides 2–4)*

1.1 Why feedback, and why a kinematic model

A desired motion is assigned to the WMR and the associated *nominal inputs* have been computed (e.g. by the planning techniques seen before). Applying those inputs in open loop would give very poor results: initial errors never decay, and disturbances, slippage and model errors accumulate. Hence we need *feedback control*.

For manipulators the feedback is designed on the *dynamic* model and produces commands at the generalized-force level. For WMRs one instead works on the *kinematic* model, for two reasons:

(1) the wheels are equipped with low-level control loops (PID on the actuators) that accept *velocities* as references;
(2) the dynamics is simpler than in manipulators and can be mostly canceled via feedback.

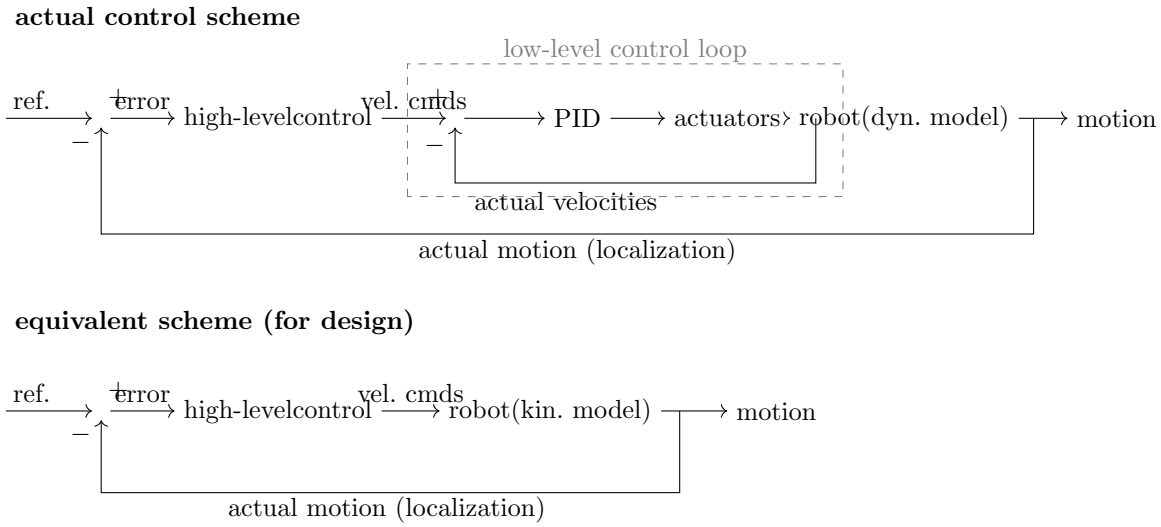


Figure 82: Actual control scheme (top) and the equivalent scheme used for design (bottom): the fast low-level loops make the velocity commands act almost instantaneously, so the robot is seen as its kinematic model.

Intuition

Think of a driver and a car. The driver (high-level control) only decides "how fast, how much steering"; the engine and steering servo (low-level loops) execute these requests much faster than the car moves on the road. So, for the purpose of planning the *path-following behaviour*, we may forget engines and masses and say that the velocities we ask for are the velocities we

get. This is exactly what makes the kinematic model (78) a legitimate design model.

1.2 Two motion control problems

Definition 1.1 (Trajectory tracking and posture regulation)**.** Two standard problems:

- *Trajectory tracking*: the robot must converge to (and then follow) a desired *Cartesian trajectory* (t).
- *Posture regulation*: the robot must converge to a desired *configuration* $q_d = (x_d, y_d, \theta_d)$.

Without loss of generality we consider the unicycle in what follows. Trajectory tracking is more relevant in practice, because moving along *planned* trajectories reduces the risk of collision; on the other hand, posture regulation does not require planning.

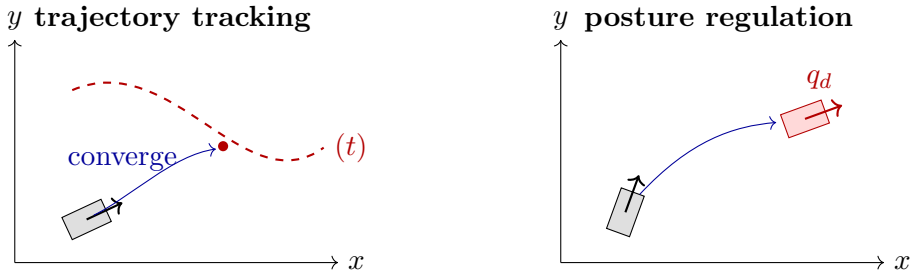


Figure 83: Tracking: follow a time-parametrized Cartesian path. Regulation: reach a final configuration (position *and* orientation), with no prescribed intermediate motion.

2 Trajectory Tracking: State Error Feedback *(slides 5–7)*

2.1 Admissible trajectories and flatness

The unicycle must track a Cartesian desired trajectory $(x_d(t), y_d(t))$ assumed to be *admissible*.

Definition 2.1 (Admissible Cartesian trajectory)**.** $(x_d(t), y_d(t))$ is admissible for the unicycle if there exist (t) and (t) such that

$$\dot{x}_d = \cos\theta_d, \qquad \dot{y}_d = \sin\theta_d, \qquad \dot{\theta}_d = .$$

The unicycle is *differentially flat* with flat output (x, y), so the whole desired state and input trajectory can be computed algebraically from $(x_d(t), y_d(t))$ and its derivatives.

Proposition 2.2 (Flatness of the unicycle)**.** *Given an admissible* $(x_d(t), y_d(t))$ *with* $\dot{x}_d^2 + \dot{y}_d^2 \neq 0$,

$$\theta_d(t) = \big(\dot{y}_d(t), \dot{x}_d(t)\big) + k\pi, \qquad k = 0, 1, \tag{61}$$

$$(t) = \pm\sqrt{\dot{x}_d^2(t) + \dot{y}_d^2(t)}, \tag{62}$$

$$(t) = \frac{\ddot{y}_d(t)\dot{x}_d(t) - \ddot{x}_d(t)\dot{y}_d(t)}{\dot{x}_d^2(t) + \dot{y}_d^2(t)}. \tag{63}$$

Proof. Dividing the second kinematic equation by the first gives $\tan\theta_d = \dot{y}_d/\dot{x}_d$, hence the expression of θ_d (the choice of k is the choice between forward and backward motion). Squaring and

adding gives $^2 = \dot{x}_d^2 + \dot{y}_d^2$, with the sign of matched to k: > 0 for $k = 0$, < 0 for $k = 1$. Finally $= \dot{\theta}_d$, and differentiating $(\dot{y}_d, \dot{x}_d)$ gives $(\ddot{y}_d\dot{x}_d - \ddot{x}_d\dot{y}_d)/(\dot{x}_d^2 + \dot{y}_d^2)$, which is independent of k. □

Admissibility is guaranteed as long as $\dot{y}_d/\dot{x}_d$ is continuous in t, i.e. the tangent direction has no jumps (*no sharp corners in time*).

Intuition

A car cannot move sideways: at every instant its velocity vector must point along its heading. So if you tell me *where* the car must be at every time (the Cartesian curve), I already know *how it must be oriented* (tangent to the curve), *how fast* it goes (the speed along the curve) and *how fast it must turn* (the curvature times the speed). There are no free choices left, apart from forward/backward (k). This is flatness: the Cartesian trajectory "contains" the full state and input history. The formulas also show why a trajectory that stops ($\dot{x}_d = \dot{y}_d = 0$) is delicate: the heading is undefined there.

2.2 The control scheme

The desired state trajectory $(t) = (x_d, y_d, \theta_d)$ is used to compute the *state error*, from which the feedback action is generated; the nominal inputs $(,)$ are used as a *feedforward* term.

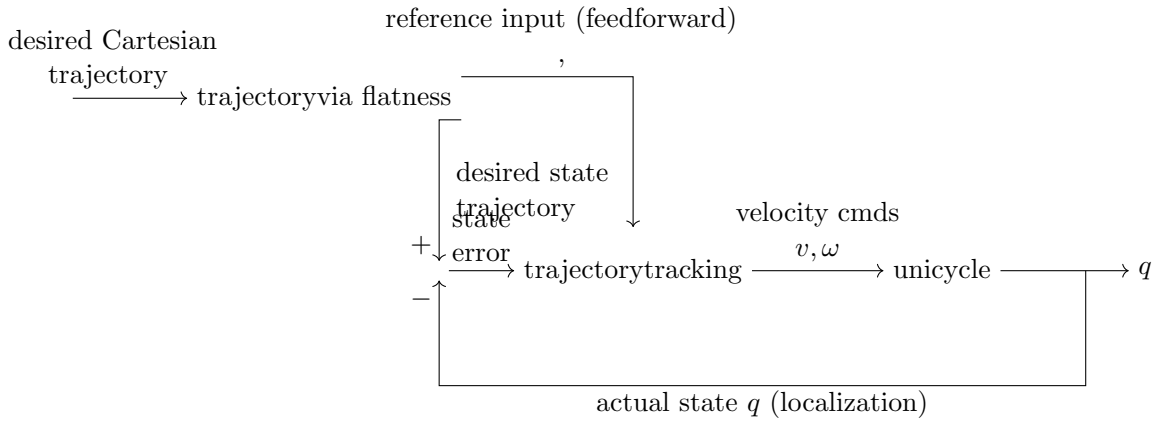


Figure 84: Trajectory tracking via state error feedback (block scheme): the desired Cartesian trajectory is turned into a desired state trajectory by flatness; the state error drives the tracking block, which also receives the nominal inputs , as feedforward.

2.3 Error coordinates and error dynamics

The objective is to drive the state error to zero. Rather than $-q$ we use its *rotated* version

$$e = \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{pmatrix}. \tag{64}$$

(e_1, e_2) is the Cartesian error $e_p = -p$ expressed in a frame rotated by θ, i.e. in the frame attached to the robot (e_1 along the sagittal axis, e_2 orthogonal to it), see Fig. 85.

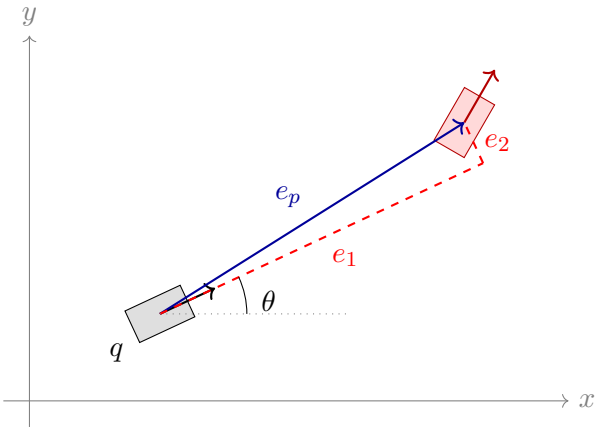


Figure 85: The rotated error: the Cartesian error e_p is decomposed along the robot's sagittal axis (e_1, "how far ahead/behind is the target") and orthogonally to it (e_2, "how far to the side").

Proposition 2.3 (Error dynamics)**.** *Along the trajectories of the unicycle* (78) *and of the reference system, the error* (64) *obeys the* nonlinear, time-varying *dynamics*

$$\dot{e}_1 = \cos e_3 - v + e_2\,\omega, \tag{65}$$
$$\dot{e}_2 = \sin e_3 - e_1\,\omega, \tag{66}$$
$$\dot{e}_3 = -\omega. \tag{67}$$

Proof. Differentiate $e_1 = \cos\theta(x_d - x) + \sin\theta(y_d - y)$:

$$\dot{e}_1 = \omega\big[-\sin\theta(x_d - x) + \cos\theta(y_d - y)\big] + \cos\theta(\dot{x}_d - \dot{x}) + \sin\theta(\dot{y}_d - \dot{y}) = \omega e_2 + \cos(\theta_d - \theta) - v,$$

where we used $\dot{x}_d = \cos\theta_d$, $\dot{y}_d = \sin\theta_d$, $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$ and $\cos\theta\cos\theta_d + \sin\theta\sin\theta_d = \cos(\theta_d - \theta)$. The computation of $\dot{e}_2$ is analogous, using $-\sin\theta\cos\theta_d + \cos\theta\sin\theta_d = \sin(\theta_d - \theta)$, and $\dot{e}_3 = -\omega$ is immediate. □

Intuition

The term $e_2\omega$ in $\dot{e}_1$ (and $-e_1\omega$ in $\dot{e}_2$) is the price of using a *rotating* frame: when the robot turns, the frame turns with it and the components of a fixed error vector mix, exactly as the components of a fixed arrow rotate when you spin your coordinate axes. In exchange, the other terms are very readable: $\cos e_3 - v$ says "if I drive slower than the reference speed (projected on my heading), the target runs away from me along e_1".

3 Approximate Linearization: Brush-up *(slides 8–9)*

The idea: to stabilize a nonlinear system at an equilibrium, stabilize its *approximate linearization* around it. Consider

$$\dot{x} = \eta(x,u) = f(x) + G(x)u, \qquad x \in \mathbb{R}^n,\ u \in \mathbb{R}^m,$$

and assume (w.l.o.g.) that the origin is the desired *unforced* equilibrium: $\eta(0,0) = 0$, or equivalently $f(0) = 0$. The Taylor expansion of η around $x = 0, u = 0$ gives

$$\eta(x,u) \approx \eta(0,0) + \left.\frac{\partial\eta}{\partial x}\right|_{\substack{x=0\\u=0}} x + \left.\frac{\partial\eta}{\partial u}\right|_{\substack{x=0\\u=0}} u.$$

The *approximate linearization* at the origin is then

$$\dot{x} = Ax + Bu, \qquad A = \left.\frac{\partial \eta}{\partial x}\right|_{0,0}, \quad B = \left.\frac{\partial \eta}{\partial u}\right|_{0,0}.$$

Let $u = Kx$, so that $\dot{x} = (A + BK)x$. If K is chosen so that $A + BK$ is *Hurwitz* (certainly possible if (A, B) is controllable), the approximate linearization is asymptotically stable.

Theorem 3.1 (Lyapunov indirect method)**.** *If $A + BK$ is Hurwitz, the origin is a* locally *asymptotically stable equilibrium of the nonlinear closed-loop system $\dot{x} = \eta(x, Kx)$. (If $A+BK$ has an eigenvalue with positive real part the origin is unstable; if the eigenvalues are only marginally stable, the linearization is inconclusive.)*

Intuition

Zoom in enough on any smooth function and it looks like a straight line (here, a linear map). If the linear caricature of the system is stable, then close enough to the equilibrium the true system behaves like it and is stable too. The word *local* is essential: the linear model says nothing about what happens far away, where the curvature (the neglected nonlinear terms) matters. That is exactly why this method will only give local guarantees for the unicycle.

4 Tracking via Approximate Linearization *(slides 10–16)*

4.1 Input transformation: making the origin an equilibrium

Apply the approximate linearization approach to the error dynamics of Proposition 2.3. In that form the origin is *not* an unforced equilibrium (even with $e = 0$, the terms $-v$ and $-\omega$ would not vanish unless the inputs equal the reference). This is rectified by the invertible *input transformation*

$$w_1 = \cos e_3 - v, \qquad w_2 = -\omega. \tag{68}$$

The error dynamics $\dot{e} = \eta(e, w)$ becomes

$$\dot{e} = \underbrace{\begin{pmatrix} e_2 \\ -e_1 + \sin e_3 \\ 0 \end{pmatrix}}_{f(e)} + \underbrace{\begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix}}_{G(e)} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}, \tag{69}$$

which is at rest when $e = 0, w = 0$, i.e. $\eta(0, 0) = 0$. Both the drift and the input term are nonlinear; in general the drift is also time-varying (through $(t), (t)$).

Intuition

The new inputs w are simply "*how far the actual velocities deviate from what is needed to keep the error at rest*". $w_1 = 0$ means we drive at the speed that makes e_1 stand still; $w_2 = 0$ means we turn at the reference rate. Feedback will generate w from the error; the actual commands (v, ω) are recovered afterwards by inverting (68). So the feedforward $(,)$ is automatically included.

4.2 The linear approximation

One easily finds

$$\frac{\partial \eta}{\partial e} = \begin{pmatrix} 0 & -w_2 & 0 \\ -+ w_2 & 0 & \cos e_3 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \frac{\partial \eta}{\partial w} = \begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix}.$$

Letting $e = 0, w = 0$ gives

$$A(t) = \begin{pmatrix} 0 & & 0 \\ - & 0 & \\ 0 & 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Note that A is actually $A(t)$ because of the dependence of and on time $\Rightarrow$ the linear approximation is still a *time-varying* system!

Define the *linear* feedback

$$w = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} -k_1 & 0 & 0 \\ 0 & -k_2 & -k_3 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = Ke.$$

The closed-loop error dynamics is still time-varying:

$$\dot{e} = (A(t) + BK)e = (t)e = \begin{pmatrix} -k_1 & & 0 \\ - & 0 & \\ 0 & -k_2 & -k_3 \end{pmatrix} e. \tag{70}$$

4.3 Gain tuning

Proposition 4.1 (Pole placement for the frozen system)**.** *Letting*

$$k_1 = k_3 = 2\zeta a, \qquad k_2 = \frac{a^2 - ^2}{\quad}, \qquad a > 0,\ \zeta \in (0, 1),$$

the characteristic polynomial of (t) *becomes time-invariant and Hurwitz:*

$$p(\lambda) = (\lambda + 2\zeta a)(\lambda^2 + 2\zeta a\lambda + a^2).$$

Proof. Expanding $\det(\lambda I -)$ along the first row,

$$\det \begin{pmatrix} \lambda + k_1 & - & 0 \\ & \lambda & - \\ 0 & k_2 & \lambda + k_3 \end{pmatrix} = (\lambda + k_1)\big(\lambda^2 + k_3\lambda + k_2\big) + ^2 (\lambda + k_3).$$

With $k_1 = k_3 = 2\zeta a$ and $k_2 = a^2 - ^2$ this becomes $(\lambda + 2\zeta a)\big(\lambda^2 + 2\zeta a\lambda + a^2 - ^2\big) + ^2 (\lambda + 2\zeta a) = (\lambda + 2\zeta a)(\lambda^2 + 2\zeta a\lambda + a^2)$. □

The polynomial has *one real negative eigenvalue* $-2\zeta a$ and *two complex eigenvalues with damping* ζ *and natural frequency* a, $\lambda = -\zeta a \pm ja\sqrt{1 - \zeta^2}$.

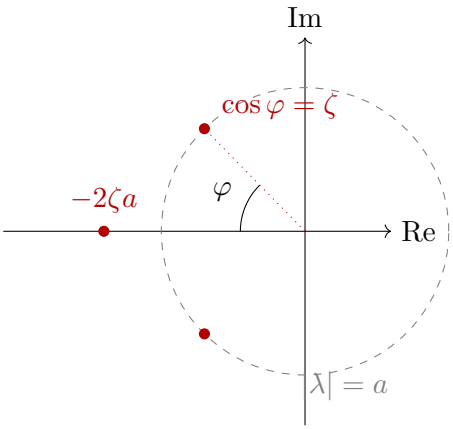


Figure 86: Closed-loop eigenvalues for $a = 2$, $\zeta = 0.7$: a real pole at $-2\zeta a$ and a complex pair on the circle of radius a with damping ζ (the angle φ from the negative real axis satisfies $\cos\varphi = \zeta$).

Intuition

The two gains k_1, k_3 are chosen equal to the damping term and k_2 absorbs the (time-varying) reference velocities ,. The trick is "gain scheduling": we *re-compute* k_2 *at every instant* so that the frozen-time eigenvalues stay at the same place. The designer then simply chooses the response speed a and damping ζ, as for a second-order system. Note the price: $k_2 = (a^2 - {}^2)/ \to \infty$ as $\to 0$.

[Caveat] Having time-invariant, Hurwitz *frozen-time* eigenvalues does *not* guarantee asymptotic stability of a time-varying system. Stability is guaranteed only if and are constant ($\Rightarrow$ constant), i.e. for rectilinear and circular reference trajectories, and then Theorem 3.1 gives only *local* asymptotic stability, not global.

4.4 Actual inputs, limitations and a global version

The actual velocity inputs (v, ω) are obtained by plugging the feedback controls w_1, w_2 in the input transformation (68):

$$v = \cos e_3 - w_1 = \cos e_3 + k_1 e_1, \qquad \omega = -w_2 = +k_2 e_2 + k_3 e_3. \tag{71}$$

Two remarks:

- $(v, \omega) \to (,)$ as $e \to 0$: the controller reduces to pure feedforward.
- $k_2 \to \infty$ as $\to 0$, so this controller can only be used with *persistent* Cartesian trajectories (stops are not allowed).

Theorem 4.2 (Global stability via a nonlinear version)**.** *Global asymptotic stability of* $e = 0$ *is obtained by the nonlinear feedback*

$$w_1 = -k_1(,)\, e_1, \tag{72}$$

$$w_2 = -k_2 \frac{\sin e_3}{e_3} e_2 - k_3(,)\, e_3, \tag{73}$$

if k_1, k_3 *are bounded, positive, with bounded derivatives (and* $k_2 > 0$ *constant), and , are bounded, with not vanishing asymptotically.*

Intuition

In the linearized controller the term $k_2 e_2$ is linear in e_3-space, i.e. it approximates $\sin e_3$ by e_3. The nonlinear version keeps $\sin e_3 / e_3$, which is bounded (between -0.22 and 1 for all e_3): large heading errors can no longer produce unreasonably large commands, and a Lyapunov argument (not repeated here) works globally. The structure is the same, and when e_3 is small the two controllers coincide.

Algorithm 4.3 (Tracking via state error feedback and approximate linearization)**.** At each control instant:

1. from (x_d, y_d) compute $\theta_d, ,$ via flatness (Prop. 2.2);
2. read (x, y, θ) from localization and compute e with (64);
3. compute gains $k_1 = k_3 = 2\zeta a$, $k_2 = (a^2 - {}^2)/;$
4. set $v = \cos e_3 + k_1 e_1$ and $\omega = +k_2 e_2 + k_3 e_3$.

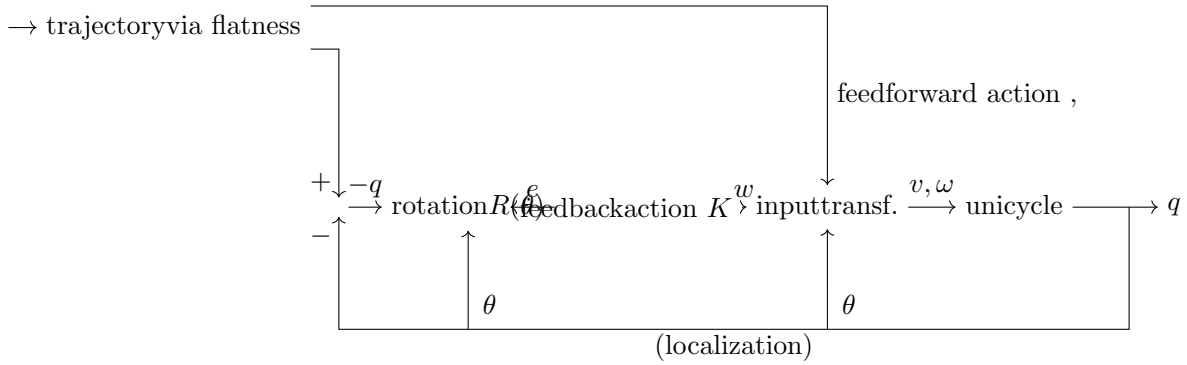


Figure 87: Detailed version of Fig. 84 for approximate linearization: a static controller on the rotated state error, with feedforward , added through the input transformation.

The final scheme (Fig. 87) is a *static* controller based on the state error; it needs ,, and needs θ both for the error rotation and for the input transformation.

5 Trajectory Tracking: Output Error Feedback *(slide 17)*

Another approach: develop the feedback action from the *output (Cartesian) error* only, *without* computing a desired state trajectory, using as *feedforward* term the velocity ˙ along the reference trajectory.

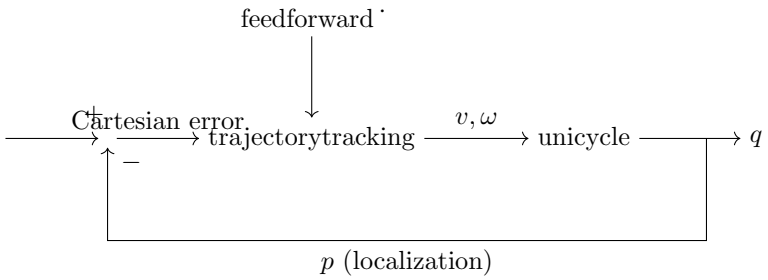


Figure 88: Output error feedback: only the Cartesian error is fed back.

The difficulty is that the unicycle has two inputs and, if only (x, y) is to be controlled, a standard linearization-by-feedback argument must be adapted. The next section recalls the tool, and the

following ones apply it in two different ways.

6 Exact I/O Linearization: Brush-up *(slides 18–19)*

Consider a driftless nonlinear system with as many outputs as inputs:

$$\dot{x} = G(x)u, \qquad y = h(x), \qquad x \in \mathbb{R}^n,\ u \in \mathbb{R}^m,\ y \in \mathbb{R}^m.$$

Since

$$\dot{y} = \frac{\partial h}{\partial x}\dot{x} = \underbrace{\frac{\partial h}{\partial x}G(x)}_{T(x)}\,u = T(x)u,$$

we call T $(m \times m)$ the *decoupling matrix*.

Proposition 6.1 (Exact input–output linearization)**.** *If $T(x)$ is invertible, the* linearizing feedback $u = T^{-1}(x)w$ *yields*

$$\dot{y} = T(x)T^{-1}(x)w = w,$$

i.e. an exactly linear (and decoupled) map between the new inputs w and the time derivative of the outputs. Given a reference output $y_d(t)$, the output error $e = y_d - y$ has dynamics $\dot{e} = \dot{y}_d - w$; the law $w = \dot{y}_d + Ke$ (feedforward + proportional feedback) gives $\dot{e} = -Ke$, i.e. global exponential stability *of $e = 0$ provided that the eigenvalues of K are in the right half-plane. The final control law is*

$$u = T^{-1}(x)\big(\dot{y}_d + Ke\big). \tag{74}$$

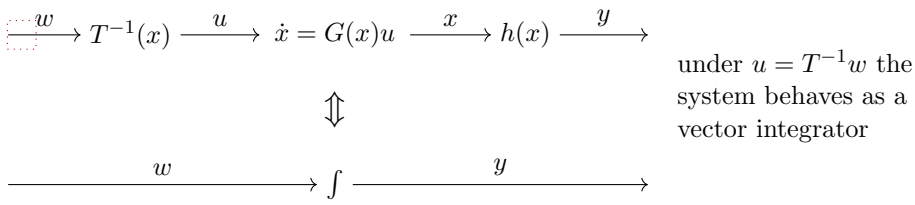


Figure 89: Exact input–output linearization: the dotted block, seen from w to y, is just a bank of integrators.

Intuition

Imagine a robot arm where you want the hand velocity $\dot{y}$ and you can command joint velocities u through a Jacobian-like matrix $T(x)$. If you multiply the desired hand velocity by the inverse Jacobian, the hand does exactly what you wanted, and the plant "disappears": what remains is $\dot{y} = w$, the simplest system in the world. Tracking becomes trivial: ask for the reference velocity plus a correction proportional to the error. Everything hinges on T being *invertible*.

7 Tracking via I/O Linearization: Static Feedback *(slides 20–24)*

7.1 The singularity for the output (x, y)

Let us apply exact i/o linearization to the unicycle with output (x, y). However, the decoupling matrix is

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix},$$

which is *singular* (its second column is zero): ω does not appear in the first derivative of (x, y). Hence exact input–output linearization is *not possible* for the output (x, y) by static feedback.

Intuition

At the contact point of the wheel, turning in place does not move the point. The steering command is invisible at the level of positions, so we cannot "invert" the map. The cure is to look at a point that *does* move when you steer.

7.2 Displacing the output to a point B

Solution: *change slightly the output* so that the new input–output map is invertible. In particular, displace the output from the contact point of the wheel to a point B along the sagittal axis, at distance b:

$$x_B = x + b\cos\theta, \qquad y_B = y + b\sin\theta.$$

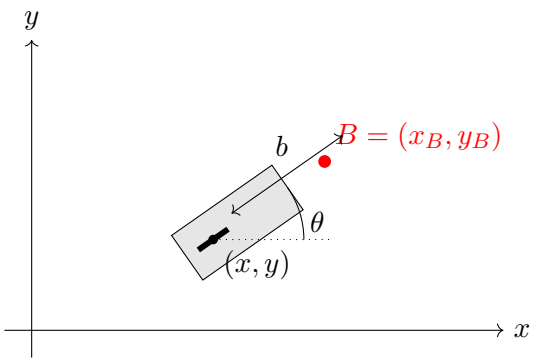


Figure 90: Point B at distance b ahead of the wheel contact point along the sagittal axis.

Differentiating with respect to time,

$$\begin{pmatrix} \dot{x}_B \\ \dot{y}_B \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -b\sin\theta \\ \sin\theta & b\cos\theta \end{pmatrix}}_{T(\theta),\ \det = b} \begin{pmatrix} v \\ \omega \end{pmatrix}.$$

If $b \neq 0$ we may set

$$\begin{pmatrix} v \\ \omega \end{pmatrix} = T^{-1}(\theta) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta/b & \cos\theta/b \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}, \tag{75}$$

obtaining the input–output linearized system

$$\dot{x}_B = w_1, \qquad \dot{y}_B = w_2, \qquad \dot{\theta} = \frac{w_2\cos\theta - w_1\sin\theta}{b}.$$

(The third equation is the part of the dynamics that is *not* seen from the output.)

We achieve *global exponential convergence* of (x_B, y_B) to the desired trajectory by letting

$$w_1 = \dot{x}_d + k_x(x_d - x_B), \qquad w_2 = \dot{y}_d + k_y(y_d - y_B), \qquad k_x, k_y > 0.$$

Intuition

Hold a broom by its middle and push the handle sideways: the bristle end (B) moves sideways even if the hand does not. In the same way, a point ahead of the axle *moves laterally when the robot steers*, so both of its velocity components can be commanded; b is the lever arm, and $\det T = b$ tells us that the shorter the lever arm, the more delicate the inversion (the $1/b$ in the second row of (75) amplifies the steering command).

7.3 What happens to θ: zero dynamics

The orientation θ is *not* controlled by this scheme, which is based on output error feedback: there is a *zero dynamics*. Still, it must evolve as dictated by flatness. The next result makes this precise.

Proposition 7.1 (Zero dynamics)**.** *Suppose B tracks the reference exactly, so $w_1 = \dot{x}_d$, $w_2 = \dot{y}_d$. Writing $\dot{} = |\dot{}|(\cos\varphi, \sin\varphi)$ and $\delta = \theta - \varphi$,*

$$\dot{\delta} = -\frac{|\dot{}|}{b}\sin\delta + (\textit{terms in } \dot{\varphi}),$$

and for the case $\varphi = const$ (straight reference) $\delta = 0$ is exponentially stable if $|\dot{}| > 0$ and $b > 0$, while $\delta = \pi$ is unstable.

Proof. Substitute w_1, w_2 in $\dot{\theta}$: $\dot{\theta} = \big(\dot{y}_d\cos\theta - \dot{x}_d\sin\theta\big)/b = \frac{|\dot{}|}{b}\sin(\varphi - \theta)$. Subtracting $\dot{\varphi}$ gives the claim; linearization at $\delta = 0$ gives $\dot{\delta} \simeq -(|\dot{}|/b)\delta$, at $\delta = \pi$ it gives $+(|\dot{}|/b)\delta'$. □

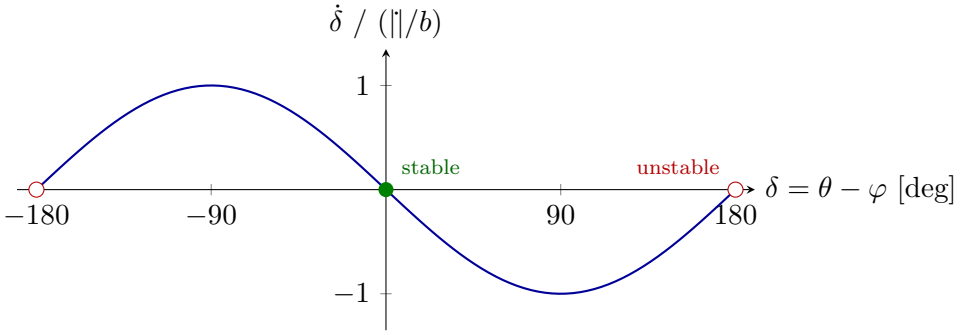


Figure 91: Phase line of the zero dynamics (straight reference): the robot's orientation relaxes to the direction of motion of B.

Intuition

It is like towing a shopping cart by its front end: if you pull B along a path, the cart (robot body) lines up behind the pulling point and follows the tangent of the path. Aligned forward ($\delta = 0$) is stable. Driving *backwards*, with the body ahead of the pulled point, is unstable: the same reason a cart pushed from behind jackknifes. This is why the scheme is used with forward motion only.

7.4 Properties and block scheme

The desired trajectory for B can be *arbitrary*; in particular, *square corners may be included*, since no desired heading has to be computed.

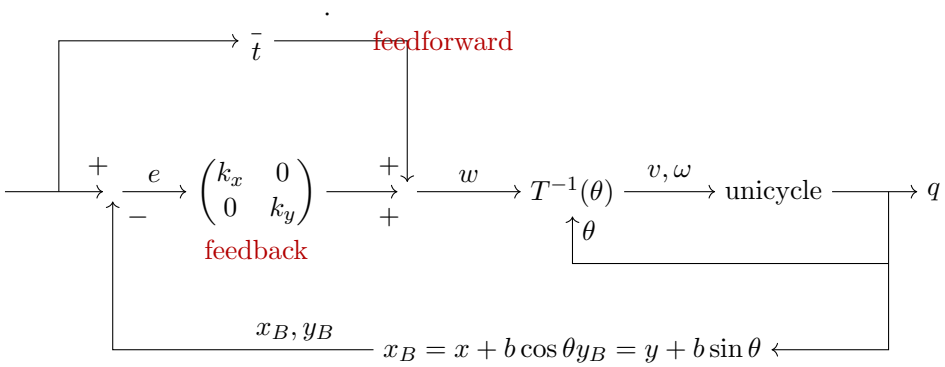


Figure 92: Tracking via output error feedback + static i/o linearization: a static controller on the output error; it needs $\dot{}$, and (x, y, θ) both for the output reconstruction and (with θ) the input transformation.

Algorithm 7.2 (Static i/o linearization tracker)**.** Given $b \neq 0$, $k_x, k_y > 0$ and the reference $(t), \dot{(t)}$:

1. compute $x_B = x + b\cos\theta$, $y_B = y + b\sin\theta$;
2. $w_1 = \dot{x}_d + k_x(x_d - x_B)$, $w_2 = \dot{y}_d + k_y(y_d - y_B)$;
3. $v = \cos\theta\, w_1 + \sin\theta\, w_2$, $\omega = (-\sin\theta\, w_1 + \cos\theta\, w_2)/b$.

[Steady-state error equals b] Because B (not the contact point (x, y)) is the point forced onto (t), in steady state the contact point trails the reference by b: the error $\| - p\|$ settles at b (circle and square examples below). Decreasing b reduces this error but increases the steering velocity, as the $1/b$ in (75) shows.

8 Tracking via I/O Linearization: Dynamic Feedback *(slides 25–28)*

Rather than displacing the controlled output to B, we can *keep the (flat) output* (x, y) and achieve exact linearization by using a *dynamic* compensator.

8.1 Dynamic extension

Perform a *dynamic extension* on the driving-velocity channel ("slow down" the input): add an integrator

$$\dot{v} = a,$$

so that the new input is the driving *acceleration* a, and v becomes a state of the compensator. We can now differentiate the output further:

$$\ddot{x} = a\cos\theta - v\sin\theta\,\omega, \qquad \ddot{y} = a\sin\theta + v\cos\theta\,\omega.$$

The new decoupling matrix is nonsingular if $v \neq 0$:

$$\begin{pmatrix} \ddot{x} \\ \ddot{y} \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -v\sin\theta \\ \sin\theta & v\cos\theta \end{pmatrix}}_{T(v,\theta),\ \det = v} \begin{pmatrix} a \\ \omega \end{pmatrix}.$$

Under this assumption we may set

$$\begin{pmatrix} a \\ \omega \end{pmatrix} = T^{-1}(\theta, v) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\dfrac{\sin\theta}{v} & \dfrac{\cos\theta}{v} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix},$$

obtaining a *fully linearized* system $\ddot{x} = w_1$, $\ddot{y} = w_2$ (two chains of two integrators; the order, 4, equals the dimension of the extended state (x, y, θ, v), so nothing is left unseen).

Intuition

Why did ω vanish from $\dot{x}, \dot{y}$ but appear in $\ddot{x}, \ddot{y}$? Because steering changes the *direction* of the velocity, and direction becomes visible one differentiation later, as an acceleration *multiplied by the current speed* v. If $v = 0$ turning does nothing to the position, hence the singularity at $v = 0$. Using an integrator on v is just a way to let the compensator "remember" the current speed, so that it can be divided by.

8.2 Control law

We achieve *global exponential convergence* of (x, y) to the desired trajectory by letting

$$w_1 = \ddot{x}_d + k_{p,x}(x_d - x) + k_{d,x}(\dot{x}_d - \dot{x}), \tag{76}$$
$$w_2 = \ddot{y}_d + k_{p,y}(y_d - y) + k_{d,y}(\dot{y}_d - \dot{y}), \tag{77}$$

with $k_{px}, k_{py}, k_{dx}, k_{dy} > 0$. The error then satisfies $\ddot{e} + k_d\dot{e} + k_p e = 0$ along each axis (e.g. $k_p = a^2$, $k_d = 2\zeta a$ for a prescribed damping and bandwidth).

Properties:

- since the original flat output is controlled, there is *no zero dynamics* with this approach;
- the desired trajectory must be *twice differentiable* and *persistent*, i.e. $\neq 0$ always (this singularity is *structural* in nonholonomic systems).

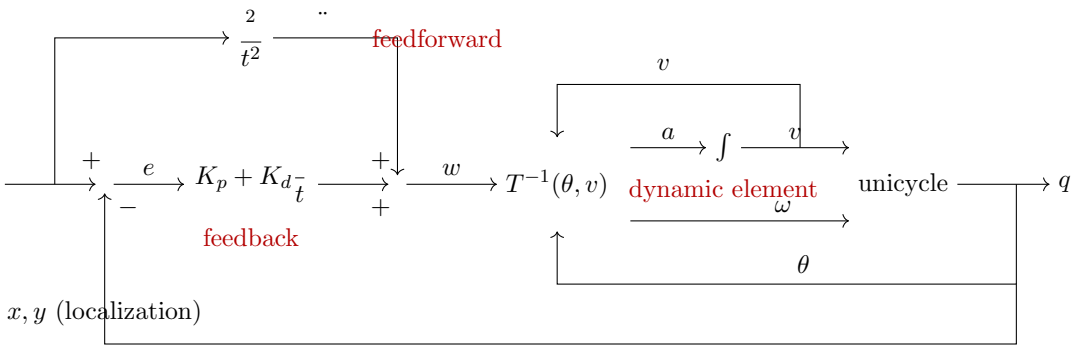


Figure 93: Tracking via output error feedback + dynamic i/o linearization: a *dynamic* controller based on the output error; it needs $\dot{}$ and $\ddot{}$ (inside K_d and the feedforward), (x, y) for the error and θ for the input transformation. The state v of the integrator is fed back to T^{-1}.

Algorithm 8.1 (Dynamic i/o linearization tracker)**.** Initialize $v(0) \neq 0$ (e.g. $v(0) = (0)$). At each instant:

1. compute w_1, w_2 as above (with $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$);
2. $a = \cos\theta\, w_1 + \sin\theta\, w_2$, $\omega = (-\sin\theta\, w_1 + \cos\theta\, w_2)/v$;
3. integrate $\dot{v} = a$ and send (v, ω) to the robot.

Intuition

Compared with the static scheme, we pay with extra machinery (one integrator, second derivatives of the reference, $v \neq 0$) but we get the real thing: the point that converges is the actual position (x, y), with zero steady-state error and no hidden heading dynamics. Static feedback tracks a *fictitious* point B; dynamic feedback tracks the *true* point.

9 Simulations *(slides 29–40)*

The slides compare the three controllers on three reference trajectories: a circle, a square (with sharp corners) and a figure-8. The qualitative findings can be summarized as follows.

@l¿X¿X¿X@

Approx. linearization	**Static i/o lin.** ($b = 0.75$ / 0.2)	**Dynamic i/o lin.**
Circle	converges; only local stability guaranteed	steady-state error $= b$; reducing b lowers the error but *increases steering velocity*	zero steady-state error, reasonable velocities
Square	only local stability; *a new transient at each corner*	error $= b$; the displaced output gives a *lookahead* behaviour (corners are cut); small b: smaller error, larger steering peaks	faster transients, reasonable velocities
Figure 8	local stability not guaranteed (persistent , vary), but good performance	error $= b$; smaller $b \Rightarrow$ larger steering velocity	zero steady-state error, reasonable velocities

Table 9: Qualitative outcome of the simulations.

Reading off the circle plots, the *initial* steering velocity grows from about 2.7 rad/s for $b = 0.75$ to about 10 rad/s for $b = 0.2$: a ratio $\approx 3.7 \approx 0.75/0.2$, as expected from the $1/b$ factor in (75).

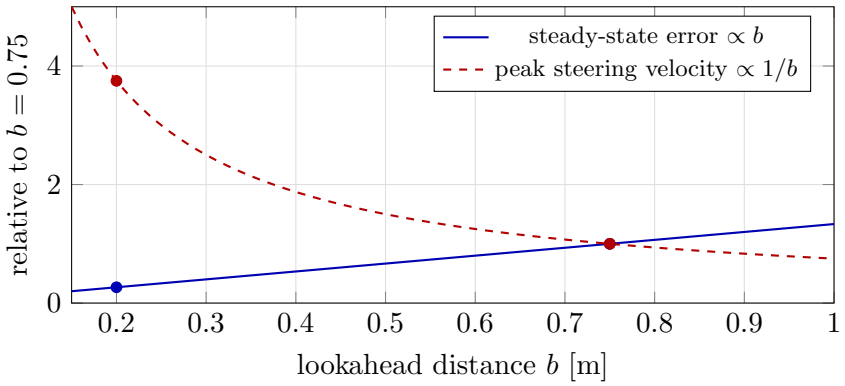


Figure 94: The trade-off of static i/o linearization (original illustration of the scaling laws; the dots mark the two cases simulated, $b = 0.75$ and $b = 0.2$). A small b gives small error but demands large steering velocities.

Example 9.1 (Circle tracking, independent simulation)**.** To visualize the comparison we simulated (Euler, $dt = 2$ ms) the tracking of a circle of radius 3 m with $= 1$ m/s ($= 1/3$ rad/s), starting from $q(0) = (1, -1, 2)$ and, for the dynamic controller, $v(0) = 1.5$ m/s. Gains: approximate linearization $a = 2, \zeta = 0.7$; static i/o $k_x = k_y = 2.5$; dynamic i/o $k_p = 6.25$, $k_d = 5$. Fig. 95 shows the paths and the error norm.



Figure 95: Original simulation of the three controllers on a circle. Approximate linearization and dynamic i/o linearization converge to zero error; static i/o linearization settles at an error equal to b.

10 Summary

@¿p2.35cm¿X¿X¿X@

State error + approx. linearization	**Output error + static i/o lin.**	**Output error + dynamic i/o lin.**
Controlled variable	state error $e = R(\theta)(-q)$	Cartesian position of B (displaced by b)	Cartesian position (x, y)
Controller type	static	static	dynamic (integrator on v)
Reference needed	, (and θ_d) via flatness	,˙ ,˙,¨
Measurements	x, y, θ	x, y, θ	x, y, θ (and v internal)
Stability	local, only for constant , (global with nonlinear version)	global exponential for B	global exponential for (x, y)
Zero dynamics	none (θ in the error)	yes: θ aligns with motion (forward only)	none
Steady-state error	0	b (contact point)	0
Restrictions	persistent trajectory ($k_2 \to \infty$ as $\to 0$); smooth direction	none on corners; $b \neq 0$; large ω if b small	persistent ($v \neq 0$); trajectory twice differentiable

Table 10: Comparison of the trajectory tracking controllers for the unicycle.

Intuition

Pick the tool by what you can accept. *State error + linearization*: simplest idea, but the guarantees are weak. *Static i/o*: guaranteed and simple, but you pay with a permanent offset b (a lookahead, which is sometimes even desirable on corners). *Dynamic i/o*: the "right" answer (true position, zero error, globally exponentially stable) at the cost of more machinery and a ban on stopping. The trade-off between offset and steering effort in the static case is shown again in Fig. 94.

Key take-aways

1. WMR motion control is designed on the *kinematic* model, since the fast low-level loops let us treat velocities as inputs; reference inputs must always be applied together with a feedback correction.

2. For an admissible Cartesian trajectory, *flatness* gives θ_d, , algebraically, and the nominal inputs act as feedforward in all schemes.

3. State error feedback with approximate linearization has a time-varying closed loop: pole placement with $k_2 = (a^2 - ^2)/$ is only rigorous for circles and lines (local stability), and requires $\neq 0$; a nonlinear variant gives global stability.

4. Exact i/o linearization of (x, y) fails with static feedback (singular decoupling matrix); displacing the output to B at distance b fixes this, at the cost of a zero dynamics and a steady-state error b with steering velocity $\propto 1/b$.

5. A dynamic extension ($\dot{v} = a$) allows tracking the true flat output (x, y) with global exponential convergence and zero steady-state error, but needs a twice-differentiable, persistent trajectory ($v \neq 0$, a structural limitation of nonholonomic systems).

Part XI
Lecture 9

Part XII
Motion control: regulation

1 Regulation: the problem and its limits *(regulation slide 2)*

Throughout this part the robot is a unicycle with configuration $q = (x, y, \theta)$ and kinematic model

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega.$$

In mobile robotics, motion control problems fundamentally divide into two broad classes:

1. **Trajectory tracking**, where the objective is to follow a time-parameterized moving reference trajectory $q_d(t)$ with non-zero reference velocities;
2. **Regulation**, which is essentially a *parking problem*: driving the vehicle from an arbitrary initial configuration q_0 to a static desired configuration q_d and bringing it to a complete stop there.

Definition 1.1 (Regulation)**.** The *regulation* problem consists in driving the unicycle, starting from a generic initial configuration, to a desired configuration q_d and keeping it there. Without loss of generality the world frame is chosen so that q_d is the origin.

Why tracking controllers fail for regulation. Since trajectory tracking techniques are well established, an obvious approach would be to design a reference trajectory that comes to rest at q_d and then track it via feedback. However, this approach fails for all the tracking controllers studied previously:

- **Linear and nonlinear controllers based on error dynamics**: they strictly require a *persistent trajectory* (i.e., $v_d(t) \geq v_{\min} > 0$ or $(v_d(t), \omega_d(t)) \neq (0, 0)$). If the reference stops ($v_d \to 0, \omega_d \to 0$), the feedback gains or denominators in the control law tend to infinity, causing the control inputs to blow up.
- **Input/output linearization via static feedback**: the controlled output is not the wheel contact point (x, y), but rather an off-axis point B located at a distance $b > 0$ along the sagittal axis (Figure 96). Consequently, this controller steers point B to the destination, leaving the contact point (x, y) displaced by b from the target.
- **Input/output linearization via dynamic feedback**: the dynamic feedback compensator requires a non-zero forward velocity ($v \neq 0$) to ensure invertibility of the decoupling matrix, so it also requires persistent trajectories.

These limitations stem directly from the nonholonomic nature of wheeled mobile robots: unlike manipulators, they do not admit *universal controllers*, i.e. controllers able to stabilize arbitrary trajectories, whether persistent or not.

Nonholonomic WMR $\implies$ No universal controller exists
(able to solve both tracking and regulation)

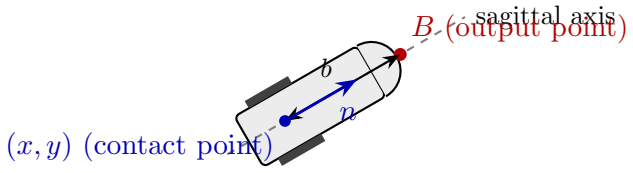


Figure 96: The unicycle wheel contact point (x, y) versus the off-axis point B at distance b along the sagittal axis: static I/O linearization drives B to the goal, leaving (x, y) displaced by b.

The obvious approach — choose a path/trajectory that stops in q_d and then track it via feedback — does *not* work.

Proposition 1.2 (Tracking a stopping trajectory fails at linear level)**.** *Let $e = (e_1, e_2, e_3)$ be the tracking error expressed in the robot frame, $e_1 = \cos\theta(x_d - x) + \sin\theta(y_d - y)$, $e_2 = -\sin\theta(x_d - x) + \cos\theta(y_d - y)$, $e_3 = \theta_d - \theta$. The linearization of the error dynamics about $e = 0$ is*

$$\dot{e} = \begin{pmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & 0 & 0 \end{pmatrix} e + \begin{pmatrix} -1 & 0 \\ 0 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} \delta v \\ \delta\omega \end{pmatrix},$$

and when the reference comes to rest ($v_d = \omega_d = 0$) it is not controllable.

Proof. With $v_d = \omega_d = 0$ the state matrix vanishes, so the controllability matrix reduces to the input matrix, whose rank is $2 < 3$. The lateral error e_2 cannot be influenced at first order. □

Intuition

Tracking controllers work because the reference keeps moving: while the robot rolls, a lateral error can be fixed by combining small steering and forward motion. Once the reference stops, the robot is a car with its wheels aligned: any *small* correction of a sideways offset is simply not available. This is the familiar parallel-parking problem, and it is why regulation needs dedicated controllers.

Roadmap. To address regulation for the unicycle, we proceed in two successive steps:

1. **Cartesian regulation**: driving the contact point (x, y) to a specified Cartesian destination $(0, 0)$, leaving the final orientation θ uncontrolled;
2. **Posture regulation**: driving the full configuration (x, y, θ) to a desired target $(0, 0, 0)$, controlling position and orientation simultaneously.

2 Cartesian regulation *(regulation slides 3–6)*

2.1 Problem and geometry *(regulation slide 3)*

The first task is to drive the unicycle to a given Cartesian position (w.l.o.g. the origin $(0, 0)$), *regardless of orientation*. The relevant quantities are:

- the *Cartesian error* $e_p = \begin{pmatrix} -x \\ -y \end{pmatrix}$ (from the robot to the origin);
- the unit vector of the sagittal axis $n = \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}$;
- the projection of e_p on the sagittal axis, $e_p^T n = -x\cos\theta - y\sin\theta$;

- the *pointing error* $\gamma = \text{Atan2}(y, x) + \pi - \theta$, i.e. the difference between the orientation of e_p and that of the unicycle (the angle is taken in $(-\pi, \pi]$).

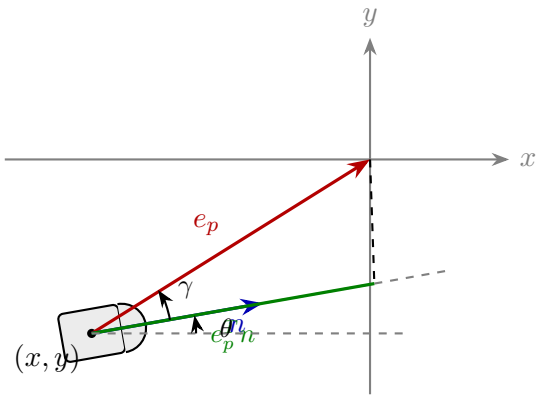


Figure 97: Geometry of Cartesian regulation: the error e_p points from the unicycle to the origin; its projection on the sagittal axis is $e_p^T n$ and the pointing error γ is the angle between n and e_p.

2.2 Control law *(regulation slide 4)*

Definition 2.1 (Cartesian regulation law)**.** For $k_1, k_2 > 0$,

$$v = -k_1 \left(x \cos\theta + y \sin\theta\right) = k_1 \, e_p^T n, \qquad \omega = k_2 \left(\text{Atan2}(y, x) - \theta + \pi\right) = k_2 \, \gamma.$$

The law has a clear geometrical interpretation:

- v is proportional to the orthogonal *projection* of the Cartesian error e_p on the sagittal axis;
- ω is proportional to the *pointing error*, i.e. the difference between the orientation of e_p and that of the unicycle.

Intuition

Imagine pulling a shopping cart towards a target: the further the target is *straight ahead*, the faster you push; you steer in proportion to how far off to the side it is. Numbers: with $k_1 = 1$, $k_2 = 2$ and start $(x, y, \theta) = (-4, -2, 0)$ we get $e_p^T n = 4$, so $v = 4$, and $\gamma = \text{Atan2}(-2, -4) + \pi \approx 0.46$ rad, so $\omega \approx 0.93$ rad/s: the robot drives forward while turning left towards the origin. If the target is *behind* the robot, $e_p^T n < 0$ and v is negative: the unicycle simply backs up.

2.3 Brush-up: Lyapunov stability of nonlinear systems *(background to regulation slide 5)*

Consider an autonomous nonlinear system $\dot{x} = f(x)$ with f locally Lipschitz and an equilibrium at the origin, $f(0) = 0$ (any other equilibrium x_e is brought to the origin by a translation of coordinates).

Why Lyapunov's indirect method (linearization) fails here. Before studying Lyapunov's direct method, let us address why stability cannot simply be proven via linearization:

- In *Lyapunov's indirect method*, one linearizes the system about an equilibrium point x_e, obtaining $\dot{\delta x} = A \, \delta x$ with $A = \frac{\partial f}{\partial x}\big|_{x_e}$. If all eigenvalues of A have strictly negative real parts

($\mathrm{Re}[\lambda_i] < 0$), the equilibrium is asymptotically stable for the nonlinear system. If at least one eigenvalue has $\mathrm{Re}[\lambda_i] > 0$, it is unstable.

- When one or more eigenvalues lie on the imaginary axis ($\mathrm{Re}[\lambda_i] = 0$) and none in the open right-half plane, the system falls into the **critical case of Lyapunov theory**. In this critical case, linearization provides no information on stability: the outcome depends entirely on higher-order nonlinear terms.

In Cartesian regulation, every configuration $(0, 0, \theta)$ is an equilibrium (the entire θ-axis is an equilibrium set), so linearizing the closed-loop system about any such point yields eigenvalues with zero real parts ($\mathrm{Re}[\lambda] = 0$). We are therefore squarely in the critical case, which renders the indirect method inconclusive and makes the **Lyapunov direct method** necessary.

Definition 2.2 (Stability notions)**.** The equilibrium $x = 0$ is
- *stable* if for every $\varepsilon > 0$ there is $\delta > 0$ such that $\|x(0)\| < \delta \Rightarrow \|x(t)\| < \varepsilon$ for all $t \geq 0$;
- *asymptotically stable* (AS) if it is stable and there is $\delta' > 0$ such that $\|x(0)\| < \delta' \Rightarrow x(t) \to 0$;
- *globally asymptotically stable* (GAS) if $x(t) \to 0$ for every initial condition;
- *exponentially stable* if $\|x(t)\| \leq k\|x(0)\|e^{-\lambda t}$ for some $k, \lambda > 0$.

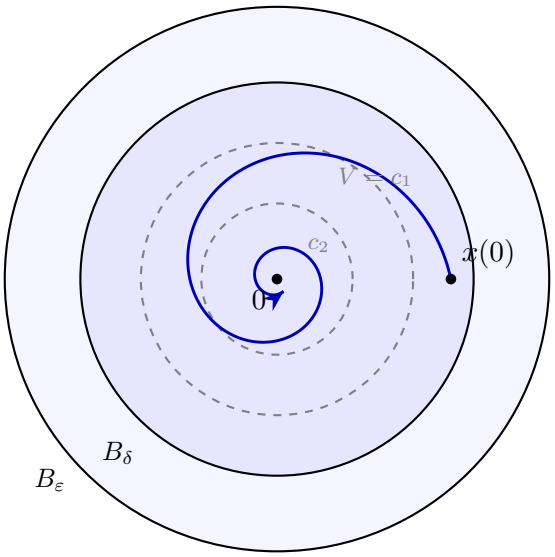


Figure 98: Stability and Lyapunov level sets: a trajectory starting in B_δ never leaves B_ε and, if $\dot{V} < 0$, crosses the level sets $V = c$ (dashed, $c_1 > c_2$) inwards towards the origin.

Definition 2.3 (Definite and semidefinite functions)**.** Let $W : D \to \mathbb{R}$ be continuous on a neighborhood $S(x_e, r)$ of x_e, with $W(x_e) = 0$. We recall that:
- W is *positive definite* (PD) in $S(x_e, r)$ if $W(x) > 0$ for all $x \in S(x_e, r) \setminus \{x_e\}$;
- W is *positive semidefinite* (PSD) in $S(x_e, r)$ if $W(x) \geq 0$ for all $x \in S(x_e, r)$ (so W may vanish at states $x \neq x_e$);
- W is *negative (semi)definite* if $-W$ is positive (semi)definite;
- W is *radially unbounded* if $W(x) \to \infty$ as $\|x\| \to \infty$.

The derivative of a C^1 function along the system trajectories is $\dot{V}(x) = \frac{\partial V}{\partial x} f(x)$.

Theorem 2.4 (Lyapunov's direct method)**.** *Let $V : D \to \mathbb{R}$ be C^1 and positive definite. Then*
(i) $\dot{V} \leq 0$ on D (NSD) $\Rightarrow 0$ is stable;
(ii) $\dot{V} < 0$ for $x \neq 0$ (ND) $\Rightarrow 0$ is asymptotically stable;

(iii) $D = \mathbb{R}^n$, V *radially unbounded and* $\dot{V}$ *ND* $\Rightarrow 0$ *is globally asymptotically stable.*

Sketch of (i). Given ε, pick $r < \varepsilon$ with $B_r \subset D$ and let $m = \min_{\|x\|=r} V > 0$ (positive definiteness). By continuity of V and $V(0) = 0$ there is δ with $V < m$ on B_δ. Along a trajectory starting in B_δ, V is nonincreasing, hence stays below m, so the trajectory cannot reach the sphere $\|x\| = r$: it remains in $B_r \subset B_\varepsilon$. □

Intuition

Think of V as the *energy* (or the height of a bowl) and of the system as a ball with friction. If energy never increases ($\dot{V} \leq 0$), a ball started close to the bottom with little energy cannot climb far: that is stability. If energy *strictly* decreases away from the bottom, friction is acting everywhere and the ball must come to rest at the bottom: asymptotic stability. The great advantage is that one never solves the differential equation: only the sign of $\dot{V} = \nabla V^T f$ is needed.

Example 2.5. $\dot{x} = -x^3$. With $V = \frac{1}{2}x^2$ (PD, radially unbounded) we get $\dot{V} = -x^4 < 0$ for $x \neq 0$: the origin is GAS (though not exponentially, since the linearization is $\dot{x} = 0$).

When $\dot{V}$ is only negative semidefinite: LaSalle versus Barbalat. Often $\dot{V}$ vanishes on a whole set of states, so Theorem 2.4(ii) does not apply directly. Usually, one resorts to *LaSalle's invariance principle* (Theorem 2.6). However, LaSalle's theorem strictly requires a **compact** (bounded and closed) positively invariant set Ω. In Cartesian regulation, our Lyapunov candidate is

$$V(q) = \tfrac{1}{2}(x^2 + y^2) \qquad (q = (x, y, \theta)^T \in \mathbb{R}^3).$$

Notice that $V(q) = 0$ for all points $(0, 0, \theta)^T$, i.e. along the *entire vertical* θ*-axis* (Figure 99). Hence V is only **positive semidefinite (PSD)** in the unicycle state space $\mathbb{R}^3$. Consequently, its sublevel sets $\{q \in \mathbb{R}^3 : V(q) \leq c\}$ are cylinders that extend infinitely along θ, violating the compactness hypothesis of LaSalle's principle! For that reason, we turn to **Barbalat's lemma**.

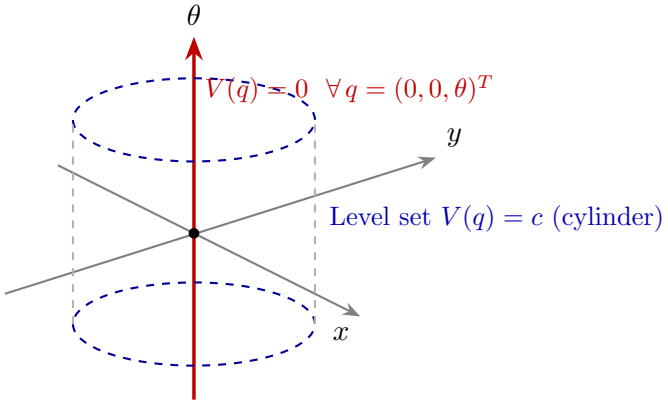


Figure 99: The Lyapunov candidate $V(q) = \frac{1}{2}(x^2 + y^2)$ in configuration space (x, y, θ): $V = 0$ along the entire vertical θ-axis, so V is only positive *semidefinite* (PSD) in $\mathbb{R}^3$. Its sublevel sets are infinite cylinders along θ, violating the compactness required by LaSalle's principle.

Theorem 2.6 (LaSalle's invariance principle)**.** *Let* Ω *be a compact positively invariant set of* $\dot{x} = f(x)$ *and* V *a* C^1 *function with* $\dot{V} \leq 0$ *on* Ω*. Let* $E = \{x \in \Omega : \dot{V}(x) = 0\}$ *and let* M *be the largest invariant set contained in* E*. Then every solution starting in* Ω *converges to* M*.*

In particular, if $M = \{0\}$ (and V is PD) the origin is asymptotically stable.

Example 2.7. Damped oscillator $\ddot{x} + \dot{x} + x = 0$, i.e. $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 - x_2$. With $V = \frac{1}{2}(x_1^2 + x_2^2)$, $\dot{V} = -x_2^2 \leq 0$ only NSD. On $E = \{x_2 = 0\}$ the dynamics give $\dot{x}_2 = -x_1$, which keeps $x_2 = 0$ only if $x_1 = 0$: the largest invariant set in E is $M = \{0\}$, so the origin is (globally) asymptotically stable.

Pedagogical motivation of Barbalat's lemma. To appreciate why Barbalat's lemma is needed, consider the following general result for bounded monotonic functions:

Proposition 2.8 (Convergence of bounded monotonic functions)**.** *If a function $f : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is always non-negative ($f(t) \geq 0$) and its time derivative is always non-positive ($\dot{f}(t) \leq 0$), then $f(t)$ is non-increasing and bounded below, so $f(t)$ tends to a finite limit $\bar{f} \geq 0$ as $t \to \infty$:*

$$f(t) \geq 0, \quad \dot{f}(t) \leq 0 \implies \lim_{t\to\infty} f(t) = \bar{f}.$$

Intuitively, one might jump to the conclusion that if $f(t)$ settles to a constant limit $\bar{f}$, its rate of change must vanish ($\dot{f}(t) \to 0$). **However, this is not true in general**:

$$f(t) \geq 0, \quad \dot{f}(t) \leq 0 \implies f(t) \to \bar{f} \not\implies \dot{f}(t) \to 0.$$

A function can converge while its derivative oscillates violently or produces vanishingly narrow spikes whose area under the curve is finite without $\dot{f}$ going to zero. For example, $f(t) = e^{-t}\sin(e^{2t})$ converges to zero, but its derivative $\dot{f}(t) = -e^{-t}\sin(e^{2t}) + 2e^{t}\cos(e^{2t})$ oscillates with an amplitude growing as e^t, which is unbounded as $t \to \infty$. Barbalat's lemma provides the missing condition: if, in addition, the second time derivative is bounded ($|\ddot{f}(t)| \leq b$, which implies that $\dot{f}(t)$ is uniformly continuous), then indeed $\dot{f}(t) \to 0$ as $t \to \infty$.

Lemma 2.9 (Barbalat)**.** *If $f : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is uniformly continuous and $\lim_{t\to\infty} \int_0^t f(\tau)\, d\tau$ exists and is finite, then $f(t) \to 0$.*

Corollary 2.10 (Lyapunov-like lemma / Barbalat in time domain)**.** *If a C^1 function $V(t)$ is bounded below ($V(t) \geq 0$), $\dot{V}(t) \leq 0$, and $\ddot{V}(t)$ is bounded ($|\ddot{V}(t)| \leq b$, so that $\dot{V}$ is uniformly continuous), then*

$$\lim_{t\to\infty} \dot{V}(t) = 0.$$

Proof. $V(t) = V(0) + \int_0^t \dot{V}$ is nonincreasing and bounded below, so it has a finite limit; apply Lemma 2.9 to $f = \dot{V}$. □

Uniform continuity cannot be dropped: $f(t) = \sin(t^2)$ has a convergent integral (Fresnel) but does not tend to zero. Note also that Barbalat's lemma only says that $\dot{V} \to 0$; it does *not* say that the state converges. One must then read the information contained in $\dot{V} \to 0$ (e.g. $\rho\cos\gamma \to 0$ and $\gamma \to 0$) and complete the argument by hand, as done below.

Intuition

The three tools form a ladder of weakening assumptions. Lyapunov's theorem needs a perfect "bowl". LaSalle accepts a bowl with flat or friction-free spots, provided the ball cannot stay on them (the largest invariant set M). Barbalat is the most permissive and also works when the system is not exactly autonomous or the vector field is not continuous, but it only tells you that the friction power $\dot{V}$ fades out, and you must deduce from it where the ball actually

Tool	Hypotheses	Needs V PD?	Conclusion
Lyapunov direct	V PD, $\dot{V} \leq 0$ (ND for AS)	yes	stability / AS / GAS
LaSalle	compact invariant Ω, $\dot{V} \leq 0$, autonomous, continuous f	not necessarily	convergence to $M \subseteq \{\dot{V} = 0\}$
Barbalat	V bounded below, $\dot{V} \leq 0$, $\ddot{V}$ bounded	no	$\dot{V} \to 0$ (state needs extra argument)

Table 11: Tools for Lyapunov-based convergence analysis.

is.

2.4 Convergence analysis *(regulation slide 5)*

Does the law work? Consider the Lyapunov candidate corresponding to the squared distance of the contact point (x, y) to the origin $(0, 0)$:

$$V = \tfrac{1}{2}(x^2 + y^2) \geq 0 \qquad \text{(positive semidefinite in } \mathbb{R}^3\text{)}.$$

Computing its time derivative along the closed-loop system trajectory:

$$\dot{V} = \tfrac{1}{2}(2x\dot{x} + 2y\dot{y}) = x\dot{x} + y\dot{y} = x(v\cos\theta) + y(v\sin\theta) = v(x\cos\theta + y\sin\theta).$$

Substituting the Cartesian regulation law $v = -k_1(x\cos\theta + y\sin\theta)$:

$$\dot{V} = -k_1(x\cos\theta + y\sin\theta)^2 \leq 0 \qquad \text{(negative semidefinite)}.$$

Since V represents the Cartesian error and $\dot{V} \leq 0$, the error never increases ($V(t) \leq V(0)$ for all $t \geq 0$); however, because V is only PSD in (x, y, θ), LaSalle's theorem cannot be applied directly. We therefore apply Barbalat's lemma through Corollary 2.10.

Proposition 2.11 (Cartesian regulation)**.** *Under the Cartesian regulation law (with γ wrapped in $(-\pi, \pi]$), the Cartesian error goes to zero:* $\lim_{t\to\infty} x(t) = 0$ *and* $\lim_{t\to\infty} y(t) = 0$.

Proof. The function $V \geq 0$ and $\dot{V} \leq 0$, so $V(t)$ converges to a finite limit and the coordinates $x(t), y(t)$ stay bounded. Letting $s = x\cos\theta + y\sin\theta$, we have $\dot{V} = -k_1 s^2$, and its second derivative is

$$\ddot{V} = -2k_1 s\dot{s}, \qquad \dot{s} = v + \omega(-x\sin\theta + y\cos\theta).$$

Since x, y, $v = -k_1 s$, and $\omega = k_2\gamma$ (with $\gamma \in (-\pi, \pi]$) are all bounded, $\dot{s}$ and hence $\ddot{V}$ are bounded. Applying Barbalat's lemma (Corollary 2.10), we obtain:

$$\lim_{t\to\infty} \dot{V}(t) = \lim_{t\to\infty} -k_1(x\cos\theta + y\sin\theta)^2 = 0 \iff \lim_{t\to\infty} (x\cos\theta + y\sin\theta) = 0.$$

Recalling that the projection of the Cartesian error on the sagittal axis is $e_p^T n = -(x\cos\theta + y\sin\theta)$, Barbalat's lemma proves that

$$\lim_{t\to\infty} e_p^T n = 0.$$

This states that the projection of e_p on the sagittal axis tends to zero (the error vector and the sagittal axis become perpendicular). However, Barbalat's lemma alone does not immediately state whether e_p itself goes to zero. To resolve this, we analyze the possible steady states where $e_p^T n = 0$ (Figure 100):

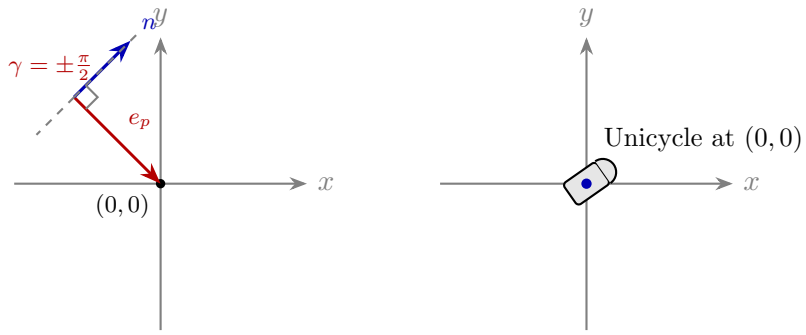
- **Scenario 1** ($e_p \neq 0$): The robot is broadside to the origin, so $e_p \perp n$. Here $v = 0$, but the pointing error is $\gamma = \pm\pi/2$. Looking at the angular velocity command:

$$\omega = k_2\gamma = \pm k_2\frac{\pi}{2} \neq 0.$$

  Because $\omega \neq 0$, the wheels must turn, rotating the unicycle away from the perpendicular configuration. Therefore, Scenario 1 is **not a steady state**.

- **Scenario 2** ($e_p = 0$): The Cartesian error is zero, meaning the wheel contact point is at the origin $(0, 0)$.

We conclude that Scenario 2 is the **only feasible steady state**, and therefore $e_p(t) \to 0$, i.e. $x(t) \to 0$ and $y(t) \to 0$. □



Scenario 1: $e_p \neq 0$ — $\gamma \neq 0 \implies \omega \neq 0$ (Not steady state!)

Scenario 2: $e_p = 0$ — $e_p = 0$ (Unique feasible steady state)

Figure 100: The two possible steady-state scenarios when $e_p^T n \to 0$: in Scenario 1 ($e_p \neq 0$) the sagittal axis n is orthogonal to e_p, so $\gamma = \pm\pi/2$ and $\omega = k_2\gamma \neq 0$, which continuously rotates the robot and breaks perpendicularity; thus Scenario 2 ($e_p = 0$) is the only feasible steady state.

Intuition

The distance can only decrease while v is nonzero, and v vanishes only when the robot is exactly broadside to the target; but there the steering term is at its maximum and immediately swings the robot back towards the target. So the robot cannot get "stuck" anywhere except at the goal. Note what is *not* controlled: θ. It ends wherever the approach direction leaves it.

2.5 Simulation *(regulation slide 6)*

- The final orientation is not controlled: it depends on the approach direction.
- There is at most one backup maneuver (when the target starts behind the robot and $v < 0$).

Intuition

The three runs end at the origin with three different headings ($\approx 51°$, $-16°$, $154°$): the controller has no mechanism to choose one. This is why a second law, which also uses the orientation error, is needed for *posture* regulation.

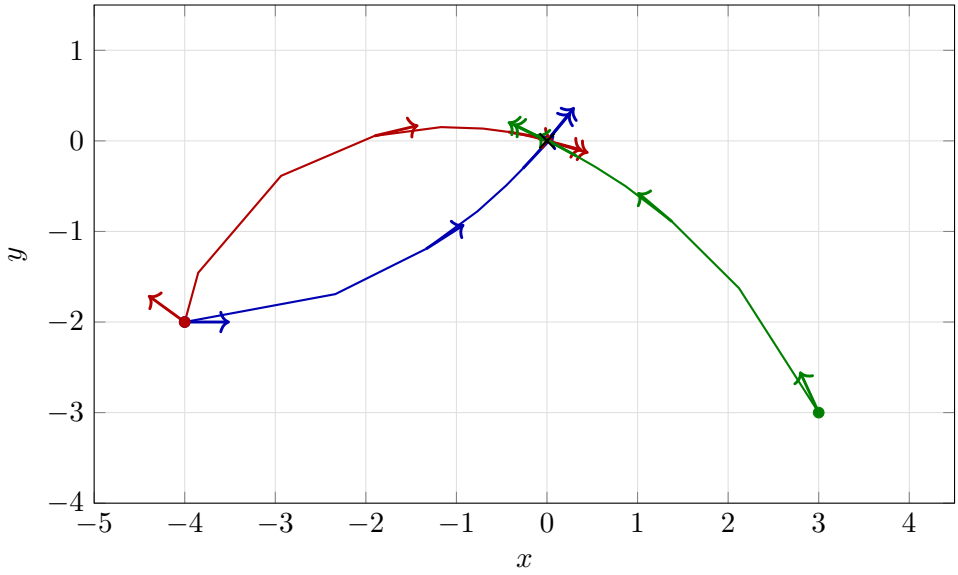


Figure 101: Cartesian regulation ($k_1 = 1$, $k_2 = 2$), three initial configurations (dots); arrows show the heading along the path. The origin is reached from every start, but the final heading differs. The second run starts with the target behind the robot and backs up first.

3 Posture regulation *(regulation slides 7–10)*

3.1 Polar coordinates *(regulation slide 7)*

We now tackle the full posture regulation problem: driving the unicycle to a desired configuration $q_d = (x_d, y_d, \theta_d)^T$, where orientation is controlled alongside position. Without loss of generality, we take the origin $q_d = (0, 0, 0)^T$ as the goal.

In Cartesian regulation, the orientation θ was left uncontrolled because it did not enter the Lyapunov candidate $V = \frac{1}{2}(x^2 + y^2)$. To design a controller that simultaneously steers position and orientation, we represent the kinematic model in **polar coordinates**. This choice decouples distance, pointing direction, and orientation error, making the control design remarkably simple.

Definition 3.1 (Polar coordinates)**.** The polar coordinates of the unicycle with respect to the origin $q_d = (0, 0, 0)^T$ are:

$$\rho = \sqrt{x^2 + y^2}, \qquad \gamma = \mathrm{Atan2}(y, x) - \theta + \pi, \qquad \delta = \gamma + \theta = \mathrm{Atan2}(y, x) + \pi.$$

Their geometric meanings are (Figure 102):

- ρ: the Euclidean *distance* from the wheel contact point to the origin;
- γ: the *pointing error*, i.e. the angular offset between the robot heading θ and the line of sight pointing to the goal;
- δ: the *orientation of the line of sight* e_p with respect to the world x axis.

The inverse transformation expressing Cartesian coordinates in terms of polar quantities is:

$$x = -\rho \cos \delta, \qquad y = -\rho \sin \delta, \qquad \theta = \delta - \gamma.$$

Note on indefiniteness at the origin. Because γ and δ are computed using $\mathrm{Atan2}(y, x)$, they are **undefined at the Cartesian origin** $(x, y) = (0, 0)$. However, if the polar quantities converge to zero ($\rho \to 0$, $\gamma \to 0$, $\delta \to 0$), then the Cartesian posture also converges to the origin: $x \to 0$, $y \to 0$, and $\theta = \delta - \gamma \to 0$.

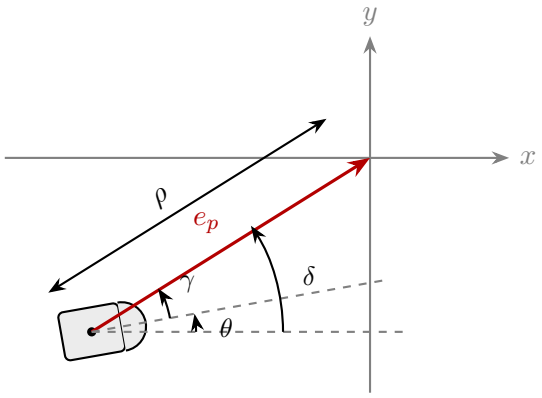


Figure 102: Polar coordinates of the unicycle with respect to the origin: ρ is the distance, γ the pointing error and $\delta = \gamma + \theta$ the orientation of e_p with respect to the x axis.

Intuition

Instead of asking "where am I in the plane", ask three questions: how far is the goal (ρ), how far off is my nose from pointing at it (γ), and from which side of the goal am I looking at it (δ). The third angle is what encodes the final orientation: at the goal, $\theta = \delta - \gamma$, so making *both* angles vanish forces $\theta = 0$. The price is that the coordinates are singular at the origin itself.

3.2 Kinematic model in polar coordinates *(regulation slide 8)*

Proposition 3.2 (Kinematic model in polar coordinates)**.** *For $\rho \neq 0$, the unicycle kinematics in polar coordinates (ρ, γ, δ) are*

$$\dot{\rho} = -v\cos\gamma, \qquad \dot{\gamma} = \frac{\sin\gamma}{\rho}\, v - \omega, \qquad \dot{\delta} = \frac{\sin\gamma}{\rho}\, v.$$

Proof. Let us derive each equation step by step.

Derivation of $\dot{\rho}$: Differentiating $\rho = (x^2 + y^2)^{1/2}$ with respect to time:

$$\dot{\rho} = \frac{d}{dt}(x^2+y^2)^{1/2} = \frac{1}{2}(x^2+y^2)^{-1/2}(2x\dot{x} + 2y\dot{y}) = \frac{x\dot{x} + y\dot{y}}{\sqrt{x^2+y^2}} = \frac{x\cos\theta\, v + y\sin\theta\, v}{\rho}.$$

To simplify this expression, introduce the polar angle $\beta = \mathrm{Atan2}(y, x)$ of the position vector (Figure 103), so that

$$x = \rho\cos\beta, \qquad y = \rho\sin\beta.$$

Substituting into $\dot{\rho}$:

$$\dot{\rho} = \frac{\rho\cos\beta\cos\theta\, v + \rho\sin\beta\sin\theta\, v}{\rho} = v(\cos\beta\cos\theta + \sin\beta\sin\theta) = v\cos(\beta - \theta).$$

Recalling that $\cos\alpha = -\cos(\alpha + \pi)$ and that $\gamma = \mathrm{Atan2}(y, x) - \theta + \pi = \beta - \theta + \pi$:

$$\cos(\beta - \theta) = -\cos(\beta - \theta + \pi) = -\cos\gamma \implies \dot{\rho} = -v\cos\gamma.$$

Derivation of $\dot{\gamma}$: From $\gamma = \text{Atan2}(y, x) - \theta + \pi$, we have

$$\dot{\gamma} = \frac{d}{dt}\text{Atan2}(y, x) - \dot{\theta} = \frac{d}{dt}\text{Atan2}(y, x) - \omega.$$

Applying the chain rule to differentiate $\text{Atan2}(y, x)$:

$$\frac{d}{dt}\text{Atan2}(y, x) = \frac{\partial \text{Atan2}(y, x)}{\partial x}\dot{x} + \frac{\partial \text{Atan2}(y, x)}{\partial y}\dot{y}.$$

Since $\frac{\partial \arctan(y/x)}{\partial x} = -\frac{y}{x^2+y^2}$ and $\frac{\partial \arctan(y/x)}{\partial y} = \frac{x}{x^2+y^2}$, we obtain

$$\frac{d}{dt}\text{Atan2}(y, x) = -\frac{y}{\rho^2}\dot{x} + \frac{x}{\rho^2}\dot{y} = \frac{x\dot{y} - y\dot{x}}{\rho^2} = \frac{x\sin\theta\, v - y\cos\theta\, v}{\rho^2}.$$

Substituting $x = \rho\cos\beta$ and $y = \rho\sin\beta$:

$$\frac{d}{dt}\text{Atan2}(y, x) = \frac{v}{\rho}(\cos\beta\sin\theta - \sin\beta\cos\theta) = -\frac{v}{\rho}(\sin\beta\cos\theta - \cos\beta\sin\theta) = -\frac{v}{\rho}\sin(\beta - \theta).$$

Using $-\sin\alpha = \sin(\alpha + \pi)$ and $\beta - \theta + \pi = \gamma$:

$$-\sin(\beta - \theta) = \sin(\beta - \theta + \pi) = \sin\gamma \implies \frac{d}{dt}\text{Atan2}(y, x) = \frac{v\sin\gamma}{\rho}.$$

Therefore,

$$\dot{\gamma} = \frac{\sin\gamma}{\rho}\, v - \omega.$$

Derivation of $\dot{\delta}$: Since $\delta = \gamma + \theta$,

$$\dot{\delta} = \frac{d}{dt}(\gamma + \theta) = \dot{\gamma} + \dot{\theta} = \left(\frac{\sin\gamma}{\rho}\, v - \omega\right) + \omega = \frac{\sin\gamma}{\rho}\, v.$$

This completes the derivation of the kinematic model. □

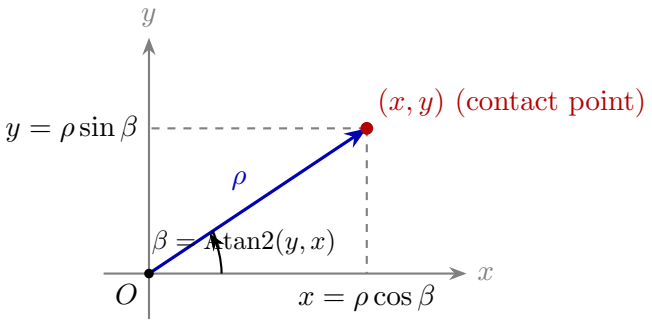


Figure 103: Auxiliary polar angle $\beta = \text{Atan2}(y, x)$: the contact point coordinates are $x = \rho\cos\beta$ and $y = \rho\sin\beta$, which simplifies the differentiation of ρ and γ.

[Behavior when reaching the origin: singularity resolution] Our ultimate control goal is to reach $(x, y, \theta) = (0, 0, 0)$, which corresponds to $(\rho, \gamma, \delta) \to (0, 0, 0)$. Looking at the polar kinematic model, ρ appears in the denominator of $\dot{\gamma}$ and $\dot{\delta}$, which might suggest a risk of divergence ($\dot{\gamma} \to \infty$, $\dot{\delta} \to \infty$) as $\rho \to 0$. However, this is **not an issue**: in the control law synthesized below, the driving velocity v is chosen proportional to ρ ($v = k_1\rho\cos\gamma$), so that the ratio

$$\frac{v}{\rho} = k_1\cos\gamma$$

remains completely bounded by k_1. Therefore, the coupling term

$$\frac{\sin\gamma}{\rho}\,v = k_1 \sin\gamma\cos\gamma$$

is smooth, well-defined, and vanishes as $\gamma \to 0$. The apparent kinematic singularity is effectively tamed by the control law.

3.3 Control law design via Lyapunov's direct method *(regulation slides 8–9)*

The Lyapunov direct method can be used not only as an a-posteriori verification tool, but as a **constructive synthesis method** to design the feedback control law.

Design procedure. We start by defining a candidate Lyapunov function representing the total error "energy":

$$V = \tfrac{1}{2}\left(\rho^2 + \gamma^2 + \delta^2\right) > 0 \qquad \text{(positive definite in polar coordinates)}.$$

Computing its time derivative along the trajectories of the polar kinematic model:

$$\dot{V} = \rho\dot{\rho} + \gamma\dot{\gamma} + \delta\dot{\delta}.$$

Plugging in the equations for $\dot{\rho}$, $\dot{\gamma}$, and $\dot{\delta}$ from Proposition 3.2:

$$\dot{V} = \rho(-v\cos\gamma) + \gamma\left(\frac{\sin\gamma}{\rho}\,v - \omega\right) + \delta\left(\frac{\sin\gamma}{\rho}\,v\right) = -\rho v\cos\gamma + \frac{\sin\gamma}{\rho}(\gamma+\delta)v - \gamma\omega.$$

We now synthesize the inputs v and ω in three consecutive steps:

1. **Step 1: Choice of v.** Examine the first term $-\rho v\cos\gamma$. To ensure this term is non-positive, we set

   $$v = k_1\rho\cos\gamma \qquad (k_1 > 0).$$

   This makes $-\rho v\cos\gamma = -k_1\rho^2\cos^2\gamma \le 0$. The parameter $k_1 > 0$ provides tuning freedom to adjust the linear approach speed. This would still work without ρ in the expression, but then the robot would slow down as it approaches the origin, which is undesirable for posture regulation, and so ρ will act as a gain.

2. **Step 2: Decomposition of ω.** With v determined by the first term, $\dot{V}$ becomes:

   $$\dot{V} = \underbrace{-k_1\rho^2\cos^2\gamma}_{\le 0} + \underbrace{\frac{\sin\gamma}{\rho}(\gamma+\delta)v}_{\text{cross coupling}} - \underbrace{\gamma\omega}_{\text{control input}}.$$

   Because v was chosen strictly to stabilize ρ, we cannot use it to govern the sign of the cross-coupling second term. We must therefore use the steering input ω. We decompose ω into two parts:

   $$\omega = k_2\gamma + \alpha \qquad (k_2 > 0),$$

   so that the control term expands to $-\gamma\omega = -k_2\gamma^2 - \gamma\alpha$, where $-k_2\gamma^2 \le 0$ acts as a stabilizing negative damping on γ.

3. **Step 3: Cross-term cancellation via α.** We choose the auxiliary term α to identically cancel the cross-coupling term:

   $$\frac{\sin\gamma}{\rho}(\gamma+\delta)v - \gamma\alpha = 0 \iff \alpha = \frac{\sin\gamma}{\gamma\rho}(\gamma+\delta)v.$$

   Substituting $v = k_1\rho\cos\gamma$:

   $$\alpha = \frac{\sin\gamma}{\gamma\rho}(\gamma+\delta)\big(k_1\rho\cos\gamma\big) = k_1\frac{\sin\gamma\cos\gamma}{\gamma}(\gamma+\delta).$$

Combining the terms yields the complete posture regulation law:

Definition 3.3 (Posture regulation law)**.** For $k_1, k_2 > 0$,

$$v = k_1\,\rho\cos\gamma, \qquad \omega = k_2\,\gamma + k_1\,\frac{\sin\gamma\cos\gamma}{\gamma}\,(\gamma+\delta),$$

where $\frac{\sin\gamma\cos\gamma}{\gamma} \to 1$ as $\gamma \to 0$. Compared with Cartesian regulation, v has the same form (since $\rho\cos\gamma = -(x\cos\theta + y\sin\theta)$), while ω incorporates an additional correction proportional to $\gamma+\delta$. Under this control law, the cross terms in $\dot{V}$ cancel identically, leaving:

$$\dot{V} = -k_1\rho^2\cos^2\gamma - k_2\gamma^2 \le 0 \qquad \text{(negative semidefinite)}.$$

Algorithm 3.4 (Posture regulation)**.** At each control instant:

1. measure (x, y, θ) and compute $\rho = \sqrt{x^2+y^2}$, $\gamma = \mathrm{Atan2}(y, x) - \theta + \pi$ (wrapped in $(-\pi, \pi]$) and $\delta = \gamma + \theta$;
2. compute $v = k_1\rho\cos\gamma$ and $\omega = k_2\gamma + k_1\frac{\sin\gamma\cos\gamma}{\gamma}(\gamma+\delta)$, using the limit value 1 for the fraction when $\gamma = 0$;
3. send (v, ω) to the unicycle.

Intuition

The new term in ω is what takes care of the final orientation. Numerical example at $(x, y, \theta) = (-4, -2, 0)$ with $k_1 = 1$, $k_2 = 3$: $\rho \approx 4.47$, $\gamma = \delta \approx 0.46$, so $v \approx 4.0$ and $\omega \approx 1.39 + 0.80 = 2.19$ rad/s, against 1.39 rad/s for the pure pointing term. The robot turns *more* than needed to point at the origin, so that it arrives rotated in the correct direction, in the style of a car aligning with a parking spot as it approaches.

3.4 Convergence analysis and Brockett's theorem *(regulation slide 9)*

Proposition 3.5 (Posture regulation)**.** *Under the posture regulation law,* $\dot{V} = -k_1\cos^2\gamma\,\rho^2 - k_2\gamma^2 \le 0$ *(negative semidefinite), and* $\rho, \gamma, \delta \to 0$*; hence* $x, y, \theta \to 0$.

Proof. Substituting the control law, the closed-loop system is

$$\dot{\rho} = -k_1\rho\cos^2\gamma, \qquad \dot{\gamma} = -k_2\gamma - k_1\frac{\sin\gamma\cos\gamma}{\gamma}\,\delta, \qquad \dot{\delta} = k_1\sin\gamma\cos\gamma.$$

Then

$$\dot{V} = \rho\dot{\rho} + \gamma\dot{\gamma} + \delta\dot{\delta} = -k_1\rho^2\cos^2\gamma - k_2\gamma^2 - k_1\sin\gamma\cos\gamma\,\delta + k_1\delta\sin\gamma\cos\gamma = -k_1\cos^2\gamma\,\rho^2 - k_2\gamma^2.$$

V is positive definite and $\dot{V}$ only negative semidefinite, so by Barbalat's lemma $\dot{V} \to 0$, i.e. $\rho\cos\gamma \to 0$ and $\gamma \to 0$; thus also $\rho \to 0$. For δ: $\gamma \to 0$ and (again by Barbalat, since $\ddot{\gamma}$ is bounded) $\dot{\gamma} \to 0$, so from the expression of $\dot{\gamma}$ the term $k_1\frac{\sin\gamma\cos\gamma}{\gamma}\delta \to 0$; as the fraction tends to 1, $\delta \to 0$. □

[Discontinuity of the control law in Cartesian coordinates] The control law was designed in polar coordinates (ρ, γ, δ). Once mapped back to the original Cartesian state $q = (x, y, \theta)^T$, the law is **discontinuous** at the origin $(0, 0, 0)$ because $\mathrm{Atan2}(y, x)$ has an essential singularity at $x = y = 0$.

[Practical behavior: why discontinuity is not an issue] In real-world implementations, **this discontinuity is never practically encountered**:

- The physical robot asymptotically converges toward the origin, but because of discrete-time sampling, sensor resolution limits, and physical disturbances, the state never lands mathematically on the exact singular point $(x, y) = (0, 0)$.
- A microscopic residual error always persists, keeping $\rho > 0$ and the control inputs well-defined at all times until the control task is terminated.

This discontinuity is by no means an accident of the chosen polar coordinates; it is a fundamental consequence of nonholonomy:

Theorem 3.6 (Brockett)**.** *Consider a driftless system $\dot{q} = G(q)u$ with* $\operatorname{rank} G = m < n$*. A necessary condition for the existence of a continuous time-invariant feedback $u = k(q)$ asymptotically stabilizing an equilibrium q_d is that the map $(q, u) \mapsto G(q)u$ be onto an open neighborhood of* 0*. This fails for nonholonomic systems; hence all posture stabilizers must be* discontinuous with respect to the state *or* time-varying*.*

Verification for the unicycle. Around $\theta = 0$ we have $\cos\theta \neq 0$. The velocity vector

$$(\dot{x}, \dot{y}, \dot{\theta})^T = (v\cos\theta,\, v\sin\theta,\, \omega)^T$$

can match $(0, \varepsilon, 0)^T$ only if $v\cos\theta = 0$, which implies $v = 0$, and therefore $\dot{y} = 0 \neq \varepsilon$. Consequently, points of the form $(0, \varepsilon, 0)^T$, however small $\varepsilon > 0$ is chosen, are never in the image of the velocity map. □

Intuition

A continuous static feedback would give small controls near the goal and thus small velocities, but a small *sideways* offset can only be fixed by moving *away* and coming back with a different heading (the parallel-parking manoeuvre), which is a large motion, not a small one. Two ways out: allow the feedback to jump (discontinuous, as here: the law is not defined at $\rho = 0$) or let it depend on time (time-varying laws oscillate to generate the sideways motion).

3.5 Simulation *(regulation slide 10)*

- The final orientation is zeroed as well, regardless of the initial configuration.
- There is at most one backup maneuver.

Part XIII

Mobile manipulators

4 Mobile manipulators *(manipulator slides 1–2)*

Definition 4.1 (Mobile manipulator)**.** A *mobile manipulator* (MM) is a robot made of a mobile base carrying one or more manipulators. In the vast majority of cases the base is *wheeled* and the manipulator is an *articulated arm*. Its configuration is

$$q = (\,,\,),$$

where $\in \mathbb{R}^{n_b}$ collects the n_b generalized coordinates of the base (e.g. (x, y, θ) for a planar vehicle) and $\in \mathbb{R}^{n_m}$ collects the n_m joint coordinates of the arm. The total number of coordinates is $n = n_b + n_m$.

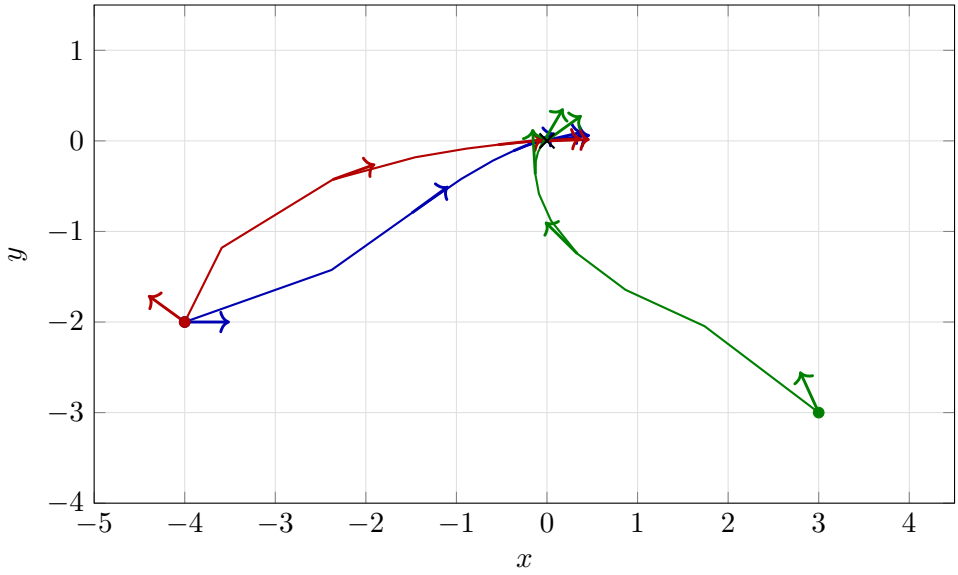


Figure 104: Posture regulation ($k_1 = 1$, $k_2 = 3$) from the same three initial configurations (dots): the robot reaches the origin *and* the final heading is $\theta = 0$ in all cases (arrows along the x axis at the origin).

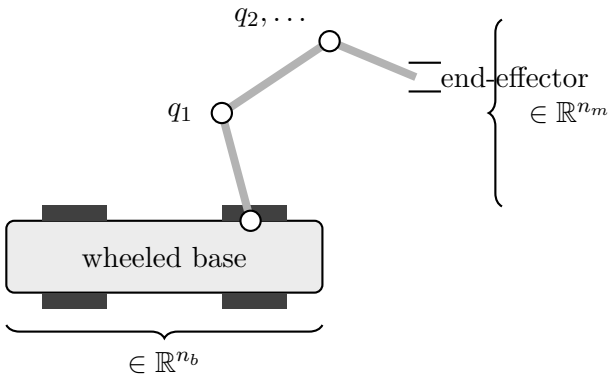


Figure 105: Structure of a mobile manipulator: the configuration splits into base coordinates and arm joint coordinates .

Intuition

Think of an MM as a "manipulator that can move its own shoulder". The base extends the workspace of the arm enormously (the arm can reach anything the base can drive to), and, as we will see, it also adds degrees of freedom that remove the singularities of the arm alone. The price is that the base is usually *nonholonomic*: it has fewer controls than coordinates, so the two halves of the robot behave very differently.

5 Kinematic model *(manipulator slide 3)*

A mobile manipulator is the mechanical composition of two fundamentally different kinematic subsystems:

1. a **constrained robot** (the mobile base), subject to nonholonomic velocity constraints;
2. an **unconstrained robot** (the manipulator arm), whose joints are directly actuated without differential constraints.

Constrained robot: the mobile base

A kinematic system is called *constrained* when its generalized velocities cannot be commanded arbitrarily in all directions, but must satisfy non-integrable (nonholonomic) constraints.

Example 5.1 (Constrained robot: unicycle base)**.** Consider a unicycle (or differential-drive vehicle) moving on a plane. Its configuration is described by the base generalized coordinates

$$= (x, y, \theta)^T \in \mathbb{R}^{n_b}, \qquad (n_b = 3),$$

where (x, y) represents the position of the wheel contact point and θ is the vehicle heading. The wheels cannot slide sideways (pure rolling and no slip condition), imposing the nonholonomic constraint in Pfaffian form:

$$A_b()\dot{} = \dot{x}\sin\theta - \dot{y}\cos\theta = \begin{pmatrix}\sin\theta & -\cos\theta & 0\end{pmatrix}\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = 0.$$

The subspace of admissible generalized velocities at configuration is the null space of $A_b()$, which has dimension $m_b = n_b - 1 = 2$. Choosing a basis for $\ker A_b()$, the admissible velocities are parameterized by $m_b = 2$ *pseudovelocities* $= (v, \omega)^T$, where v is the driving linear velocity and ω is the steering angular velocity:

$$\dot{} = \begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = \begin{pmatrix}\cos\theta & 0\\ \sin\theta & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}v\\ \omega\end{pmatrix} = ()\,.$$

Because $m_b = 2 < 3 = n_b$, the base is kinematically **constrained**: instantaneous motion in the lateral direction $\begin{pmatrix}-\sin\theta & \cos\theta & 0\end{pmatrix}^T$ is strictly forbidden, even though the vehicle can reach any configuration (x, y, θ) through appropriate maneuvers.

Unconstrained robot: the manipulator arm

In contrast, a standard articulated manipulator mounted on the base has joints that are independently driven by individual actuators.

Example 5.2 (Unconstrained robot: articulated manipulator arm)**.** Consider a serial manipulator with n_m revolute (or prismatic) joints, with joint coordinates

$$= (q_1, q_2, \ldots, q_{n_m})^T \in \mathbb{R}^{n_m}.$$

Unlike the wheeled base, the arm is subject to **no nonholonomic constraints**. Each joint velocity $\dot{q}_j$ can be independently and directly commanded by the joint velocity inputs:

$$= (\dot{q}_1, \dot{q}_2, \ldots, \dot{q}_{n_m})^T \in \mathbb{R}^{m_m}, \qquad (m_m = n_m).$$

Consequently, the kinematic model of the arm is simply:

$$\dot{} = I_{n_m}\ = .$$

Because the number of velocity inputs equals the dimension of the configuration space ($m_m = n_m$), the robot is kinematically **unconstrained**: at any configuration , the arm can instantaneously move with any generalized velocity $\dot{} \in \mathbb{R}^{n_m}$ without restriction. For example, for a planar 2R arm ($n_m = 2$), $= (q_1, q_2)^T$ and $\dot{} = \begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}\dot{q}_1\\ \dot{q}_2\end{pmatrix}$.

Combined kinematic model

Definition 5.3 (Kinematic model of a mobile manipulator)**.** Stacking the constrained base subsystem and the unconstrained arm subsystem, the complete mobile manipulator configuration is $q = (,) \in \mathbb{R}^n$ with $n = n_b + n_m$, driven by the stacked velocity inputs $u = (,) \in \mathbb{R}^m$ with $m = m_b + n_m < n$:

$$\dot{q} = \begin{pmatrix} \dot{} \\ . \end{pmatrix} = (()) = \begin{pmatrix} () & O \\ O & I_{n_m} \end{pmatrix} () = G(q)\, u.$$

The overall system is a *driftless nonlinear control system.* The matrix $G(q)$ has dimension $n \times m = (n_b + n_m) \times (m_b + n_m)$ and is block-diagonal.

Example 5.4 (Unicycle base with a planar 2R arm)**.** For a mobile manipulator composed of a unicycle base ($n_b = 3, m_b = 2$) and a planar 2R arm ($n_m = 2, m_m = 2$), the state has dimension $n = 5$ and the input has dimension $m = 4$. The explicit kinematic model is:

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{q}_1 \\ \dot{q}_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 & 0 & 0 \\ \sin\theta & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ \omega \\ \dot{q}_1 \\ \dot{q}_2 \end{pmatrix} = \begin{pmatrix} () & O_{3\times 2} \\ O_{2\times 2} & I_{2\times 2} \end{pmatrix} () .$$

Notice how the upper 3×2 block embodies the nonholonomic constraint of the base ($m_b < n_b$), while the lower 2×2 block is the unconstrained identity matrix ($m_m = n_m$).

Proposition 5.5 (Controllability of an MM)**.** *The MM kinematic model is controllable if and only if the base kinematic model is controllable.*

Proof. The system is driftless and (in the cases of interest) analytic, so by Chow's theorem it is controllable iff the Lie algebra generated by its input vector fields has dimension n at every configuration. The input vector fields are

$$g_i = \begin{pmatrix} g_{b,i}() \\ 0 \end{pmatrix} \ (i = 1, \dots, m_b), \qquad h_j = \begin{pmatrix} 0 \\ e_j \end{pmatrix} \ (j = 1, \dots, n_m),$$

where $g_{b,i}$ are the columns of and e_j the canonical basis of $\mathbb{R}^{n_m}$. Since the g_i do not depend on and the h_j are constant, one computes

$$[g_i, g_k] = \begin{pmatrix} [g_{b,i}, g_{b,k}] \\ 0 \end{pmatrix}, \qquad [g_i, h_j] = 0, \qquad [h_j, h_k] = 0.$$

Hence the Lie algebra of the MM is the direct sum of the Lie algebra $\mathcal{L}_b$ of the base (extended with zeros) and $\operatorname{span}\{h_j\}$, which has dimension n_m at every point. Its dimension is $\dim \mathcal{L}_b() + n_m$, which equals $n_b + n_m$ iff $\dim \mathcal{L}_b() = n_b$, i.e. iff the base is controllable. □

Intuition

The arm joints are *unconstrained* actuators: each one can be moved independently, so they contribute their n_m directions for free and never create any obstruction. Moreover, base motions and joint motions never "interfere" in the Lie-bracket sense (the brackets between them vanish), so no new directions can be generated by mixing them. Everything that can go wrong is therefore concentrated in the base — and if the base is, say, a unicycle (controllable), the whole MM is.

6 Path/trajectory planning *(manipulator slide 4)*

The kinematic model (or its geometric version, which only describes the admissible *paths*) can be used to plan paths and trajectories for the MM.

- **Planning in configuration space** (between two configurations q_{start}, q_{goal}) *naturally separates* into planning for the base and planning for the arm, because G is block-diagonal. Both sub-problems can be solved with the methods already introduced (e.g. flat-output or chained-form planning for the base, standard joint-space interpolation for the arm).
- **Planning under task constraints**: an MM has an end-effector, so one is often interested in a prescribed end-effector motion. In classical robotics this is the *inverse differential kinematics* problem, and it requires an appropriate differential kinematic map (next section).

Intuition

Without task constraints the arm and the base are two independent planning problems, like moving a car and an arm on it separately. As soon as the hand must follow something (a line on a table, a door handle), base and arm are coupled through the end-effector pose, and this coupling is precisely what the Jacobian describes.

7 Differential kinematics *(manipulator slide 5)*

Denote the end-effector pose by $= (p_e, \phi_e)$.

Definition 7.1 (Direct and differential kinematics of an MM)**.** The *direct kinematics* is

$$= k_e(q) = k_e(,).$$

Differentiating, and using the kinematic model,

$$\dot{y}_e = \frac{\partial k_e}{\partial}\dot{q}_b + \frac{\partial k_e}{\partial}\dot{q}_m = (q)\,() + (q)\,,$$

or, in compact form,

$$\dot{y}_e = \begin{pmatrix}(q)() & (q)\end{pmatrix}\begin{pmatrix}\\ \end{pmatrix} = (q)\,u,$$

where $= \partial k_e/\partial$ and $= \partial k_e/\partial$.

Intuition

For a fixed-base arm, $\dot{y}_e = \dot{q}_m$. Here we add one more term: the base moves too and drags the whole arm with it. The key point is that base motion enters through *pseudovelocities* (e.g. v, ω), not through $\dot{}$: this is why appears between and . The wheels simply cannot realise an arbitrary $\dot{}$.

8 Mobile manipulator Jacobian *(manipulator slide 6)*

Definition 8.1 (MM Jacobian)**.** The $6 \times (m_b + n_m)$ matrix $(q) = \begin{pmatrix} & \end{pmatrix}$ is called the *mobile manipulator Jacobian*.

- Typically $m_b + n_m > 6$, so mobile manipulators are often *kinematically redundant* with respect to a generic spatial task.
- is used for *singularity analysis* and for solving the *inverse differential kinematics* problem.
- In spite of the name, some elements of are *not* partial derivatives: the columns associated with the pseudovelocities are , linear combinations of partial derivatives with the coefficients of .

Intuition

Numbers: a unicycle base ($m_b = 2$) with a 6R arm gives 8 inputs for a 6-dimensional end-effector task, i.e. 2 degrees of redundancy; a unicycle with a 2R arm and a planar positioning task (2 outputs, 4 inputs) has 2 degrees of redundancy as well. The unusual fact is that u_b is *not* the derivative of any set of coordinates (the nonholonomic constraints are not integrable), so cannot be the gradient of anything: it is a purely differential object.

9 Example: unicycle base with a 2R arm *(manipulator slides 7–8)*

Example 9.1. Consider an MM made of a unicycle base (e.g. a differential-drive vehicle) carrying a planar 2R arm, mounted on the sagittal axis at distance d from the wheel center. Then

$$q = (,), \quad = (x, y, \theta), \quad = (q_1, q_2),$$

with velocity inputs $= (v, \omega)$ and $= (\dot{q}_1, \dot{q}_2)$. Here $n = 5$, $m = 4$, and

$$= \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{pmatrix}.$$

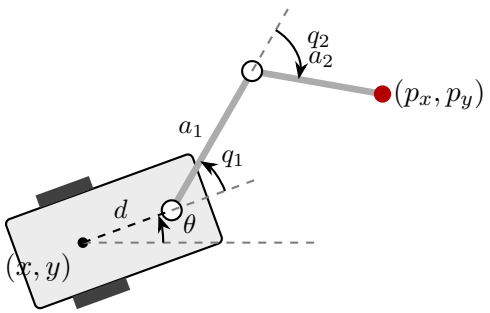


Figure 106: Unicycle base with a planar 2R arm mounted on the sagittal axis at distance d from the wheel center (top view).

Direct kinematics for the positioning task.

$$\begin{pmatrix} p_x \\ p_y \end{pmatrix} = \begin{pmatrix} x + d\,c_\theta + a_1 c_{\theta q_1} + a_2 c_{\theta q_1 q_2} \\ y + d\,s_\theta + a_1 s_{\theta q_1} + a_2 s_{\theta q_1 q_2} \end{pmatrix},$$

where $s_{\alpha\ldots\gamma} = \sin(\alpha + \cdots + \gamma)$ and $c_{\alpha\ldots\gamma} = \cos(\alpha + \cdots + \gamma)$.

Jacobians. Differentiating with respect to and ,

$$= \begin{pmatrix} 1 & 0 & -ds_\theta - a_1 s_{\theta q_1} - a_2 s_{\theta q_1 q_2} \\ 0 & 1 & dc_\theta + a_1 c_{\theta q_1} + a_2 c_{\theta q_1 q_2} \end{pmatrix}, \qquad = \begin{pmatrix} -a_1 s_{\theta q_1} - a_2 s_{\theta q_1 q_2} & -a_2 s_{\theta q_1 q_2} \\ a_1 c_{\theta q_1} + a_2 c_{\theta q_1 q_2} & a_2 c_{\theta q_1 q_2} \end{pmatrix}.$$

Multiplying by (first column: $c_\theta\, e_x + s_\theta\, e_y$; second column: third column of), the 2×4 MM Jacobian is

$$= \begin{pmatrix} c_\theta & -ds_\theta - a_1 s_{\theta q_1} - a_2 s_{\theta q_1 q_2} & -a_1 s_{\theta q_1} - a_2 s_{\theta q_1 q_2} & -a_2 s_{\theta q_1 q_2} \\ s_\theta & dc_\theta + a_1 c_{\theta q_1} + a_2 c_{\theta q_1 q_2} & a_1 c_{\theta q_1} + a_2 c_{\theta q_1 q_2} & a_2 c_{\theta q_1 q_2} \end{pmatrix}.$$

Proposition 9.2. *The MM of Example 9.1 has no kinematic singularities if $d > 0$.*

Proof. Call $c_1, \dots, c_4$ the columns of and let $u = (c_\theta, s_\theta)$ be the heading direction, $u^\perp = (-s_\theta, c_\theta)$, and $e_k^\perp$ the unit vector perpendicular to the direction of link k. Then

$$c_1 = u, \qquad c_2 = d\,u^\perp + a_1 e_1^\perp + a_2 e_2^\perp, \qquad c_3 = a_1 e_1^\perp + a_2 e_2^\perp, \qquad c_4 = a_2 e_2^\perp.$$

Therefore $c_3 - c_2 = -d\,u^\perp$. Since u and $u^\perp$ are orthogonal unit vectors and $d > 0$, the vectors $c_1 = u$ and $c_3 - c_2 = -d\,u^\perp$ are linearly independent, so rank $= 2$ for *every* configuration. □

Comparison with the arm alone. A direct computation gives

$$\det = a_1 a_2 \sin q_2,$$

so the 2R arm alone is singular whenever $q_2 = 0$ (arm fully stretched) or $q_2 = \pm\pi$ (arm folded).

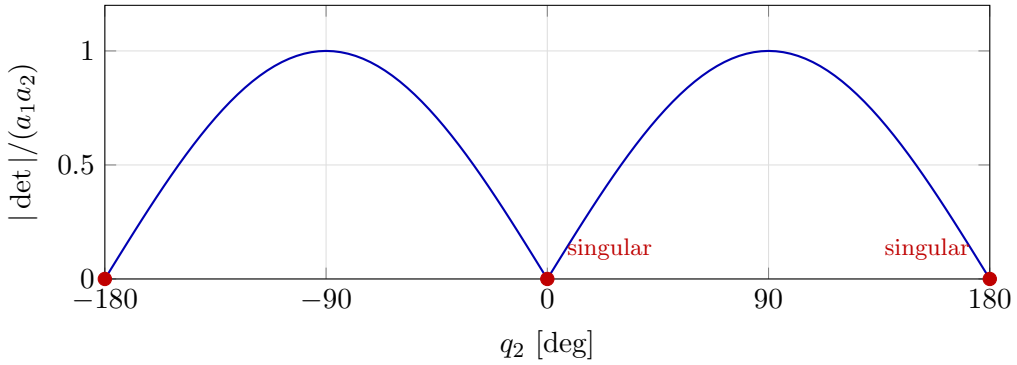


Figure 107: Determinant of the 2R arm Jacobian as a function of q_2: the arm alone is singular at $q_2 \in \{0, \pm\pi\}$. With the unicycle base ($d > 0$) these singularities disappear.

Intuition

At a stretched-out arm configuration the hand cannot move along the arm direction. A mobile base removes this obstruction: driving forward moves the hand along the heading u, while *rotating* the base about the wheel center moves the hand along $u^\perp$ with lever arm d (the column $c_3 - c_2 = -du^\perp$ in the proof). A unicycle cannot translate sideways, but if the arm is mounted ahead of the wheel axis, a rotation produces a sideways hand velocity anyway. If $d = 0$ the lever arm vanishes and this advantage is lost. In words: *adding the degrees of freedom of the base allows removing the singularities of the 2R arm alone.*

10 Inverse differential kinematics *(manipulator slides 9–10)*

10.1 Redundancy resolution

Methods for solving inverse differential kinematics in the presence of redundancy (*redundancy resolution*) extend to MMs with minimal adaptation.

Proposition 10.1 (Pseudoinverse solution with null-space term)**.** *Let (q) have full row rank and let $\dot{y}_d$ be the desired task velocity. Every input u with $u = \dot{y}_d$ can be written as*

$$u =^{\dagger} (q)\,\dot{y}_d + \big(I -^{\dagger} (q)(q)\big)u_N,$$

for a suitable u_N, where $^{\dagger} =^{T} (^{T})^{-1}$ is the pseudoinverse. The first term is the minimum-norm solution; the second term lies in ker *and does not affect the task.*

Proof. Let $P = I - ^{\dagger}$. Since $^{\dagger} = I$, we get $P = - = O$, so $(^{\dagger}\dot{y}_d + Pu_N) = \dot{y}_d$ for any u_N. Conversely, if $u = \dot{y}_d$, take $u_N = u$: then $^{\dagger}\dot{y}_d + Pu =^{\dagger} u + u -^{\dagger} u = u$. Finally P is symmetric and idempotent, i.e. the orthogonal projector onto ker, and $^{\dagger}\dot{y}_d \in \text{range}(^{T}) \perp \ker$, so $\|u\|^2 = \|^{\dagger}\dot{y}_d\|^2 + \|Pu_N\|^2$ is minimised by $Pu_N = 0$. □

Intuition

The set of inputs producing the desired hand velocity is a flat "affine subspace": one particular solution plus anything in the kernel. The pseudoinverse picks the smallest one (least effort), and the term Pu_N spends the leftover freedom — the *self-motion* of the robot — on a secondary goal $H(q)$ (stay away from joint limits, keep the arm well-conditioned, avoid obstacles), without disturbing the hand at all. Think of holding a cup steady while your elbow and torso reposition.

10.2 Why the gradient cannot be used directly

The null-space term u_N should improve a performance criterion $H(q)$ without affecting the task. For a fixed-base manipulator one would simply take $u_N = \pm\alpha\, H(q)$ (sign + to maximise, − to minimise). For an MM this is not possible: the first m_b components of u are *pseudovelocities*, not generalized velocities, so they cannot be set equal to components of a gradient. Put differently, the nonholonomic constraints on the base prevent motion in the direction of the gradient.

Intuition

Imagine H increases if the vehicle moves sideways. The gradient points sideways, but a unicycle cannot slide sideways; and even the dimensions do not match (H has n entries, u only m). The gradient is a "wish" in configuration space; the controls live in a different, smaller space.

10.3 The right choice of u_N

To find a suitable u_N, look at the time derivative of H along the system motion:

$$\dot{H}(q) = \frac{\partial H}{\partial q}\dot{q} =^{T} H(q)\, G(q)\, u.$$

By the Cauchy–Schwarz inequality this is maximised (for given $\|u\|$) when u is aligned with $G^{T}(q)H(q)$.

Proposition 10.2 (Maximum local improvement)**.** *Choosing*

$$u_N = \pm\alpha\, G^T(q)\, H(q) = \pm\alpha \begin{pmatrix} {}^T() & O \\ O & I \end{pmatrix} H(q), \qquad \alpha > 0,$$

yields the maximum local improvement of $H(q)$, with corresponding generalized velocity

$$\dot{q}_N = G(q)u_N = \pm\alpha\, G(q)G^T(q)H(q),$$

which is the projection of $\pm\alpha H(q)$ onto the subspace of admissible generalized velocities range $G(q)$.

If the columns of G are orthonormal, GG^T is exactly the orthogonal projector onto range G. This is the case for the unicycle, where $= \begin{pmatrix} c_\theta & 0 \\ s_\theta & 0 \\ 0 & 1 \end{pmatrix}$ and

$${}^T\nabla H = \begin{pmatrix} c_\theta\, \partial_x H + s_\theta\, \partial_y H \\ \partial_\theta H \end{pmatrix}.$$

So the forward velocity v follows the component of the gradient along the heading, ω follows $\partial_\theta H$ (turn to make H grow), and each joint velocity follows $\partial_{q_j} H$. The component of the gradient orthogonal to the heading is simply discarded.

[Effect of the projection] In the resulting control the null-space velocity is actually Pu_N, not u_N. The resulting improvement of H due to this term is $\pm\alpha\, \|PG^T H\|^2$, because ${}^T H\, GPG^T H = \|PG^T H\|^2$ (P symmetric idempotent). Hence the projection may reduce the improvement, but never reverses its sign.

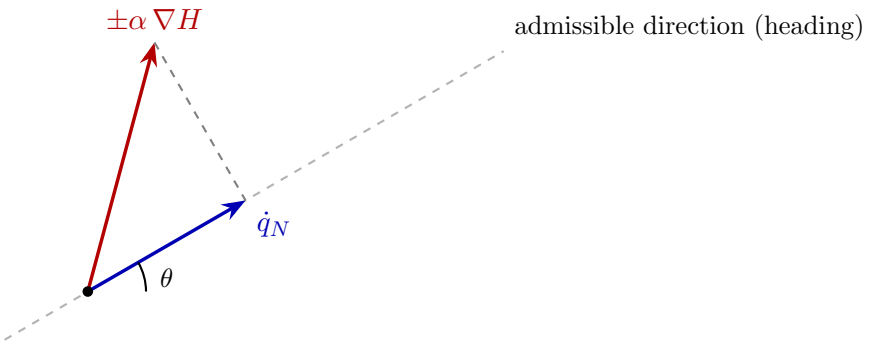


Figure 108: Base-only cartoon in the (x, y) plane: the wished motion $\pm\alpha\nabla H$ is projected onto the only direction the unicycle can actually move along (its heading). The orthogonal component is lost.

Intuition

The recipe is "*do the best you can with the motions you are allowed*". Compute the gradient as if everything were free, then map it through G^T to obtain the controls that make $\dot{H}$ as large as possible. Numerical example for the unicycle at $\theta = 0$ with $\nabla H = (2, 5, 1)$: the vehicle gets $v = 2$ (forward component), $\omega = 1$, and the sideways "5" is ignored. The result is not the ideal improvement ($\|H\|^2$) but the best *admissible* one ($\|G^T H\|^2$).

Algorithm 10.3 (Redundancy resolution for an MM)**.** At each control instant:
1. compute $(q) = (\ \ )$ and its pseudoinverse †;
2. compute $u_N = \pm\alpha\, G^T(q)H(q)$;

3. output $u = {}^{\dagger}\dot{y}_d + (I - {}^{\dagger})u_N$ and apply to the base, to the arm.

11 Kinematic control *(manipulator slide 11)*

In general, as for manipulators, robust feedback control laws for MMs should be designed on the basis of their *dynamic* model. Kinematic control of MMs gives acceptable performance only if the required velocities are small, so that the inertial couplings between the coordinates of the base and those of the manipulator remain negligible.

A closed-loop inverse kinematic scheme (*kinematic control*) is obtained by adding an error feedback term $K(y_d - y_e)$ to the feedforward term $\dot{y}_d$ in the control of Proposition 10.1:

$$u = {}^{\dagger}(q)\big[\dot{y}_d + K(y_d -)\big] + \big(I - {}^{\dagger}\big)u_N, \qquad = k_e(q).$$

Proposition 11.1 (Task error dynamics)**.** *If has full row rank along the motion and K is a symmetric positive-definite matrix, the task error $e = y_d -$ obeys $\dot{e} = -Ke$ and converges exponentially to zero, irrespective of u_N.*

Proof. $\dot{e} = \dot{y}_d - u = \dot{y}_d - {}^{\dagger}[\dot{y}_d + Ke] - Pu_N = \dot{y}_d - \dot{y}_d - Ke = -Ke$, using ${}^{\dagger} = I$ and $P = O$. □

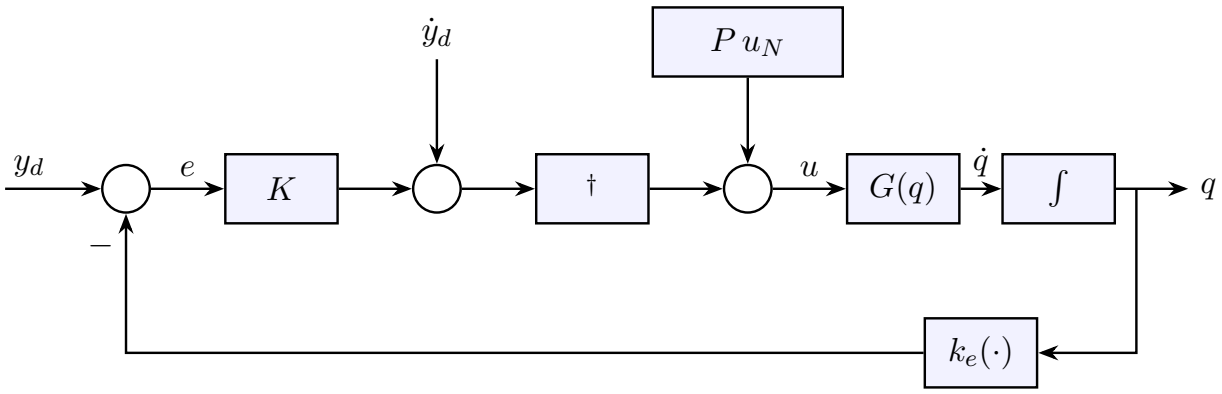


Figure 109: Kinematic control scheme of an MM: error feedback plus feedforward through †, with a null-space term for the secondary criterion.

Intuition

It is a "proportional controller in task space": the error between desired and actual hand pose is turned into a corrected desired hand velocity, which the pseudoinverse then distributes among base and arm. Because everything is expressed through †, the same law drives the base wheels and the arm joints in a coordinated way; the null-space part moves the robot "behind the scenes". The assumption of small velocities is the price of ignoring dynamics: if the base accelerates hard, its inertia reacts on the arm and the kinematic prediction is no longer accurate.

Near singularities † becomes very large. A common practical remedy (not covered here) is the damped pseudoinverse ${}^{T}({}^{T} + \lambda^2 I)^{-1}$. By Proposition 9.2, for the unicycle with a 2R arm and $d > 0$ this issue does not arise for the positioning task.

12 Summary

Regulation

	Cartesian regulation	**Posture regulation**
Goal	reach $(0,0)$, any orientation	reach $(0,0,0)$
Coordinates	e_p, projection on n, pointing error γ	polar: $\rho, \gamma, \delta = \gamma + \theta$
Design approach	geometric projection on sagittal axis	constructive Lyapunov design (cross-term cancellation)
Control law	$v = k_1 \rho \cos\gamma$, $\omega = k_2 \gamma$	$v = k_1 \rho \cos\gamma$, $\omega = k_2\gamma + k_1 \frac{\sin\gamma\cos\gamma}{\gamma}(\gamma + \delta)$
Lyapunov function	$V = \frac{1}{2}(x^2 + y^2)$ (PSD in $\mathbb{R}^3$, zero on θ-axis)	$V = \frac{1}{2}(\rho^2 + \gamma^2 + \delta^2)$ (PD in polar coordinates)
$\dot{V}$	$-k_1(x\cos\theta + y\sin\theta)^2 \leq 0$	$-k_1 \cos^2\gamma\, \rho^2 - k_2\gamma^2 \leq 0$
Invariance tool	Barbalat (LaSalle fails: sublevel sets non-compact)	Barbalat (+ closed-loop analysis for δ)
Steady state	broadside scenario ($\gamma = \pm\pi/2 \implies \omega \neq 0$) ruled out	full state convergence $\rho, \gamma, \delta \to 0$
Final heading	uncontrolled	$\theta \to 0$
Maneuvers	at most one backup	at most one backup
Continuity	discontinuous (Atan2 jump)	discontinuous at origin (Brockett); never met in practice

Table 12: Cartesian versus posture regulation of the unicycle.

Mobile manipulators

Key take-aways

1. Nonholonomic WMRs admit no universal controllers: tracking schemes need persistent trajectories ($v_d(t) \geq v_{\min} > 0$), and tracking a trajectory that stops in q_d fails (linearization is uncontrollable at rest, and error-dynamics laws tend to infinity), so regulation needs dedicated laws.

2. Cartesian regulation uses $v = k_1 e_p^T n$ and $\omega = k_2\gamma$; V is only semidefinite in $\mathbb{R}^3$ (unbounded sublevel sets along θ prevent LaSalle), so Barbalat's lemma proves $e_p^T n \to 0$; analyzing steady state rules out broadside alignment ($\omega \neq 0$) and proves $(x, y) \to 0$, while the final orientation is not controlled.

3. Posture regulation uses polar coordinates and a constructive Lyapunov synthesis canceling the cross-coupling term in $\dot{V}$; Barbalat's lemma proves full convergence $x, y, \theta \to 0$. By Brockett's theorem any time-invariant posture stabilizer must be discontinuous, though the origin singularity is never met in practice.

4. A mobile manipulator has a block-diagonal kinematic model, is controllable iff its base is, and has = ( ); the base DOFs can remove arm singularities (unicycle + 2R with $d > 0$).

Topic	Result	Key point
Configuration	$q = (,)$, $n = n_b + n_m$	wheeled base + articulated arm
Kinematic model	$\dot{q} = \mathrm{diag}(,I)\,u$, $u = (,)$	driftless; base has $m_b < n_b$ pseudovelocities
Controllability	iff the base is controllable	arm joints add free directions
Differential kinematics	$\dot{y}_e = (\ \ )\,u = u$	some entries are not partial derivatives
Redundancy	typically $m_b + n_m > 6$	secondary criterion $H(q)$
Singularities (unicycle+2R)	none if $d > 0$	arm alone singular at $\sin q_2 = 0$
Inverse diff. kinematics	$u =^{\dagger} \dot{y}_d + (I - ^{\dagger})u_N$	min. norm + null-space term
Null-space term	$u_N = \pm\alpha\, G^T H$	*not* $\pm\alpha H$ (pseudovelocities)
Kinematic control	$u =^{\dagger} [\dot{y}_d + K(y_d -)] + \ldots$	$\dot{e} = -Ke$; low speeds only

Table 13: Summary of the main results on mobile manipulators.

	Fixed-base manipulator	Mobile manipulator
Inputs	$\dot{q}_m$	$(,)$, pseudovelocities for the base
Task Jacobian		$= (\ \ )$
Null-space term	$\pm\alpha H$	$\pm\alpha\, G^T H$
Workspace	limited by the arm	extended by base motion
Singularities	arm singularities	removable (e.g. $d > 0$)

Table 14: Fixed-base versus mobile manipulators.

5. Redundancy is resolved with $u =^{\dagger} \dot{y}_d + (I - ^{\dagger})u_N$ with $u_N = \pm\alpha\, G^T H$ (not the raw gradient), and kinematic control with $K(y_d -)$ gives $\dot{e} = -Ke$ at low speeds.

Part XIV

Lecture 10

Conventions. Slide ranges in the section headers refer to the *Odometric Localization* deck (Sections 1–4) or to the *Extended Kalman Filter* deck (Sections 5–10). Section 11 is the application worked out in the companion book chapter (Sect. 7.7.2 there), merged at the point where the EKF has just been introduced. The robot is a unicycle with configuration $= (x, y, \theta)$, kinematic inputs v (driving velocity) and ω (steering velocity), and sampling interval $= t_{k+1} - t_k$.

1 Why localization is needed odometry deck, slide 2

Planning and, above all, feedback control need the robot configuration at every time instant: a trajectory-tracking law computes its commands from the *error* between the desired and the actual configuration, so it cannot even be written down without an estimate of .

For a robot manipulator this is easy: joint encoders measure the joint variables, which *are* the configuration. A wheeled mobile robot (WMR) is different. Its incremental encoders measure only how much each wheel has rotated, *not* where the vehicle is with respect to a fixed world frame.

Definition 1.1 (Localization)**.** *Localization* is a procedure that estimates, typically in real time, the configuration (t) of a robot with respect to a fixed world frame from the available sensor data.

Intuition

A joint encoder is like a GPS: it tells you directly *where* you are. A wheel encoder is like a pedometer: it tells you how many steps you took, from which you can *reconstruct* where you are only if you know where you started and how long and in which direction each step was. The whole lecture follows this analogy: first we add up the steps (odometry), then we learn to correct the sum by looking around (Kalman filtering with exteroceptive sensors).

2 Odometric localization: integrating the kinematic model odometry deck, slides 3–6

2.1 Setting: constant inputs over a sampling interval

Recall the unicycle kinematic model

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{78}$$

In a digital implementation the velocity inputs are held constant over each sampling interval: $v(t) = v_k$, $\omega(t) = \omega_k$ for $t \in [t_k, t_{k+1})$. Then $\theta(t) = \theta_k + \omega_k(t - t_k)$, so the heading changes at a constant rate, and the robot moves along an arc of circle of radius $|v_k/\omega_k|$ (a line segment if $\omega_k = 0$).

Problem. Given $_k = (x_k, y_k, \theta_k)$ and (v_k, ω_k), compute $_{k+1}$ by integrating (78) over $[t_k, t_{k+1}]$.

Intuition

Constant "gas" and constant "steering wheel" means driving on a circle. Over one sampling interval the robot therefore covers a short arc; the only question is how faithfully we want to

reproduce that arc with a formula.

2.2 Three integration schemes

Algorithm 2.1 (Euler integration)**.**

$$\begin{aligned} x_{k+1} &= x_k + v_k \cos\theta_k, \\ y_{k+1} &= y_k + v_k \sin\theta_k, \\ \theta_{k+1} &= \theta_k + \omega_k. \end{aligned} \tag{79}$$

x_{k+1} and y_{k+1} are *approximate* (computed as if the orientation stayed equal to θ_k throughout the interval); θ_{k+1} is *exact*.

Algorithm 2.2 (Second-order Runge–Kutta integration)**.**

$$\begin{aligned} x_{k+1} &= x_k + v_k \cos\Big(\theta_k + \frac{\omega_k}{2}\Big), \\ y_{k+1} &= y_k + v_k \sin\Big(\theta_k + \frac{\omega_k}{2}\Big), \\ \theta_{k+1} &= \theta_k + \omega_k. \end{aligned} \tag{80}$$

The *average orientation* over $[t_k, t_{k+1}]$ is used; x_{k+1}, y_{k+1} are still approximate but more accurate.

Proposition 2.3 (Exact integration)**.** *Under constant v_k, ω_k with $\omega_k \neq 0$,*

$$\begin{aligned} x_{k+1} &= x_k + \frac{v_k}{\omega_k}\big(\sin\theta_{k+1} - \sin\theta_k\big), \\ y_{k+1} &= y_k - \frac{v_k}{\omega_k}\big(\cos\theta_{k+1} - \cos\theta_k\big), \\ \theta_{k+1} &= \theta_k + \omega_k. \end{aligned} \tag{81}$$

For $\omega_k \to 0$ the first two formulas tend to the Euler (and Runge–Kutta) ones, so they remain defined at $\omega_k = 0$.

Proof. Since $\theta(t) = \theta_k + \omega_k(t - t_k)$ we have $dt = d\theta/\omega_k$, hence

$$x_{k+1} - x_k = \int_{t_k}^{t_{k+1}} v_k \cos\theta \, dt = \frac{v_k}{\omega_k} \int_{\theta_k}^{\theta_{k+1}} \cos\theta \, d\theta = \frac{v_k}{\omega_k}(\sin\theta_{k+1} - \sin\theta_k),$$

and similarly for y. For the limit, $\sin(\theta_k+\omega_k) - \sin\theta_k = \omega_k \cos\theta_k + O(\omega_k^2)$, so $x_{k+1} - x_k \to v_k \cos\theta_k$ (and likewise for y) as $\omega_k \to 0$. □

Intuition

For $\omega_k \approx 0$ the exact formula is a "0/0" expression: numerically it divides a tiny number by a tiny number. In an implementation one therefore adds a conditional instruction: if $|\omega_k|$ is below a small threshold use the straight-line (Euler/Runge–Kutta) update, otherwise the closed form.

2.3 How different are they? Geometry and error order

Figure 110 compares the three reconstructions of $_{k+1}$ on a single arc.

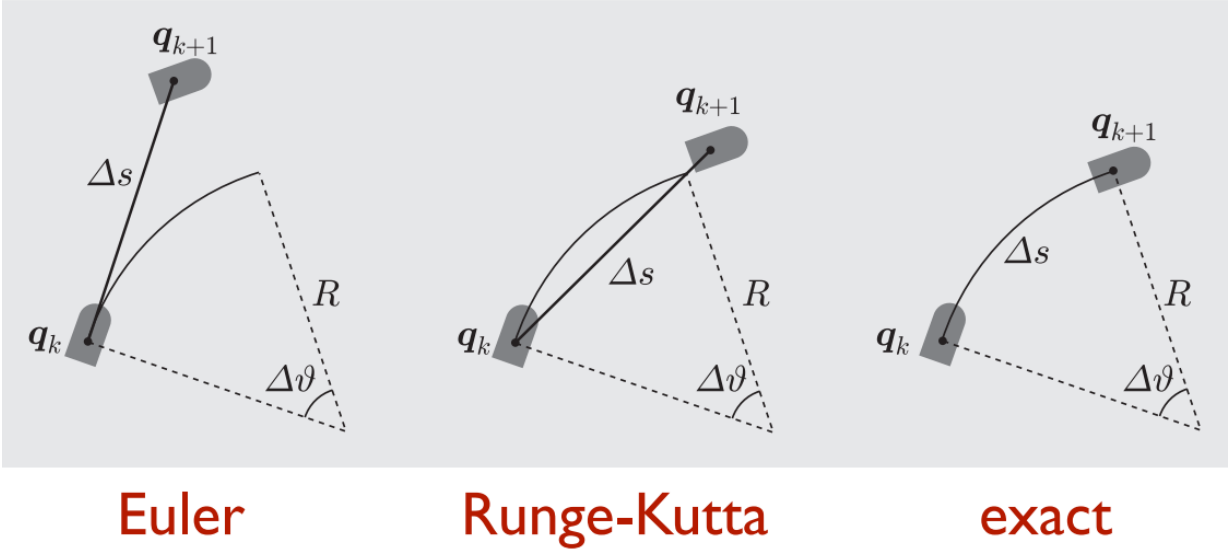


Figure 110: Odometric localization of a unicycle over one arc of circle (redrawn). Euler moves along the initial heading; Runge–Kutta moves along the *mean* heading; the exact formula follows the arc. The small grey circle marks the exact end point. The arc here is exaggerated ($\Delta\theta = 60°$); at realistic sampling rates the three points almost coincide.

Proposition 2.4 (Leading-order error of the schemes)**.** *Let $a = \omega_k/2$. The exact update equals the Runge–Kutta one with the step length scaled by $(a) = \sin a/a$:*

$$\begin{aligned} x_{k+1} - x_k &= v_k\,(a)\cos(\theta_k + a), \\ y_{k+1} - y_k &= v_k\,(a)\sin(\theta_k + a). \end{aligned} \tag{82}$$

Consequently, to leading order the position error per step *is $\frac{1}{2}v_k\omega_k^2$ for Euler and $\frac{1}{24}v_k\omega_k^{23}$ for Runge–Kutta.*

Proof. Use $\sin\alpha - \sin\beta = 2\cos\frac{\alpha+\beta}{2}\sin\frac{\alpha-\beta}{2}$ with $\alpha = \theta_{k+1}$, $\beta = \theta_k$: $\frac{v_k}{\omega_k}(\sin\theta_{k+1} - \sin\theta_k) = \frac{2v_k}{\omega_k}\sin(a)\cos(\theta_k + a) = v_k(a)\cos(\theta_k + a)$. The y formula is analogous with $\cos\beta - \cos\alpha = 2\sin\frac{\alpha+\beta}{2}\sin\frac{\alpha-\beta}{2}$. Now $(a) = 1 - a^2/6 + \cdots = 1 - (\omega_k)^2/24 + \ldots$, so Runge–Kutta (which sets $= 1$) errs by about $v_k\,(\omega_k)^2/24$. Euler uses the heading θ_k instead of $\theta_k + a$: its step vector is rotated by the angle a with respect to the exact one, giving an error of about $v_k\,a = \frac{1}{2}v_k\omega_k^2$. □

Intuition

Euler commits to the *starting* heading, Runge–Kutta to the *average* heading, and the exact formula additionally knows that a chord is shorter than its arc (the factor). Taking the heading at the middle of the interval is what removes the dominant error: halve and the Euler error per step drops by a factor 4, the Runge–Kutta one by a factor 8 (Fig. 111). This is why the differences "tend to disappear as the sampling rate increases".

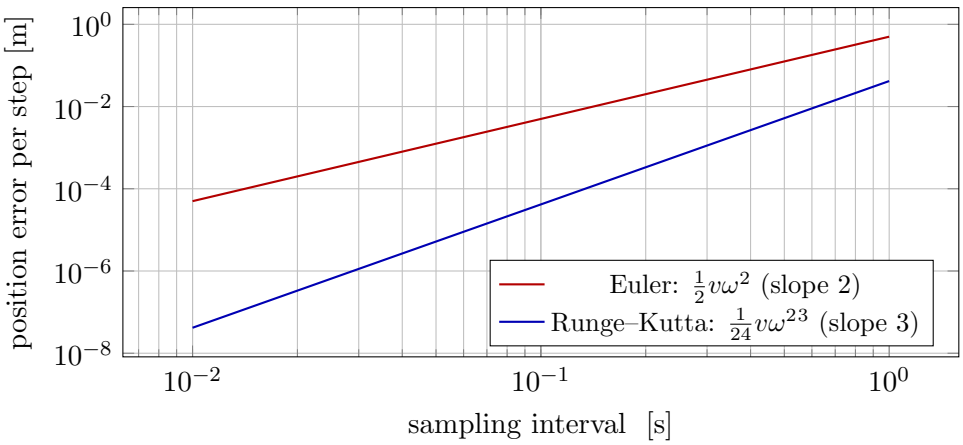


Figure 111: Leading-order local error of the two approximate schemes for $v = 1$ m/s, $\omega = 1$ rad/s (illustrative asymptotics from Prop. 2.4).

Example 2.5 (A coarse step, numerically)**.** Take $\theta_k = 0$, $v_k = 1$ m/s, $\omega_k = \pi/2$ rad/s and $= 1$ s (a quarter turn in one step, deliberately coarse).

Scheme	$(x_{k+1} - x_k,\ y_{k+1} - y_k)$ [m]	distance from exact
Euler	$(1,\ 0)$	0.733
Runge–Kutta	$(0.707,\ 0.707)$	0.100
Exact	$(2/\pi,\ 2/\pi) = (0.637,\ 0.637)$	0

Check with (82): $a = \pi/4$, $(a) = 0.900$, and $0.900 \cdot 0.707 = 0.637$.

3 Reconstructing the inputs from proprioceptive sensors odometry deck, slide 7

The formulas above assume that v_k, ω_k are known. But no actuation system is ideal: the *commanded* inputs are not what the robot actually executes. It is therefore better to reconstruct v_k and ω_k from their *effect*:

$$v_k = \Delta s, \qquad \omega_k = \Delta\theta, \qquad \frac{v_k}{\omega_k} = \frac{\Delta s}{\Delta\theta}, \tag{83}$$

where Δs is the signed distance traveled and $\Delta\theta$ the total orientation change over the interval, both measured by *proprioceptive* sensors.

Proposition 3.1 (Differential drive)**.** *For a differential-drive robot with wheel radius r and wheel separation d, let $\Delta\phi_R, \Delta\phi_L$ be the total rotations of the right and left wheel measured by the encoders during the sampling interval. Then*

$$\Delta s = \frac{r}{2}(\Delta\phi_R + \Delta\phi_L), \qquad \Delta\theta = \frac{r}{d}(\Delta\phi_R - \Delta\phi_L). \tag{84}$$

Proof. The wheel rim speeds are $r\dot{\phi}_R$ and $r\dot{\phi}_L$. The robot center moves with the average rim speed, $v = \frac{r}{2}(\dot{\phi}_R + \dot{\phi}_L)$, and the robot turns at the rim-speed difference divided by the separation, $\omega = \frac{r}{d}(\dot{\phi}_R - \dot{\phi}_L)$. Integrating over the sampling interval gives (84). □

With (83)–(84) the three schemes can be written without :

$$\text{Euler: } x_{k+1} = x_k + \Delta s \cos\theta_k, \qquad \text{RK: } x_{k+1} = x_k + \Delta s \cos\big(\theta_k + \tfrac{\Delta\theta}{2}\big),$$
$$\text{exact: } x_{k+1} = x_k + \tfrac{\Delta s}{\Delta\theta}(\sin\theta_{k+1} - \sin\theta_k),$$

and analogously for y; $\theta_{k+1} = \theta_k + \Delta\theta$. Similar formulas can be derived for other WMRs, e.g. the bicycle.

Algorithm 3.2 (One odometry update)**.** Given $_k$ and the encoder increments $\Delta\phi_R, \Delta\phi_L$:

1. compute Δs and $\Delta\theta$ from (84);
2. set $\theta_{k+1} = \theta_k + \Delta\theta$;
3. if $|\Delta\theta| < \varepsilon$ (conditional instruction) use the straight-line/Runge–Kutta update for x_{k+1}, y_{k+1}; otherwise use the exact formulas (81) with $v_k/\omega_k = \Delta s/\Delta\theta$.

Example 3.3. With $r = 0.05$ m, $d = 0.30$ m, $\Delta\phi_R = 0.50$ rad, $\Delta\phi_L = 0.30$ rad: $\Delta s = \frac{0.05}{2}(0.80) = 0.020$ m and $\Delta\theta = \frac{0.05}{0.30}(0.20) = 0.0333$ rad $\approx 1.9°$. The robot advanced 2 cm while turning slightly left, because the right wheel rolled more than the left.

Intuition

The encoders "see" only the wheels. The sum of the two wheel rotations says how far the midpoint of the axle moved; their *difference* says how much the vehicle swung around, and it is divided by the small separation d — a lever arm effect that makes heading very sensitive to wheel errors (we will see the consequence in the next section).

4 Dead reckoning and odometric drift odometry deck, slides 8–9

Definition 4.1 (Odometric localization / dead reckoning)**.** Maintaining an estimate of the robot configuration by *iterative integration of the kinematic model*, with velocities reconstructed from wheel encoders, starting from an estimate of $_0$.

Its accuracy cannot be better than that of $_0$. Moreover, odometric errors *accumulate over time* (*odometric drift*), so accuracy deteriorates along sufficiently long paths. The causes are:

- the *integration error* of Euler/Runge–Kutta (reducible, Sect. 2);
- *wheel slippage* and irregular ground (a model perturbation);
- *inaccurate calibration* of kinematic parameters such as the wheel radius (a model uncertainty);
- *encoder noise* (a measurement uncertainty).

The last three are inevitable, and the achievable performance also depends on the kinematic arrangement of the robot (e.g. differential drive is usually better than synchro drive). Effective localization therefore uses exteroceptive sensing as well as proprioceptive sensing (Part II).

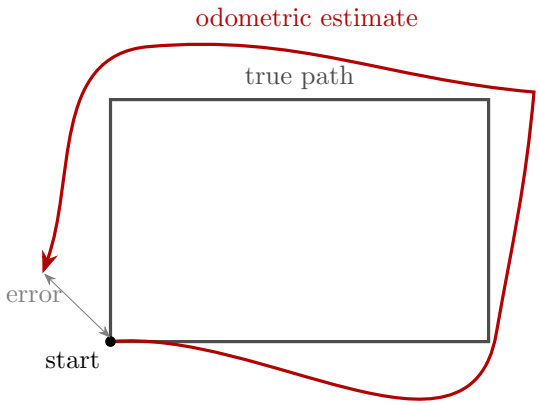


Figure 112: Conceptual sketch of odometric drift (original): the robot drives around a rectangle and returns to its start, but the integrated estimate does not close the loop; the gap keeps growing with distance.

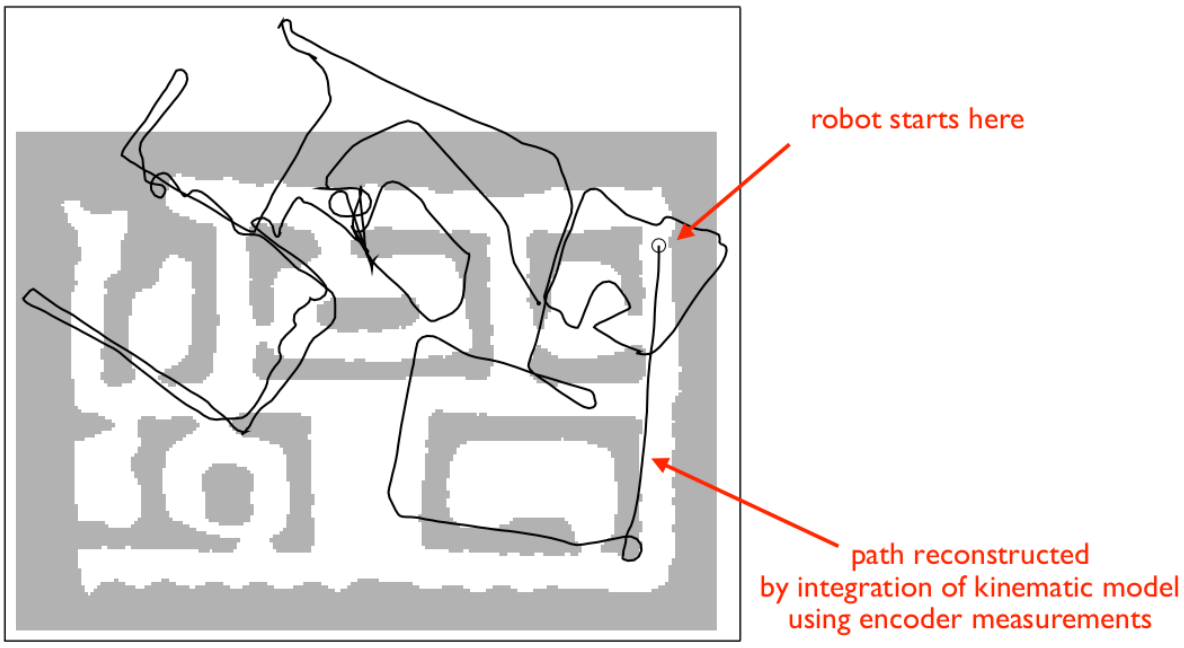


Figure 113: Real experiment: path reconstructed by integrating the kinematic model with encoder measurements, drawn over the environment map

Example 4.2 (Why a 1% calibration error is not small)**.** Differential drive with $d = 0.30$ m. Suppose the true right wheel is 1% larger than the nominal radius used by the odometry, and the robot is commanded to go straight, so both encoders report the same rotation $\Delta\phi$. The odometry concludes $\Delta\theta = 0$, but physically $\Delta\theta = \frac{0.01\, r\Delta\phi}{d} \approx \frac{0.01}{0.30}\,\Delta s \approx 0.033\,\Delta s$: the robot actually follows an arc with curvature ≈ 0.033 m^{-1} (radius ≈ 30 m). After $s = 10$ m the heading is off by ≈ 0.33 rad ($\approx 19°$) and the lateral position by $\approx s^2\kappa/2 \approx 1.7$ m, while the estimate still believes the robot is on the straight line.

Intuition

A tiny mismatch between the wheels is divided by the small wheelbase d and becomes a heading error; a heading error then is turned into a sideways position error that grows with the distance traveled. Errors are never "forgotten" by integration — they only add up. To stop this we need a measurement that does not depend on the past: a landmark, a wall, a map.

5 From dead reckoning to probabilistic localization EKF deck, slide 2

Dead reckoning keeps a *single hypothesis* about the configuration, and its error diverges over time. *Probabilistic localization* instead maintains a probability distribution over the space of all possible hypotheses. If a map of the environment is known, the actual readings of an exteroceptive sensor can be compared with the readings predicted from the current estimate; the mismatch is used to reduce the odometric drift. This idea is formalized by Bayesian estimation, and a tractable instance is the *Extended Kalman Filter* (EKF).

Intuition

Walking in a dark corridor you count steps (odometry) and your belief about your position spreads out. When your hand touches a door frame you know exists on the map, the belief suddenly shrinks. The filter is the bookkeeping that tells you exactly how much to trust the step count versus the touch.

6 Basic concepts: mean, covariance, Gaussians EKF deck, slides 3–4

Definition 6.1 (Mean, covariance, multivariate Gaussian)**.** Let $\boldsymbol{X} \in^n$ be a random vector with density $f_{\boldsymbol{X}}()$. Its *expected value* and *covariance matrix* are

$$\bar{\boldsymbol{X}} = (\boldsymbol{X}) = \int_n f_{\boldsymbol{X}}()\, d, \qquad _X = \big((\boldsymbol{X} - \bar{\boldsymbol{X}})(\boldsymbol{X} - \bar{\boldsymbol{X}})\big).$$

$\boldsymbol{X}$ is *multivariate Gaussian* if

$$f_{\boldsymbol{X}}() = \frac{1}{\sqrt{(2\pi)^n |_X|}} \exp\Big(- \tfrac{1}{2}(-\bar{\boldsymbol{X}})^{-1}_X(-\bar{\boldsymbol{X}})\Big).$$

Proposition 6.2 (Geometry of a Gaussian)**.** *The equidensity contours of a Gaussian are ellipsoids* $(-\bar{\boldsymbol{X}})^{-1}_X(-\bar{\boldsymbol{X}}) = c$. *Their principal axes are directed along the eigenvectors of* $_X$*, and the squared lengths of the axes are proportional to the corresponding eigenvalues.*

Proof. Write $_X = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}$ with $\mathbf{U}$ orthogonal. In the coordinates $\boldsymbol{z} = \mathbf{U}(-\bar{\boldsymbol{X}})$ the contour is $\sum_i z_i^2/\lambda_i = c$, an axis-aligned ellipsoid with semi-axes $\sqrt{c\lambda_i}$ along the columns of $\mathbf{U}$. □

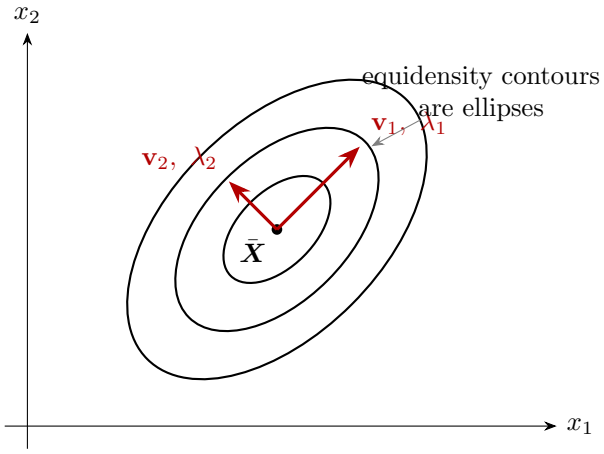


Figure 114: Geometric interpretation of a covariance matrix (redrawn): ellipses centered at the mean, with axes along the eigenvectors $\mathbf{v}_i$ of $_X$. Drawn for $_X = 21$
12, $\lambda_1 = 3$, $\lambda_2 = 1$.

Intuition

The covariance matrix is the "shape of the cloud": a long thin ellipse means "I am quite sure in one direction and quite unsure in the other". Example: $_X = 21$
12 has eigenvalues 3 and 1 with eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$, so the cloud is stretched along the diagonal, and its length along that direction is $\sqrt{3} \approx 1.7$ times the length across it. The off-diagonal term $1 > 0$ says that x_1 and x_2 tend to be large or small together. Filters will propagate exactly this kind of ellipse.

7 Kalman filter without noise EKF deck, slides 5–9

7.1 Recursive observer: prediction and correction

Consider the linear discrete-time system *without noise*

$$_{k+1} = _k \ _k + _k \ _k, \qquad _k = _k \ _k. \tag{85}$$

We want a *recursive observer*: an estimate $_{k+1}$ of $_{k+1}$ computed from $_k$, $_{k+1}$ and the previous estimate $_k$, in two steps:

1. **prediction**: build an intermediate estimate $_{k+1|k}$ by propagating $_k$ through the process dynamics,
$$_{k+1|k} = _k \ _k + _k \ _k; \tag{86}$$
2. **correction (update)**: correct the prediction using the difference between the measured and the predicted output.

Even without noise a correction is needed, because the exact initial state $_0$ is unknown.

7.2 The correction as a constrained optimization

To be consistent with the measured output, $_{k+1}$ must belong to the hyperplane

$$\Omega = \{: \ _{k+1} = _{k+1}\}, \tag{87}$$

so the correction Δ must satisfy $_{k+1}(_{k+1|k}+\Delta) = _{k+1}$. There are infinitely many such corrections; intuitively the best is the *closest* to the prediction, which we believe is accurate (Fig. 115).

Proposition 7.1 (Minimum-norm correction)**.** *Assume* $_{k+1}$ *has full row rank. The solution of*

$$\min \|\Delta\| \quad s.t. \quad _{k+1}\Delta = _{k+1} - _{k+1k+1|k}$$

is

$$\Delta = _{k+1} \; _{k+1}, \qquad _{k+1} = _{k+1} \; (_{k+1k+1})^{-1}, \qquad _{k+1} = _{k+1} - _{k+1k+1|k}, \tag{88}$$

where *is the* pseudoinverse *and* $_{k+1}$ *is the* innovation.

Proof. Lagrangian $\frac{1}{2}\Delta\Delta + \boldsymbol{\lambda}(-\Delta)$. Stationarity gives $\Delta = \boldsymbol{\lambda}$; substituting in the constraint, $= \boldsymbol{\lambda}$, so $\boldsymbol{\lambda} = ()^{-1}$ (invertible by full row rank) and $\Delta = ()^{-1}$. □

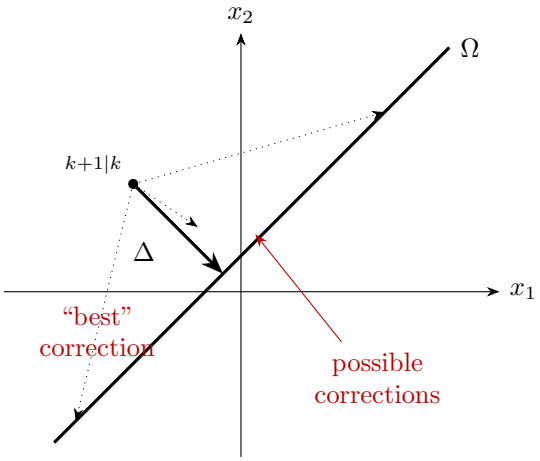


Figure 115: Naive correction (redrawn): among all points of the measurement hyperplane Ω, pick the one closest to the prediction; Δ is orthogonal to Ω.

Intuition

The measurement tells you "the true state is somewhere on this line". You were already standing at the prediction, so you take the shortest step onto the line — a perpendicular projection. *Worked numbers:* $\in^2$, $= (1 \;\; 1)$ (we only measure the sum $x_1 + x_2$), $_{k+1|k} = (1, 0)$ and $y_{k+1} = 3$. Then $\nu = 3 - 1 = 2$, $= (\frac{1}{2}, \frac{1}{2})$, $\Delta = (1, 1)$ and $_{k+1} = (2, 1)$, whose sum is indeed 3.

7.3 Why this naive observer is not enough

The resulting two-step observer is $_{k+1|k} = _k \; _k + _k \; _k$, $_{k+1} = _{k+1|k} + _{k+1k+1}$. In general $_{k+1}$ will *not* converge to $_{k+1}$: the correction is naive because estimation errors directed as Ω are not corrected. In particular the gain does not exploit the observability of the system (unlike, e.g., a Luenberger observer).

Intuition

Take $_k = $, $= (1 \;\; 1)$, true state $(1, 2)$ and $_0 = (0, 0)$. The first correction gives $(1.5, 1.5)$, which already reproduces $y = 3$, so every later innovation is zero and the error $(-0.5, 0.5)$ never disappears: a correction that only looks at the "compatible with the measurement" direction cannot fix errors along Ω. (This pair is unobservable, so no observer could do better; the point is only to see the mechanism.) The remedy is to choose the gain in a principled way using noise statistics.

8 Kalman filter with process noise only EKF deck, slides 10–15

Now include *process noise*:

$$_{k+1}=_k\ _k+_k\ _k+_k, \qquad _k=_k\ _k, \tag{89}$$

where $_k$ is *white* (successive samples are uncorrelated), *Gaussian*, zero-mean with covariance $_k$. The system is now a random process, so we compute both an estimate $_{k+1}$ and its estimation-error covariance $_{k+1}$, keeping the prediction/correction structure.

8.1 Prediction

The state prediction is unchanged, $_{k+1|k}=_k\ _k+_k\ _k$, because $_k$ has zero mean.

Lemma 8.1 (Covariance prediction)**.** $_{k+1|k}=_k\ _{kk}+_k$.

Proof. $_{k+1}-_{k+1|k}=_k\ (_k-_k)+_k$. Expanding $_{k+1|k}=\big((_{k+1}-_{k+1|k})(\cdot)\big)$ gives $_k\big((_k-_k)(_k-_k)\big)_k+(_{kk})$ plus cross terms. By linearity of and the independence of $_k$ from $_k$ and $_k$ (and zero mean), the cross terms vanish. Hence $_{k+1|k}=_k\ _{kk}+_k$. □

The predicted state distribution is Gaussian with mean $_{k+1|k}$ and covariance $_{k+1|k}$.

Intuition

Propagating through $_k$ stretches or rotates the uncertainty ellipse; adding $_k$ *fattens* it: every step, the world shakes the robot a little and we lose a bit of knowledge. With no measurements, can only grow — the covariance-level picture of odometric drift.

8.2 State correction: Mahalanobis distance and weighted pseudoinverse

We should choose Δ so as to get the *most likely* in Ω, i.e. the that maximizes the Gaussian $p()\propto\exp\big(-\frac{1}{2}(-_{k+1|k})^{-1}_{k+1|k}(-_{k+1|k})\big)$. Since p is maximized when the exponent is minimized, the correction minimizes the squared *Mahalanobis distance*

$$\|\Delta\|^2_M=\Delta^{-1}_{k+1|k}\Delta, \tag{90}$$

which penalizes deviations along directions where the precision is higher.

Proposition 8.2 (Weighted correction)**.** *The solution of* $\min\|\Delta\|_M$ *s.t.* $_{k+1}\Delta=_{k+1}-_{k+1k+1|k}$ *is*

$$\Delta=_{k+1,M}\ _{k+1}, \qquad _{k+1,M}=_{k+1|k}\ _{k+1}\big(_{k+1k+1|kk+1}\big)^{-1}, \tag{91}$$

the weighted pseudoinverse *of* $_{k+1}$.

Proof. As in Prop. 7.1 with the Lagrangian $\frac{1}{2}\Delta^{-1}\Delta+\boldsymbol{\lambda}(-\Delta)$: stationarity gives $\Delta=\boldsymbol{\lambda}$, the constraint gives $=\boldsymbol{\lambda}$, hence $\Delta=()^{-1}$. □

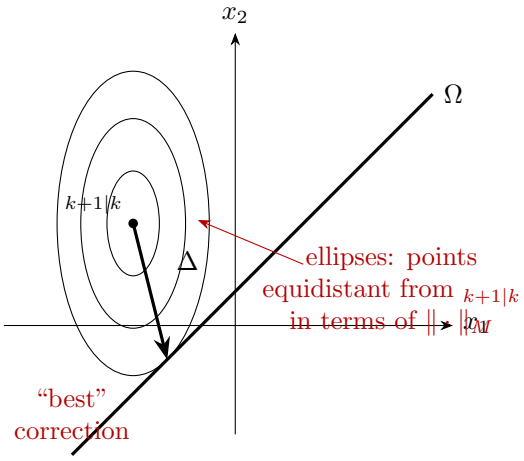


Figure 116: Weighted correction (redrawn): the best correction is the point of Ω reached by the smallest ellipse of the current covariance estimate, i.e. where an ellipse is tangent to Ω. Here $_{k+1|k} = (0.55^2, 1.1^2)$, $= (-1 \;\; 1)$.

Intuition

Now "close" is measured with the ruler of the filter's own uncertainty: a unit of error along a well-known direction costs a lot, along a poorly known direction it is cheap. So the filter moves mostly along the direction it is *least sure* about. *Worked numbers:* same $= (1 \;\; 1)$, $\nu = 2$ as before, but $_{k+1|k} = (4, 1)$ (x_1 is more uncertain). Then $= 5$ and $\Delta = \nu/5 = (1.6,\ 0.4)$ instead of the isotropic $(1, 1)$: the uncertain coordinate absorbs 80% of the correction.

8.3 Covariance correction and its flaw

Proposition 8.3 (Covariance correction)**.** *With the correction* (91)*,* $_{k+1} = _{k+1|k} - _{k+1,M} _{k+1} _{k+1|k}$.

Proof. Since $_{k+1} = _{k+1} \; _{k+1}$, $_{k+1} = _{k+1} \; (_{k+1} - _{k+1|k})$ and the error after the correction is $(-_M)(_{k+1} - _{k+1|k})$ (indices dropped). With $=$, $_M = ^{-1}$ and $\mathbf{X} := ^{-1}$ (symmetric), we get $_{k+1} = (-_M)(-_M) = -\mathbf{X} - \mathbf{X} + ^{-1} \; ^{-1} = -\mathbf{X} = -_M$. □

The resulting two-step filter is: prediction (86) and $_{k+1|k} = _k \; _k + _k$; correction $_{k+1} = _{k+1|k} + _{k+1,M} _{k+1}$ and $_{k+1} = _{k+1|k} - _{k+1,M} _{k+1} _{k+1|k}$.

Problem: with no measurement noise the covariance estimate becomes *singular*: $_{k+1} = - ^{-1} = \mathbf{0}$, so there is no uncertainty left in the direction normal to the measurement hyperplane.

Intuition

Continue the example $_{k+1|k} = (4, 1)$, $= (1 \;\; 1)$: $_M = \frac{1}{5} 164$
41, so $_{k+1} = 0.8 - 0.8$
$- 0.80.8$, with determinant 0. The filter claims to know $x_1 + x_2$ *exactly* (variance 0 along $(1, 1)$) — reasonable if the sensor is perfect, but a real sensor never is. That is why we now add measurement noise.

9 Kalman filter: the full version EKF deck, slides 16–21

Finally include also *measurement (sensor) noise*:

$$_{k+1}=_k\ _k+_k\ _k+_k, \qquad _k=_k\ _k+_k, \tag{92}$$

with $_k$,$_k$ white Gaussian, zero mean, covariances $_k$,$_k$. The dynamic equation is unchanged, so the predictions are the same: $_{k+1|k}=_k\ _k+_k\ _k$ and $_{k+1|k}=_k\ _k_k+_k$.

9.1 State correction with a noisy measurement

Because of the sensor noise the output is no longer certain: we only know that $_{k+1}$ is drawn from a Gaussian with mean $_{k+1k+1}$ and covariance $_{k+1}$. The correction therefore proceeds as follows.

1. Compute the *most likely output value* $^*_{k+1}$, given the predictions and the measured output $_{k+1}$.
2. Compute the associated *most likely hyperplane* $\Omega^*=\{:\ _{k+1}=^*_{k+1}\}$, a compromise between prediction and measurement.
3. Compute Δ as before, but using Ω^* in place of Ω.

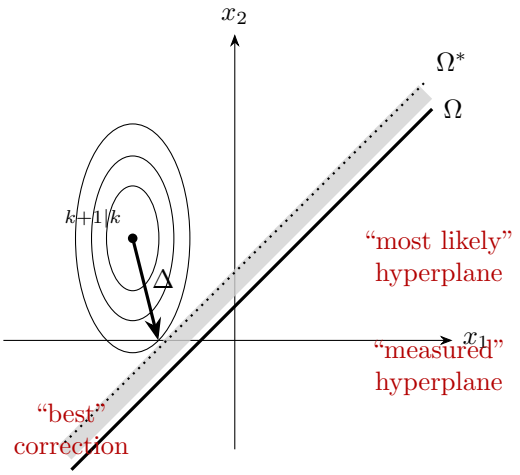


Figure 117: Correction with sensor noise (redrawn): the measured hyperplane Ω is uncertain (grey band); the correction targets the most likely hyperplane Ω^*, which lies between the prediction and Ω. Illustrative numbers for $=0.5$.

Theorem 9.1 (Kalman filter)**.** *For the system* (92) *the Kalman filter (KF) is*

$$\begin{aligned}
_{k+1|k}&=_k\ _k+_k\ _k,\\
_{k+1|k}&=_k\ _k_k+_k,\\
_{k+1}&=_{k+1|k}+_{k+1k+1}, \qquad _{k+1}=_{k+1}-_{k+1k+1|k},\\
_{k+1}&=_{k+1|k}-_{k+1k+1k+1|k},
\end{aligned} \tag{93}$$

with the Kalman gain matrix

$$_{k+1}=_{k+1|k}\ _{k+1}\big(_{k+1}_{k+1|kk+1}+_{k+1}\big)^{-1}. \tag{94}$$

Sketch of the gain. Let $=_{k+1|k}$ be the covariance of the predicted output. Fusing the predicted output $\mathcal{N}(_{k+1|k},\ )$ with the measurement (noise) gives the most likely output $^{*} =_{k+1|k} +(+)^{-1}$. Applying Prop. 8.2 with Ω^{*}: $\Delta =^{-1} (^{*}-_{k+1|k}) =^{-1} (+)^{-1} =_{k+1}$. If $\to 0$, $_{k+1} \to_{k+1,M}$ and we recover the previous filter. □

Intuition

decides *how much of the surprise to believe.* If is "large" the measurements are more reliable than the prediction; if is "small" the prediction is more reliable. Scalar example: x with $c = 1$, prior variance $P_{k+1|k} = 4$, sensor noise $W = 1$. Then $R = \frac{4}{4+1} = 0.8$. If the prediction is 0 and the sensor reads 5, the update is $\hat{x}_{k+1} = 0 + 0.8 \cdot 5 = 4$ (mostly the sensor), and $P_{k+1} = 4 - 0.8 \cdot 4 = 0.8$, smaller than both 4 and 1: fusing two uncertain pieces of information always yields something more certain than either. With a very noisy sensor ($W = 100$) we would get $R \approx 0.04$ and the prediction would dominate.

9.2 Properties of the KF

- relatively low computational load, plus a lot of quantities can be precomputed; no measurements need to be stored;
- it is a *minimum mean-square error* (MMSE) estimator: it minimizes $\|(_{k+1} - _{k+1})\|^2$ for all k;
- it is *correct*: it provides mean value and covariance of the *posterior* Gaussian distribution;
- if the noises are non-Gaussian, the KF is still the best *linear* estimator, but more accurate nonlinear filters may exist;
- if the system is time-invariant and detectable, the mean-square estimation error stays bounded; it generally does not converge to zero in the presence of persistent process noise.

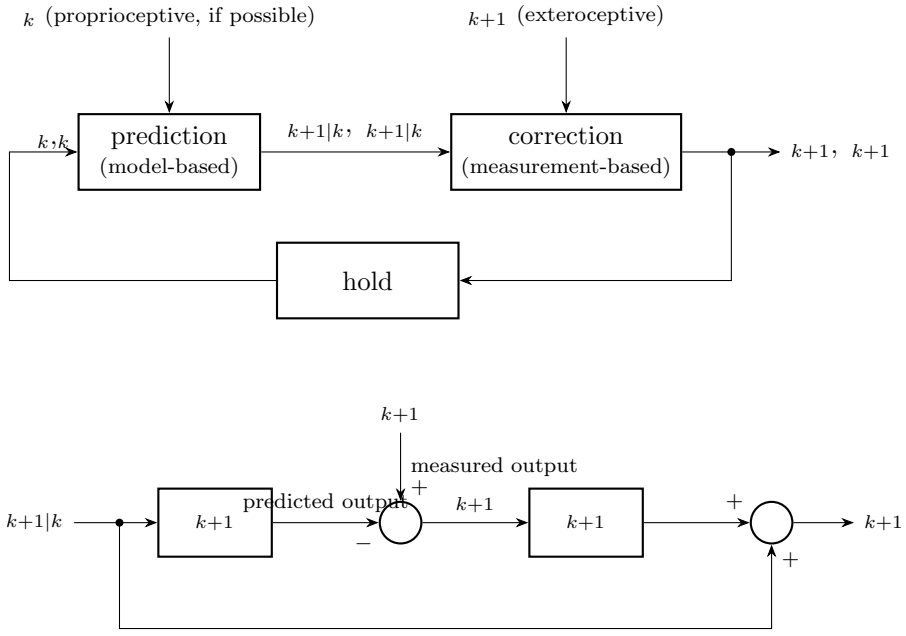


Figure 118: Wrapping up (redrawn). Top: the filter alternates a model-based prediction, driven by proprioceptive data when available, and a measurement-based correction driven by exteroceptive data; the output is held for one step and fed back. Bottom: the correction block: the innovation $_{k+1}$ (measured minus predicted output) is weighed by the gain $_{k+1}$ and added to the prediction.

10 The Extended Kalman Filter EKF deck, slides 22–24

Consider a *nonlinear* discrete-time system with noise

$$_{k+1}= \boldsymbol{f}_k(_k,_k)+_k, \qquad _k = \boldsymbol{h}_k(_k)+_k, \tag{95}$$

where $\boldsymbol{f}_k$ and $\boldsymbol{h}_k$ are C^1 functions. One simple way to build a filter: *linearize* the equations around the current estimate and apply the KF equations to the linear approximation. However, the nonlinear model is still used for the prediction, $_{k+1|k} = \boldsymbol{f}_k(_k,_k)$, and for the innovation, $_{k+1} =_{k+1} - \boldsymbol{h}_{k+1}(_{k+1|k})$.

Algorithm 10.1 (Extended Kalman Filter)**.**

$$\begin{aligned} _{k+1|k} &= \boldsymbol{f}_k(_k,_k), \\ _{k+1|k} &=_k \,_k\,_k+_k, \\ _{k+1} &=_{k+1|k} +_{k+1}\,_{k+1}, \\ _{k+1} &=_{k+1|k} -_{k+1}\,_{k+1}\,_{k+1|k}, \end{aligned} \tag{96}$$

with the Jacobians and gain

$$_k = \left.\frac{\partial \boldsymbol{f}_k}{\partial }\right|_{=_k}, \qquad _{k+1} = \left.\frac{\partial \boldsymbol{h}_{k+1}}{\partial }\right|_{=_{k+1|k}},$$
$$_{k+1} =_{k+1|k}\,_{k+1}\big(_{k+1}\,_{k+1|k}\,_k\,_{k+1} +_{k+1}\big)^{-1}.$$

Remarks. Prediction and innovation come from the nonlinear equations, while the covariance is propagated using their *linearizations*. Unlike the KF for linear-Gaussian systems, the EKF is *not* guaranteed to be optimal: even with Gaussian initial uncertainty and noises, the nonlinearity makes the state distribution non-Gaussian, and the EKF approximates it by a Gaussian. Accuracy depends on the validity of the linearization, hence on the size of the estimation uncertainty and on the system nonlinearities.

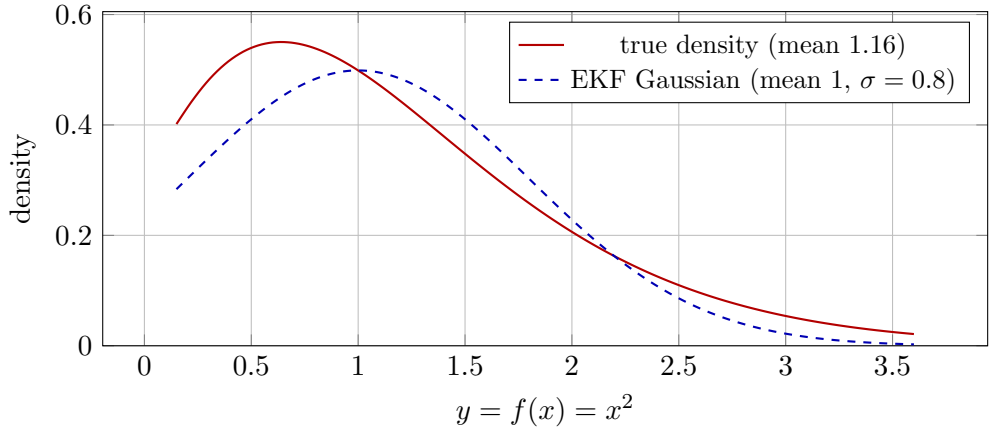


Figure 119: Why the EKF is only approximate (illustrative toy example, original): $x \sim \mathcal{N}(1, 0.4^2)$ is passed through $f(x) = x^2$. The true output density is skewed with mean $\mu^2 + \sigma^2 = 1.16$; the EKF replaces it with a Gaussian centered at $f(\mu) = 1$ and with standard deviation $|f'(\mu)|\sigma = 0.8$.

Intuition

Linearizing is like replacing a curved road by its tangent line at the current estimate: fine as long as the uncertainty cloud is small compared with the curvature, misleading when it is large.

This is why an EKF that has become overconfident or has a very large initial uncertainty can fail, and why the quality of the filter hinges on how good the estimate already is.

11 EKF localization of a unicycle with range-and-bearing landmarks book, Sect. 7.7.2

We now apply the EKF to a unicycle equipped with a sensor that measures *range* (relative distance) and *bearing* to *landmarks*: easily recognizable elements of the environment, natural (features, objects) or artificial (devices added for localization), whose positions are known to the robot. Depending on the robot configuration and the field of view, only a subset of the landmarks is visible (Fig. 120).

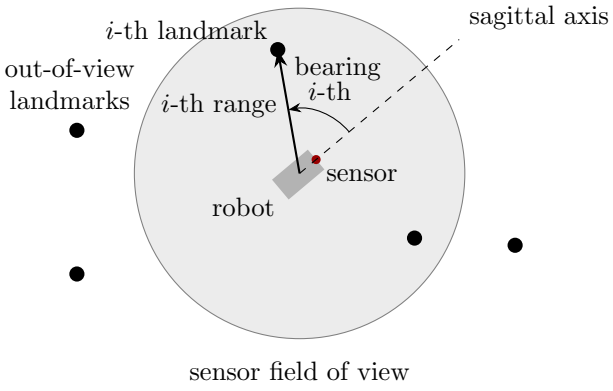


Figure 120: A typical setting for EKF-based localization with range and bearing measurements to known landmarks (redrawn). The bearing is measured from the robot's sagittal axis; the landmarks outside the field of view do not produce measurements.

11.1 Motion model

Any of the odometric models (79), (80) or (81) can serve as the discrete-time motion model, with v_k, ω_k reconstructed from the encoders as in Sect. 3. Taking the Euler one and adding *motion noise* $_k = (v_{1,k}, v_{2,k}, v_{3,k})$ (white, Gaussian, zero mean, covariance $_k$):

$$\begin{aligned} x_{k+1} &= x_k + v_k \cos\theta_k + v_{1,k}, \\ y_{k+1} &= y_k + v_k \sin\theta_k + v_{2,k}, \\ \theta_{k+1} &= \theta_k + \omega_k + v_{3,k}. \end{aligned} \tag{97}$$

11.2 Measurement model

Let L be the number of landmarks, $\boldsymbol{p}_{l,i} = (x_{l,i}, y_{l,i})$ the known position of the i-th one, and L_k the number of landmarks actually seen at t_k. Assume the identity of each observed landmark is known (landmarks are tagged, e.g. by shape or color), which allows building the *association map*

$$a : \{1, \ldots, L_k\} \to \{1, \ldots, L\},$$

where $a(i)$ is the identity of the i-th observed landmark. Assuming the sensor sits at the center of the wheel axle, the measurement model is

$$_k= \begin{pmatrix} \boldsymbol{h}_1(_k, a(1)) \\ \vdots \\ \boldsymbol{h}_{L_k}(_k, a(L_k)) \end{pmatrix} + \begin{pmatrix} _{1,k} \\ \vdots \\ _{L_k,k} \end{pmatrix}, \qquad \boldsymbol{h}_i(_k, a(i)) = \begin{pmatrix} \sqrt{(x_k - x_{l,a(i)})^2 + (y_k - y_{l,a(i)})^2} \\ (y_{l,a(i)} - y_k,\ x_{l,a(i)} - x_k) - \theta_k \end{pmatrix}, \tag{98}$$

with $_k$ white, Gaussian, zero mean, covariance $_k$. Both the motion and the measurement model are nonlinear.

11.3 The EKF equations for this problem

Prediction.

$$_{k+1|k} = \begin{pmatrix} \hat{x}_{k+1|k} \\ \hat{y}_{k+1|k} \\ \hat{\theta}_{k+1|k} \end{pmatrix} = \begin{pmatrix} \hat{x}_k + v_k \cos\hat{\theta}_k \\ \hat{y}_k + v_k \sin\hat{\theta}_k \\ \hat{\theta}_k + \omega_k \end{pmatrix}, \qquad _{k+1|k} =_k \ _k\,_k+_k,$$

$$_k= \frac{\partial \boldsymbol{f}}{\partial_k}\Big|_{_k} = \begin{pmatrix} 1 & 0 & -v_k \sin\hat{\theta}_k \\ 0 & 1 & v_k \cos\hat{\theta}_k \\ 0 & 0 & 1 \end{pmatrix}. \tag{99}$$

Correction. $_{k+1} =_{k+1|k} +_{k+1}\,_{k+1}, \ _{k+1} =_{k+1|k} -_{k+1}\,_{k+1}\,_{k+1|k}$, with $_{k+1} =_{k+1|k} \ _{k+1}(_{k+1}\,_{k+1|k}\,_{k+1}+_{k+1})^{-1}$ and the innovation $_{k+1} =_{k+1} -\boldsymbol{h}_{k+1}(_{k+1|k})$ (actual minus predicted measurements). The matrix $_{k+1}$ stacks the L_k blocks

$$\frac{\partial \boldsymbol{h}_i}{\partial_k}\Big|_{_{k+1|k}} = \begin{pmatrix} \dfrac{\hat{x}_{k+1|k} - x_{l,a(i)}}{\rho_i} & \dfrac{\hat{y}_{k+1|k} - y_{l,a(i)}}{\rho_i} & 0 \\ -\dfrac{\hat{y}_{k+1|k} - y_{l,a(i)}}{\rho_i^2} & \dfrac{\hat{x}_{k+1|k} - x_{l,a(i)}}{\rho_i^2} & -1 \end{pmatrix}, \qquad \rho_i = \|\boldsymbol{p}_{l,a(i)} - \hat{\boldsymbol{p}}_{k+1|k}\|, \tag{100}$$

$i = 1, \ldots, L_k$, where $\hat{\boldsymbol{p}}_{k+1|k} = (\hat{x}_{k+1|k}, \hat{y}_{k+1|k})$.

In the bearing row of (100) the denominators are ρ_i^2. This follows from differentiating (Y, X), whose partial derivatives are $-Y/(X^2+Y^2)$ and $X/(X^2+Y^2)$, with $X = x_l - x_k$, $Y = y_l - y_k$.

Intuition

Reading $_k$. The third column says that a heading error $\delta\theta$ moves the predicted position sideways by $v_k\,\delta\theta$ per step: heading uncertainty is turned into position uncertainty, and the longer the robot drives, the more it accumulates (exactly the drift mechanism of Sect. 4, now quantified inside).

Reading . The first row is the unit vector pointing from the landmark to the robot: moving the robot directly away from the landmark increases the range at rate 1, moving sideways does not change it (to first order). *Worked numbers:* predicted pose $(0, 0, 0)$, landmark at $(3, 4)$, so $\rho = 5$: predicted range 5, predicted bearing $(4, 3) - 0 = 0.927$ rad, and $_i = -0.6 - 0.80$ $0.16 - 0.12 - 1$. Moving the robot 1 m toward the landmark along x shortens the range by about 0.6 m (first entry -0.6); a heading error of $+0.1$ rad shifts the bearing by -0.1 rad (last entry -1). A single landmark gives two scalar equations for three unknowns (x, y, θ), so generally at least two landmarks are needed to constrain the full pose from one set of readings; the stacked is what combines them.

@l¿X¿X¿X@

Integration scheme	Update of x_{k+1} (and y)	Local error	Comment
Euler	$x_k + v_k \cos\theta_k$	$\approx \frac{1}{2} v\omega^2$	cheapest; uses the initial heading
Runge–Kutta (2nd order)	$x_k + v_k \cos(\theta_k + \frac{\omega_k}{2})$	$\approx \frac{1}{24} v\omega^{23}$	uses the mean heading; no division
Exact	$x_k + \frac{v_k}{\omega_k}(\sin\theta_{k+1} - \sin\theta_k)$	none (constant inputs)	needs a conditional for $\omega_k \approx 0$

Table 15: Odometric integration schemes (all have exact $\theta_{k+1} = \theta_k + \omega_k$).

@¿p2.6cm¿X¿X¿X@

Estimator	Model / sensors	What it provides	Main limitation
Dead reckoning (odometry)	nonlinear kinematics; proprioceptive only	only; exact only if $_0$, model and encoders are perfect	unbounded drift
Naive observer (no noise)	linear; exact output	via (min-norm)	errors along Ω not corrected; ignores uncertainty
KF, process noise only	linear; exact output	, via weighted $_M$	becomes singular
Kalman filter	linear; noisy output, Gaussian noises	, with gain ; MMSE, correct posterior	needs linear-Gaussian setting
Extended Kalman filter	nonlinear C^1 $\boldsymbol{f}, \boldsymbol{h}$; noisy exteroceptive data (e.g. landmarks)	,; drift removed	approximate (linearization, Gaussian assumption); no optimality guarantee

Table 16: From dead reckoning to the EKF.

12 Summary

12.1 Comparison of methods

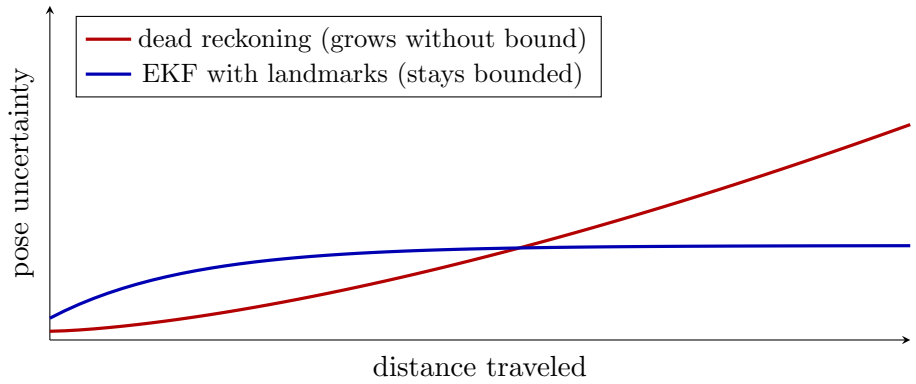


Figure 121: Qualitative sketch (not measured data): pure odometry accumulates uncertainty with the distance traveled, while the EKF alternates growth during prediction with reductions at every correction, so the uncertainty settles at a bounded level.

12.2 Key take-aways

- **Odometry is integration.** Under piecewise-constant inputs the unicycle moves on arcs; Euler, Runge–Kutta and exact formulas differ only in how well they follow the arc (error per step $\propto^2$, ^{3}, none), and the inputs should be reconstructed from encoders $(\Delta s, \Delta\theta)$, not taken from the commands.
- **Dead reckoning always drifts.** Slippage, calibration errors and encoder noise are integrated, and small wheel errors become large heading errors through the small wheelbase; accuracy can never be better than that of $_0$.
- **Predict, then correct.** The Kalman filter propagates a Gaussian (mean and covariance)

through the model, then pulls the mean toward the measurement along the direction the filter is least sure about, in the sense of the Mahalanobis distance.

- **The gain $= (+)^{-1}$ balances prediction and sensor.** Large : trust the measurements; small : trust the model. The filter is MMSE and correct for linear-Gaussian systems.
- **The EKF linearizes.** It keeps the nonlinear model for prediction and innovation and uses Jacobians for the covariance; this is what makes range-and-bearing localization with known landmarks workable, at the price of approximation and no optimality guarantee.